

3X-UIパネル ユーザーマニュアル

 العربية ·  English ·  Español ·  فارسی ·  Bahasa Indonesia ·  日本語 ·  Português ·  Русский ·  Türkçe ·  Українська ·  Tiếng Việt ·  简体中文 ·  繁體中文

3X-UIバージョン: 3.5.0。 本マニュアルはこのバージョンを基に作成されており、このバージョンに対応しています。3.4.2からの3.5.0変更点の概要は「[3.5.0の新機能](#)」セクションをご参照ください。

3X-UI ウェブパネル（Xray-core管理）の詳細な日本語マニュアルです。インターフェース上のすべてのフィールドと スイッチを解説しながら、機能・設定・運用について説明しています。

名称とラベルはパネルのインターフェースに準拠しています。*inbound / outbound* は 翻訳せずにそのまま使用します。

目次

- 3.5.0の新機能
- 1. はじめに、要件、インストール
 - 1.1. 3X-UIとは
 - 1.2. 対応オペレーティングシステムとアーキテクチャ
 - 1.3. インストール方法
 - 1.4. 初回起動とデフォルト認証情報
 - 1.5. ファイルの配置
 - 1.6. 管理コマンド `x-ui` (スクリプトメニュー)
 - 1.7. `x-ui` のサブコマンド (インタラクティブメニューなし)
 - 1.8. SQLite → PostgreSQLの移行
- 2. パネルへのログインとアクセスセキュリティ
 - 2.1. ログインフォーム
 - 2.2. 二要素認証 (2FA / TOTP)
 - 2.3. ログイン試行の制限 (ログインリミッター / ブルートフォース保護)
 - 2.4. 管理者のログイン名とパスワードの変更
 - 2.5. シークレットパス (URIパス / webBasePath) とパネルポート
 - 2.6. セッション有効期間 (タイムアウト)
 - 2.7. LDAP (同期と認証)
- 3. 概要 / ダッシュボード
 - 3.1. データ収集の基本原則
 - 3.2. CPU
 - 3.3. メモリ (RAM)
 - 3.4. スワップ (Swap)
 - 3.5. ディスク (Storage)
 - 3.6. システム稼働時間 (Uptime)
 - 3.7. システム負荷 (Load average)
 - 3.8. ネットワーク: 速度および総トラフィック量
 - 3.9. サーバーIPアドレス
 - 3.10. TCP/UDP接続数
 - 3.11. Xrayのステータスとプロセス管理
 - 3.12. パネルの更新 (3X-UI)
 - 3.13. ジオファイルの更新 (GeoIP / GeoSite)
 - 3.14. データベースのバックアップと復元
 - 3.15. その他のインターフェース要素
- 4. Inbounds: 作成と共通パラメータ
 - 4.1. 共通フォームフィールド
 - 4.2. Sniffing (スニффイング)
 - 4.3. Allocate (ポート割り当て戦略)
 - 4.4. External Proxy (外部プロキシ)
 - 4.5. Fallbacks (フォールバック)
 - 4.6. 定期的なトラフィックリセット
 - 4.7. 受信の JSON (上級者向け)
 - 4.8. Inbound の操作: QR / Edit / Reset / Delete と統計

- 5. プロトコル
 - 5.1. サポートされているプロトコルの一覧
 - 5.2. TLS / REALITY / トランスポートをサポートするプロトコル
 - 5.3. VLESS
 - 5.4. VMess
 - 5.5. Trojan
 - 5.6. Shadowsocks
 - 5.7. Dokodemo-door / Tunnel (透過フォワーダー)
 - 5.8. SOCKS / HTTP (プロトコル `mixed`)
 - 5.9. WireGuard (inbound)
 - 5.10. Hysteria (デフォルト v2)
 - 5.11. MTPROTO (Telegram プロキシ)
 - 5.12. プロトコル選択クイックリファレンス
- 6. トランスポート (Stream Settings)
 - 6.1. 転送ネットワークの選択
 - 6.2. RAW / TCP (`tcpSettings`)
 - 6.3. mKCP (`kcpSettings`)
 - 6.4. WebSocket (`wsSettings`)
 - 6.5. gRPC (`grpcSettings`)
 - 6.6. HTTPUpgrade (`httpupgradeSettings`)
 - 6.7. XHTTP / SplitHTTP (`xhttpSettings`)
 - 6.8. Hysteria トランスポート (`hysteriaSettings`)
 - 6.9. 付随パラメータ
- 7. 接続セキュリティ: TLS、XTLS、REALITY
 - 7.1. 違いの説明: TLS vs XTLS vs REALITY
 - 7.2. 「なし」モード (`none`)
 - 7.3. TLS モード
 - 7.4. REALITY モード
 - 7.5. 設定の実践的な推奨事項
- 8. クライアント
 - 8.1. クライアントのフィールド
 - 8.2. inbound への紐付け
 - 8.3. クライアントの操作
 - 8.4. 一括操作
 - 8.5. 検索・フィルター・並べ替え
 - 8.6. バッジとステータス
- 9. クライアントグループ
 - 9.1. クライアントグループとは何か、なぜ必要か
 - 9.2. グループとクライアント、inbound、ノード、プロトコルの関係
 - 9.3. グループ辞書と「空のグループ」
 - 9.4. グループのフィールドと列
 - 9.5. グループの作成
 - 9.6. グループの名前変更
 - 9.7. グループへのクライアントの追加
 - 9.8. グループからのクライアントの削除 (クライアント自体は削除しない)

- 9.9. グループのトラフィックリセット
- 9.10. グループの削除とグループのクライアントの削除
- 9.11. 「クライアント」 ページとの連携
- 9.12. API エンドポイントのまとめ
- 9.13. グループ別のトラフィック
- 10. サブスクリプション (Subscription)
 - 10.1. subId とは何か、リンクの構成方法
 - 10.2. サブスクリプションサーバーの設定
 - 10.3. 出力フォーマット
 - 10.4. サブスクリプション情報ページと QR コード
 - 10.5. サブスクリプションページのカスタムテンプレート
- 11. Xray: ルーティング、outbounds、DNS、および拡張機能
 - 11.1. エディターの構造: タブ/モード
 - 11.2. 一般設定 (General)
 - 11.3. ルーティングルール (routing)
 - 11.4. Outbounds (送信接続)
 - 11.5. バランサー (Balancers)
 - 11.6. DNS
 - 11.7. Fake DNS
 - 11.8. WireGuard / WARP / NordVPN
 - 11.9. リバースプロキシと TUN
 - 11.10. ログと統計 (Stats, metrics)
 - 11.11. 保存、再起動、および自動変換
 - 11.12. サブスクリプションからの Outbound (自動更新付き)
 - 11.13. WARP での IP ローテーション
- 12. ノード (マルチパネル、マスター/スレーブ)
 - 12.1. リスト上部のサマリー
 - 12.2. ノードの追加と編集
 - 12.3. TLS 検証 (https ノード向け)
 - 12.4. 各ノードに表示される内容
 - 12.5. ノードに対するアクション
 - 12.6. メトリクス履歴
 - 12.7. inbound とクライアントの同期方法
 - 12.8. ノードチェーン (サブノード/トランジティブノード)
 - 12.9. 3.5.0 の修正 (ノードの安定性)
 - 12.10. ノード: 3.3.0 での新機能
- 13. パネル設定
 - 13.1. パネルの保存と再起動
 - 13.2. 一般設定 (「パネル」 タブ / *General*)
 - 13.3. パネルへのアクセス: IP、ポート、パス、ドメイン、証明書
 - 13.4. セッション、パネルプロキシおよびトラステッドプロキシ (「プロキシとサーバー」 タブ / *Proxy and Server*)
 - 13.5. Telegram ボット (「Telegram ボット」 タブ / *Telegram Bot*)
 - 13.6. 日付と時刻 (「日付と時刻」 タブ / *Date and Time*)
 - 13.7. 外部トラフィックと Xray の動作 (「外部トラフィック」 タブ / *External Traffic*)
 - 13.8. その他: Xray 設定テンプレートとテスト URL

- 13.9. 管理者アカウントと API トークン
- 13.10. 3.3.0 での API の変更（インテグレーションに重要）
- 14. Telegramボット
 - 14.1. ボットの有効化と設定
 - 14.2. メインメニューとボタン
 - 14.3. ボットのコマンド
 - 14.4. クライアント管理（管理者のみ）
 - 14.5. 通知とレポート
 - 14.6. バックアップとログ
 - 14.7. 動作の特性
- 15. ジオデータベース（geoip / geosite およびカスタム）
 - 15.1. geoip.dat と geosite.dat とは
 - 15.2. 標準ジオファイルとその更新
 - 15.3. Xray による Geodata 自動更新（Geodata Auto-Update）
 - 15.4. バリデーションと制限事項
 - 15.5. パネル起動時の自動チェック
 - 15.6. ルーティングルールでのジオデータベースの使用
- 16. 運用：バックアップ、ログ、更新、CLI
 - 16.1. データベースのバックアップと復元
 - 16.2. ログの閲覧
 - 16.3. Xrayのログレベルとログ設定
 - 16.4. Xrayの管理：停止と再起動
 - 16.5. パネルの再起動と更新
 - 16.6. 定期タスク（cron）
 - 16.7. コンソールメニューとCLI（`x-ui`）
 - 16.8. パネルのアンインストール
 - 16.9. `x-ui migrateDB` コマンド

3.5.0の新機能

バージョン3.5.0は大規模なリリースです: MTProtoがマルチクライアントモデルに移行し (mtg-multiエンジン、クライアントごとのシークレット・クォータ・ad-tag)、管理対象ホストがグループ化され (1つのレコードに複数のinboundとアドレス)、PostgreSQLパネルの復元がSQLiteバックアップを受け付けるようになり、outboundには「Target Strategy」、「Real delay」テスト、Egress/Country列が追加され、バランサーは別のバランサーをfallbackとして使用できるようになりました。同梱のXrayコアは26.7.11です。以下では、3.4.2からの変更点をマニュアルのセクション別にまとめています。

セクション1の変更 – はじめに、要件、インストール

- Xray コアが **26.7.11** に更新されました。付随する自動マイグレーション: Shadowsocks の暗号方式 `none/plain` と VMess の `none/zero` がコアから削除され (保存済みの設定は自動的に書き換えられます)、公開アドレスへの暗号化されていない VLESS/Trojan outbound は保存時に拒否されます。
- 新しいコマンド `x-ui pgclient` [バージョン] と、PostgreSQL メニューの項目 **10. Install/Upgrade client tools (pg_dump/pg_restore)** が追加されました – PostgreSQL クライアントツールのインストール/更新を行います。
- スクリプトの修正: RHEL 系での PostgreSQL と fail2ban のインストール (EPEL)、Arch では完全な `pacman -Syu` を実行しない、32 ビット ARM での Xray バイナリの正しい名前 (`xray-linux-arm32`)、IP 証明書の発行前に自動検出された IPv4 を確認、デフォルトの ACME ポート選択時の誤った「Your input is invalid」を修正。

セクション2の変更 – パネルへのログインとアクセスセキュリティ

- IP リミット: 「死んだ」接続は 10 秒ごとのスキンのたびではなく **1 回だけ** BAN されるようになりました – fail2ban のカウンターが不要に増加しなくなり、`maxretry` を大きめに設定する必要はなくなりました。

セクション4の変更 – Inbounds: 作成と共通パラメータ

- inbound リストに **検索** (備考・ポート・プロトコルによる) が追加され、ノードのドロップダウンリスト (「デプロイ先」、「ノード」フィルター) が検索対応になりました。

セクション5の変更 – プロトコル

- **MTProto がマルチクライアントモデルに移行しました** (mtg-multi エンジン) : MTProto のユーザーは、独自のシークレット・クォータ・有効期限・ad-tag・パーソナルな `tg://proxy` リンクを持つ通常のクライアントになりました。inbound レベルの「Secret」フィールドは削除され (既存の inbound は自動的に変換されます)、「FakeTLS domain」は新しいシークレットのデフォルトドメインになりました。新しい inbound フィールド: **Max connections** (接続数の制限)、**Public IPv4/IPv6** (ad-tag middle proxy 用)。クライアントの変更は、他のユーザーの Telegram セッションを切断せずに「オンザフライ」で適用されます。
- WireGuard: inbound メニューにクライアント操作の完全なセット (Export All URLs、紐付け/紐付け解除、グループ) が追加され、エクスポートが **Config** と **Links** のタブに分割され、「WireGuard 許可

IP」フィールドが編集可能 になり（カンマ区切りで複数エントリ）、ノード上の inbound のクライアント設定では Endpoint がノードのアドレスを指すようになりました。

セクション7の変更 – 接続セキュリティ: TLS、XTLS、REALITY

- Finalmask + REALITY の組み合わせは保存時に拒否 されるようになりました（初回接続時に Xray-core がクラッシュする原因になっていました）。minClientVer のプレースホルダーは 26.3.27 に更新されました。
- Finalmask の新しい TCP マスクタイプ **XMC (Minecraft)**: ストリームを Minecraft トラフィックに偽装します (Hostname、Usernames、自動生成付きの必須 Password)。

セクション8の変更 – クライアント

- 新しい「速度」列 – 各クライアントのライブ速度（↑/↓、約 5 秒間の移動平均）。
- クライアント検索が再び Telegram ID で検索できるようになりました。クライアントフォームでは無効化された inbound が紐付けリストに表示されなくなり、「紐付け解除」ウィンドウでの重複エントリの蓄積が修正されました。
- MTProto クライアントには専用フィールドがあります: 「MTProto secret」（再生成付き）と 「Ad-tag（スポンサーチャンネル）」（32 の hex 文字）。クォータと有効期限が MTProto にも実際に適用されるようになりました。

セクション9の変更 – クライアントグループ

- クライアント情報ウィンドウにクライアントの グループ が表示されるようになりました。

セクション10の変更 – サブスクリプション (Subscription)

- 管理対象ホストがグループ化されました: 1 つのレコードが 複数の inbound（マルチ選択）と 複数のアドレス（タグ入力、各エントリに独自の :ポート を指定可能; アドレスの自動補完; 空の場合は inbound のアドレスを継承）をカバーします。リストの列はアドレスと inbound のチップ（「+N」付き）を表示し、操作と API はグループ（groupId）単位で動作し、一括の POST /panel/api/hosts/bulk/add が追加されました。ホストの並び順はグローバルになりました（並び順、次に備考順）。
- お知らせ（subAnnounce）のテキストがサブスクリプション情報ページにバナーとして表示されるようになり、カスタムテンプレートで変数 announce が利用可能になりました。
- ブラウザでの情報ページが JSON/Clash リンク でも開くようになりました（メインリンクだけではなくりました）。
- ホスト設定の Final Mask と Allow insecure が、それぞれ raw リンク（fm=）と Hysteria2（insecure=1 / skip-cert-verify: true）にも適用されるようになりました。
- 「更新間隔」（subUpdates）の範囲が 0-525600 に修正されました（従来の UI 上限 168 は 2.x からのアップグレード後に設定の保存をブロックしていました）。
- ネイティブ WireGuard クライアントが Clash および JSON サブスクリプションに含まれる ようになりました（以前は raw のみ）。

セクション11の変更 – Xray: ルーティング、outbounds、DNS、および拡張機能

- outbound エディター: 新しい「**Target Strategy**」フィールド（AsIs から ForceIPv4 まで 11 の値）、テストモード「**Real delay**」（トンネル確立を含む合計時間; HTTP モードは「ウォーム」接続で測定されるようになりました）、HTTP/Real テスト後の **Egress** 列（「目」アイコンの背後の出口 IP）と **Country** 列（国旗 + 国名、WARP ラベル）。
- バランサーの **Fallback** に別のバランサーを指定できるようになりました: パネルは非表示の loopback オブジェクト（`_bl_...`）を自動的に構築し、循環参照や使用中のバランサーの削除から保護します。プレフィックス `_bl_` は予約されています。
- 「基本ルーティング」タブに「**Default Outbound**」セクターが追加されました – どのルールにもマッチしなかったトラフィックを処理する outbound です（選択したものが先頭に移動されます）。
- プライベート IP 上の DNS サーバーが `geoip:private` ルールでブロックされなくなりました – パネルが管理された許可ルールを自動的に維持します。
- dial 設定 (sockopt) の Happy Eyeballs が実際に有効化されるようになりました。「Try delay」のデフォルトは 250 ms になり、明示的な 0 は保持されます。
- outbound サブスクリプションのインポート: `?plugin=` や末尾の `/` を持つ `ss://` リンクのポートが正しく解析されるようになりました。

セクション12の変更 – ノード（マルチパネル、マスター/スレーブ）

- 修正パッケージ: 変更なしのクライアント保存がノード inbound の稼働中トラフィックを壊さなくなりました。ノードの Host オーバーライドが初回承認時にマスターへ取り込まれます。自動更新が新しいクォータウィンドウを開きます。マスターでのクライアント削除がノード上でも完全に削除するようになりました。ノードの inbound は初回承認前に一掃されなくなりました。1 つの不正な inbound がノードのトラフィック同期を止めなくなりました。ポート競合チェックは自分のノードに限定されました。

セクション14の変更 – Telegramボット

- ボットのコマンドメニューに `/usage`、`/inbound`、`/restart` と、新しい管理者コマンド `/clearall`（全クライアントのトラフィックリセット、確認付き）が追加されました。
- オンラインクライアントのリストは `email - inbound` の備考としてラベル付けされます。バックアップと BAN ログのメッセージにホスト名が含まれます。Telegram ID による検索は設定のフォーマットに依存せず機能します。

セクション16の変更 – 運用: バックアップ、ログ、更新、CLI

- PostgreSQL パネルの復元が SQLite ファイルを受け付けるようになりました: 通常の `.db` バックアップまたはマイグレーション用 `.dump` が直接 PostgreSQL にインポートされます（単一トランザクション、Xray 停止前のチェック付き）。ファイル選択ダイアログは両方の DBMS で `.dump`、`.db` を受け付けます。「マイグレーションファイルをダウンロード」は PostgreSQL パネルにのみ残りました。
- `pg_dump` アーカイブの復元前に、パネルはダンプの読み取り可能性を確認し、バージョン不一致の場合は正確なコマンド `x-ui pgclient <バージョン>` を提示します。
- 起動時の自動修復: オーバーフローしたトラフィックカウンターがクランプされて復元されます。inbound ポートの古い UNIQUE 制約が削除されます（multi-node の妨げになっていました）。

- Xray ログ: 新しいジョブが 10 分ごとに、access ログと error ログが **64 MiB** を超えた場合に切り詰めます。毎日のクリーンアップは両方をクリアするようになりました。
- Docker: 証明書の自動更新が修正されました（crond が起動し、acme.sh の状態がボリュームに保存されます）。

1. はじめに、要件、インストール

1.1. 3X-UIとは

3X-UI は **Xray-core** サーバー向けのオープンソース・ウェブ管理パネルです。単一のVPSから複数ノードの分散構成まで、幅広いプロキシおよびVPNプロトコルの展開・設定・監視を行う統一された多言語ウェブインターフェースを提供します。

3X-UIはオリジナルのX-UIプロジェクトの拡張フォークです。より多くのプロトコルのサポート、安定性の向上、クライアントごとのトラフィック計測、そして多数の便利な機能が追加されています。

主な機能:

- **各種プロトコルのInbound** – VLESS、VMess、Trojan、Shadowsocks、WireGuard、Hysteria2、HTTP、SOCKS (Mixed)、Dokodemo-door / Tunnel、TUN、**MTPROTO** (Telegramプロキシ、3.3.0で追加)。
- **最新のトランスポートと暗号化** – TCP (Raw)、mKCP、WebSocket、gRPC、HTTPUpgrade、XHTTPに対応し、TLS、XTLS、REALITYで保護。
- **Fallback** – Xrayのfallback機能により1つのポートで複数プロトコルを処理 (例: 443番ポートでVLESSとTrojan)。
- **クライアントごとの管理** – トラフィッククォータ、有効期限、IPリミット、オンラインステータスの表示、ワンクリック招待リンク、QRコード、サブスクリプション。
- **トラフィック統計** – 各inbound、クライアント、outboundごとにリセット機能付きで集計。
- **複数ノード (ノード) のサポート** – 1つのパネルから複数サーバーを管理・スケール。
- **Outboundとルーティング** – WARP、NordVPN、カスタムルーティングルール、ロードバランサー、プロキシチェーン。
- **組み込みサブスクリプションサーバー** (複数の出力フォーマットに対応)。
- **Telegramボット** によるリモート監視と管理。
- **REST API** (Swaggerドキュメント内蔵)。
- **柔軟なストレージ** – SQLite (デフォルト) またはPostgreSQL。
- **13言語のインターフェース**、ダークテーマとライトテーマ。
- **Fail2ban連携** によるクライアントごとのIPリミット適用。

重要: このプロジェクトは個人利用のみを目的としています。違法な目的や本番環境での使用は推奨しません。

1.2. 対応オペレーティングシステムとアーキテクチャ

オペレーティングシステム

インストールスクリプトは `/etc/os-release` (または `/usr/lib/os-release`) の `ID` フィールドからディストリビューションを判別します。公式にサポートされているもの:

Ubuntu、Debian、Armbian、Fedora、CentOS、RHEL、AlmaLinux、Rocky Linux、Oracle Linux、Amazon Linux、Virtuozzo、Arch、Manjaro、Parch、openSUSE (Tumbleweed / Leap)、Alpine、およびWindows。

AlpineファミリーのシステムではOpenRCサービス（`rc-service` / `rc-update`）が使用され、それ以外ではsystemdが使用されます。CentOS 7ではパッケージのインストールに `yum` が、新しいリリースでは `dnf` が使用されます。ディストリビューションが認識されない場合、スクリプトはデフォルトで `apt-get` パッケージマネージャーの使用を試みます。

CPUアーキテクチャ

アーキテクチャは `uname -m` の出力から判別され、サポートされている値の1つに変換されます:

<code>uname -m</code> の値	3X-UIアーキテクチャ
<code>x86_64</code> , <code>x64</code> , <code>amd64</code>	<code>amd64</code>
<code>i*86</code> , <code>x86</code>	<code>386</code>
<code>armv8*</code> , <code>arm64</code> , <code>aarch64</code>	<code>arm64</code>
<code>armv7*</code> , <code>arm</code>	<code>armv7</code>
<code>armv6*</code>	<code>armv6</code>
<code>armv5*</code>	<code>armv5</code>
<code>s390x</code>	<code>s390x</code>

このリストにないアーキテクチャの場合、スクリプトは「Unsupported CPU architecture!」というメッセージを表示してインストールを中止します。

基本依存パッケージ

パネルのインストール前に、スクリプトは基本パッケージを自動でインストールします（ディストリビューションによって名前が異なります）：`cron` / `cronie` / `dcron`、`curl`、`tar`、`tzdata` / `timezone`、`socat`、`ca-certificates`、`openssl`。

1.3. インストール方法

方法1. インストールスクリプト（推奨）

rootとして1つのコマンドでインストールを実行します:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh)
```

スクリプトはrootの権限を必須とします: root以外で実行すると「Please run this script with root privilege」と表示されてエラーで終了します。

インストーラーの手順:

1. OSとアーキテクチャを判別します。
2. 基本依存パッケージをインストールします。

3. リリースアーカイブ `x-ui-linux-<arch>.tar.gz` をダウンロードし、`/usr/local/x-ui` ディレクトリに展開します。
4. 管理スクリプト `x-ui.sh` をダウンロードし、コマンド `/usr/bin/x-ui` としてインストールします。
5. ログディレクトリ `/var/log/x-ui` を作成します。
6. 初期設定を実行します: データベースの選択、認証情報の生成、ポートの選択、任意のSSL設定。
7. 自動起動サービス (systemdユニット `x-ui.service`、またはAlpine向けのOpenRC initスクリプト) をインストールして起動します。

インストール時のデータベース選択。 インストーラーは以下を提示します:

- 1) SQLite (デフォルト、クライアント数 < 500の場合に推奨) – `/etc/x-ui/x-ui.db` の単一ファイル、設定不要。
- 2) PostgreSQL (クライアント数が多い場合や複数ノードの場合に推奨)。PostgreSQLはローカルにインストールできます (専用ユーザーと `xui` という名前のデータベースが作成されます)、または既存サーバーへのDSNを指定できます。接続パラメーターはサービスの環境ファイル (ディストリビューションに応じて `/etc/default/x-ui`、`/etc/conf.d/x-ui`、または `/etc/sysconfig/x-ui`) に `XUI_DB_TYPE` および `XUI_DB_DSN` 変数として書き込まれます。

例: PostgreSQLパラメーターをサービス環境ファイルに書き込む。 PostgreSQLを選択しDSNを指定すると、インストーラーは環境ファイルにおよそ次のような行を追加します:

```
XUI_DB_TYPE=postgres
XUI_DB_DSN=postgres://xui:S3cretPass@127.0.0.1:5432/xui?sslmode=disable
```

ここで `xui` はユーザー名とデータベース名、`127.0.0.1:5432` はサーバーのアドレスとポート、`sslmode=disable` はローカル接続に適しています (リモートサーバーには通常 `require` を使用します)。

特定の (古い) バージョンのインストール。 バージョンタグを明示的に指定することができます – インストーラーは対応するリリースをダウンロードします:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/v2.4.0/install.sh)
v2.4.0
```

このインストール方法で許容される最低バージョンは `v2.3.5` です。より古いバージョンを指定すると「Please use a newer version (at least v2.3.5)」と表示されます。

dev ビルドのインストール。 バージョンタグに加えて、インストーラーは引数 `dev-latest` (エイリアス `dev`) を受け付けます – これにより `main` ブランチの最新コミットのローリングdevビルドがインストールされます:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh) dev-
latest
```

devビルドはコミットごとのプレリリース (タグ `dev-latest`) であり、安定版ではないため、最低バージョンの確認は行われません。実行時に「Installing the rolling dev build (tag: dev-latest). This is a per-

commit pre-release, not a stable version.」という警告が表示されます。引数なしの場合、インストーラーは最新の安定リリースをインストールします。devビルドはまだリリースされていない修正を確認する目的でのみ使用してください。通常の運用では安定版をインストールしてください。

方法2. Docker

デフォルトのSQLiteデータベースで起動する場合:

```
docker compose up -d
```

組み込みのPostgreSQLサービスで起動するには、`docker-compose.yml` 内の `XUI_DB_*` 行のコメントを解除し、プロファイルを指定して起動します:

```
docker compose --profile postgres up -d
```

イメージにはFail2ban（デフォルトで有効）が含まれており、クライアントごとのIPリミットを適用します。Fail2banは `iptables` を通じて違反者をブロックするため、`NET_ADMIN` ケーパビリティが必要です。`docker-compose.yml` では `cap_add` を通じてすでに提供されています。バージョン3.5.0 から、コンテナ内で SSL メニューから発行した証明書の **自動更新** が修正されました:entrypoint が `cron` を起動して `acme.sh` の cron ジョブを再登録し、`docker-compose.yml` には `acme.sh` の状態用の専用ボリュームが追加されました – 更新はコンテナの再作成後も維持されます（以前は証明書が約 90 日で通知なく期限切れになっていました）。`docker run` で手動起動する場合はケーパビリティを自分で追加する必要があります。そうしないとブロックはログに記録されるだけで適用されません:

例: 完全な `docker run` コマンド。 パネルポートのマッピング、ネットワークケーパビリティ、データベース用の永続ボリュームを含む最小構成:

```
docker run -d \
  --name 3x-ui \
  --restart unless-stopped \
  --cap-add=NET_ADMIN --cap-add=NET_RAW \
  -v $PWD/db:/etc/x-ui \
  -v $PWD/cert:/root/cert \
  -p 2053:2053 \
  ghcr.io/mhsanaei/3x-ui:latest
```

ボリューム `/etc/x-ui` はコンテナの再起動をまたいで `x-ui.db` ファイルを保持します。これがないと設定とアカウントが失われます。

```
docker run -d --cap-add=NET_ADMIN --cap-add=NET_RAW ... ghcr.io/mhsanaei/3x-ui
```

Dockerでは、パネルはコンテナのメインプロセスです: 自動起動はコンテナ内のサービスではなくコンテナの再起動ポリシー（例: `restart: unless-stopped`）によって制御されます。

1.4. 初回起動とデフォルト認証情報

初回インストール時（まだデフォルト認証情報が使用されている場合）、インストーラーはユーザー名、パスワード、ウェブパス、およびポートのランダムな値を生成します：

パラメーター	インストール時の生成方法	備考
ユーザー名 (Username)	10文字のランダム文字列	自動生成
パスワード (Password)	10文字のランダム文字列	自動生成
パネルのウェブパス (WebBasePath)	18文字のランダム文字列	ルートURLからパネルを発見されにくくする
パネルのポート (Port)	デフォルトは1024～62000の範囲のランダムなポート。手動で指定することも可能	<code>webPort</code> の「工場出荷時」の値は 2053 ですが、インストーラーが上書きします

インストールの最後に、スクリプトは最終サマリーを表示します: ユーザー名、パスワード、ポート、ウェブパス、APIトークン、および次の形式のアクセスリンク (Access URL) :

```
https://<ドメインまたはIP>:<ポート>/<ウェブパス>
```

SSL証明書が設定されていない場合、リンクは `http://` となり、スクリプトはSSLの設定（メニュー項目 19）が必要であるという警告を表示します。

認証情報の変更について。ログインとパスワードはランダムに生成されるため、インストール直後に保存してください。いつでもメニューの「Reset Username & Password」（下記参照）またはパネルのウェブインターフェースの設定から変更できます。リセット後、スクリプトは「Please use the new login username and password to access the X-UI panel. Also remember them!」と表示します。

インストール後、管理メニューを開くには `x-ui` コマンドを使用します（セクション1.6を参照）。

1.5. ファイルの配置

パス	用途
<code>/usr/local/x-ui/</code>	パネルのインストールディレクトリ（バイナリ <code>x-ui</code> 、スクリプト <code>x-ui.sh</code> ）
<code>/usr/local/x-ui/bin/xray-linux-<arch></code>	Xray-coreバイナリ（armv5/armv6/armv7では <code>xray-linux-arm</code> にリネームされます）
<code>/usr/bin/x-ui</code>	管理スクリプト（コマンド <code>x-ui</code> ）
<code>/etc/x-ui/x-ui.db</code>	SQLiteデータベースファイル（デフォルト）
<code>/var/log/x-ui/</code>	パネルのログディレクトリ
<code>/etc/systemd/system/x-ui.service</code>	サービスのsystemdユニット（Alpine以外）

パス	用途
/etc/init.d/x-ui	OpenRC initスクリプト（Alpineのみ）
/etc/default/x-ui ・ /etc/conf.d/x-ui ・ /etc/ sysconfig/x-ui	サービスの環境変数ファイル（パスはディストリビューションによって異なります）。XUI_DB_TYPE / XUI_DB_DSN はここに書き込まれます

データベースディレクトリは環境変数 `XUI_DB_FOLDER`（デフォルト `/etc/x-ui`）で変更でき、Xrayバイナリのディレクトリは `XUI_BIN_FOLDER`（デフォルトはパネルディレクトリからの相対パス `bin`）で変更できます。データベースファイル名は `x-ui.db` です。

例: データベースを別のディスクに移動する。 `x-ui.db` を `/etc/x-ui` ではなくマウントされたディスク `/data` などに保存するには、サービス環境ファイルに変数を設定してパネルを再起動します:

```
echo 'XUI_DB_FOLDER=/data/x-ui' >> /etc/default/x-ui
mkdir -p /data/x-ui
systemctl restart x-ui
```

データベースへの完全なパスは `/data/x-ui/x-ui.db` になります。

主な環境変数

変数	用途	デフォルト
<code>XUI_DB_TYPE</code>	データベースバックエンド: <code>sqlite</code> または <code>postgres</code>	<code>sqlite</code>
<code>XUI_DB_DSN</code>	PostgreSQL接続文字列（ <code>XUI_DB_TYPE=postgres</code> の場合）	—
<code>XUI_DB_FOLDER</code>	SQLiteデータベースファイルのディレクトリ	<code>/etc/x-ui</code>
<code>XUI_INIT_WEB_BASE_PATH</code>	ウェブパネルの初期URIパス（初回初期化時のみ）	<code>/</code>
<code>XUI_DB_MAX_OPEN_CONNS</code>	最大オープン接続数（PostgreSQLプール）	—
<code>XUI_DB_MAX_IDLE_CONNS</code>	最大アイドル接続数（PostgreSQLプール）	—
<code>XUI_ENABLE_FAIL2BAN</code>	Fail2banによるIPリミット適用を有効にする	<code>true</code>
<code>XUI_LOG_LEVEL</code>	ログレベル（ <code>debug</code> 、 <code>info</code> 、 <code>warning</code> 、 <code>error</code> ）	<code>info</code>
<code>XUI_DEBUG</code>	デバッグモード	<code>false</code>

例: 詳細ログを一時的に有効にする。 問題の診断のためにログレベルを `debug` に上げてサービスを再起動します:

```
echo 'XUI_LOG_LEVEL=debug' >> /etc/default/x-ui
systemctl restart x-ui
x-ui log      # デバッグログを表示
```

診断後は、ログが肥大化しないように `info` に戻してください。

環境変数によるウェブパネルの初期パス。変数 `XUI_INIT_WEB_BASE_PATH` は設定の初回初期化時にウェブパネルのURIパス（`webBasePath`）を設定します。これはDockerまたはsystemdでの展開時に、パネルのログインパスをあらかじめ固定するのに便利です。値は自動的に正規化されます – 先頭と末尾のスラッシュは必要に応じて追加され、空白のみまたは空の値は無視されます（その場合はデフォルトパス `/` が適用されます）。この変数は初回初期化時にのみ影響します: 設定がすでに作成されている場合は、ウェブインターフェースまたはメニュー項目「Reset Web Base Path」でパスを変更します。

1.6. 管理コマンド `x-ui` (スクリプトメニュー)

インストール後、コマンド `x-ui` (rootとして実行) は「3X-UI Panel Management Script」のインタラクティブメニューを開きます。項目は番号 (0~27の範囲) を入力することで選択します。多くの項目はスクリプト向けのサブコマンドとしても使用できます (セクション1.7を参照)。

メニューはテーマ別のブロックに分かれています。

インストールと更新

- **1. Install** – パネルのインストール (`install.sh` を実行)。インストール前にパネルがまだインストールされていないことを確認します。
- **2. Update** – `x-ui`のすべてのコンポーネントを最新バージョンに更新します。データは失われません。更新後、パネルは自動的に再起動されます。確認が必要です。
- **3. Update Menu** – パネルを再インストールせずに、管理スクリプト (`x-ui.sh` / コマンド `x-ui`) のみを最新バージョンに更新します。
- **4. Legacy Version** – 指定した (古い) バージョンのパネルをインストールします。スクリプトはバージョン番号 (例: `2.4.0`) を要求し、対応するリリースをダウンロードします。
- **5. Uninstall** – パネルをXrayと一緒に完全に削除します。サービスが停止・無効化され、ディレクトリ `/etc/x-ui/` と `/usr/local/x-ui/`、サービス環境ファイル、管理スクリプト自体が削除されます。確認が必要です (デフォルトは「いいえ」)。

認証情報と設定

- **6. Reset Username & Password** – パネルのユーザー名とパスワードをリセットします。独自の値を入力するか、空欄のままにするとランダム生成されます (ランダムなユーザー名は10文字、ランダムなパスワードは18文字)。設定済みの場合、二要素認証 (2FA) の無効化も提案されます。リセット後、パネルが再起動されます。
- **7. Reset Web Base Path** – パネルのウェブパスをリセットします: 新しいランダムパス (18文字) が生成され、パネルが再起動されます。以前のパスが漏洩したか忘れた場合に使用します。
- **8. Reset Settings** – パネルのすべての設定をデフォルト値にリセットします。認証情報 (ユーザー名とパスワード) とアカウントデータは失われません。確認が必要です。リセット後、パネルが再起動されます。
- **9. Change Port** – ウェブパネルのポートを変更します。ポート番号 (1~65535) を要求します。設定後、ポートを有効にするには再起動が必要です。
- **10. View Current Settings** – 現在の設定を表示します (`x-ui setting -show`)。使用中のデータベースバックエンド (SQLite、またはDSN内のパスワードがマスクされたPostgreSQL) とアクセスリンク

(Access URL) も表示されます。SSLが設定されていない場合、IPアドレス用のLet's Encrypt証明書の発行を提案します。

サービス管理

- **11. Start** – パネルサービスを起動します。パネルがすでに動作している場合、再起動は不要であるというメッセージが表示されます。
- **12. Stop** – パネルサービスを停止します。
- **13. Restart** – パネルサービスを再起動します。
- **14. Restart Xray** – パネル自体を再起動せずにXray-coreカーネルのみを再起動します (`systemctl reload x-ui`、Dockerでは `USR1` シグナルをパネルプロセスに送信) 。
- **15. Check Status** – サービスの状態を確認します (`systemctl status x-ui` または `rc-service x-ui status`) 。
- **16. Logs Management** – ログ管理: デバッグログの表示 (Debug Log、 `journalctl` 経由) 、および Alpine以外ではすべてのログのクリア (Clear All logs) 。

自動起動

- **17. Enable Autostart** – OS起動時のパネルの自動起動を有効にします (`systemctl enable x-ui` または `rc-update add`) 。
- **18. Disable Autostart** – OS起動時の自動起動を無効にします。

Dockerでは自動起動はコンテナの再起動ポリシーによって制御されるため、これらの項目は対応するヒントを表示するだけです。

セキュリティとネットワーク

- **19. SSL Certificate Management** – acme.shを通じたSSL証明書の管理: ドメイン用の証明書発行、失効、強制更新、既存ドメインの表示、パネル用の証明書パスの指定、IPアドレス用の短命 (約6日間、自動更新付き) 証明書の発行。
- **20. Cloudflare SSL Certificate** – CloudflareのDNS検証を通じたSSL証明書の発行。
- **21. IP Limit Management** – クライアントごとのIPリミット管理 (Fail2banベース) : ブロックの表示と解除など。
- **22. Firewall Management** – ファイアウォール管理 (ポートの開放/閉鎖とルールを表示) 。
- **23. SSH Port Forwarding Management** – SSHトンネルを通じてローカルマシンからパネルを開くためのSSHポートフォワーディングの設定。

パフォーマンスとメンテナンス

- **24. Enable BBR** – TCP BBR輻輳制御アルゴリズムの有効化/無効化 (Enable BBR / Disable BBR のサブメニュー) 。
- **25. Update Geo Files** – ジオデータベース (`.dat` ファイル) をソース選択付きで更新: Loyalsoldier (`geoip.dat`、 `geosite.dat`)、chocolate4u (`geoip_IR.dat`、 `geosite_IR.dat`)、runetfreedom (`geoip_RU.dat`、 `geosite_RU.dat`)、またはAll (すべて一度に)。更新後、パネルが再起動されます。

- **26. Speedtest by Ookla** – Speedtest by Ooklaによるネットワーク速度テストを実行します。
- **27. PostgreSQL Management** – 組み込み/接続済みのPostgreSQLインスタンスの管理（有効化と関連操作）。
- **0. Exit Script** – メニューを終了します。

1.7. x-ui のサブコマンド（インタラクティブメニューなし）

スクリプトでの使用のために、コマンド `x-ui` は直接サブコマンドをサポートします（引数なしの `x-ui` はメニューを開きます）：

コマンド	動作
<code>x-ui</code>	管理メニューを開く
<code>x-ui start</code>	パネルを起動する
<code>x-ui stop</code>	パネルを停止する
<code>x-ui restart</code>	パネルを再起動する
<code>x-ui restart-xray</code>	Xrayを再起動する
<code>x-ui status</code>	サービスの現在の状態
<code>x-ui settings</code>	現在の設定
<code>x-ui enable</code>	OS起動時の自動起動を有効にする
<code>x-ui disable</code>	自動起動を無効にする
<code>x-ui log</code>	ログを表示する
<code>x-ui banlog</code>	Fail2banのブロックログを表示する
<code>x-ui update</code>	パネルを更新する
<code>x-ui update-all-geofiles</code>	すべてのgeoファイルを更新する
<code>x-ui migrateDB [file]</code>	<code>.db</code>  <code>.dump</code> の変換 (SQLite)
<code>x-ui legacy</code>	旧バージョンをインストールする
<code>x-ui install</code>	パネルをインストールする
<code>x-ui uninstall</code>	パネルを削除する

1.8. SQLite → PostgreSQLの移行

既存のSQLiteインストールをPostgreSQLに移行できます：

```
x-ui migrate-db --dsn "postgres://xui:password@127.0.0.1:5432/xui?sslmode=disable"
# 次に /etc/default/x-ui に XUI_DB_TYPE と XUI_DB_DSN を設定して再起動します：
systemctl restart x-ui
```

元のSQLiteファイルはそのまま残ります – 新しいバックエンドの動作を確認した後でのみ手動で削除してください。

例: PostgreSQLへの切り替えを確認する。 移行後、設定表示コマンドでパネルが実際に新しいバックエンドで動作していることを確認します – 出力にPostgreSQLが表示されているはずです（DSN内のパスワードはマスクされます）：

```
x-ui settings | grep -i -E 'db|dsn'
```

パネルが開き、アカウントが存在すれば、元の `x-ui.db` を削除できます。

2. パネルへのログインとアクセスセキュリティ

このセクションでは、3X-UIパネルの管理者認証に関するすべての事項を説明します。ログインフォーム、二要素認証（TOTP）、ブルートフォース攻撃からの保護、認証情報の変更、シークレットパスおよびパネルポートの変更、セッション有効期間、そしてLDAP経由の同期・認証について扱います。

2.1. ログインフォーム

ログインページはパネルのシークレットパス（webBasePath）のルートから提供されます。ユーザーがすでにログイン済みの場合は、自動的に `.../panel/` にリダイレクトされます。このページにはテーマ切り替え、インターフェイス言語の選択、およびフォーム本体があります。

フォームのフィールド:

フィールド	ラベル/ヘッダー	必須	説明
ユーザー名	«Имя пользователя»	はい	管理者のログイン名。空欄はクライアント側で拒否され、サーバー側では「Введите имя пользователя」というメッセージで拒否されます。
パスワード	«Пароль»	はい	管理者のパスワード。空欄は「Введите пароль」というメッセージで拒否されます。
2FAコード	«Код 2FA»	2FA有効時のみ	二要素認証がパネルで有効になっている場合にのみ表示されます。認証アプリからの6桁のコードです。

«Войти» ボタンはフォームを `POST /login` に送信します。

動作とメッセージ:

- ログイン成功時には「Вход выполнен успешно」と表示され、`.../panel/` に遷移します。
- 認証情報のエラーや誤った2FAコードの場合、サーバーは統一されたメッセージを返します:
「Неверные данные учетной записи。」（英: *Invalid username or password or two-factor code.*）。これは意図的な設計であり、パネルはログイン名・パスワード・コードのいずれが誤りかを示しません。ブルートフォース攻撃を容易にしないためです。
- «Код 2FA»フィールドは、`POST /getTwoFactorEnable` へのリクエストに基づいてパネルが表示・非表示を切り替えます。このリクエストは認証前に現在の2FAステータスを返します。
- サーバーセッションが期限切れになった場合、次のリクエスト時に「Сессия истекла. Войдите в систему снова」と表示され、ユーザーはログインページにリダイレクトされます。

CSRFに関する注記: フォーム送信前にクライアントはCSRFトークン（`GET /csrf-token`）を取得します。`/login` および `/logout` はCSRF検証で保護されています。

例: APIを使ったログイン。 2FAが無効の場合、ログイン名とパスワードだけで十分です。2FAが有効な場合は `twoFactorCode` フィールドを追加します:

```
# 2FAなし
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=ВашПароль'

# 2FA有効時 - 6桁のコードを追加
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=ВашПароль&twoFactorCode=123456'
```

成功するとサーバーはセッションcookieを含む Set-Cookie を返します。このcookieを以降の /panel/api/... へのリクエストに使用します。

2.2. 二要素認証 (2FA / TOTP)

3X-UIの2FAは**TOTP**標準に基づいて実装されており、任意の認証アプリ（Google Authenticator、Aegis、FreeOTPなど）と互換性があります。パラメーターはコードに固定されています: アルゴリズム**SHA1**、**6**桁、期間**30**秒、発行者（issuer）**3x-ui**、ラベル **Administrator**。

例: QRコードがエンコードするotpauth URI。 認証アプリでカメラが使えない場合、次のリンクを使ってトークンを手動で追加できます（JBSWY3DPEHPK3PXP の部分を実際のBase32シークレットに置き換えてください）：

```
otpauth://totp/3x-ui:Administrator?secret=JBSWY3DPEHPK3PXP&issuer=3x-
ui&algorithm=SHA1&digits=6&period=30
```

パラメーター **algorithm=SHA1**、**digits=6**、**period=30** はパネルに固定された値と対応しており、変更する必要はありません。

設定は**Настройки** → **Учетная запись**セクションの**«Двухфакторная аутентификация»**タブにあります。

要素	テキスト	説明
トグル	«Включить 2FA»	二要素認証を有効/無効にします。
説明	«Добавляет дополнительный уровень аутентификации для повышения безопасности.»	トグルの下ヒント。

2FAを有効にする方法

トグルをオンにすると、パネルはローカルで新しいシークレットを生成します。これはBase32エンコード（A-Zと2-7のアルファベット）のランダムな文字列です。「Включить двухфакторную аутентификацию」というウィンドウが開き、手順が表示されます：

1. 「認証アプリでこのQRコードをスキャンするか、QRコードの隣に表示されているトークンをコピーしてアプリに貼り付けてください」。QRコードの下にシークレットがテキスト形式で表示され、QRコードをクリックするとシークレットがクリップボードにコピーされます（「Скопировано」と表示されます）。

2. 「アプリからコードを入力してください」 – アプリが生成した6桁のコードを入力する必要があります。コードはブラウザ側で検証されます。パネルは生成したシークレットを使って現在のTOTPを計算し、入力値と比較します。コードが正しくない場合は「Неверный код」と表示されます。フィールドは正確に6桁の数字のみを受け付けます。

確認が成功した場合のみ、シークレットと有効フラグが保存されます。保存時には「Двухфакторная аутентификация была успешно установлена」と表示されます。

重要: 設定セクションの変更は共通の**«Сохранить»ボタンで適用されます。通常、パネルの再起動が必要です（「Сохраните изменения и перезапустите панель для их применения」）。2FAを初めて有効にした際、サーバーはすべてのアクティブなセッションを無効化**します（「ログインエポック」を増加させます）。そのため設定適用後は再度ログインが必要で、その際は2FAコードが必要になります。

2FAを無効にする方法

トグルを再度押すと「Отключить двухфакторную аутентификацию」ウィンドウが開き、「Введите код из приложения, чтобы отключить двухфакторную аутентификацию.」というヒントが表示されます。正しいコードを入力すると（3.4.2 からはコードがサーバー側で検証されます）フラグとシークレットがクリアされ、「Двухфакторная аутентификация была успешно удалена」と表示されます。

ログイン時のコード検証

ログイン時、サーバーは保存されたシークレットを取得し、現在のTOTPと送信された2FAコードを比較します。一致しない場合はログイン失敗とみなされますが、ユーザーには同じ統一メッセージ「Неверные данные учетной записи。」が表示されます。

アクセスの回復 (recovery)

3X-UIには独立した「リカバリーコード」の仕組みはありません。認証アプリへのアクセスを失った場合、パネルのインターフェイスからログインを回復することはできません。唯一の方法は、サーバー上のデータベースで直接2FAを無効化することです。設定テーブルの twoFactorEnable キーを false にリセットし（必要に応じて twoFactorToken もクリアし）、パネルを再起動します。そのため、2FA有効化時にシークレット（Base32トークン）を安全な場所に保存することを推奨します。

例: サーバー上での緊急の2FA無効化。 SSHでサーバーにアクセスできる場合は、パネルを停止し、設定テーブルのキーをリセットして、パネルを再起動します:

```
x-ui stop
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='false' WHERE key='twoFactorEnable';"
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='' WHERE key='twoFactorToken';"
x-ui start
```

この後、ログインはログイン名とパスワードのみで行われ、必要であれば2FAを再設定できます。

認証情報の変更との関連: ログイン名/パスワードを変更した場合（2.4参照）、古いシークレットが新しい認証情報でのアクセスをブロックしないよう、サーバーは2FAを自動的に無効化します。

2.3. ログイン試行の制限（ログインリミッター / ブルートフォース保護）

パネルにはログイン失敗の組み込みリミッターがあります（アプリケーションレベルのfail2ban相当）。パラメーターはコードに固定されており、インターフェイスからは設定できません:

パラメーター	値	用途
最大失敗回数	5	ウィンドウ内で許容される失敗試行回数。
カウントウィンドウ	5分	失敗がカウントされるスライディングウィンドウ（古いものは破棄されます）。
ブロック時間（cooldown）	15分	閾値超過後にキーがブロックされる時間。

仕組み:

- ブロックキーは**「IP + ログイン名」の組み合わせ**から構成されます（ログイン名は小文字に変換され、スペースはトリムされます）。つまり、ブロックはパネル全体ではなく、特定の「アドレス + ユーザー名」の組み合わせに適用されます。
- 失敗試行（誤ったログイン名/パスワードまたは誤った2FAコード）のたびにカウンターが増加します。5分以内に5回失敗すると、そのキーは15分ブロックされます。ブロック中は、たとえ認証情報が正しくても、その組み合わせによる試行は同じメッセージ「Неверные данные учетной записи.»で即座に拒否されます。
- ログイン成功時にカウンターは即座にリセットされ、その組み合わせのブロックが解除されます。
- クライアントのIPアドレスは信頼されたプロキシを考慮して決定されます（trustedProxyCIDRsを参照）。X-Real-IP および X-Forwarded-For ヘッダーは、信頼されたアドレスからのリクエストの場合のみ受け付けられます。そうでない場合は実際の接続アドレスが使用され、取得できない場合は文字列 unknown が使用されます。

すべての試行がログに記録されます。失敗時はユーザー名、IP、原因、ブロック中は blocked_until 時刻とともにサーバーログに警告が書き込まれます。Telegramボット経由のログイン通知（tgNotifyLogin – «Уведомление о входе»）が有効な場合、管理者はユーザー名、IP、および成功・失敗・ブロックされたすべての試行の時刻を通知として受け取ります。

例: Telegramへのログイン通知。 tgNotifyLogin が有効な場合、各試行後に管理者はおおよそ次のようなメッセージを受け取ります:

```
Уведомление о входе
Пользователь: admin
IP: 203.0.113.45
Время: 2026-06-10 14:32:07
Статус: успешно
```

ブロックされた「IP + ログイン名」の組み合わせの場合、ステータスにはリミッターによって試行が拒否されたことが示されます。

2.4. 管理者のログイン名とパスワードの変更

Настройки → Учетная запись セクション、**«Учетные данные администратора»**タブ。フィールド:

フィールド	テキスト	説明
現在のログイン名	«Текущий логин»	現在のユーザー名。現在のログイン名と一致する必要があります。一致しない場合は変更が拒否されます。
現在のパスワード	«Текущий пароль»	本人確認のための現在のパスワード。
新しいログイン名	«Новый логин»	新しいユーザー名。空欄にはできません。
新しいパスワード	«Новый пароль»	新しいパスワード。空欄にはできません。

変更は**«Подтвердить»**ボタンで適用され、POST /panel/setting/updateUser に送信されます。

サーバーのロジックとメッセージ:

- 「現在のログイン名」が実際のものと一致しないか「現在のパスワード」が誤りの場合 – 「Произошла ошибка при изменении учетных данных администратора.» と 「Неверное имя пользователя или пароль」 の説明が表示されます。
- 新しいログイン名または新しいパスワードが空欄の場合 – 「Новое имя пользователя и новый пароль должны быть заполнены」 と表示されます。
- 成功時 – 「Вы успешно изменили учетные данные администратора.」。パスワードはbcryptハッシュとして保存されます。

2FA コードによる確認 (3.4.2 から)。パネルで二要素認証が有効になっている場合、ログイン名/パスワードの変更には認証アプリの **現在のコードの入力** が追加で必要になります。「**認証情報の変更**」ウィンドウが開き、「Введите код из приложения, чтобы изменить учётные данные администратора.」というヒントが表示されます。コード (twoFactorCode) はサーバー側で検証され、コードが誤っているか空の場合は変更が適用されません。この同じ確認は、2FA の無効化時にも必要になりました (2.2 参照)。

例: APIを使った認証情報の変更。 リクエストには有効なセッションcookie (ログイン時に取得) と現在のログイン名/パスワードの確認が必要です:

```
curl -X POST https://panel.example.com:2053/мой-секрет/panel/setting/updateUser \
  -b 'session=БАША_СЕССИОННАЯ_COOKIE' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'oldUsername=admin&oldPassword=СтарыйПароль&newUsername=root&newPassword=НовыйСложныйПароль'
```

成功後、現在のセッションは無効化されます。新しい認証情報で再度ログインする必要があります。

認証情報変更の重要な影響:

- **すべての既存セッションが無効化されます**（ユーザーの `login_epoch` カウンターが増加します）。そのため変更後、パネルは自動的にログアウトしてログインページにリダイレクトします。再度ログインが必要です。
- 変更時に**2FAが有効だった場合、自動的に無効化されます**（フラグとシークレットがリセットされます）。ログイン名/パスワード変更後、二要素認証を再設定する必要があります。

2FAが有効な場合、フォーム送信前に「Изменить учетные данные」ウィンドウが開き、「Введите код из приложения, чтобы изменить учетные данные администратора.」というヒントが表示されます。現在の2FAコードを確認した場合のみ認証情報を変更できます。

2.5. シークレットパス（URIパス / `webBasePath`）とパネルポート

これらのパラメーターは**Настройки** → **Панель**セクションにあり、パネルの「隠蔽性」とアクセス可能性に直接影響します。保存とパネルの再起動後に適用されます。

フィールド	テキスト	デフォルト値	説明
パネルポート	«Порт панели»（ <code>panelPort</code> ）、ヒント「Порт, на котором работает панель」	2053	ウェブインターフェイスのTCPポート。
URIパス	«URI-путь»（ <code>panelUrlPath</code> ）、ヒント「Должен начинаться с '/' и заканчиваться '/'」	/	シークレットベースパス（ <code>webBasePath</code> ）。パネルはこのパスからのみアクセスできます（例: <code>/мой-секрет/</code> ）。
パネル管理用IPアドレス	«IP-адрес для управления панелью»（ <code>panelListeningIP</code> ）、ヒント「Оставьте пустым для подключения с любого IP」	空欄	パネルがリッスンするアドレス。空欄 = すべてのインターフェイス。
パネルドメイン	«Домен панели»（ <code>panelListeningDomain</code> ）、ヒント「Оставьте пустым для подключения с любых доменов и IP」	空欄	ドメイン（Host）によるアクセス制限。
パネル証明書の公開鍵パス	<code>publicKeyPath</code> 、ヒント「Введите полный путь, начинающийся с '/'」	空欄	パネルへのHTTPSアクセス用のTLS証明書。
パネル証明書の秘密鍵パス	<code>privateKeyPath</code> 、同じヒント	空欄	TLS秘密鍵。

ベースパス（`webBasePath`）の動作:

- 値は自動的に正規化されます。/ で始まらない場合は先頭に追加され、/ で終わらない場合は末尾に追加されます。つまり実際のパスは常に `/.../` の形式になります。
- ベースパスはパネル本体、アセット、セッションcookieに適用されます（cookieはこのパスに対してのみ発行されます）。

セキュリティの推奨事項（«Предупреждения безопасности»セクション）：パネルは設定が「公開されすぎている」場合に警告を表示します：

- 「パネルが通常のHTTPで動作しています – 本番環境ではTLSを設定してください。」
- 「標準ポート2053は広く知られています – ランダムなポートに変更してください。」
- 「デフォルトのベースパス「/」は広く知られています – ランダムなパスに変更してください。」

つまり、本番サーバーでは非標準ポート、非自明なURIパス、TLS証明書を設定する必要があります。

例: 本番環境向けの「隠蔽された」パネル設定。Настройки → Панельセクションで次のような値を設定します：

フィールド	値
パネルポート	34571（2053の代わりにランダムなポート）
URIパス	/aXf9Qm2/（非自明、/で始まり/で終わる）
パネル証明書の公開鍵パス	/etc/letsencrypt/live/panel.example.com/fullchain.pem
パネル証明書の秘密鍵パス	/etc/letsencrypt/live/panel.example.com/privkey.pem

保存と再起動後、パネルは `https://panel.example.com:34571/aXf9Qm2/` からのみアクセスでき、セキュリティ警告は消えます。

2.6. セッション有効期間（タイムアウト）

«Продолжительность сессии»フィールド（sessionMaxAge）はパネル/インターバルの設定にあります。

フィールド	テキスト	デフォルト値	単位	説明
セッション継続時間	«Продолжительность сессии」、ヒント「Продолжительность сессии в системе (значение: минута)」	360	分	管理者のセッションcookieの有効期間。

動作：

- 値は分単位で指定します（デフォルト360分 = 6時間）。cookieの設定時には秒に変換されます。
- 値が0より大きい場合、セッションcookieには対応するMaxAgeが設定されます。この期間が過ぎるとcookieは無効になり、次のリクエスト時にユーザーは「Сессия истекла. Войдите в систему снова」を受け取ります。
- セッションは、認証情報の変更や2FAの初回有効化時（login_epochメカニズムによる。2.4および2.2を参照）、または明示的なログアウト（POST /logout）によっても早期に無効化されます。
- セッションcookieにはHttpOnlyが設定され、SameSite=Laxポリシーが適用されます。パネルへの直接HTTPS接続時にはSecureフラグが設定されます。

タイムアウト自体に加えて、関連する通知があります: **«Задержка уведомления об истечении сессии»** (`expireTimeDiff`、ヒント「Получение уведомления об истечении срока действия сессии до достижения порогового значения (значение: день)」、デフォルト `0`) – 期限切れ前に事前警告を受け取ることができます。

2.7. LDAP (同期と認証)

LDAPセクションでは2つの機能が利用できます: (1) ローカルパスワードが一致しなかった場合にLDAPで管理者ログインを認証する、および(2) ディレクトリからクライアントの状態 (VLESSフラグの有効/無効) を定期的に同期する。

ログイン時の動作: サーバーはまずローカルのbcryptハッシュを確認します。一致しない場合でLDAPが有効な場合、パネルはディレクトリでユーザーを認証しようとします。Bind DN が設定されている場合はサービスbindを実行し、フィルターと属性でユーザーレコードを検索し、見つかったDNと入力されたパスワードでbindを試みます。成功するとログインが許可されます。(LDAP認証が成功した後も、2FAが有効な場合はTOTPコードが引き続き確認されます。)

セクションのフィールド:

フィールド	テキスト	デフォルト値	説明
LDAP同期を有効にする	«Включить LDAP-синхронизацию» (<code>enable</code>)	false	LDAP統合のメインスイッチ。
LDAPホスト	«LDAP-хост» (<code>host</code>)	空欄	LDAPサーバーのアドレス。
LDAPポート	«Порт LDAP» (<code>port</code>)	389	ポート。LDAPSの場合は通常636。
TLS (LDAPS) を使用する	«Использовать TLS (LDAPS)» (<code>useTls</code>)	false	有効にすると、 <code>ldaps://</code> スキームが使用されます (デフォルトではサーバー証明書の検証付き)。

フィールド	テキスト	デフォルト値	説明
TLS 証明書の検証をスキップする	«Пропускать проверку TLS-сертификата» (<code>ldapInsecureSkipVerify</code>)	<code>false</code>	3.4.2 で追加。LDAPS 時にサーバー証明書の検証を無効にします – 内部/自己署名 CA 向けです。ヒント: 「安全ではありません – サーバー証明書の検証を無効にします。内部/信頼されていない CA のみ使用してください」。このトグルは「TLS (LDAPS) を使用する」が有効な場合のみ 利用できます。
Bind DN	«Bind DN» (<code>bindDn</code>)	空欄	初期bind/検索用のサービスアカウントのDN。空欄の場合はbindを実行しません（匿名検索）。
Bindパスワード	ヒント: 「Настроено; оставьте пустым, чтобы сохранить текущий пароль.» / 「Не настроено.» / 「Настроено – введите новое значение для замены」	空欄	<code>Bind DN</code> のパスワード。個別に保存されます。既存のパスワードを保持するにはフィールドを空欄にします。
Base DN	«Base DN» (<code>baseDn</code>)	空欄	検索を行うサブツリーのルート（検索はサブツリー全体に対して再帰的に行われます）。
ユーザーフィルター	«Фильтр пользователя» (<code>userFilter</code>)	(<code>objectClass=person</code>)	アカウントを選択するLDAPフィルター。認証時にログイン名はエスケープされてフィルターに埋め込まれます。
ユーザー属性 (username/ email)	«Атрибут пользователя (username/ email)» (<code>userAttr</code>)	<code>mail</code>	ログイン名/クライアント識別子と照合する属性（例: <code>mail</code> または <code>uid</code> ）。
VLESSフラグ属性	«Атрибут VLESS-флага» (<code>vlessField</code>)	<code>vless_enabled</code>	クライアントのVLESSアクセスを有効にするかどうかを決定する属性。

フィールド	テキスト	デフォルト値	説明
共通フラグ属性 (オプション)	«Общий атрибут флага (опц.)» (<code>flagField</code>)、ヒント「Если задано, переопределяет флаг VLESS – напр. <code>shadowInactive</code> .」	空欄	設定されている場合は <code>vless_enabled</code> の代わりに使用されます。
Truthyな値	«Truthy-значения» (<code>truthyValues</code>)、ヒント「Через запятую; по умолчанию: <code>true,1,yes,on</code> 」	<code>true,1,yes,on</code>	「有効」として解釈されるフラグ属性の値のリスト。
フラグを反転する	«Инвертировать флаг» (<code>invertFlag</code>)、ヒント「Включите, когда атрибут означает «отключено» (напр. <code>shadowInactive</code>).」	<code>false</code>	フラグの意味を反転します。
同期スケジュール	«Расписание синхронизации» (<code>syncSchedule</code>)、ヒント「Строка типа cron, напр. <code>@every 1m</code> 」	<code>@every 1m</code>	cron形式の同期頻度。
inboundタグ	«Теги входящих» (<code>inboundTags</code>)、ヒント「Входящие, на которых LDAP-синхронизация может авто-создавать или авто-удалять клиентов.」	空欄	自動操作が許可されるinboundを制限します。inboundがない場合:「Входящие не найдены. Сначала создайте входящий.」
クライアントの自動作成	«Авто-создание клиентов» (<code>autoCreate</code>)	<code>false</code>	ディレクトリにクライアントが現れた場合、指定されたinboundに自動的に作成します。
クライアントの自動削除	«Авто-удаление клиентов» (<code>autoDelete</code>)	<code>false</code>	ディレクトリからクライアントが消えた場合、自動的に削除します。
デフォルトの容量 (GB)	«Объём по умолчанию (ГБ)» (<code>defaultTotalGb</code>)	<code>0</code>	自動作成クライアントのトラフィック制限 (0 = 無制限)。
デフォルトの期間 (日数)	«Срок по умолчанию (дни)» (<code>defaultExpiryDays</code>)	<code>0</code>	自動作成クライアントの有効期間 (0 = 無期限)。
デフォルトのIP制限	«Лимит IP по умолчанию» (<code>defaultIpLimit</code>)	<code>0</code>	同時接続IP数の制限 (0 = 無制限)。

同期フラグのロジックの詳細: フラグ属性 (`flagField` 、デフォルト `vless_enabled`) を読み取る際、その値がtruthyな値のリストに含まれていれば「有効」とみなされます。反転が有効な場合、結果は逆になります。ユーザー属性 (`userAttr`) は照合キー (email/名前) として使用されます。この属性の値を持たないレコードはスキップされます。

セキュリティ: bindパスワードや検証されるパスワードが平文で送信されないよう**TLS (LDAPS) **を有効にすること、および Bind DN には読み取りに必要な最小限の権限を持つアカウントを使用することを推奨します。

例: LDAP同期の典型的な設定 (Active Directory)。 アクセスステータスが userAccountControl に類似したフラグ属性に保存され、照合がメールで行われるディレクトリのフィールド入力例:

フィールド	値
LDAPホスト	ldap.example.com
LDAPポート	636
TLS (LDAPS) を使用する	有効
Bind DN	CN=svc-3xui,OU=Service,DC=example,DC=com
Base DN	OU=Users,DC=example,DC=com
ユーザーフィルター	(objectClass=person)
ユーザー属性 (username/email)	mail
VLESSフラグ属性	vless_enabled
Truthyな値	true,1,yes,on
同期スケジュール	@every 5m

この設定では、5分ごとにパネルが OU=Users のサブツリーを走査し、mail でクライアントを照合し、vless_enabled の値に基づいてVLESSアクセスを有効/無効にします。

3. 概要 / ダッシュボード

ダッシュボード（英語インターフェースでは *Overview*）はパネルのスタートページです。サーバーおよび Xray プロセスの状態をリアルタイムで表示します。すべての指標はサーバー側から取得されます。バックグラウンドスケジューラが2秒ごとにスナップショットを再構築し、WebSocket経由で開いているすべてのタブに配信します。1分ごとに蓄積されたメトリクスの行がディスクに書き出されます。HTTPエンドポイント `GET /status` は最後にキャッシュされたスナップショットを返します。

以下では、ページ上の各指標および各コントロール要素について説明します。

バージョン3.4.2 から、パネルのサイドメニュー（およびモバイルのドロワー）に「ドキュメント」ボタン（本のアイコン）があり、公式ドキュメント <https://docs.sanaei.dev/> を開きます。また 3.4.2 では全面的な **アクセシビリティ** の改善が行われ、アイコンボタンやクリック可能な要素に aria ラベルと Enter/Space による操作が追加されました（スクリーンリーダーおよびキーボードナビゲーション向け）。

3.1. データ収集の基本原則

- スナップショットは `gopsutil` ライブラリによって収集されます。特定の測定値の取得に失敗した場合、そのフィールドはゼロのままとなり、ログに警告（`get cpu percent failed`、`get uptime failed` など）が記録されます。これはダッシュボード全体を壊すものではなく、該当するタイルが 0/N-A を表示するだけです。
- 「瞬間」速度（CPU %、ネットワーク、ディスク I/O）は、現在のスナップショットと前回のスナップショットの差を秒単位のインターバルで割ることで計算されます。そのため、ページの初回読み込み時は2回目の計測値が蓄積されるまで速度値がゼロになることがあります。
- 履歴は「システム履歴」（*System History*）セクションで確認できます。グラフは以下に説明するのと同じデータ行から構築されます（3.12節参照）。

3.2. CPU

「CPU」タイルはプロセッサの現在の使用率（%）およびプロセッサのパラメータを表示します。

指標	説明
CPU使用率 (%)	直近のインターバルにおける占有プロセッサ時間の割合。指数移動平均（EMA、係数 <code>alpha = 0.3</code> ）でスムージングされ、急激な変動でインジケーターが揺れないようにしています。値は常に 0～100 % の範囲に収められます。最初の計測では基準点を初期化するため 0 を返します。
論理プロセッサ数	Hyper-Threading を含む論理コア数。
物理コア数	物理コアの数。
動作周波数	プロセッサのベース周波数（MHz）。遅延評価でキャッシュされます。最初の成功した計測値が保存され、再試行は最低5分間隔で行われ、リクエスト自体は1.5秒のタイムアウトがあります（一部のシステムでは周波数のリクエストが遅い場合があります）。

CPU使用率のアルゴリズム: ネイティブのプラットフォーム実装がある場合はそれを使用し、ない場合はプロセッサ時間カウンタのデルタ (busy / total) によって計算します。GuestおよびGuestNice時間は二重計上を避けるため除外されます。

3.3. メモリ (RAM)

「メモリ」 (RAM) タイルは使用量と合計量を表示します。「使用中 / 合計」および/または使用率 (%) の形式で表示されます。履歴には使用率 (%) が記録されます。

3.4. スワップ (Swap)

「スワップ」 (Swap) タイルは使用量と合計量を表示します。スワップファイル/パーティションが設定されていない場合 (合計 = 0)、値はゼロとなります。スワップが存在しない場合、履歴の行には 0 が記録されます。

3.5. ディスク (Storage)

「ディスク」 (Storage) タイルは使用量と合計量を表示しますが、**ルートパーティション / のみ**が対象です。「ディスク使用量」 (Disk Usage) の履歴には使用率 (%) が記録されます。ディスク I/O (読み込み / 書き込み、バイト/秒) もインターバルのカウンタデルタとして別途収集され、履歴の「ディスク I/O」タブに表示されます。

3.6. システム稼働時間 (Uptime)

「システム稼働時間」 (Uptime) 指標は、パネルや Xray の稼働時間ではなく、**サーバー全体**の起動からの経過時間 (秒) を表します。Xray プロセスの稼働時間は別途保持されます (3.9節参照)。また、パネルのスレッド数 (「スレッド」 / Threads) も表示されます。

パネルが使用するメモリ

パネルプロセスの指標の横に、3X-UI プロセス自体が占有するRAM容量が表示されます。この値はプロセスの実際のRSS (オペレーティングシステムが認識する値) から取得されており、システムユーティリティが表示する値と一致します。メモリが解放されると数値は減少します。以前のパネルは Go の内部カウンタを表示していたため、メモリ消費量が過大に報告されていました (例: 1クライアントのアイドル状態のサーバーで約300MB) が、これは決して減少しませんでした。このアーティファクトは現在解消されています。また、定期的なバックグラウンドプロセスが未使用のメモリをオペレーティングシステムに返却することで、指標が実際の消費量を反映するようにしています。

3.7. システム負荷 (Load average)

「システム負荷」 (System Load) ブロックは [Load1, Load5, Load15] の3つの数値の配列です。ツールチップの説明: 「過去1分、5分、15分間のシステム平均負荷」 (System load average for the past 1, 5, and 15 minutes)。履歴グラフの名称は「システム平均負荷 (1 / 5 / 15 分)」です。履歴の行には load1、load5、load15 が個別に記録されます。

これは標準的な Unix の指標で、実行キューに並んでいるプロセスの平均数を表します。目安として物理コア数と比較します。負荷が物理コア数を継続的に上回っている場合はオーバーロードの状態です。

3.8. ネットワーク：速度および総トラフィック量

物理インターフェースのみが対象です。仮想インターフェースおよびトンネルインターフェースは除外されます： `lo` / `lo0`、および `loopback`、`docker`、`br-`、`veth`、`virbr`、`tun`、`tap`、`wg`、`tailscale`、`zt` で始まるもの。残りのすべてのインターフェースの値が合算されます。

総合速度 (Overall Speed) – インターバルのカウンタデルタによる瞬間速度：

指標	説明
アップロード (表示ラベル「アップロード」 / <i>Upload</i>)	送信速度 (バイト/秒)。
ダウンロード (表示ラベル「ダウンロード」 / <i>Download</i>)	受信速度 (バイト/秒)。

総トラフィック量 (Total Data) – システム起動からの累積カウンタ：

指標	説明
送信済み (表示ラベル「送信済み」 / <i>Sent</i>)	合計送信バイト数。
受信済み (表示ラベル「受信済み」 / <i>Received</i>)	合計受信バイト数。

パケット速度 (パケット/秒) および累積パケットカウンタも別途収集され、履歴の「ネットワークパケット」 (*Network Packets*) タブに表示されます。ネットワーク履歴の行： `netUp`、`netDown`、`pktUp`、`pktDown`。

3.9. サーバーIPアドレス

「サーバーIPアドレス」 (*IP Addresses*) ブロックは `IPv4` および `IPv6` を表示します。外部アドレスは外部サービス (`IPv4` には `api4.ipify.org`、`ipv4.icanhazip.com`、`v4.api.ipinfo.io/ip`、`ipv4.myexternalip.com/raw`、`4.ident.me`、`check-host.net/ip`、`IPv6` には対応するサービス) を通じて取得されます。最初に正常な応答が返るまでリストが順に試行されます。各リクエストのタイムアウトは3秒です。

特徴：

- 結果はプロセスの生存期間中キャッシュされます。正常に取得されたアドレスは再度リクエストされません。
- どのサービスも応答しなかった場合、フィールドには `N/A` が残ります。`IPv6` が最初に `N/A` になった場合、`IPv6` のないネットワークでの無駄な時間を避けるため `IPv6` へのリクエストは完全に無効化されます。
- 隣に目のアイコンボタンがあり、アドレスの表示/非表示を切り替えられます。ツールチップ：「サーバーIPアドレスの表示/非表示を切り替える」 (*Toggle visibility of the IP*)。これはインターフェース上での視覚的な非表示のみ (スクリーンショット用など) であり、アドレス自体には影響しません。

3.10. TCP/UDP接続数

「接続数」 (*Connection Stats*) ブロックは、サーバー上のアクティブなTCPおよびUDP接続の総数を表示します (Xray だけでなくシステム全体)。履歴グラフは「アクティブ接続数 (TCP / UDP)」 (*Active Connections*) で、行は `tcpCount`、`udpCount` です。

3.11. Xrayのステータスとプロセス管理

「Xray」カードはXray-coreプロセスの状態を表示し、管理機能を提供します。

ステータス

値	表示ラベル	説明	設定される条件
running	「実行中」	<i>Running</i>	Xray プロセスが実行中。
stop	「停止中」	<i>Stopped</i>	プロセスが実行されておらず、起動エラーも記録されていない。
error	「エラー」	<i>Error</i>	プロセスが実行されておらず、起動エラーが記録されている。エラーのテキストは「Xray実行中にエラーが発生しました」 (<i>An error occurred while running Xray</i>) というタイトルのポップアップウィンドウに表示されます。
–	「不明」	<i>Unknown</i>	ステータスがまだ取得されていない間表示されます。

ステータスの横に **Xrayのバージョン** が表示されます。

操作ボタン

- **停止 (Stop)**。 `POST /stopXrayService` を呼び出します。成功すると、パネルは WebSocket 経由で新しい状態 `stop` と通知「Xrayが正常に停止されました」 (*Xray service has been stopped*) を配信します。エラーの場合はエラーテキストとともに `error` 状態が配信されます。重要: パネルが Xray 自体を経由してアクセスされている場合、Xray を停止するとパネルへの接続が切断される可能性があります。パネルへ直接接続している場合は問題ありません。
- **再起動 (Restart)**。 `POST /restartXrayService` を呼び出します。操作前に「xrayを再起動しますか?」という確認が表示され、「保存された設定でxrayサービスをリロードします」という説明が添えられます。成功すると `running` 状態と通知「Xrayが正常に再起動されました」 (*Xray service has been restarted successfully*) が表示されます。再起動は現在保存されている設定を適用します。設定を変更した後はこれを使用してください。

注記。このフォークでは、すべての認証タイプに対してダッシュボードに完全な Start / Stop / Restart 管理機能が追加されています。元の 3x-ui UI には独立した「スタート」ボタンはなく、起動は再起動によって実行されます。

Xrayログ表示ボタン

Xrayカードには Xray ログ表示ボタン (Logs) があります。このボタンは、Xrayの設定でアクセスログが設定されている場合にのみ表示されます。組み込みのビューアはこのファイルを読み込むため、アクセスログがない場合はボタンが非表示になります。ボタンの表示は `accessLogEnable` という個別のフラグに紐付けられており、IPの上限には依存しなくなりました。オンラインリストとIPアドレス制限はアクセスログなしでも引き続き動作します (8節参照)。

Xrayバージョンの選択

「バージョン選択」 (Version) セクションでは、Xray-core を別のリリースに切り替えることができます。バージョンリストは `GET /getXrayVersion` で読み込まれます:

- ソースは XTLS/Xray-core リポジトリの GitHub API (/releases) です。リクエストは**15分間** キャッシュされます。GitHub への接続が失敗した場合は最後に正常取得したリストが返され、ピッカーが空にならないようにしています。
- リストには `x.y.z` 形式のリリースのみが含まれ、**26.4.25より古いものは除外**されます。

ツールチップ: 「切り替えたいバージョンを選択してください」 (Choose the version you want to switch to.) および警告「重要: 古いバージョンは現在の設定と互換性がない場合があります」 (Choose carefully, as older versions may not be compatible with current configurations.) 。

切り替え: `POST /installXray/:version`。手順:

例。特定のバージョンのXray-coreに切り替える (セッションのcookieは事前に認証済みである必要があります):

```
curl -X POST 'https://panel.example.com:2053/xpanel/installXray/v25.6.8' \
  -b cookie.txt
```

ここで `v25.6.8` は `GET /getXrayVersion` が返すリストのタグです。バージョンはこのリストに存在する必要があり、そうでない場合パネルは拒否します。バージョン3.4.2 から、インストール可能な Xray の最小バージョンは **26.6.27** に引き上げられ、3.5.0 には **Xray 26.7.11** コアが同梱されています。コアの付随的な変更: Shadowsocks の暗号方式 `none/plain` と VMess の `none/zero` は削除され (保存済みの設定は自動的に書き換えられます: SS はサポートされている暗号方式に、VMess は `auto` に)、公開アドレスへの暗号化されていない VLESS/Trojan outbound は保存時に拒否されます – そのような設定ではコアが起動できないためです。

1. 選択されたバージョンが最新のリリースリストに存在するか確認されます (存在しない場合は拒否)。
2. Xray が停止されます。
3. 現在のOSとアーキテクチャ向けの `Xray-<os>-<arch>.zip` アーカイブが GitHub からダウンロードされます (amd64/64、arm64-v8a、arm32-v7a/v6/v5、386/32、s390x に対応。Windows の場合は `xray.exe`)。アーカイブとバイナリのサイズは 200 MB に制限されています。
4. バイナリがアトミックに置き換えられます (一時ファイル経由でリネーム) し、実行可能フラグが付与されます。
5. Xray が再起動されます。

切り替え前に「Xrayバージョンを切り替えますか？」（*Do you really want to change the Xray version?*）というダイアログが表示され、「Xrayのバージョンを #version# に変更します」という説明が添えられます。成功すると「Xrayが正常に更新されました」（*Xray updated successfully*）という通知が表示されます。

3.12. パネルの更新 (3X-UI)

パネルの更新確認ブロック。データは `GET /getPanelUpdateInfo` 経由で取得されます：

フィールド	説明
現在のパネルバージョン	インストールされているパネルのバージョン。
最新パネルバージョン	GitHubから取得した3x-uiの最新リリース。
アップデートの有無	最新バージョンが現在のバージョンより新しいことを示すフラグ。更新が不要な場合は「パネルは最新です」/「更新済み」と表示されます。

「パネルを更新」 ボタン (*Update Panel*) は `POST /updatePanel` を実行します。ツールチップ：「3X-UIを最新リリースに更新し、パネルサービスを再起動します」。実行前に「本当にパネルを更新しますか？」という確認が表示され、「3X-UIを #version# バージョンに更新し、パネルサービスを再起動します」というテキストが添えられます。

特徴と制限：

- 自動更新は**Linuxのみ**サポートされます（他のOSではエラーが返されます）。
- 更新スクリプトは公式リポジトリ（`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`、制限 2 MB）からダウンロードされ、`bash` で実行されます。可能な場合は `systemd-run` で分離されます。
- 正常に起動した場合は「パネルの更新が開始されました」（*Panel update started*）が表示されます。更新確認に失敗した場合は「パネルの更新確認に失敗しました」が表示されます。インストール中は「インストール中です。ページを更新しないでください」という警告が表示されます。

3.13. ジオファイルの更新 (GeoIP / GeoSite)

ジオデータベース更新のボタン/ダイアログは `POST /updateGeofile`（全ファイル）または `POST /updateGeofile/:fileName`（1ファイル）を呼び出します。更新はファイル名とソースの厳格なホワイトリストに基づいて動作します：

ファイル	ソース
<code>geoip.dat</code> , <code>geosite.dat</code>	<code>Loyalsoldier/v2ray-rules-dat</code> (latest)
<code>geoip_IR.dat</code> , <code>geosite_IR.dat</code>	<code>chocolate4u/iran-v2ray-rules</code> (latest)
<code>geoip_RU.dat</code> , <code>geosite_RU.dat</code>	<code>runetfreedom/russia-v2ray-rules-dat</code> (latest)

動作:

- ファイル名が検証されます: `..`、スラッシュ、絶対パスは禁止。 `[a-zA-Z0-9._-]+.dat` のみ許可。ホワイトリスト外のファイルはダウンロードされません。
- 条件付きリクエスト `If-Modified-Since` が使用されます。ソースサーバー上でファイルが変更されていない場合 (HTTP 304)、再ダウンロードは行われず、タイムスタンプのみ更新されます。
- ダウンロード後、Xray は再起動されます (新しいデータベースを読み込むため)。

例。他のファイルに影響を与えずにロシアのジオデータベースのみを更新する:

```
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geoip_RU.dat' -b cookie.txt
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geosite_RU.dat' -b cookie.txt
```

ホワイトリスト内の全ファイルを一度に更新するには、ファイル名なしで `POST /updateGeofile` を呼び出してください。

- ダイアログ: 1ファイルの場合は「本当にジオファイルを更新しますか?」に「ファイル #filename# を更新します」、「すべて更新」ボタンの場合は「すべてのジオファイルを更新します」というテキストが添えられます。成功すると「ジオファイルが正常に更新されました」と表示されます。

3.14. データベースのバックアップと復元

「バックアップ&復元」 (*Backup & Restore*) ブロック。動作は使用しているDBMS (デフォルトのSQLite またはPostgreSQL) によって異なります。

データベースのエクスポート (バックアップ)

「データベースをエクスポート」/「バックアップ」ボタン (*Back Up*) は `GET /getDb` を呼び出します。ファイルは添付ファイルとして返されます:

- **SQLite:** 最初にチェックポイント (WALのフラッシュ) が実行され、その後 `x-ui.db` ファイルがダウンロードされます。ツールチップ: 「現在のデータベースのバックアップを含む .db ファイルをダウンロードするにはクリックしてください...」。
- **PostgreSQL:** カスタム形式 (`pg_dump --format=custom --no-owner --no-privileges`) で `x-ui.dump` ダンプがダウンロードされます。サーバーにPostgreSQLクライアントツールがインストールされている必要があります。ない場合は `pg_dump` がないというエラーが表示されます。

データベースのインポート (復元)

「データベースをインポート」/「復元」ボタン (*Restore*) は `POST /importDB` (フォームフィールド `db`) 経由でファイルをアップロードします。ツールチップ: 「バックアップからデータベースを復元するために .db ファイルを選択してアップロードするにはクリックしてください...」。

SQLite の場合のシナリオはロールバック付きの安全な手順です：

1. ファイルのSQLiteフォーマットが確認され、一時ファイルに保存された後、整合性チェックが行われます。
2. Xray が停止され、現在のDBが閉じられ *.backup にリネームされます（フォールバック用）。
3. 新しいファイルが作業DBの場所に配置され、初期化とマイグレーションが実行されます。問題が発生した場合はフォールバックが復元されます。
4. Xray が再起動されます。

PostgreSQL の場合、3.5.0 から「復元」は **3 種類**のファイルを受け付けます（種類は拡張子ではなく内容で判定されます）：`pg_dump` アーカイブ（PGDMP）は `pg_restore --clean --if-exists --single-transaction ...` で適用されます。**SQLite データベース .db**（通常のバックアップ）と **SQLite マイグレーション用 .dump** は、検証され、必要に応じて再構築されて、`x-ui migrate-db --dsn` と同じエンジンによって **単一トランザクション** で PostgreSQL にインポートされます（エラー時は PostgreSQL は変更されません）。ファイル選択ダイアログは両方の DBMS で `.dump, .db` を受け付けます。SQLite パネルに `pg_dump` アーカイブをアップロードすると分かりやすいエラーが返されます。ツールチップには「これにより現在のすべてのデータが置き換えられます」という明示的な警告が含まれています。

メッセージ：「データベースが正常にインポートされました」、「データベースのインポート中にエラーが発生しました」、「...データベースの読み込み中」、「...データベースの取得中」。

マイグレーションファイル（SQLiteとPostgreSQL間）

「マイグレーションファイルをダウンロード」ボタン (*Download Migration*) は `GET /getMigration` を呼び出します。3.5.0 からこのボタンは **PostgreSQLパネルにのみ** 表示され、`x-ui.db` – PostgreSQLのデータから構築された既製のSQLiteデータベース（「SQLiteに戻る」ための経路）– をダウンロードします。SQLiteパネルではボタンは冗長なため削除されました：通常の `.db` バックアップがそのままPostgreSQLパネルに直接復元できるようになったためです。

3.15. その他のインターフェース要素

- **オンラインクライアントインジケーター**。ダッシュボードは `online` (*Online Clients* / 「オンラインクライアント」) の行を管理します。これはアクティブな接続を持つクライアント数です。Xray が動作中の場合に計測されます（そうでない場合は 0）。同じ2秒間隔で履歴に記録されます。グラフは「オンライン」タブです。
- **システム履歴（グラフ）**。「グラフ」→「システム履歴」ボタン/セクションで、タブは「帯域幅」、「パケット」、「ディスク I/O」、「オンライン」、「負荷」、「接続数」、「ディスク使用量」です。データは `GET /history/:metric/:bucket` から取得されます。許可される集計インターバル (bucket、秒)：**2、30、60、180、360、720、1440、2880、10080**。各タブには最大60ポイントが表示されます。ページ上の時間範囲セレクターには **2m、1h、3h、6h、12h、24h、2d、7d** のボタンがあります (bucket はそれぞれ 2、60、180、360、720、1440、2880、10080)。2d および 7d の長い時間範囲では、軸の時刻ラベルに `MM-DD HH:MM` 形式の日付が付加されます。ストレージは3段階のロールダウン方式で管理されます：最新データは直近**1時間**を2秒間隔で保持、次に**48時間**を1分間隔で平均化、さらに**7日間**を10分間隔で平均化します。そのためグラフ (CPU、RAM、トラフィック、パケット、接続数、ディスク、オンライン、負荷) を**最大7日間**（以前は最大48時間）遡って確認でき、過去に遡るほど詳細度は粗くなります。許可されるメトリクス：`cpu、mem、swap、netUp、`

netDown、pktUp、pktDown、diskRead、diskWrite、diskUsage、tcpCount、udpCount、online、load1、load5、load15。「直近2分間」というラベルは bucket = 2（リアルタイムモード）に対応します。

例。 直近約2分間のCPU使用率の行（bucket = 2秒、最大60ポイント）および同じ行を5分間隔で集計したものを取得する：

```
curl 'https://panel.example.com:2053/xpanel/history/cpu/2' -b cookie.txt
curl 'https://panel.example.com:2053/xpanel/history/cpu/300' -b cookie.txt
```

メトリクスは許可された任意の値（mem、netUp、tcpCount、load1 など）に置き換えることができます。ホワイトリスト 2、30、60、180、360、720、1440、2880、10080 以外の bucket は拒否されます。

- **Xrayメトリクス** – XrayのメモリとGCの消費を示す個別のブロック（行：xrAlloc、xrSys、xrHeapObjects、xrNumGC、xrPauseNs）と「Observatory」（送信接続の状態）。Xrayの設定に metrics ブロック（listen 127.0.0.1:11111、タグ metrics_out）が設定されている場合のみ動作します。設定されていない場合は「Xrayメトリクスエンドポイントが設定されていません」と表示されます。Xrayメトリクスウィンドウには独自の時間範囲セクターがあり、2m、1h、3h、6h、12h のボタン（bucket 2、60、180、360、720）があります。

例： Xrayメトリクスタイルを有効にするブロック。Xrayの設定セクションには metrics（タグ付き）と、そのタグをリッスンする inbound の両方が必要です：

```
{
  "metrics": {
    "tag": "metrics_out"
  },
  "inbounds": [
    {
      "listen": "127.0.0.1",
      "port": 11111,
      "protocol": "dokodemo-door",
      "settings": { "address": "127.0.0.1" },
      "tag": "metrics_out"
    }
  ]
}
```

アドレス 127.0.0.1:11111 は意図的に外部に公開されていません。パネルはローカルでポーリングします。

- **ダークテーマの切り替え。** ダッシュボード自体ではなく共通メニュー/ヘッダーにあります。選択肢：「テーマ」（Theme）に「ダーク」と「ウルトラダーク」（Ultra Dark）があります。これは純粋に視覚的なテーマ設定であり、パネルの動作には影響しません。
- **ダッシュボード周辺のその他のリンク**（メニュー/フッターパネルから）：「ログ」、「設定」 – Xrayの最終的なJSONを表示（GET /getConfigJson）、「ドキュメント」。

4. Inbounds: 作成と共通パラメータ

「受信」（英語: *Inbounds*）セクションは、クライアントが接続するすべての Xray エントリポイントの一覧です。各 inbound には「パネル側」フィールド（備考、トラフィック制限、リセットスケジュール）と、Xray 設定の生 JSON ブロック（`settings`、`streamSettings`、`sniffing`）が格納されています。

作成は**「接続を作成」（*Add Inbound*）ボタン、編集は「接続を変更」**（*Modify Inbound*）ボタンから行います。どちらの操作も API エンドポイント `POST /add` および `POST /update/:id` に送信されます。

以下では、特定のプロトコル設定（クライアント、暗号化、REALITY/TLS）や、トランスポート / ストリーム設定（「ストリーム」タブ、「セキュリティ」タブ）には属さないフォームフィールドをすべて説明します。これらは別セクションで扱います。

4.1. 共通フォームフィールド

Remark（備考）

パラメータ	値
フィールド	<code>remark</code>
型	文字列
デフォルト	空

inbound の人が読める名前で、一覧やダイアログのタイトル（「接続 "{remark}" を削除しますか？」など）に表示されます。フィールドのラベルは**「備考」**です。Xray の動作には影響しませんが、管理の利便性のために存在します。エクスポートされるファイル名や一括操作の確認ダイアログに使われるため、一意でわかりやすい名前を設定することを推奨します。

Protocol（プロトコル）

パラメータ	値
フィールド	<code>protocol</code>
ラベル	「プロトコル」
バリデーション	<code>required,oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun</code>

inbound プロトコルのドロップダウンリストです。使用可能な値:

値	備考
<code>vmess</code>	
<code>vless</code>	
<code>trojan</code>	

値	備考
shadowsocks	
wireguard	
hysteria	Hysteria v2 は streamSettings.version = 2 を持つ hysteria であり、独立したプロトコルはありません
http	
mixed	1 ポートで socks/http を同時処理
tunnel	
tun	バリデーターで受け付けられますが、独立したプロトコル定数はありません

このフィールドは必須（required）です。プロトコルの選択によって、利用可能なクライアント設定フィールドとポートが決まります（プロトコル固有のセクションを参照）。

重要: 保存時にサービスは streamSettings を正規化します。ポート設定は vmess、vless、trojan、shadowsocks、hysteria プロトコルにのみ保持され、それ以外（http、mixed、tunnel、wireguard、tun）では streamSettings フィールドが強制的にクリアされます。

tunnel/TProxy タイプの inbound で、streamSettings ブロックに security キーが含まれない（ポートなし）場合、フォームは streamSettings.security Invalid input というバリデーションエラーなしで開閉できます。

Listen IP（リッスン IP）

パラメータ	値
フィールド	listen
型	文字列
デフォルト	空 → Xray は 0.0.0.0（すべての IP）でリッスン

inbound が接続を受け付ける IP アドレスです。フィールドのヒント:

「すべての IP アドレスでリッスンするには空のままにしてください。」

設定の生成時、空の値は 0.0.0.0 に置き換えられます。IP アドレスの他に、このフィールドは Unix ソケットパスも受け付けます。ヒント:

「Unix ソケットパス（例: /run/xray/in.sock）や、@ プレフィックスを付けた抽象ソケット名（例: @xray/in.sock）を指定して、TCP ポートの代わりにソケットをリスンすることもできます。その場合はポートを 0 に設定してください。」

つまり、このフィールドは 2 種類の Unix ソケット形式を受け付けます: ファイルシステム上のパス（ /run/xray/in.sock ）と、@ プレフィックス付きの抽象ソケット名（ @xray/in.sock ）。どちらの場合も Port を 0 に設定してください。

このフィールドを変更するのは、inbound を特定のインターフェース（例: Nginx のフォールバック先として動作する inbound の 127.0.0.1 ）に制限したい場合、または inbound が Unix ソケットをリスンする場合です。

例: ローカルインターフェースのみをリスンする inbound（典型的な Nginx 後段のフォールバック先）と Unix ソケット:

```
listen = 127.0.0.1    port = 8443
listen = /run/xray/in.sock    port = 0
```

Port（ポート）

パラメータ	値
フィールド	port
ラベル	「ポート」
バリデーション	gte=0,lte=65535
デフォルト	–（ユーザーが設定）

リスンする TCP/UDP ポートです。0 ~ 65535 の値が使用できます。0 は Unix ソケットリスンと組み合わせる場合にのみ使用します（上記参照）。

保存時にサービスはポートの競合を確認します: 2 つの inbound が同じトランスポート（TCP/UDP）で listen:port が重複することはできません。トランスポートはプロトコルと streamSettings / settings から判定されます: たとえば hysteria と wireguard は常に UDP を使用し、kcp / quic は UDP、その他のほとんどは TCP です。競合が検出された場合、保存はエラーで拒否されます。

また、パネルは内部 Xray API の予約ポート（タグ api、デフォルト 127.0.0.1 上の 62789）を使用することも禁止しています: ループバック上でこのポートと重複するローカル TCP inbound は同様のポート競合エラーで拒否されます。実際の API ポートは Xray 設定テンプレートから読み取られ（フォールバック値は 62789）、ノードではこの制限は適用されません（ノードは独自の Xray を持つため）。

Xray タグ（Tag、一意）はポートとトランスポートから in-<ポート>-<tcp|udp|tcpudp|any> 形式で自動生成されます。ノードにデプロイされた inbound の場合は n<nodeId>- プレフィックスが付

きます。競合時は -2、-3 などが末尾に付加されます。通常、ユーザーがタグを編集することはありません。

Total traffic（総トラフィック,GB）

パラメータ	値
フィールド	total（バイト単位）
ラベル	「総使用量」
デフォルト	0

inbound の総トラフィック制限です。フォームではギガバイト単位で入力し、データベースにはバイト単位で保存されます。フィールドのヒント:

「= 無制限。（単位: GB）」

つまり **0** は無制限を意味します。これは inbound 全体のレベルの制限（個別クライアントではなく）であり、実際の使用済みトラフィックはフィールド **up**（送信）と **down**（受信）に保存され、**total** と比較されます。

Expiry date / Duration（有効期限 / 期間）

パラメータ	値
フィールド	expiryTime（Unix タイムスタンプ）
ラベル	「有効期限」（英語: <i>Duration</i> ）
デフォルト	空 / 0

inbound の有効期限です。ヒント:

「無期限にするには空のままにしてください。」

空の値（0）は期限なしの inbound を意味します。値は Unix タイムスタンプとして保存されます。フォームでは具体的な日付と、現在時刻からの相対的な日数（英語のフィールドラベル *Duration*）のどちらも設定できます。

Enabled（有効化）

パラメータ	値
フィールド	enable
ラベル	「有効化」（英語: <i>Enabled</i> ）

パラメータ	値
デフォルト	作成時に設定

inbound のアクティブ状態を示すフラグです。一覧でこのフラグを切り替えると、完全な更新ではなく専用の「軽量」エンドポイント `POST /setEnabled/:id` が呼び出されます。これは意図的な設計で、数千のクライアントを持つ inbound でトグルを切り替えるたびに `settings` ブロック全体（すべてのクライアント）を再シリアル化しないようにするためです。無効化すると inbound は実行中の Xray から削除され、有効化すると再追加されます。

Node / Deploy to（ノード / デプロイ先）

パラメータ	値
フィールド	<code>nodeId</code>
ラベル	「デプロイ先」、「ローカルパネル」
デフォルト	空（ローカルパネル）

inbound が物理的に動作する場所（ローカルパネルまたは登録済みノードのいずれか）を選択します。バージョン3.5.0 から、「デプロイ先」リスト（および inbound リスト上部の「ノード」フィルター）は入力による検索に対応し、inbound リスト自体の上部には検索フィールド（備考・ポート・プロトコルによる。ノードフィルターと併用できます）が追加されました。実装上の注意: `nodeId = 0` は `nil` に正規化されます。`0` は有効なノード ID ではなく、フォームバインディングのアーティファクトであり、`nil / 0` はローカルパネルを意味します。オフラインノードに inbound を保存した場合、「ノードが再接続したときに変更が同期されます」というトーストが表示されることがあります。バージョン3.4.2 から、このトーストはノードが実際にオフラインまたは無効の場合にのみ表示されます（以前はオンラインのノードに保存した際にも表示されることがありました）。

共有リンクのアドレス戦略（Share address strategy）

パラメータ	値
フィールド	戦略 + （任意）カスタムアドレス
ラベル	「共有リンクのアドレス戦略」（英語: <i>Share address strategy</i> ）
デフォルト	「Inbound のリッスンアドレス」（ <i>Inbound listen</i> ）

このドロップダウンリストは、この inbound のエクスポートされた共有リンクおよび QR コードにどのアドレスを使用するかを決定します。値:

値	ラベル	使用されるアドレス
<code>node</code>	「ノードアドレス」（ <i>Node address</i> ）	inbound が動作するノードのアドレス
<code>listen</code>	「Inbound のリッスンアドレス」（ <i>Inbound listen</i> ）	inbound 自身のリッスンアドレス

値	ラベル	使用されるアドレス
custom	「カスタム」 (Custom)	「カスタム共有アドレス」 (Custom share address) フィールドに入力した独自アドレス

「カスタム」を選択すると「カスタム共有アドレス」フィールドが表示され、スキームとポートを除いたホスト名または IP を入力します（バリデーションあり）。「ノードアドレス」オプションは、この inbound が動作できる有効なノードが存在する場合にのみリストに表示されます。存在しない場合は非表示となり、値は「Inbound のリッスンアドレス」にリセットされます。

この戦略は**直接**の共有リンクと QR コードに**のみ**影響します。サブスクリプションの出力には影響せず、サブスクリプションのアドレスは引き続きパネルの通常のロジックで決定されます。

4.2. Sniffing（スニффイング）

「スニッフイング」タブは、Xray 設定の `sniffing` ブロックを編集します（生 JSON として保存）。スニッフイングにより、Xray はルーティング目的で接続内の実際のドメイン名やプロトコルを「覗き見る」ことができます。

サブフィールド	ラベル	用途
<code>enabled</code>	(タブのトグル)	inbound のスニッフイングを有効/無効にします
<code>destOverride</code>	—	宛先アドレスをインターセプトするプロトコルのリスト: <code>http</code> 、 <code>tls</code> 、 <code>quic</code> 、 <code>fakedns</code>
<code>metadataOnly</code>	「メタデータのみ」	ペイロードを読まずに接続メタデータのみを使用します
<code>routeOnly</code>	「ルーティングのみ」	スニッフイング結果を宛先アドレスの書き換えなしにルーティングにのみ使用します
<code>domainsExcluded</code>	「除外ドメイン」	スニッフイングから除外するドメイン
(除外 IP)	「除外 IP」	スニッフイングから除外する IP アドレス

- **`destOverride`** — スニファのセット: `http` (HTTP Host ヘッダーからドメインを取得)、`tls` (SNI から取得)、`quic` (QUIC ClientHello から取得)、`fakedns` (FakeDNS プールとの照合)。ドメイン検出には通常 `http` と `tls` を有効にします。

`sniffing` ブロックの例 (HTTP と TLS でドメインを検出し、ローカルネットワークはそのままルーティングのみに使用)：

```
{
  "enabled": true,
  "destOverride": ["http", "tls"],
  "routeOnly": true,
  "domainsExcluded": ["courier.push.apple.com"]
}
```

- **`metadataOnly`** — 有効にすると、Xray は最初のパケットの内容を読まずにメタデータのみに依存します。データを「覗き見る」ことができないプロトコルを壊さないために有効です。

- **routeOnly** – スニффイング結果はルーティングルールにのみ使用されます。outbound への接続アドレスは検出されたドメインに書き換えられません。

注意: パネルは sniffing を不透明な JSON ブロックとして保存し、保存時に何も追加しません。これらのチェックボックスのデフォルト値はクライアントアプリケーション側で形成されます。生の形式では、以下の「受信の JSON」セクションから直接編集できます。

4.3. Allocate（ポート割り当て戦略）

streamSettings 内の allocate ブロックは、Xray がリスンポートをどのように割り当てるかを制御します。これは Xray 設定の一部で、パネルは streamSettings /JSON inbound の一部として保存・送信します。パラメータ（Xray-core の用語に従う）:

サブフィールド	用途	値 / デフォルト
strategy	ポート割り当て戦略	always – 常に指定ポートでリスン（デフォルト）; random – ポート範囲内でリスンポートを定期的に変更
refresh	random 時のポート変更間隔（分）	整数の分数（推奨値 5; 最小 2）
concurrency	random 時に同時に開くポート数	整数（デフォルト 3; ポート範囲の 3 分の 1 以下）

strategy: always は inbound を 1 つのポートに固定します（標準モード）。strategy: random はアンチブロッキングシナリオで inbound がポート範囲内を定期的に「ジャンプ」する場合に使用します。その場合、refresh と concurrency が意味を持ちます。これらの値を変更するのは、ランダムポートモードを意図的に使用する場合のみです。

allocate ブロックの例（streamSettings 内、ランダムポートモード: 3 ポートを同時に開き、5 分ごとに変更）:

```
{
  "allocate": {
    "strategy": "random",
    "refresh": 5,
    "concurrency": 3
  }
}
```

これを機能させるには、inbound の port を範囲（例: 20000–20100）で指定します。

4.4. External Proxy（外部プロキシ）

「External Proxy」フィールドは招待リンク生成の設定に関するもので、inbound の streamSettings に保存されます。これは、クライアントリンクに inbound の実際の listen:port の代わりに代入される外部アドレスの代替リスト（ホスト/ポート、必要に応じて強制 TLS – **「強制 TLS」**付き）を指定します。

クライアントがサーバーに直接接続するのではなく、外部プロキシ/リバース/CDN 経由で接続する必要がある場合に使用します: その場合、共有リンクにはそのようなフロントエンドのパブリックアドレスが指定されます。Xray の接続受け付けプロセス自体には影響しません – これは生成されるリンクの「見た目」の変更です。関連するフォームフィールド: 「**強制 TLS**」、 「**Fingerprint**」、各エントリのラベル。

4.5. Fallbacks (フォールバック)

****「フォールバック」****セクションは、inbound のクライアントにマッチしなかった接続のリダイレクトルールを設定します。TLS トランスポート (VLESS/Trojan TCP-TLS) のマスター inbound で利用できます。エンドポイント GET `/:id/fallbacks` / POST `/:id/fallbacks` で管理されます。

セクションのヒント:

「この inbound でクライアントにマッチしなかった接続は別の場所にリダイレクトされます。以下で子 inbound を選択すると、ルーティングフィールド (SNI / ALPN / Path / xver) がそのポートから自動入力されます。または選択を空のままにして Dest を直接指定 (例: 8080 または 127.0.0.1:8080) し、Nginx などの外部サーバーにリダイレクトすることもできます。各子 inbound は 127.0.0.1 で security=none としてリッスンする必要があります。」

フォールバックセクションは、セキュリティが TLS または REALITY の RAW (TCP) 上の VLESS/Trojan inbound にのみ表示されます。新しい inbound は security=none で起動するため、このセクションは最初は表示されない場合があります。この状態 (VLESS/Trojan、RAW/TCP、セキュリティ未設定) では、セクションの代わりに内蔵のヒントが表示されます: フォールバックは****「セキュリティ」****タブで TLS または Reality を選択した後に利用可能になります。

フォールバック行のフィールド

フィールド	デフォルト	説明
(子 inbound)	—	子 inbound の選択 (ラベル **「Inbound を選択」**)。選択した場合、Name/Alpn/Path/Dest フィールドがそのポートから自動入力される場合があります
Name	空 (= 任意)	名前 (SNI/名前) によるマッチング条件。「任意」のラベル – 「任意」
Alpn	空	ALPN によるマッチング条件
Path	空	パスによるマッチング条件 (子 inbound の WS/HTTP トランスポート用)
Dest	自動	リダイレクト先。プレースホルダー **「自動 (子の listen:ポート)」** 。ポート (8080) または <code>host:port</code> (<code>127.0.0.1:8080</code>) を指定可能
Xver	0	PROXY プロトコルバージョン (「 Xver 」): 0 – 無効、 1 または 2 – 対応する PROXY protocol バージョン
(順序)	位置順	ルール適用順序。「上へ」/ **「下へ」** ボタンで設定

保存ロジック: マスターのフォールバックリスト全体がアトミックに置き換えられます。子 inbound が選択されておらず (`childId <= 0`)、`Dest` も指定されていない行はスキップされます。選択された子 inbound がマスター自身の ID と同じ場合はリセットされます。最終 JSON 生成時: `Dest` が空の場合は子 inbound の `listen:port` から計算され、`0.0.0.0 / :: / :::0` は `127.0.0.1` に置き換えられます。空の `name / alpn / path` は出力 JSON に含まれません。 `xver` は 0 より大きい場合のみ追加されます。

最終的な `settings.fallbacks` の例 (`alpn=h2` のトラフィックはパス `/ws` の WS ターゲットへ、それ以外はポート 8080 のローカル Nginx へ) :

```
{
  "fallbacks": [
    { "alpn": "h2", "path": "/ws", "dest": "127.0.0.1:2001", "xver": 1 },
    { "dest": 8080 }
  ]
}
```

`name / alpn / path` のない最後の行は「デフォルト」ルールで、それ以外すべてをキャッチします。

フォールバックセクションのボタンとヒント

- 「フォールバックを追加」 - 行を追加; 「フォールバックはまだありません」 - 空の状態。
- 「適切なものをすべてすばやく追加」 / 「すべて追加」 - まだ接続されていない適切な各 inbound にフォールバック行を追加します。結果: 「{n} 件のフォールバックを追加しました」または「新しい適切な inbound がありません」。
- 「子から自動入力」 - 選択された子 inbound のトランスポートからルーティングフィールド (SNI/ALPN/Path/xver) を再取得します。実行後 - 「子から入力しました」。
- 「ルーティングフィールドを変更」 / 「詳細設定を非表示」 - 行の詳細フィールドを表示/非表示。
- ** 「マッチする条件」と「デフォルト - それ以外すべてをキャッチ」 ** というラベルが各行の発動条件を説明します。

フォールバックを保存すると、サーバーは Xray の再起動を開始し、新しい `settings.fallbacks` を有効化します。

4.6. 定期的なトラフィックリセット

** 「トラフィックリセット」 ** ブロックは、スケジュールに従って inbound のトラフィックカウンターを自動リセットするように設定します。説明:

「指定した間隔でトラフィックカウンターを自動リセットします。」

パラメータ	値
フィールド	<code>trafficReset</code>
バリデーション	<code>omitempty, oneof=never hourly daily weekly monthly</code>
デフォルト	<code>never</code>

パラメータ	値
関連フィールド	<code>lastTrafficResetTime</code> – 最終リセットのタイムスタンプ（ラベル**「最終リセット」**）

ドロップダウンリスト:

値	ラベル
<code>never</code>	「なし」
<code>hourly</code>	「毎時」
<code>daily</code>	「毎日」
<code>weekly</code>	「毎週」
<code>monthly</code>	「毎月」

各期間に対して、対応するスケジュール（`@hourly`、`@daily`、`@weekly`、`@monthly`）で実行される cron ジョブが登録されています。ジョブは指定された `trafficReset` を持つすべての inbound を選択し、それぞれについて inbound 自体のカウンター（`up=0`、`down=0`）およびすべてのクライアントのトラフィックをリセットします。つまり、定期リセットは inbound とそのクライアント両方に影響します。

フィールド値の例。 カウンターを毎月 1 日にリセットするには、フォームで**「毎月」**を選択します。これは次のように保存されます:

```
{ "trafficReset": "monthly" }
```

値 `never`（デフォルト）は自動リセットを完全に無効にします。

4.7. 受信の JSON（上級者向け）

「受信の JSON セクション」は inbound の生 JSON ブロックへの直接アクセスを提供します。説明:

「受信の完全な JSON と、settings、sniffing、streamSettings の個別エディター。」

利用可能なエディター:

タブ	ラベル	編集対象
すべて	「1つのエディターですべてのフィールドを含む受信の完全なオブジェクト」	Inbound オブジェクト全体
設定	「Xray settings ブロックのラッパー」	<code>settings</code> フィールド
Sniffing	「Xray sniffing ブロックのラッパー」	<code>sniffing</code> フィールド
Stream	「Xray stream ブロックのラッパー」	<code>streamSettings</code> フィールド

これらのフィールドはネストされた JSON オブジェクトとしてシリアル化されます: 空のブロックは `null` として返され、有効な JSON でないテキストはデータが失われないよう文字列にラップされます。保存時のパースエラーは**「拡張 JSON」**プレフィックス付きで表示されます。

「受信の JSON」表示ウィンドウも inbound インポートウィンドウも、通常のテキストフィールドではなく JSON 構文ハイライト付きの本格的なコードエディターを使用します: 設定の表示はハイライトされた読み取り専用モード、インポートは編集可能モードで、読み取りと編集が容易です。

4.8. Inbound の操作: QR / Edit / Reset / Delete と統計

一覧と inbound カード内では、以下の操作が利用できます（「メニュー」）:

トラフィック統計

inbound の集計トラフィックが表示されます: 「送信/受信」（up / down フィールド）、「総トラフィック」、「総接続数」。カード内にはさらに**「作成日時」、「更新日時」、「有効期限」**も表示されます。

inbound 一覧には **Speed** 列があり、各 inbound の現在のトラフィック速度（送信/受信）がポーリング間のカウンター増分から計算されてリアルタイム表示されます。同じライブ速度が inbound の統計ウィンドウにも表示されます。次のポーリングで増分がない場合、速度の値はリセットされます。

inbounds ページのクライアントサマリーでは、ステータスは「消費済み/終了」の優先度で決定されます: 期限切れまたはトラフィック消費済みのクライアント（自動タスクによって `enable` が解除されたもの）は、グレーの**「無効」（*Disabled*）ではなく「消費済み/終了」**（*Depleted/Ended*）ステータスに分類され、二重カウントされません。この分類はクライアントカード自体に表示されるものと一致し、複数の inbound に紐付けられたクライアントも正しく考慮されます。

QR コードとリンクのコピー

- 「詳細」 – 接続リンクとサブスクリプションリンクを展開します。
- クライアントの QR コード: ヒント**「QR コードをクリックしてコピー」**。
- 「リンクをコピー」（英語: *Copy URL*）、「リンクをエクスポート」。バージョン 3.4.2 から、ページ単位のアクション「すべてのリンクをエクスポート」（GET `/panel/api/inbounds/allLinks`; ウィンドウ「inbound のすべてのリンクをエクスポート」、ファイル「All-Inbounds」）がサブスクリプションエンジン経由でリンクを生成し、各クライアントにリマークテンプレートを適用して管理対象の Host エンドポイントを優先するようになりました（以前は固定のリマーク `inbound-email` が使われていました）。

Edit（変更）

「接続を変更」 – 編集フォームを開きます（POST `/update/:id`）。更新時にサービスは既存の行を再読み込みし、変更されたフィールドを移行し、必要に応じてタグを再生成（古いタグが自動生成されていた場合）して Xray ランタイムと同期します。成功時 – トースト**「接続が正常に更新されました」**。

Reset Traffic (トラフィックをリセット)

「トラフィックをリセット」 – この inbound のカウンター `up / down` のみをゼロにします (`POST /:id/resetTraffic`、`up=0, down=0` に設定)。確認ダイアログ:

「"{remark}" のトラフィックをリセットしますか? 」 / 「この接続の送信/受信カウンターを 0 にリセットします。」

inbound のトラフィックリセットはそのクライアントのカウンターには影響しません (クライアント用には別の「クライアントトラフィックをリセット」アクションがあります)。リセット後に Xray の再起動が開始されます。成功時 – トースト**「受信トラフィックがリセットされました」**。一括バリエーションもあります – 「すべての接続のトラフィックをリセット」 (`POST /resetAllTraffics`)。

Delete (削除)

「接続を削除」 (`POST /del/:id`)。確認ダイアログ:

「接続 "{remark}" を削除しますか? 」 / 「接続とそのすべてのクライアントが削除されます。この操作は元に戻せません。」

削除により inbound は実行中の Xray から削除されます (必要に応じて再起動)。成功時 – トースト**「接続が正常に削除されました」**。一括削除は `POST /bulkDel` で、要素ごとのレポートと最大 1 回の Xray 再起動で行われます。

Inbound クライアントのその他の操作

メニューには以下も含まれます: 「クローン」 (新しいポートと空のクライアントリストを持つ inbound のコピー)、 「すべてのクライアントを削除」 (`POST /:id/deleteAllClients` – すべてのクライアントを削除し、inbound 自体は保持)、 「無効なクライアントを削除」、 「クライアントをリンク/リンク解除」、 「インポート」 / 「接続をエクスポート」 (`POST /import`)。クライアント操作の詳細はクライアントに関するセクションを参照してください。

5. プロトコル

inbound を作成する際、最初に**プロトコル**（「Protocol」）を選択します。プロトコルは、Xray-core がこの inbound に対してどの認証・暗号化方式を適用するか、**settings** でどのフィールドを入力する必要があるか、また利用可能なトランスポート（**network**）とセキュリティ種別（TLS / REALITY）を決定します。

プロトコルフィールドは inbound 作成時に一度だけ設定され、**編集時には変更できません**（編集フォームではドロップダウンがロックされます）。プロトコルを変更するには、新しい inbound を作成する必要があります。

5.1. サポートされているプロトコルの一覧

サーバーは **Protocol** フィールドに以下の値を受け付けます：

```
oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun mtproto
```

バージョン **3.3.0** 以降、**mtproto**（Telegram プロキシ）がリストに追加されました。

設定ファイルの値	用途	クライアントモデル
vless	主要プロキシプロトコル（inbound 作成時のデフォルト）	UUID を持つクライアント。flow とポスト量子暗号をサポート
vmess	Xray のクラシックプロキシプロトコル	UUID と security パラメーターを持つクライアント
trojan	通常の HTTPS に偽装するプロキシ	パスワードを持つクライアント
shadowsocks	Shadowsocks プロキシ（SIP022 / 2022-blake3 を含む）	単一ユーザーまたは複数ユーザー（2022）
wireguard	WireGuard inbound	クライアントではなく peer
hysteria	Hysteria inbound（デフォルトはバージョン 2）	auth トークンを持つクライアント
http	クラシック HTTP プロキシ（フォワードプロキシ）	user/pass アカウント。トラフィック計測なし
mixed	SOCKS + HTTP 複合プロキシ	user/pass アカウント
tunnel	透過フォワーダー（xray dokodemo-door）	クライアントなし
tun	TUN インターフェース（既存のもののレンダリングのみ）	クライアントなし
mtproto	Telegram プロキシ（MTPROTO）。3.3.0 で追加。Xray ではなく別プロセス mtg が処理	クライアントなし（シークレットでアクセス）

`tun` に関する注意: この値は後方互換性および以前に保存された inbound の表示のためにリストに残されていますが、現バージョンの backend では作成は推奨されておらず、サポートは非推奨とされています。このタイプの新しい inbound を作成しても意味はありません。

Hysteria 2 に関する注意: 「hysteria2」という独立したプロトコルは存在しません。これは `streamSettings.version = 2` を持つ `hysteria` プロトコルです。share リンクを生成する際、ストリームバージョンが 2 の場合に `hysteria2://` スキームが自動的に選択されます。

すべてのプロトコルがノード (nodes) への配布をサポートするわけではありません。ノードへ展開できるのは `vless`、`vmess`、`trojan`、`shadowsocks`、`hysteria`、`wireguard` のみです。`http`、`mixed`、`tunnel`、`tun`、`mtproto` はローカルパネルでのみ動作します。

5.2. TLS / REALITY / トランスポートをサポートするプロトコル

特定のセキュリティレイヤーやトランスポートを有効にできるかどうかは、プロトコルと選択したネットワーク (`streamSettings.network`) によって異なります:

機能	利用可能なプロトコル	許可されるネットワーク (network)
TLS	<code>vmess</code> 、 <code>vless</code> 、 <code>trojan</code> 、 <code>shadowsocks</code> (および <code>hysteria</code> は常時)	<code>tcp</code> 、 <code>ws</code> 、 <code>http</code> 、 <code>grpc</code> 、 <code>httpupgrade</code> 、 <code>xhttp</code>
REALITY	<code>vless</code> 、 <code>trojan</code>	<code>tcp</code> 、 <code>http</code> 、 <code>grpc</code> 、 <code>xhttp</code>
flow (xtls-rprx-vision)	<code>vless</code> のみ	<code>tcp</code> のみ。security = <code>tls</code> または <code>reality</code> の場合
Stream / トランスポート (「ストリーム」タブ)	<code>vmess</code> 、 <code>vless</code> 、 <code>trojan</code> 、 <code>shadowsocks</code> 、 <code>hysteria</code>	—

`http`、`mixed`、`tunnel`、`tun`、`wireguard` プロトコルではトランスポートタブは利用できません。これらには Xray のストリーム設定がありません。

5.3. VLESS

用途: 主要なモダンプロキシプロトコル。XTLS-Vision (`flow`)、REALITY、および VLESS レベル自体のポスト量子暗号 (`decryption` / `encryption` フィールド) をサポートします。新しい inbound のデフォルトとして使用されます。

`settings` ブロックのフィールド:

フィールド	デフォルト値	説明
clients	[]	クライアントのリスト。各クライアントは次を持ちます: id (UUID)、email (必須)、flow、制限 (limitIp、totalGB、expiryTime)、enable、tgId、subId、comment、reset
decryption	none	サーバー側の復号パラメーター。UI のラベル: 「復号」 (英語: 「Decryption」)
encryption	none	対になる暗号化パラメーター (クライアントリンクに含まれます)。ラベル: 「暗号化」 (英語: 「Encryption」)
fallbacks	[]	fallback のリスト (fallback に関するセクションを参照)。network = tcp かつ security が TLS または REALITY の場合に利用可能
testseed	(4つの数値: 900、500、 900、256)	「Vision testseed」 – XTLS-Vision パディング用の4つの正の整数。flow が xtls-rprx-vision のクライアントにのみ適用され、それ以外の場合は無視されます

flow (xtls-rprx-vision)

flow は inbound ではなくクライアント側で設定され、次の3つの値のいずれかを取ります:

値	意味
`` (空)	XTLS-flow なし (デフォルト)
xtls-rprx-vision	XTLS-Vision – TCP+TLS/REALITY 上の VLESS に推奨されるモード
xtls-rprx-vision-udp443	同じ Vision だが、UDP/443 (QUIC) の処理を含む

flow フィールドは、プロトコルが vless、network = tcp、かつ security が tls または reality のすべての条件が満たされた場合にのみ選択可能です。フォーム内の **Vision testseed** フィールドも同じ条件の場合のみ表示されます。

XHTTP の例外: VLESS の network = xhttp で VLESS ポスト量子認証 (encryption / decryption、vlessenc) が有効な場合、セキュリティレイヤー (REALITY を含む) に関わらず flow xtls-rprx-vision も使用できます。この場合、パネルは share リンクとサブスクリプション (Clash/Mihomo 形式を含む) に xtls-rprx-vision を正しく渡すため、クライアントは Vision を含む設定を受け取ります。

復号 / 暗号化 (VLESS ポスト量子認証)

decryption と encryption フィールドは、VLESS レベル自体での認証です (トランスポートの TLS/ REALITY とは別)。デフォルトでは両方とも none です。フォームにはこれらのフィールドの下に**「キー生成」ブロックがあり、モードのドロップダウンリストと「生成」ボタン (隣に「クリア」**ボタン) が含まれています。ドロップダウンリストには6つのオプションがあります: **X25519 (native)**、

X25519 (xorpub)、**X25519 (random)**、**ML-KEM-768 (native)**、**ML-KEM-768 (xorpub)**、**ML-KEM-768 (random)**
 – つまり2種類のキー（クラシックな X25519 とポスト量子の ML-KEM-768）それぞれ3つのモード：

- **native** – 選択したタイプのベースキーペア；
- **xorpub** – 公開部分に追加処理を施した派生モード；
- **random** – ランダム成分を持つ派生モード。

リストから必要なモードを選択し**「生成」をクリックします：パネルはそのモード用の準備済みキーペアで両方のフィールド（**decryption** と **encryption**）を埋めます。「クリア」**ボタンは両フィールドを **none** にリセットします。

ブロックの下には**「選択中: ...」**というステータス行が表示され、生成された文字列からキーの種類（X25519 または ML-KEM-768）とモード（native / xorpub / random）の両方を認識して表示します。空フィールドまたは **none** は「None」と表示されます。

技術的には、ボタンは `GET /panel/api/server/getNewVlessEnc`（`xray vlessenc` によるキー生成）にアクセスし、`mlkem768x25519plus.native.<rtt>.<role>` 形式のペア値（例: `decryption = mlkem768x25519plus.native.600s.server-x25519`、`encryption = mlkem768x25519plus.native.0rtt.client-x25519`）で両方のフィールドを埋めます。`decryption` パラメーターはサーバー側に残り、`encryption` はクライアントリンクに含まれます。

重要: inbound の Xray 設定を生成する際、パネルは余分なものを削除します: `settings` に `encryption`（クライアント側に属するもの）が残っている場合、それはサーバー設定から除外されます。サーバー側には `decryption` のみが残ります。

VLESS を選ぶ場面: 新しい inbound のデフォルト推奨オプションです。特に REALITY（独自証明書不要）や TLS + XTLS-Vision との組み合わせに適しています。

例: 1クライアントと XTLS-Vision を持つ VLESS-inbound の `settings` ブロック。 `flow` フィールドはクライアントに設定され、`decryption` はサーバー側に残ります:

```
{
  "clients": [
    {
      "id": "d342d11e-d424-4583-b36e-524ab1f0afa4",
      "email": "user1",
      "flow": "xtls-rprx-vision",
      "limitIp": 2,
      "totalGB": 0,
      "expiryTime": 0,
      "enable": true
    }
  ],
  "decryption": "none"
}
```

REALITY との組み合わせでは、対応する `streamSettings` ブロック（「Transport」タブ → Security: REALITY）は次のようになります:

```
{
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "dest": "www.microsoft.com:443",
    "serverNames": ["www.microsoft.com"],
    "privateKey": "<приватный ключ X25519>",
    "shortIds": ["", "6ba85179e30d4fc2"]
  }
}
```

5.4. VMess

用途: Xray のクラシックプロキシプロトコル。UUID による認証。クライアント側でペイロードの暗号化方式（`security`）を追加設定します。

`settings` ブロックのフィールド:

フィールド	デフォルト値	説明
<code>clients</code>	<code>[]</code>	クライアントのリスト

各 VMess クライアントの追加フィールド（共通フィールド `email`、制限、`enable`、`tgId`、`subId`、`comment`、`reset` 以外）:

クライアントフィールド	デフォルト値	説明
<code>id</code>	—	クライアントの UUID
<code>security</code>	<code>auto</code>	VMess ペイロードの暗号化方式。許可される値: <code>aes-128-gcm</code> 、 <code>chacha20-poly1305</code> 、 <code>auto</code> 、 <code>none</code> 、 <code>zero</code>

`security` の値:

- `auto` — Xray がプラットフォームに応じて暗号を自動選択（推奨）;
- `aes-128-gcm`、`chacha20-poly1305` — 固定の AEAD 暗号;
- `none` — ペイロードの暗号化なし（TLS の上でのみ意味を持つ）;
- `zero` — ペイロードの暗号化も認証もなし。

後方互換性: 古いレコードでは `security: ""` が保存されている場合があります。読み込み時、空文字列は `auto` に変換されます。サーバー設定生成時、VMess クライアントの `security` フィールドは `settings` から削除されます。inbound には不要なためです。

VMess を選ぶ場面: 古いクライアントや既存の設定との互換性が必要な場合。新規展開では通常 VLESS が推奨されます。

5.5. Trojan

用途: 通常の HTTPS トラフィックに偽装するプロキシ。パスワードによる認証。VLESS と同様に、`fallback` と (`network = tcp` の場合) `REALITY`/TLS をサポートします。

`settings` ブロックのフィールド:

フィールド	デフォルト値	説明
<code>clients</code>	<code>[]</code>	クライアントのリスト
<code>fallbacks</code>	<code>[]</code>	fallback のリスト (<code>network = tcp</code> かつ TLS/REALITY の場合に利用可能)

各 Trojan クライアントのキーフィールド:

クライアントフィールド	デフォルト値	説明
<code>password</code>	–	クライアントのパスワード (必須、最低1文字)
<code>email</code>	–	クライアントの一意識別子

その他のクライアントフィールドは共通 (`limitIp`、`totalGB`、`expiryTime`、`enable`、`tgId`、`subId`、`comment`、`reset`)。

Trojan を選ぶ場面: ポート 443 での HTTPS 偽装が必要な場合、特に予期しない接続に対して Web サーバー (Nginx) への `fallback` が必要な場合。

例: ローカル Web サーバーへの `fallback` を持つ Trojan の `settings` ブロック。有効なパスワードなしの予期しない接続は、`127.0.0.1:8080` をリスンする Nginx に転送されます:

```
{
  "clients": [
    { "password": "S3cret-Pass-1", "email": "user1" }
  ],
  "fallbacks": [
    { "dest": "127.0.0.1:8080" }
  ]
}
```

`fallback` には `network = tcp` と `Security = TLS` または `REALITY` が必要です。それ以外の場合、`fallbacks` フィールドは利用できません。

5.6. Shadowsocks

用途: 軽量の Shadowsocks プロキシ。旧来の AEAD 暗号と新しい SIP022 方式 (`2022-blake3-*`) の両方をサポートします。単一ユーザーまたは複数ユーザーモードで動作できます。

`settings` ブロックのフィールド:

フィールド	デフォルト値	説明
method	2022-blake3-aes-256-gcm	inbound の暗号化方式。UI のラベル: 「暗号化方式」 (英語: 「Encryption method」)
password	``	inbound のパスワード (2022 方式では選択した方式に合わせて自動生成)
network	tcp,udp	トランスポート。ラベル: 「ネットワーク」 (英語: 「Network」)。選択肢: tcp,udp (TCP、UDP)、tcp、udp
clients	[]	クライアントのリスト
ivCheck	false (オフ)	「ivCheck」スイッチ - IV の再利用防止

暗号化方式 (method)

許可される値のセット:

方式	カテゴリ
aes-256-gcm	旧来の AEAD
chacha20-poly1305	旧来の AEAD
chacha20-ietf-poly1305	旧来の AEAD
xchacha20-ietf-poly1305	旧来の AEAD
2022-blake3-aes-128-gcm	SS 2022 (推奨)
2022-blake3-aes-256-gcm	SS 2022 (デフォルト)
2022-blake3-chacha20-poly1305	SS 2022、単一ユーザー

方式に関するパネルのロジック:

- **2022 方式** (2022-blake3-*) は「SS 2022」とみなされます。方式 2022-blake3-chacha20-poly1305 は**単一ユーザー** (複数ユーザー不可); 他の 2022 方式は複数クライアントを許可します。パスワードフィールド (方式に合わせてキー長を調整する生成ボタン付き) は 2022 方式の場合にフォームに表示されます。
- **旧来の暗号** (aes-*、chacha20-*) はクラシックな「1方式 + 1パスワード」方式で動作します。

Xray 起動前の正規化: 旧来の暗号では、各クライアントが inbound の方式と一致する method を持つ必要があります (そうでないと Xray は「unsupported cipher method:」でクラッシュします)。2022 方式では逆に、クライアントの method フィールドは空でなければなりません (そうでないと Xray は「users must have empty method」で inbound を拒否します)。パネルは方式切り替え時に自動的にデータを整合させます。

キーサイズ変更時のクライアントキー再生成: Shadowsocks-2022 で暗号化方式を別のキーサイズを持つ方式 (例: 2022-blake3-aes-256-gcm と 2022-blake3-aes-128-gcm の間) に変更すると、パネルは inbound 保存時に自動的にクライアント PSK を新しい長さに再生成します。そうしないと古

いキーが古い長さのまま残り、Xray に拒否されます。その結果：影響を受けたクライアントはサブスクリプションを再取得する必要があります。古いリンクは接続できなくなります。

Shadowsocks を選ぶ場面：TLS 偽装なしのシンプルな展開向け。現代的な選択は 2022-blake3 方式です。

例：2022-blake3 方式の Shadowsocks settings ブロック（複数ユーザーモード）。 inbound は独自のパスワード（必要な長さの base64 キー）を持ち、各クライアントは独自のパスワードを持ち、クライアントの method フィールドは空です：

```
{
  "method": "2022-blake3-aes-256-gcm",
  "password": "d2hhdGV2ZXItMzItYnl0ZS1iYXNlNjQta2V5LWhlcU=",
  "network": "tcp,udp",
  "clients": [
    {
      "email": "user1",
      "password": "Y2xpZW50LWtleS0zMilieXRlcy1iYXNlNjQtaGVyZQ==",
      "method": ""
    }
  ]
}
```

レガシー暗号（aes-256-gcm 等）では逆です：パスワードは inbound で1つ、クライアントの method は inbound の方式と一致する必要があります。

5.7. Dokodemo-door / Tunnel（透過フォワーダー）

用途：透過フォワーダー（パネルでは dokodemo-door の動作を実装するプロトコル tunnel）。認証やクライアントなしに、トラフィックを受け取り指定のアドレス/ポートに転送します。

settings ブロックのフィールド：

フィールド	デフォルト値	説明
rewriteAddress	(なし)	「アドレスを書き換える」（英語：「Rewrite address」）－トラフィックがリダイレクトされる転送先アドレス
rewritePort	(なし)	「ポートを書き換える」（英語：「Rewrite port」）－転送先ポート（0～65535）
allowedNetwork	tcp,udp	「許可ネットワーク」（英語：「Allowed network」）。選択肢：tcp,udp、tcp、udp
portMap	{}	「ポートマッピング」－ポート→ポートのマップ（Record<string,string>）
followRedirect	false（オフ）	「リダイレクトに従う」（英語：「Follow redirect」）－インターセプトされた接続から元の転送先アドレスを使用

Tunnel の「Transport」タブ：このタイプの inbound では**「Transport」**タブが利用可能ですが、sockopt 設定のみに限定されています。これは **TProxy** モード（sockopt.tproxy を通じた透過プ

ロキシ/リダイレクト)に十分です。トランスポート選択 (network) のドロップダウンリストと Tunnel の「Security」タブは非表示になっています。このタイプでは TLS/REALITY がサポートされないためです。

選ぶ場面: 内部サービスへの透過プロキシ/ポートリダイレクトが必要な場合。

「ポートを書き換える」フィールド (rewritePort) は空のままにできます: 値をクリアするとその値は inbound 設定から除外されるだけで、保存エラーにはなりません。(以前はこのフィールドをクリアすると settings.rewritePort のバリデーションエラーが発生し、JSON タブからの保存も含めて保存がブロックされていました。)

5.8. SOCKS / HTTP (プロトコル mixed)

このビルドには独立した socks プロトコルはありません。SOCKS と HTTP プロキシはプロトコル mixed (SOCKS + HTTP 複合) に統合されています。また、独立した純粋な http プロキシも存在します。

5.8.1. Mixed (SOCKS + HTTP)

settings ブロックのフィールド:

フィールド	デフォルト値	説明
auth	password	「Auth」 – 認証モード。選択肢: password (ログイン/パスワード) または noauth (認証なし)
accounts	(オプション)	「アカウント」 – user/pass ペアのリスト。auth = noauth の場合、フィールドは設定ファイルに書き込まれません
udp	false (オフ)	「UDP」スイッチ – SOCKS を通じた UDP サポート
ip	127.0.0.1	「UDP IP」 – UDP アソシエーション用のローカルアドレス。udp が有効な場合のみ表示

アカウントは「追加」ボタンで追加されます。追加時にランダムなログイン (8文字) とパスワード (12文字) が生成され、編集可能です。

5.8.2. HTTP (純粋なプロキシ)

用途: クラシックなフォワード HTTP プロキシ。Xray レベルではクライアントを「課金対象」として追跡しません (email/制限なし)。アカウントリストのみがあります。

settings ブロックのフィールド:

フィールド	デフォルト値	説明
accounts	[]	「アカウント」 – user/pass ペアのリスト (両フィールド必須)

フィールド	デフォルト値	説明
allowTransparent	false (オフ)	「透過を許可」 (英語: 「Allow transparent」) – 元の Host ヘッダーでリクエストを転送

SOCKS/HTTP を選ぶ場面: 複雑な偽装なしのローカルまたはサービス用プロキシアクセスが必要な場合。
mixed は1つのポートで SOCKS と HTTP クライアントの両方を処理できる点で便利です。

5.9. WireGuard (inbound)

用途: WireGuard inbound – 偽装プロキシではなく WireGuard VPN トンネルです。トランスポートと TLS/REALITY は適用されません。

バージョン3.4.2 から WireGuard はマルチクライアントモデルに移行しました (VLESS/VMess/Trojan/Shadowsocks/Hysteria と同様)。inbound 自体はサーバー側のみを保持します。peer デバイスは通常のクライアント (セクション8参照) として追加されるようになりました – トンネル内アドレスの自動割り当て、サブスクリプション対応、トラフィック/期限の制限、グループに対応します。inbound フォームの従来のインライン「Peer / peer を追加」リストは削除されました: 新しい inbound は peer なしで作成され、各クライアントにはアドレスが自動的に割り当てられます。

inbound のフィールド (settings ブロック) :

フィールド	デフォルト値	説明
secretKey	–	サーバーのプライベートキー (必須)。隣に生成ボタン。公開キー (Public Key) はそこから自動的に導出されます (読み取り専用; 別途保存されていた pubKey は削除されました)
mtu	1420	インターフェースの MTU
dns	1.1.1.1, 1.0.0.1	3.4.2 で追加。クライアントの .conf の DNS = 行に書き込まれる DNS
noKernelTun	false (オフ)	「カーネルなし TUN」 – カーネルモードの代わりにユーザースペース TUN を使用
domainStrategy	(オプション)	「Domain Strategy」: ForceIP、ForceIPv4、ForceIPv4v6、ForceIPv6、ForceIPv6v4

クライアントの WireGuard パラメーター – クライアントの追加/編集ウィンドウの「認証情報」タブ (紐付けられた inbound が WireGuard の場合に表示されます) :

フィールド	説明
「WireGuard プライベートキー」	クライアントのプライベートキー (編集可能、「再生成」ボタンあり)。入力するとそこから公開キーが自動的に導出されます
「WireGuard 公開キー」	読み取り専用; プライベートキーから計算されます

フィールド	説明
「WireGuard 共有キー」 (PSK)	任意の追加の事前共有キー
「WireGuard 許可 IP」	3.5.0 から 編集可能 (プレースホルダー <code>10.0.0.2/32</code> 、ヒント「自動割り当ての場合は空のままにしてください; エントリはカンマで区切ります」)。空の場合はアドレスが自動的に割り当てられます; サーバーは各エントリ (IP または CIDR) を検証し、単一アドレスは <code>/32</code> に正規化し、この inbound の別のクライアントが使用中のアドレスは拒否します

トンネル内アドレスは、inbound のサブネット (デフォルト `10.0.0.0/24`: サーバーが `.1` を占め、クライアントは `.2` から) からサーバーによって自動的に割り当てられます。既存のクライアントが別の `/24` を使用している場合、新しいアドレスは同じサブネットから割り当てられます。

Domain Strategy と Transport タブ

上記のほか、WireGuard inbound には **Domain Strategy** フィールド (ドメイン解決戦略: `ForceIP`、`ForceIPv4`、`ForceIPv4v6`、`ForceIPv6`、`ForceIPv6v4`) があります。このフィールドはオプションで、設定された場合のみ設定ファイルに書き込まれます。

Workers フィールド (`workers`、ワーカースレッド数) は WireGuard フォーム (inbound と outbound の両方) から削除されました: xray-core v26.6.22 以降、エンジンはこれを使用せず、wireguard-go の内部メカニズムに依存しています。以前保存された設定はそのまま動作します。解析時にフィールドは単に無視され、移行は不要です。

WireGuard には**「Transport」**タブも利用可能ですが、縮小版です: `sockopt` と **Finalmask** 難読化の設定のみ。WireGuard は常に UDP でリッスンするため、トランスポート選択 (`network`) のドロップダウンリストは非表示です。ノイズレコード (`noise`) では Finalmask が別フィールドで **Rand Range** (バリデーション付きの 0~255 バイト範囲) を提供し、WireGuard と Hysteria の難読化方式として **Salamander** が利用可能です。

inbound の操作とエクスポート (3.5.0 から)

WireGuard inbound の行メニューには、完全な「マルチクライアント」操作セットが含まれるようになりました – 「すべてのリンクをエクスポート」、サブスクリプションによるエクスポート、クライアントの紐付け/紐付け解除、グループへの追加、全クライアントの削除です (以前はエクスポート/リセット/クローン/削除のみでした)。リストにはクライアント数のカウンターが表示されます。WireGuard の「すべてのリンクをエクスポート」ウィンドウは 2 つのタブに分割されました: **Config** (クライアントごとに 1 つずつ連結された `.conf` ブロック) と **Links** (パーソナルな `wireguard://` リンク) です。「コピー」と「ダウンロード」は開いているタブに対して機能します。ノード上に配置された inbound では、クライアントの `.conf` /QR の **Endpoint** はメインパネルではなく **ノードのアドレス** を指すようになりました。

WireGuard クライアントの設定と共有

WireGuard クライアントの「情報」/QR ウィンドウには、**折りたたみ可能な設定カード** があります: 完全な `.conf` (`PrivateKey / Address / DNS / MTU` を含む `[Interface]` と、 `PublicKey / PresharedKey / AllowedIPs = 0.0.0.0/0, ::/0 / Endpoint / PersistentKeepalive` を含む `[Peer]`) と、**コピー**、**ダウンロード**、**QR** のボタンが付いています。クライアントアプリが完全な `.conf` に展開する `wireguard:///wg://` 形式のリンクもサポートされています。

WireGuard を選ぶ場面: 偽装プロキシではなく本格的な WireGuard VPN トンネルが必要な場合。

5.10. Hysteria (デフォルト v2)

用途: QUIC 上の Hysteria inbound。パネルはデフォルトでバージョン 2 で動作します。各クライアントは UUID/パスワードの代わりに `auth` トークンで認証します。Hysteria では TLS は常に利用可能です (5.2 の機能表を参照)。

`settings` ブロックのフィールド:

フィールド	デフォルト値	説明
<code>version</code>	2	プロトコルバージョン (最小 1; パネルのデフォルト 2)
<code>clients</code>	<code>[]</code>	クライアントのリスト

各クライアントのキーフィールドは `auth` (トークン、必須) と共通フィールド (`email`、制限、`enable`、`tgId`、`subId`、`comment`、`reset`) です。

追加パラメーターは `streamSettings.hysteriaSettings` で設定します:

フィールド	値 / 選択肢	説明
<code>version</code>	2 で固定 (フィールドはロック)	「バージョン」 (英語: 「Version」)
<code>udpIdleTimeout</code>	(整数 ≥ 1、秒)	「UDP アイドルタイムアウト (秒)」 – UDP アイドルタイムアウト
<code>masquerade</code>	デフォルトで無効	「Masquerade」 – 「予期しない」 リクエストに対して通常の Web サーバーに偽装

`masquerade` を有効にするとタイプ (`type`) を選択できます:

- `''` – デフォルト (404 ページ) ;
- `proxy` – リバースプロキシ (「Upstream URL」、 「Host を書き換える」、 「TLS verify をスキップ」 フィールド) ;
- `file` – ディレクトリ配信 (「ディレクトリ」 フィールド、例: `/var/www/html`) ;
- `string` – 固定レスポンス (「ステータスコード」、 「Body」、 「ヘッダー」 フィールド) 。

Hysteria を選ぶ場面: QUIC トランスポートが必要で、不安定/モバイル回線での安定性が求められる場合。偽装はエントリーポイントの秘匿性を高めます。

5.11. MTProto (Telegram プロキシ)

バージョン **3.3.0** で追加。プロトコルの値は `mtproto`。

MTProto は Telegram 独自のプロキシプロトコルです。3X-UI では、この inbound は **Xray** ではなく**別プロセス `mtg`** によって処理され、パネル自身が管理します。パネルは定期的に有効な MTProto inbound と実行中の `mtg` プロセスを照合し、不足しているものを起動し、余分なものを停止し、`mtg` のメトリクスからトラフィックカウンターを取得します。そのため、この inbound のトラフィック計測は通常のプロトコルと同様に機能します。

バージョン**3.5.0** から **MTProto はマルチクライアントプロトコル**です（エンジンはフォーク `mtg-multi` に変更されました）: 1 つの inbound が多数のユーザーを処理し、各ユーザーは、独自の FakeTLS シークレット、トラフィッククォータ、有効期限、有効化スイッチ、任意の ad-tag、パーソナルな `tg://proxy` リンクを持つ通常の **クライアント**（セクション8参照）です。従来の「1 つの inbound = 1 つのシークレット」モデルは廃止されました: inbound レベルの「Secret」フィールドはフォームから削除され、パネルの更新時に既存の単一シークレットは自動的にこの inbound の最初のクライアントに変換されます（手動での操作は不要です）。

結果として:

- 「**Transport**」タブ (**Stream Settings**) はこの inbound には適用されません。クライアントは他のプロトコルと同様に管理されるようになりました – 紐付け/紐付け解除、制限、グループ、サブスクリプション。
- クライアントの変更（追加、削除、シークレットの変更、有効化/無効化、ad-tag）は、`mtg` の management API を通じて「**オンザフライ**」で適用され、他のユーザーの Telegram セッションを切断しません。inbound の構造的な変更（アドレス、domain fronting、接続数制限、パブリック IP）のみがプロセスを再起動します。
- MTProto inbound は**メインパネルでのみ起動**されます。子ノード (nodes) には展開されません（NodeID が指定された inbound はスキップされます）。
- MTProto の**「Sniffing」**タブは非表示です – このプロトコルは Xray ではなく `mtg` プロセスによって処理されるため、スニффイングは適用できません。

フィールド。inbound の `settings` に保存されます:

UI のフィールド	キー	説明
Remark	<code>remark</code>	inbound のラベル。
Listen IP	<code>listen</code>	リッスン IP (空 = 全インターフェース)。
Port	<code>port</code>	プロキシポート。

UI のフィールド	キー	説明
偽装ドメイン (FakeTLS)	<code>settings.fakeTlsDomain</code>	デフォルトは <code>www.cloudflare.com</code> 。3.5.0 から、これは 新しいクライアントのシークレット生成に使えるデフォルトドメイン です（ヒント: 「Default FakeTLS domain used to generate a new client's secret. Each client can front its own domain」）。各クライアントは自分のシークレットを通じて独自のドメインで偽装できます。
Max connections	<code>settings.throttleMaxConnections</code>	3.5.0 の新機能 。全ユーザーに対する同時接続数の制限（公平に分配されます）。 <code>0</code> は無制限。
Public IPv4 / Public IPv6	<code>settings.publicIpv4</code> / <code>settings.publicIpv6</code>	3.5.0 の新機能 。ad-tag middle proxy 用のサーバーのパブリックアドレス（プレースホルダー <code>1.2.3.4</code> / <code>2001:db8::1</code> ）。空の場合は <code>mtg</code> が自動判定します。

シークレットの形式（FakeTLS）。シークレットは **各クライアントごと** に保持されるようになりました（クライアントフォームの「**MTProto secret**」フィールド、再生成ボタン付き。その隣には任意の「**Ad-tag（スポンサーチャンネル）**」、ちょうど 32 の hex 文字）。シークレットの形式は従来どおりです：`ee` + 32 の hex 文字 + 偽装ドメインの hex コード（`ee<hex32><hex(ドメイン)>`）。新しいクライアントを MTProto inbound に紐付けると、シークレットは inbound のデフォルトドメインから自動的に生成されます。3.5.0 から、クライアントのトラフィッククォータと有効期限は（`mtg-multi` の制限を通じて）**実際に適用されます**：枯渇または期限切れのクライアントは無効化され、トラフィックのリセットで即座にアクセスが回復します。

Domain-fronting と mtg の高度なオプション

MTProto inbound には `mtg` プロセスの追加パラメーターがあります。**Domain fronting IP**、**Domain fronting port**、**Domain fronting PROXY protocol** フィールドは、`mtg` が Telegram 以外のトラフィック（例：フェイク NGINX サイトへ）を送信する先を指定します：IP を空にすると DNS 経由で FakeTLS ドメインが使用され、デフォルトポートは `443` です。追加で **Accept PROXY protocol**（リスナー用）、**IP preference**（`prefer-ipv6` / `prefer-ipv4` / `only-ipv6` / `only-ipv4`）、**Debug logging** が利用可能です。各値は設定された場合のみ `mtg-<id>.toml` ファイルに書き込まれます。

Xray を通じた Telegram トラフィックのルーティング

「Route through Xray」スイッチ（デフォルトでオフ）とオプションの **Outbound** フィールドにより、Telegram の egress を Xray のルーターに委ねることができます。有効にすると、パネルは Xray 設定に inbound 自身のタグを持つローカル SOCKS ブリッジを組み込み、`mtg` はそれを通じて Telegram トラフィックを送信します。その後、「Routing」タブのルールでトラフィックをマッチングしたり、**Outbound** フィールド（空の場合はルーティングルールが決定）で選択した outbound またはバランサーに強制転送できます。

ユーザーへの配布方法。MTProto inbound に対して、パネルは招待リンクを生成します：

例: **FakeTLS** シークレットと完成したリンク。偽装ドメインが `www.cloudflare.com` の場合、シークレットは `ee + 32` の hex 文字 + ドメインの hex コードとして構築されます。例:

```
secret = ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

完成した招待リンク (Telegram でユーザーに送信するリンクと QR コード) :

```
tg://proxy?
server=203.0.113.10&port=443&secret=ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

```
tg://proxy?server=<アドレス>&port=<ポート>&secret=<シークレット>
```

(同等: `https://t.me/proxy?server=...&port=...&secret=...`)。3.5.0 からリンクは **パーソナル** です – 特定のクライアントのシークレットから構築され、フラグメント `#remark` はリンクから削除されました (Telegram の厳密でないパーサーがそれをシークレットと連結してインポートを壊していました)。備考は情報ウィンドウに別のラベルとして表示されます。このリンクと QR コードを Telegram ユーザーに送信します。開くとプロキシがアプリにすぐ追加されます。リンクはサブスクリプションサーバーからも提供されます。

使用場面。 Telegram ブロック回避の標準的な方法。FakeTLS 偽装 (偽装ドメイン) により、トラフィックは指定されたサイトへの通常のアクセスのように見えます。

5.12. プロトコル選択クイックリファレンス

- **VLESS** – デフォルトの選択; REALITY または TLS + XTLS-Vision との組み合わせに最適。ポスト量子認証をサポート。
- **Trojan** – Web サーバーへの fallback を持つ HTTPS 偽装。
- **VMess** – 古いクライアントとの互換性。
- **Shadowsocks** – TLS なしのシンプルなプロキシ; 現代的な選択は `2022-blake3-*` 方式。
- **Hysteria** – QUIC、不安定な回線での安定性。
- **mixed / http** – サービス用 SOCKS/HTTP プロキシ。
- **WireGuard** – 本格的な VPN トンネル。
- **tunnel** – 透過的なポートリダイレクト。
- **MTProto** – Telegram ブロック回避プロキシ (FakeTLS); 別プロセス `mtg`。

6. トランスポート (Stream Settings)

トランスポート（パネルのインターフェースではフィールド名「**Transmission**」）は、Xray-core が inbound 内でデータを転送する方法、すなわち TLS/Reality の上位で使用するネットワークプロトコルとトラフィックのフレーミング方式を定義します。これらのパラメータは Xray の設定オブジェクト `streamSettings` に保存され、inbound エディターのトランスポートタブで設定します。暗号化（TLS / Reality）については別のセクションで説明します。ここではネットワークの選択とそのパラメータのみを説明します。

6.1. 転送ネットワークの選択

ネットワークはドロップダウンリスト「**Transmission**」（`streamSettings.network`）で選択します。デフォルト値は `tcp`（リストでは **RAW** と表示）です。利用可能な選択肢は次のとおりです。

リスト上の表示	<code>network</code> フィールド	トランスポート
RAW	<code>tcp</code>	通常の TCP（新バージョンの Xray では RAW に改名）。オプションで HTTP 難読化を使用可能
mKCP	<code>kcp</code>	信頼性のある UDP トランスポート mKCP
WebSocket	<code>ws</code>	HTTP(S) 上の WebSocket
gRPC	<code>grpc</code>	gRPC トンネル (HTTP/2)
HTTPUpgrade	<code>httpupgrade</code>	HTTP Upgrade
XHTTP	<code>xhttp</code>	XHTTP / SplitHTTP – 現代的な多重化トランスポート

値を変更すると、パネルは前のネットワークの設定ブロックをクリアし、新しいネットワークのブロックをそのスキーマのデフォルト値で埋めます。そのため、サブフォームの各フィールドには常に意味のある初期値が設定されます。

重要。 本バージョンのパネルでは **HTTP/2 トランスポート（h2）** はリストに存在しません – これはネットワークセットから除外されています。双方向 HTTP/2 に類似したトンネルには gRPC を、現代的な HTTP マスキングトランスポートには XHTTP を使用してください。**Hysteria** トランスポート（`hysteria`）はこのリストから選択するものではありません。Hysteria プロトコルに固定されており、inbound 自体が Hysteria プロトコルで作成されたときに自動的に有効になります（6.8 項参照）。

以下では、各ネットワークとそのフィールドを個別に説明します。

6.2. RAW / TCP（`tcpSettings`）

基本的な TCP トランスポートです。デフォルトではトラフィックをそのまま転送します。オプションで通常の HTTP/1.1 通信に見せかけることができます。

フィールド	デフォルト値	説明
Proxy Protocol (<code>acceptProxyProtocol</code>)	<code>false</code> (無効)	上位のロードバランサー/プロキシからの PROXY protocol ヘッダーを受け入れる
HTTP-обфускация (<code>header.type</code>)	<code>none</code> (無効)	トラフィックを HTTP/1.1 に見せかける難読化を有効にする

Proxy Protocol

スイッチ「**Proxy Protocol**」(`acceptProxyProtocol`)。有効にすると、Xray は受信接続で PROXY protocol ヘッダーを待受し、そこからクライアントの実際の IP を取得します。パネルの前にリバースプロキシ/ロードバランサー (HAProxy や `send-proxy` を設定した nginx など) があり、そのヘッダーを付加している場合にのみ有効にします。デフォルトでは無効です。

HTTP 難読化 (camouflage)

スイッチ「**HTTP Обфускация**」。 `header` フィールドを制御します。

- **無効** → `header.type = "none"` (ワイヤー上では `header` フィールドが存在しない)。クリーンな TCP。
- **有効** → `header.type = "http"`。トラフィックが HTTP/1.1 のリクエストとレスポンスに見せかけてフレーミングされます。有効にすると、パネルはすぐに `request` と `response` のサブオブジェクトをデフォルト値で埋めます。

HTTP 難読化が有効の場合、模倣するリクエストとレスポンスの設定フィールドが表示されます。

リクエストヘッダー (`header.request`) :

フィールド	キー	デフォルト値	説明
リクエストバージョン	<code>request.version</code>	<code>1.1</code>	リクエスト開始行の HTTP バージョン
リクエストメソッド	<code>request.method</code>	<code>GET</code>	模倣するリクエストの HTTP メソッド
リクエストパス	<code>request.path</code>	<code>/</code>	パス (複数可)。カンマ区切りで入力し、ワイヤー上は文字列の配列になります。空の場合は <code>/</code> が使用されます
リクエストヘッダー	<code>request.headers</code>	<code>{}</code> (空)	HTTP ヘッダーの「名前/値」テーブル。名前 → [値] のマップとして保存されます (1 つの名前に複数の値を指定可能)

レスポンスヘッダー (`header.response`) :

フィールド	キー	デフォルト値	説明
レスポンスバージョン	<code>response.version</code>	<code>1.1</code>	レスポンス開始行の HTTP バージョン

フィールド	キー	デフォルト値	説明
レスポンスステータス	<code>response.status</code>	200	模倣するレスポンスの HTTP ステータスコード
レスポンス理由句	<code>response.reason</code>	OK	ステータスのテキスト説明 (reason-phrase)
レスポンスヘッダー	<code>response.headers</code>	{ } (空)	レスポンスヘッダーの「名前/値」テーブル (名前 → [値] のマップ)

ヘッダーフィールドは行ごとに編集します。各行でヘッダー名（名前）と値（値）を指定します。これらのパラメータはトラフィックの外観を偽装するためだけに使用されており、暗号化には影響しません。デフォルト値（GET / HTTP/1.1、レスポンス 200 OK）はほとんどのシナリオに適しています。特定のサイト/サービスを模倣する必要がある場合にのみ変更してください。

HTTP 難読化を使用した RAW の `streamSettings` 例:

```
{
  "network": "tcp",
  "tcpSettings": {
    "acceptProxyProtocol": false,
    "header": {
      "type": "http",
      "request": {
        "version": "1.1",
        "method": "GET",
        "path": ["/"],
        "headers": {
          "Host": ["www.example.com"]
        }
      },
    },
    "response": {
      "version": "1.1",
      "status": "200",
      "reason": "OK"
    }
  }
}
```

注意: ワイヤー上の `path` は文字列の配列であり、各ヘッダーも値の配列です（Host → ["www.example.com"]）。

6.3. mKCP (`kcpSettings`)

mKCP は UDP 上の信頼性のあるトランスポートです。パケットロスや高遅延のチャンネルに有用ですが、オーバーヘッドトラフィックが増加します。すべてのデフォルト値は xray-core の推奨値と一致しています。

フィールド	キー	デフォルト	許容値	説明
MTU	<code>mtu</code>	1350	576–1460	最大パケットサイズ（バイト）。断片化の問題がある場合は減らします
TTI (ms)	<code>tti</code>	20	10–100	送信間隔（ms）。小さいほど遅延が低くなりますが、オーバーヘッドが増加します
Uplink (MB/s)	<code>uplinkCapacity</code>	5	≥ 0	アップロード方向の推定帯域幅（MB/s）
Downlink (MB/s)	<code>downlinkCapacity</code>	20	≥ 0	ダウンロード方向の推定帯域幅（MB/s）
CWND 乗数	<code>cwndMultiplier</code>	1	≥ 1	輻輳ウィンドウ（congestion window）の乗数
最大送信ウィンドウ	<code>maxSendingWindow</code>	2097152	≥ 0	送信ウィンドウの最大サイズ

フィールドについての注意事項：

- **Uplink / Downlink capacity** は mKCP がチャンネルをどれだけ積極的に使用するかを設定します。実際のチャンネル帯域幅に合わせて設定してください。高すぎると余分なトラフィックが発生し、低すぎるとチャンネルが十分に活用されません。
- **TTI** は「遅延  オーバーヘッド」のトレードオフに直接影響します。小さい値は遅延を減らしますが、制御パケットの量が増加します。
- **MTU** は mKCP パケット 1 つのサイズを制限します。大きな UDP パケットが切断または損失するチャンネルでは縮小が有効です。

本バージョンのパネルでは、mKCP サブフォームに「**seed**」（mKCP 難読化パスワード）およびヘッダータイプ/難読化タイプのドロップダウンリスト（none, srtp, utp, wechat-video, dtls, wireguard）が独立したフィールドとして存在しません。トランスポート層の難読化は、mkcp-legacy モードを含む共通の「FinalMask」メカニズムに移行されており、該当するセクションで説明されています。「congestion」パラメータも独立したチェックボックスとして表示されず、輻輳制御は `cwndMultiplier` と `maxSendingWindow` で設定します。

6.4. WebSocket (`wsSettings`)

HTTP(S) 上の WebSocket トランスポートです。CDN やリバースプロキシをよく通過でき、通常の Web トラフィックに見せかけることができます。

フィールド	キー	デフォルト	説明
Proxy Protocol	<code>acceptProxyProtocol</code>	false	上位プロキシからの PROXY protocol ヘッダーを受け入れる (6.2 項参照)

フィールド	キー	デフォルト	説明
ホスト	host	"" (空)	HTTP Host ヘッダーの値。CDN/ドメインフロンティング経由で動作する場合に指定します
パス	path	/	WebSocket ハンドシェイクのリクエスト行のパス
Heartbeat 周期	heartbeatPeriod	0	heartbeat フレームの送信間隔 (秒)。0 で heartbeat を無効化します
ヘッダー	headers	{ } (空)	ハンドシェイクの追加 HTTP ヘッダー。名前 → 値 のマップ (配列なし、文字列値のみ)

注意事項:

- **パス** はサーバー (inbound) とクライアントで一致する必要があります。Web サーバー側ではこのパスをエントリーポイントとして隠すことが多いです。
- **ホスト** は、inbound が CDN の背後にある場合やドメインフロンティングを使用している場合に設定します。それ以外は空のまま構いません。
- **Heartbeat 周期** は、非アクティブなセッションを積極的に切断するプロキシ/CDN を通過する際に接続を「生きた」状態に保ちます。デフォルト (0) では heartbeat は無効です。
- RAW とは異なり、WebSocket のヘッダーテーブルは 名前 → 値 の「フラット」形式 (ヘッダーごとに 1 行の値) を使用します。

CDN の背後での WebSocket の streamSettings 例:

```
{
  "network": "ws",
  "wsSettings": {
    "acceptProxyProtocol": false,
    "host": "cdn.example.com",
    "path": "/ray",
    "heartbeatPeriod": 0,
    "headers": {
      "User-Agent": "Mozilla/5.0"
    }
  }
}
```

host と path の値はクライアントと一致する必要があります。RAW とは異なり、ここでのヘッダー値は配列ではなく通常の文字列です。

6.5. gRPC (grpcSettings)

パラメータ数が最も少ないトランスポートです。gRPC コール内 (HTTP/2 上) でトラフィックをトンネリングします。gRPC をサポートする CDN との互換性が高いです。ヘッダー難読化はありません。

フィールド	キー	デフォルト	説明
Service Name	<code>serviceName</code>	<code>""</code> (空)	gRPC サービス名 (実質的にはトンネルの「パス」)。サーバーとクライアントで一致する必要があります
Authority	<code>authority</code>	<code>""</code> (空)	<code>:authority</code> 擬似ヘッダーの値 (HTTP/2 における <code>Host</code> に相当)。CDN/ドメイン経由で動作する場合に指定します
Multi Mode	<code>multiMode</code>	<code>false</code> (無効)	1 つの接続内で複数の並列 gRPC ストリームを多重化することを有効にします

注意事項:

- **Service Name** は gRPC チャンネルの主要な識別子であり、両側で同一である必要があります。空の値も可能ですが、通常は偽装のために分かりにくい文字列を設定します。
- **Authority** は HTTP/2 フレームで送信される `:authority` に影響します。主に CDN 経由でプロキシする場合に必要です。
- **Multi Mode** を使用すると、複数の論理ストリームが 1 つの物理接続を経由できます。サーバーとクライアントの両方がサポートしている場合、パフォーマンス向上のために有効にします。

gRPC の `streamSettings` 例:

```
{
  "network": "grpc",
  "grpcSettings": {
    "serviceName": "GunService",
    "authority": "grpc.example.com",
    "multiMode": false
  }
}
```

`serviceName` (ここでは `GunService`) はトンネルの「パス」として機能し、サーバーとクライアントで一致する必要があります。

6.6. HTTPUpgrade (`httpupgradeSettings`)

HTTP の `Upgrade` メカニズム (WebSocket と同様ですが、WebSocket プロトコル自体は使用しない) に基づくトランスポートです。プロキシや CDN もよく通過できます。フィールド構成は WebSocket と同様ですが、heartbeat 周期がありません。

フィールド	キー	デフォルト	説明
Proxy Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	上位プロキシからの PROXY protocol ヘッダーを受け入れる
ホスト	<code>host</code>	<code>""</code> (空)	HTTP <code>Host</code> ヘッダーの値
パス	<code>path</code>	<code>/</code>	<code>Upgrade</code> ヘッダーを伴う HTTP リクエストのパス

フィールド	キー	デフォルト	説明
ヘッダー	headers	{ } (空)	追加の HTTP ヘッダー。名前 → 値 の「フラット」マップ (WebSocket と同様)

ホスト、パス、ヘッダー の各フィールドの意味は WebSocket (6.4 項) と同じです。Heartbeat は HTTPUpgrade では提供されていません – これは WebSocket 固有の機能です。

6.7. XHTTP / SplitHTTP (xhttpSettings)

XHTTP (SplitHTTP と呼ばれる) は xray-core の現代的な多重化 HTTP トランスポートです。アップリンクとダウンリンクのストリームを別々の HTTP リクエストに分割するため、接続時間に制限のある CDN や環境に適しています。エディターにすべてのフィールドが同時に表示されるわけではありません。一部のフィールドは選択したモード (mode) と有効にしたスイッチに応じて表示されます。

基本フィールド (常に表示)

フィールド	キー	デフォルト	説明
ホスト	host	"" (空)	HTTP Host ヘッダーの値
パス	path	/	HTTP リクエストのベースパス
Mode	mode	auto	転送モード (下記参照)
Server Max Header Bytes	serverMaxHeaderBytes	0	サーバー側のリクエストヘッダーサイズ制限 (バイト)。0 は xray-core のデフォルト値
Padding Bytes	xPaddingBytes	100-1000	サイズ分析を困難にするためのランダムなパディングの範囲 (バイト単位、最小-最大 形式)
ヘッダー	headers	{ } (空)	追加の HTTP ヘッダー。名前 → 値 の「フラット」マップ
HTTP-метод Uplink	uplinkHTTPMethod	"" (デフォルト = POST)	アップリンクリクエストの HTTP メソッド。選択肢: 空 (デフォルトは POST)、POST、PUT、GET (最後のものは packet-up モードのみ使用可能)
Padding Obfs Mode	xPaddingObfsMode	false (無効)	拡張パディング難読化を有効にし、追加フィールドを表示します (下記参照)
No SSE Header	noSSEHeader	false (無効)	Content-Type: text/event-stream (SSE) ヘッダーを送信しません。中間ノードを通過する際に問題がある場合に有効にします

「Mode」フィールド (mode)

ドロップダウンリストの値:

値	説明
auto	モードの自動選択（デフォルト）
packet-up	アップリンクストリームを個別の HTTP リクエストに分割（パケットごとに 1 リクエスト）
stream-up	アップリンクストリームを 1 つの長いストリーミングリクエストで転送
stream-one	1 つの共有双方向ストリーミングリクエスト

モードの選択によって、表示される追加フィールドが決まります。

mode = packet-up の場合にのみ表示されるフィールド:

フィールド	キー	デフォルト	説明
最大バッファリング送信数	scMaxBufferedPosts	30	アップリンクストリームで同時にバッファリングされる POST リクエストの最大数
最大送信サイズ (バイト)	scMaxEachPostBytes	1000000	1 つのアップリンク POST リクエストの最大サイズ (バイト)
Uplink Data Placement	uplinkDataPlacement	"" (デフォルト = body)	アップリンクデータの配置場所: body、header、cookie、query
Uplink Data Key	uplinkDataKey	""	uplink データのキー/ヘッダー名。 uplinkDataPlacement が設定されており body 以外の場合にのみ表示されます

mode = stream-up の場合にのみ表示されるフィールド:

フィールド	キー	デフォルト	説明
Stream-Up Server	scStreamUpServerSecs	20-80	サーバー側のストリーミング接続保持時間の範囲 (秒単位、最小-最大 形式)

パディング難読化フィールド (xPaddingObfsMode = 有効の場合に表示)

フィールド	キー	デフォルト	説明
Padding Key	xPaddingKey	"" (プレースホルダー x_padding)	パディングのキー名
Padding Header	xPaddingHeader	"" (プレースホルダー X-Padding)	パディングを転送する HTTP ヘッダー名
Padding Placement	xPaddingPlacement	"" (デフォルト = queryInHeader)	パディングの配置場所: queryInHeader、header、cookie、query
Padding Method	xPaddingMethod	"" (デフォルト = repeat-x)	パディング生成メソッド: repeat-x または tokenish

セッションとシーケンスの配置（常に表示）

フィールド	キー	デフォルト	説明
Session ID Placement	sessionIDPlacement	""（デフォルト = path）	セッション識別子の転送場所: path、header、cookie、query
Session ID Key	sessionIDKey	""（プレースホルダー x_session）	セッションキー名。sessionIDPlacement が設定されており path 以外の場合にのみ表示されます
Session ID Table	sessionIDTable	""（プレースホルダー Base62）	セッション識別子の生成に使用する文字セット。オートコンプリートのドロップダウンから事前定義済みの値（ALPHABET、Alphabet、BASE36、Base62、HEX、alphabet、base36、hex、number）を選択するか、任意の ASCII 文字列を入力できます。空の場合は xray-core のデフォルト値が使用されます
Session ID Length	sessionIDLength	""（空）	生成される識別子の長さまたは範囲（例: 8-16）。Session ID Table が設定されている場合にのみ表示されます。最小値は 0 より大きい必要があります
Sequence Placement	seqPlacement	""（デフォルト = path）	パケットのシーケンス番号の転送場所: path、header、cookie、query
Sequence Key	seqKey	""（プレースホルダー x_seq）	シーケンスキー名。seqPlacement が設定されており path 以外の場合にのみ表示されます

セッションフィールドは xray-core v26.6.22 で名称が変更されました。以前は **Session Placement / Session Key**（sessionPlacement / sessionKey）と呼ばれていましたが、現在は **Session ID Placement / Session ID Key**（sessionIDPlacement / sessionIDKey）になっています。古い名称はコアでは認識されなくなっています。更新前に保存された inbound は新しいキーに自動的に移行されます – 再保存は不要です。

推奨事項:

- ほとんどの構成では、**Mode = auto** のまま パス/ホスト を設定し、CDN 経由で動作する場合はクライアントと一致させるだけで十分です。
- 配置フィールド（*Placement / *Key）およびパディング難読化フィールドは、特定のアンチ DPI/CDN シナリオに対する微調整が必要な場合にのみ使用します。空の場合は括弧内に示されている xray-core のデフォルト値が使用されます。
- クライアント/outbound 側のパラメータ（POST 再試行間隔、チャンクサイズなど）は inbound フォームには表示されません – サーバーリスナーはそれらを無視します。一方、XMUX 多重化は inbound フォームで利用可能です（下記参照）。
- サービス用デフォルト値は設定されません。** パネルは XHTTP 設定のサービス用デフォルト値 scMaxEachPostBytes と scMinPostsIntervalMs をコンフィグに書き込まなくなりました – xray-core の内部値が適用されます。これにより、以前トラフィックのブロックに利用されていた固定の DPI シグネチャ（リテラル scMinPostsIntervalMs=30）が除去されます。すでに保存された inbound

では、xray-core のデフォルトと一致する値はリンクやサブスクリプションに出力されません (inbound の再保存は不要)。手動で設定した値は保持されます。

XHTTP (auto モード) の streamSettings 例:

```
{
  "network": "xhttp",
  "xhttpSettings": {
    "host": "xhttp.example.com",
    "path": "/yourpath",
    "mode": "auto",
    "xPaddingBytes": "100-1000"
  }
}
```

ほとんどの構成ではこの 4 つのフィールドで十分です。セッション/シーケンスの配置フィールドとパディング難読化フィールドは空のままにしておくと、xray-core のデフォルト値が使用されます。

XMUX (接続の多重化)

スイッチ **XMUX** (enableXmux) を有効にすると、並列リクエストを少数の物理接続のプールに分散する多重化レイヤーが有効になります。有効にすると、6 つの設定可能なフィールドが表示されます (xhttpSettings.xmux に保存)。

フィールド	キー	デフォルト	説明
Max Concurrency	maxConcurrency	16-32	1 接続あたりの最大同時リクエスト数 (最小-最大 範囲)
Max Connections	maxConnections	6	最大物理接続数 (0 は制限なし)。バージョン3.4.2 から、新しい inbound は値 6 で作成されます; それ以前に作成されたものは自身の値を保持します。
Max Reuse Times	cMaxReuseTimes	"" (空)	接続を再利用する回数
Max Request Times	hMaxRequestTimes	600-900	接続あたりの最大リクエスト数 (範囲)
Max Reusable Secs	hMaxReusableSecs	1800-3000	接続が再利用可能な時間 (秒、範囲)
Keep Alive Period	hKeepAlivePeriod	"" (空)	接続を維持するための keep-alive 周期

重要。 **Max Connections** と **Max Concurrency** を同時に設定することはできません – xray-core はそのような設定を拒否します。デフォルトでは XMUX を有効にした際にパネルが Max Concurrency = 16-32 を設定します。 **Max Connections** に 0 より大きい値を設定した場合、パネルは競合を避けるために Max Concurrency のデフォルト値を削除します。

6.8. Hysteria トランスポート（`hysteriaSettings`）

Hysteria トランスポートは「Transmission」リストから選択するものではありません。inbound が Hysteria プロトコルで作成されたときに自動的に有効になり、他のプロトコルでは非表示になります（Hysteria プロトコルから離れると、ネットワークは強制的に `tcp` に戻ります）。パラメータ:

フィールド	キー	デフォルト	説明
バージョン	<code>version</code>	2（固定、フィールドはロック済み）	Hysteria のバージョン。Hysteria 2 のみサポート
UDP idle timeout（秒）	<code>udpIdleTimeout</code>	60	UDP セッションのアイドルタイムアウト（秒）。有効範囲は 2-600。範囲外の値は xray-core の起動時に拒否されます
Masquerade	<code>masquerade</code>	無効（不在）	プローブ時にリスナーを HTTP/3 サーバーに偽装することを有効にします

Masquerade を有効にすると、タイプ（`type`）の選択とそれに依存するフィールドが表示されます:

- `""` – **default（404 page）**：標準の 404 ページを返します（追加フィールド不要）。
- **proxy（reverse proxy）**：外部サイトへのリバースプロキシ。
 - `url`（**Upstream URL**、プレースホルダー `https://www.example.com`） – ターゲットアドレス；
 - `rewriteHost`（**Rewrite Host**、デフォルト `false`） – Host ヘッダーを書き換える；
 - `insecure`（**TLS verify をスキップ**、デフォルト `false`） – アップストリームの TLS 証明書を検証しない。
- **file（serve directory）**：ディレクトリからファイルを提供する。
 - `dir`（ディレクトリ、プレースホルダー `/var/www/html`）。
- **string（fixed body）**：固定 HTTP レスポンス。
 - `statusCode`（ステータスコード、デフォルト 0、範囲 0-599）；
 - `content`（**Body**） – レスポンスボディ；
 - `headers`（ヘッダー） – 名前 → 値 のマップ。

Masquerade を使用すると、Hysteria ベースの inbound がアクティブプローブに対して通常の HTTP/3 サーバーのように見えるため、隠密性が向上します。デフォルトでは無効です。

`masquerade` → `proxy` を使用した `hysteriaSettings` 例:

```
{
  "version": 2,
  "udpIdleTimeout": 60,
  "masquerade": {
    "type": "proxy",
    "url": "https://www.example.com",
    "rewriteHost": true,
    "insecure": false
  }
}
```

この設定では、プローブ時にリスナーが `https://www.example.com` からのレスポンスを返し、通常の HTTP/3 サイトに偽装します。

6.9. 付随パラメータ

ネットワークの選択に加えて、同じタブには特定のトランスポートに依存しない 2 つの共通ブロックがあります（詳細はそれぞれのセクションを参照）：

- **External Proxy** (`externalProxy`) – パネル自体のアドレスの代わりにサブスクリプションリンクに代入される外部アドレス/ポートのリスト。
- **Sockopt** (`sockopt`) – 低レベルのソケットオプション（TCP Fast Open、マーク、ドメイン戦略、透過プロキシなど）。

Real client IP（CDN/リレー背後での実際の IP 検出）

inbound が仲介者（Cloudflare などの CDN、L4 トンネル/リレー、または別のパネル）の背後にある場合、Xray は実際の訪問者ではなく仲介者のアドレスを認識します。このアドレスはオンラインクライアントリストに表示され、クライアントの IP 制限にも使用されるため、プロキシ背後では両方が機能しなくなります。実際の IP を復元するために、inbound フォームの **Sockopt** セクションには、`acceptProxyProtocol` と `trustedXForwardedFor` の設定を組み合わせた **Real client IP** プリセット選択があります：

プリセット	動作	適用場面
Off / direct	両フィールドをクリアします。	inbound にクライアントが直接アクセスする場合
Cloudflare CDN	<code>sockopt.trustedXForwardedFor = ["CF-Connecting-IP"]</code> を設定します。	Cloudflare CDN（オレンジ色のクラウド）背後の WebSocket / HTTPUpgrade / XHTTP / gRPC
L4 relay / Spectrum (PROXY)	<code>acceptProxyProtocol = true</code> を有効にします。	inbound 前の L4 トンネル/リレーまたは Cloudflare Spectrum

プリセットは相互排他的です。1 つを選択すると他のフィールドがクリアされるため、古い `trustedXForwardedFor` が PROXY プロトコルで復元された IP を上書きすることはありません。プリセットの下には「生の」**Proxy Protocol** スイッチと **Trusted X-Forwarded-For** リストが引き続き表示されます。プリセットはそれらを自動で埋めるだけで、必要に応じて手動で変更できます。選択したプリセットが現在のトランスポートでサポートされていない場合（例：mKCP での PROXY プロトコル）、フォームに警告が表示されます。これらのフィールドはサーバー側のみに適用され、サブスクリプションでクライアントに送信されることはありません。

どちらか一方のみ使用してください。 `acceptProxyProtocol` は L4 PROXY プロトコルヘッダーから実際の IP を読み取り、`trustedXForwardedFor` は HTTP リクエストヘッダーから読み取ります。仲介者のチェーンが要求する場合にのみ、手動で組み合わせて使用してください。

- **FinalMask** (`finalmask`) – mKCP のレガシー難読化を含むトランスポート層の共通難読化メカニズム。各ネットワークのサブフォーム内の個別の「seed」 / 「header type」フィールドを置き換えるものです。

7. 接続セキュリティ: TLS、XTLS、REALITY

transport ストリームを通じた通信をサポートする各 inbound (VMess、VLESS、Trojan、Shadowsocks、Hysteria) のエディターには、**「セキュリティ」**タブがあります。このタブでは、トランスポートチャネルの暗号化とカモフラージュの方法を設定します。ラジオボタンで切り替えられる 3 つのモードが使用できます:

モード	UI 上の表記	利用可能な場合
none	なし	常時利用可能 (TLS が必須の Hysteria を除く)
tls	TLS	tcp、ws、http、grpc、httpupgrade、xhttp ネットワーク上の VMess/VLESS/Trojan/Shadowsocks; Hysteria では常時
reality	Reality	tcp、http、grpc、xhttp ネットワーク上の VLESS/Trojan のみ

プロトコルが Hysteria の場合、**なし**ボタンは表示されません (TLS が必須のため)。**Reality** ボタンは、プロトコルとネットワークの組み合わせが有効な場合にのみ表示されます (上記の表を参照)。

モードを変更すると、パネルは `streamSettings` ブロックを完全に再構築します。前のモードの `tlsSettings` と `realitySettings` が削除され、選択したモードのデフォルト値が適用されます。特に **Reality** を選択した場合、パネルは即座に自動的に: 組み込みの人気ドメインリストからランダムな `target` と `serverNames` (SNI) のペアを設定し、ランダムな `shortIds` を生成し、サーバーに新しい X25519 キーペア (`privateKey/publicKey`) をリクエストします。

7.1. 違いの説明: TLS vs XTLS vs REALITY

- **TLS** – TLS 1.2/1.3 プロトコルによるクラシックなトランスポート暗号化。サーバーには有効な証明書 (独自ドメイン+チェーン) が必要です。トラフィックは通常の HTTPS として見えますが、積極的な検閲者に対しては、あなたのドメインへの特徴的な TLS ハンドシェイクが認識されます。SNI によるブロックや信頼された証明書がない場合、接続はブロックされるかエラーになります。
- **XTLS (Vision)** – これは「セキュリティ」リストの独立したモードではなく、TLS または Reality の上に重なる `flow` メカニズムです。inbound のクライアント側で **Flow** フィールドを `xtls-rprx-vision` (または `xtls-rprx-vision-udp443`) に設定することで有効になります。Vision は、`security = tls` または `security = reality` の `tcp` ネットワーク上の VLESS で利用可能です。また、VLESS 暗号化 (`vlessenc/ML-KEM`) が有効な `xhttp` トランスポート上の VLESS でも利用でき、この場合 **Flow** フィールドを `xtls-rprx-vision` に設定でき、値は `vless://` リンク (`flow=xtls-rprx-vision`) に正しく含まれます。Vision はハンドシェイク後にペイロードを直接渡すことで「二重暗号化」(TLS-in-TLS) を削減し、転送速度を向上させてカモフラージュを改善します。そのため、**VLESS + Reality + Flow xtls-rprx-vision** の組み合わせが現代の推奨設定とされています。

Vision flow の自動復元。 VLESS/XHTTP-inbound の暗号化 (ML-KEM、`decryption/encryption` フィールド) がクライアント追加後に有効になると、その inbound は flow 対応となります。この状況では、パネルは flow が必要なクライアントのうち **Flow** フィールドが空になっているものに対して、自動的に `flow = xtls-rprx-vision` を復元します。以前はこのシナリオで Vision が設定、共有リン

ク、サブスクリプションから黙って消えていました（特に中継 inbound で顕著でした）。手動での操作は不要です：修正は inbound の保存時およびパネル更新時に一度自動的に適用されます。動作は保守的です。パネルは flow を作り出したり、クライアントが明示的に設定した値を上書きしたりしません。

- **REALITY** – 独自の証明書なしでカモフラージュを行うメカニズム。サーバーは実在するサードパーティサイト（`target/serverNames`）の TLS ハンドシェイクを「借用」するため、観察者からはそのサイトへのアクセスと区別できず、証明書は一切不要です。認証は X25519 キーペアと `shortIds` セットに基づきます。REALITY は SNI に実際の外部ドメインを指定するため、能動的プロービング（`active probing`）や SNI によるブロックに強いです。代償として設定要件が厳しくなります（ポートを含む正確な `target`、クライアントとのキー同期）。

モード選択の簡単なルール：

- 独自ドメインと有効な証明書があり、シンプルな HTTPSに見せたい場合 → **TLS**（可能であれば Vision を使用）；
- ドメイン・証明書がない場合、または DPI に対する最大限の隠蔽が必要な場合 → **REALITY**（VLESS/TCP には Vision を使用）。

7.2. 「なし」モード（`none`）

トランスポートは TLS ラッパーなしで送信されます：`streamSettings` から `tlsSettings` と `realitySettings` が除外されます。このモードには追加フィールドがありません。以下の場合に適しています：

- inbound が `127.0.0.1` のみをリッスンし、fallback ターゲットとして機能する場合（パネルのルールでは、fallback の子 inbound は `security=none` で `127.0.0.1` をリッスンする必要があります）；
- 暗号化・カモフラージュを外部レイヤー（パネル前の Nginx リバースプロキシなど）が担当する場合；
- 内部ネットワークで独自暗号化を持つプロトコル（Shadowsocks）を使用する場合。

外部からアクセス可能な inbound には「なし」モードは推奨されません。

例： `tcp` ネットワーク上の TLS 向け `streamSettings` ブロック（VLESS/Trojan/VMess）。**TLS** モードを選択し、SNI と証明書のパスを入力した後の結果：

```

"streamSettings": {
  "network": "tcp",
  "security": "tls",
  "tlsSettings": {
    "serverName": "vpn.example.com",
    "minVersion": "1.2",
    "maxVersion": "1.3",
    "alpn": ["h2", "http/1.1"],
    "settings": { "fingerprint": "chrome" },
    "certificates": [
      {
        "certificateFile": "/root/cert/vpn.example.com.crt",
        "keyFile": "/root/cert/vpn.example.com.key",
        "ocspStapling": 3600,
        "usage": "encipherment"
      }
    ]
  }
}
}

```

7.3. TLS モード

`tlsSettings` ブロックのフィールド。デフォルト値はパネルのスキーマから取得されます。

基本パラメーター

フィールド (ラベル)	デフォルト値	説明
SNI (<code>serverName</code>)	<code>""</code> (空)	Server Name Indication – TLS ハンドシェイクで提示されるドメイン名。証明書のドメインと一致する必要があります。英語のプレースホルダーヒント: 「Server Name Indication」。
Cipher Suites (<code>cipherSuites</code>)	<code>""</code> → 自動	許可された暗号スイートのリスト。デフォルトは空 – 選択は Xray/Go に委ねられます (自動オプション)。暗号を明示的に制限する必要がある場合のみ変更します。
最小/最大バージョン (<code>minMaxVersion</code>)	最小 = <code>1.2</code> 、最大 = <code>1.3</code>	TLS の最小・最大バージョン。利用可能な値: <code>1.0</code> 、 <code>1.1</code> 、 <code>1.2</code> 、 <code>1.3</code> 。 <code>1.2</code> – <code>1.3</code> のままにすることを推奨; 最小を 1.0/1.1 に下げるとは非推奨 (古い、安全でないバージョン)。
uTLS (<code>settings.fingerprint</code>)	<code>chrome</code> (フォームでは None = <code>""</code> が選択可能)	クライアントハローの偽装 TLS フィンガープリント (uTLS fingerprint)。ハンドシェイクが人気ブラウザのように見えるようにします。以下のリストを参照。TLS では先頭に偽装を無効化する None (<code>""</code>) があります。
ALPN (<code>alpn</code>)	<code>["h2", "http/1.1"]</code>	TLS でネゴシエートするアプリケーション層プロトコルのリスト (複数選択)。利用可能な値: <code>h3</code> 、 <code>h2</code> 、 <code>http/1.1</code> 。デフォルトは <code>h2</code> と <code>http/1.1</code> 。

uTLS fingerprint の利用可能な値 (TLS と REALITY で共通) : `chrome`、`firefox`、`safari`、`ios`、`android`、`edge`、`360`、`qq`、`random`、`randomized`、`randomizednoalpn`、`unsafe`。TLS フォームには追加で空の **None** オプションがあります (フィンガープリント偽装を適用しない)。

Cipher Suites の利用可能な値 (TLS 1.3 および ECDHE スイート) : `TLS_AES_128_GCM_SHA256`、`TLS_AES_256_GCM_SHA384`、`TLS_CHACHA20_POLY1305_SHA256`、`TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA`、`TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA`、`TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA`、`TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA`、`TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256`、`TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384`、`TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256`、`TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384`、`TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256`、`TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256`。

TLS スイッチ

スイッチ	デフォルト	説明
不明な SNI を拒否 (<code>rejectUnknownSni</code>)	オフ (<code>false</code>)	有効にすると、クライアントが提示した SNI が証明書の名前と一致しない場合にサーバーが接続を切断します。隠蔽性が向上しますが (サーバーが「他者」のリクエストに応答しない)、クライアントの SNI が正確に一致する必要があります。
System Root を無効化 (<code>disableSystemRoot</code>)	オフ (<code>false</code>)	信頼されたルート証明書のシステムストアの使用を無効にします。
セッション再開 (<code>enableSessionResumption</code>)	オフ (<code>false</code>)	TLS セッション再開 (session resumption / session tickets) を有効にします。

TLS の追加パラメーター (vcn、曲線、キーログ、ECH Sockopt)

基本の TLS 設定の下に追加フィールドがあります。

フィールド (ラベル)	デフォルト	説明
Verify Peer Cert By Name (<code>settings.verifyPeerCertByName</code>)	""	クライアントが SNI の代わりにサーバー証明書を検証するための名前 (カンマ区切り)。これは 2026-06-01 以降の Xray から削除された <code>allowInsecure</code> フィールドの現代的な代替です。パネル専用の値: xray サーバー設定には書き込まれませんが、招待リンクとサブスクリプションに (<code>vcn=...</code>) として含まれ、クライアントが適用できるようにします。プレースホルダー: <code>example.com</code> 。
Curve Preferences (<code>curvePreferences</code>)	""	TLS 鍵交換曲線の制限と優先順位 (例: <code>X25519MLKEM768</code> 、 <code>X25519</code>)。空の場合は Xray-core のデフォルト値が使用されます。

フィールド (ラベル)	デフォルト	説明
Master Key Log (masterKeyLog)	""	SSLKEYLOGFILE 形式で TLS マスターキーを記録するパス (Wireshark でのデバッグ用トラフィック復号化)。プレースホルダー: <code>/path/to/sslkeylog.txt</code> 。本番環境では空のままにしてくださいー このファイルで全トラフィックが復号化されます。
ECH Sockopt (echSockopt)	オフ	Xray が ECH config list をリクエストする接続のソケットパラメーターのスイッチ。有効にすると利用可能: Dialer Proxy (dialerProxy - タグで指定した outbound 経由でリクエストを送信)、 Domain Strategy (domainStrategy)、 TCP Fast Open (tcpFastOpen)、 Multipath TCP (tcpMptcp)。必要がなければオフのままにします。

フィールド `verifyPeerCertByName`、`curvePreferences`、`masterKeyLog`、`echSockopt` は `tlsSettings` のトップレベルに配置され、設定保存時のパネルフィールドの「トリミング」後も残ります。

証明書

SSL 証明書セクション (UI の見出し「SSL-сертификат」) はリストとして設定されます: + ボタンで新しい証明書エントリを追加し、- 削除ボタンで削除します (削除ボタンはエントリが 2 つ以上の場合のみ利用可能)。デフォルトでは TLS を有効にすると空のエントリが 1 つ作成されます。

各エントリには入力モードスイッチ (`useFile`) があります:

- **証明書パス** (`useFile = true`、デフォルト) - サーバー上のファイルのパスを指定します:
 - **公開鍵** (`certificateFile`) - 証明書ファイルのパス (`.crt` / `.pem`);
 - **秘密鍵** (`keyFile`) - 秘密鍵ファイルのパス (`.key`)。
- **証明書コンテンツ** (`useFile = false`) - コンテンツをフィールドに直接貼り付けます (複数行テキストエリア):
 - **公開鍵** (`certificate`) - 証明書の PEM コンテンツ;
 - **秘密鍵** (`key`) - 鍵の PEM コンテンツ。

「証明書パス」モードのフィールドの下に 2 つのボタンがあります:

- **パネルの証明書を設定** - パネル自身の SSL 証明書のパスをフィールドに入力します。中央パネル上の inbound の場合はそのパネルの証明書 (`POST /panel/setting/all` → `webCertFile` / `webKeyFile`) が、ノードに割り当てられた inbound の場合はそのノード自身の証明書 (`GET /panel/api/nodes/webCert/{nodeId}`) が使用されます。これは中央パネルのパスがノードに存在しないためです。証明書が設定されていない場合、警告が表示されます: 「パネルに証明書が設定されていません。まず設定で証明書を設定してください。」 (パネル自身の証明書は「設定 → セキュリティ」セクションで設定します)。
- **クリア** - 両方のパスを消去します。

各証明書エントリの追加フィールド:

フィールド	デフォルト	説明
OCSP Stapling (ocspStapling)	0 (オフ)	OCSP ステープリングの更新間隔 (秒) (最小 0)。新しい inbound ではデフォルトでオフ (0) : これにより OCSP レスポンダーがない証明書 (例: OCSP を廃止した Let's Encrypt) での xray ログのエラーが除去されます。ステープリングをサポートする証明書にのみ有効にしてください。
一回限りのロード (oneTimeLoading)	オフ (false)	有効にすると、証明書は起動時に一度だけディスクから読み取られ、ファイルが変更されても再読み取りされません。
使用オプション (usage)	encipherment	証明書の用途。利用可能な値: encipherment (暗号化 – 通常のサーバー証明書)、 verify (検証)、 issue (発行 – サーバー自身が証明書に署名・発行)。
Build Chain (buildChain)	オフ (false)	usage = issue の場合のみ表示されます。証明書チェーンを構築します。

inbound エディターには自己署名証明書の個別ボタンはありません: パネルは inbound 用の自己署名証明書をその場で生成しません。証明書はパスまたはコンテンツで指定するか、「パネルの証明書を設定」ボタンでパネルの設定から取得します。パネル自身の SSL 証明書の発行・取得 (ファイルのアップロードやドメインへの紐付けを含む) は設定 → セキュリティセクションで行います。ここには inbound 用の ACME/Let's Encrypt エンドポイントはあります。

ECH と証明書ピンニング (TLS の拡張フィールド)

フィールド	デフォルト	説明
ECH key (echServerKeys)	""	Encrypted Client Hello のサーバーキー。
ECH config (settings.echConfigList)	""	ECH config list (クライアント側、リンクに含まれる)。
ピアの証明書の SHA-256 (settings.pinnedPeerCertSha256)	[]	ピアの証明書の SHA-256 ハッシュ (16 進文字列、カンマ区切り)。ヒント: 「ピアの証明書の SHA-256 ハッシュ (16 進文字列形式、例: e8e2d3...)、カンマ区切り。パネル専用 – xray サーバー設定には書き込まれませんが、クライアントが証明書をピンできるように招待リンクに含まれます。」

ボタン: **ピアの証明書の SHA-256** フィールドの隣には 2 つの自動入力ボタンがあります:

- **Fill from this inbound's certificate** (シールドアイコン) – この inbound 自身の証明書の SHA-256 ハッシュを入力します (**getCertHash** エンドポイント経由でローカルに取得) 。
- **Fetch the hash by pinging the SNI (xray tls ping)** (ダウンロードアイコン) – 指定した SNI への TLS 接続を実行してサーバーの証明書のライブハッシュを取得します (サーバーで **getRemoteCertHash** を呼び出します)。**SNI** フィールド (**serverName**) が入力されている必要があります – 入力されていない場合、ヒント「*Set the SNI (serverName) first to ping the remote certificate.*」が表示されます。

取得したハッシュはフィールドに（カンマ区切りで）追加され、クライアントが証明書をピンできるように招待リンクに含まれます。

- **新しい ECH 証明書を取得** – 現在の SNI に対する新しい ECH ペアをサーバーにリクエストします（`POST /panel/api/server/getNewEchCert`、サーバーで `xray tls ech --serverName <SNI>` を実行）；**ECH key** と **ECH config** フィールドに入力します。
- **クリア** – 両方の ECH フィールドをクリアします。

7.4. REALITY モード

`realitySettings` ブロックのフィールド。REALITY は SSL 証明書を使用しません。代わりに外部ドメインからの TLS ハンドシェイクを借用し、X25519 キーペアを使用します。

カモフラージュパラメーター

フィールド (ラベル)	デフォルト値	説明
Show (<code>show</code>)	オフ (<code>false</code>)	Xray ログへの REALITY デバッグ出力。通常はオフのままにします。
Xver (<code>xver</code>)	<code>0</code>	バックエンドに渡す PROXY プロトコルのバージョン (<code>0</code> – オフ)。最小値 <code>0</code> 。
uTLS (<code>settings.fingerprint</code>)	<code>chrome</code>	偽装 TLS フィンガープリント (TLS と同じリストだが、空の <code>None</code> オプションはない)。
ターゲット (<code>target</code>)	<code>""</code> (3.4.2 から REALITY 有効化時も空のまま)	必須フィールド 。REALITY が TLS ハンドシェイクを借用する実際のドメイン。ヒント: 「必須。ポートを含める必要があります (例: <code>example.com:443</code>)。ポートがないと <code>Xray-core</code> が起動しません。」パネルのバリデーションはポートの有無と正確性を確認します; そうでない場合は「REALITY ターゲットは必須です」/ 「REALITY ターゲットにはポートが必要です...」/ 「REALITY ターゲットのポートが無効です」などのエラーが表示されます。フィールドの隣には「 スキャン 」(現在のターゲットを「ライブで」チェック)と「 ターゲットを検索 」(REALITY ターゲットスキャナーを開く)ボタンがあります; 下記参照。
SNI (<code>serverNames</code>)	<code>[]</code> (ターゲットと一緒に設定される)	許可された SNI のリスト (タグによる複数入力)。ターゲットのドメインと一致する必要があります。ターゲットのスキャンに成功すると、SNI はその証明書から入力されます。
最大時刻差 (ミリ秒) (<code>maxTimediff</code>)	<code>0</code>	クライアントとサーバーのクロック差の最大許容値 (ミリ秒) (<code>0</code> – 制限なし)。最小値 <code>0</code> 。
最小クライアントバージョン (<code>minClientVer</code>)	<code>""</code>	Xray クライアントの最小バージョン (プレースホルダーは 3.5.0 から <code>26.3.27</code>)。空の場合は制限なし。
最大クライアントバージョン (<code>maxClientVer</code>)	<code>""</code>	Xray クライアントの最大バージョン。空の場合は制限なし。

フィールド (ラベル)	デフォルト値	説明
Short IDs (shortIds)	☐ (有効化時に生成される)	クライアントを区別する短い識別子 (16 進数) のリスト。タグによる複数入力; 更新ボタンはランダムなセットを生成します。
SpiderX (settings.spiderX)	/	外部サイトへのアクセスを模倣する際に使用される「スパイダー」パス (REALITY のクライアント側)。招待リンクに含まれます。

バージョン3.4.2 から、REALITY を有効にしても **ターゲット** (target) と **SNI** (serverNames) は自動的に入力されなくなりました – 両フィールドは空のまま、ターゲットはライブスキャナーで選定します (下記参照)。TLS 1.3 と HTTP/2 をサポートし、あなた自身のサーバーの背後にない、安定した大手のサードパーティ HTTPS サイトを選んでください。

REALITY ターゲットの検索 (ライブスキャナー)

バージョン3.4.2 から、従来の「ランダムなターゲット」ボタン (静的な組み込みリスト) は、**ターゲット** フィールドの隣にある 2 つのアクションに置き換えられました:

- 「**スキャン**」 – 現在のターゲットを「ライブで」チェックします: TLS 接続を確立し、ターゲットが適切であればその証明書から SNI を入力します。トースト: 「Цель пригодна – заполнены цель и SNI.» / 「Цель доступна, но не пригодна для REALITY.» / 「Не удалось просканировать цель REALITY.»。
- 「**ターゲットを検索**」 – 「**REALITY ターゲットスキャナー**」 ウィンドウを開きます: ドメイン、**IP または CIDR 範囲** (この場合、パネルは生きているホストの証明書からターゲットを検索します) を指定するか、フィールドを空のままにする (組み込みの候補がチェックされます) ことができます。結果はランク付けされます; 列は「ステータス」/「適合」、「鍵交換」、「証明書」、TLS、ALPN、「遅延」です。「**使用**」ボタンで、選択した行が inbound のフィールドに代入されます。

ターゲットは、サーバーが **TLS 1.3**、**HTTP/2 (ALPN h2)**、**X25519/X25519MLKEM768** の鍵交換をネゴシエートし、**信頼された** (ワイルドカードでない) 証明書を提示する場合に **適切** と見なされます。スキャンはパネルのサーバー側で実行されます (`POST /panel/api/server/scanRealityTarget` および `.../scanRealityTargets`)。

例: **tcp** ネットワーク上の **REALITY** 向け **streamSettings** ブロック (VLESS)。証明書は不要 – 代わりに借用したドメインと X25519 キーペアを使用します:


```

"streamSettings": {
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "show": false,
    "xver": 0,
    "dest": "www.nvidia.com:443",
    "serverNames": ["www.nvidia.com"],
    "privateKey": "YOUR_X25519_PRIVATE_KEY",
    "shortIds": [ "", "6ba85179e30d4fc2" ],
    "settings": {
      "publicKey": "YOUR_X25519_PUBLIC_KEY",
      "fingerprint": "chrome",
      "spiderX": "/"
    }
  }
}
}

```

ここで、パネルの**ターゲット**（target）フィールドは Xray の最終設定で `dest` に対応します。REALITY-inbound が `dest` キーに destination を持つ形で作成されていた場合（古いバージョンのパネル、API、または外部ツール経由）、`target` が空の場合にパネルは解析時に `dest` → `target` と正規化します。そのため、このような inbound は正しく読み込まれ、**ターゲット**フィールドは空になりません。また再保存しても動作中の REALITY が壊れません。

REALITY キー（X25519）

フィールド	デフォルト	説明
公開鍵 (settings.publicKey)	""	X25519 公開鍵（クライアントが設定・リンクに使用します）。
秘密鍵 (privateKey)	""	X25519 秘密鍵（サーバーのみに保存されます）。

キーの下ボタン：

- **新しい証明書を取得** – サーバーに新しいキーペアをリクエストします（GET /panel/api/server/getNewX25519Cert；サーバーで `xray x25519` を実行）。**秘密鍵**と**公開鍵**に入力します。このペアは REALITY モードを初めて有効にした際にも自動的に生成されます。

例：API 経由で X25519 キーペアを取得する方法（フォーム外、例えばスクリプト用）。リクエストは秘密鍵と公開鍵を返します：

```

curl -s -b cookie.txt https://your-panel:2053/panel/api/server/getNewX25519Cert
# レスポンス:
# {"success":true,"obj":{"privateKey":"...","publicKey":"..."}}

```

`cookie.txt` – POST /login でログイン後に取得したセッション Cookie ファイル。

- **クリア** – 両方のキーをクリアします。

ポスト量子署名 ML-DSA-65 (mldsa65)

REALITY の追加（オプション）ポスト量子認証レイヤー：

フィールド	デフォルト	説明
mldsa65 Seed (mldsa65Seed)	""	ML-DSA-65 キーのサーバーシード。
mldsa65 Verify (settings.mldsa65Verify)	""	検証値（クライアント側、リンクに含まれる）。

ボタン：

- **新しい Seed を取得** – 新しいペアをリクエストします（GET /panel/api/server/getNewmldsa65 ; サーバーで xray mldsa65 を実行）。**mldsa65 Seed** と **mldsa65 Verify** に入力します。
- **クリア** – 両方のフィールドをクリアします。

fallback 速度制限と REALITY キーログ

REALITY の設定には fallback トラフィックの速度制限があります – これにより能動的なプローブがサーバーを借用ドメインへの無料チャネルとして利用するのを防ぎます。設定は 2 つの方向 – **Limit Fallback Upload** と **Limit Fallback Download** (limitFallbackUpload / limitFallbackDownload) – に対して個別に設定され、各方向に同じフィールドセットがあります：

フィールド (ラベル)	デフォルト	説明
After Bytes (afterBytes)	0	制限を開始する前にフル速度で通過させるバイト数。0 – 最初のバイトから制限します。
Bytes Per Sec (bytesPerSec)	0	しきい値を超えた後の fallback トラフィックの速度上限（バイト/秒）。0 – 制限なし（この方向を無効化）。
Burst Bytes Per Sec (burstBytesPerSec)	0	定常速度を超えた一時的なバーストのための余裕（token-bucket のサイズ）。Bytes Per Sec より小さい場合はその値まで引き上げられます。

同じ場所に **Master Key Log** (masterKeyLog) フィールドもあります – Wireshark でのデバッグ用 SSLKEYLOGFILE 形式で TLS マスターキーを記録するパスです。本番環境では空のままにしてください。

7.5. 設定の実践的な推奨事項

1. **VLESS + Reality (推奨)** : tcp ネットワーク上に VLESS-inbound を作成し、「セキュリティ」タブで **Reality** を選択してください – パネルが自動的にランダムな target /SNI、shortIds を設定し、X25519 キーを生成します。必要に応じて「新しい証明書を取得」を押して独自のキーペアを取得してください。VLESS クライアントには **Flow = xtls-rprx-vision** (XTLS Vision) を有効にしてください – これにより最大のパフォーマンスと隠蔽性が得られます。

例：VLESS + Reality + Vision の最終クライアントリンク。 このような inbound に対してパネルが生成する招待リンクの形式（キー/ID の値はイラスト用）：

```
vless://uuid-клиента@1.2.3.4:443?  
type=tcp&security=reality&pbk=ПУБЛИЧНЫЙ_КЛЮЧ&fp=chrome&sni=www.nvidia.com&sid=6ba85179e3  
0d4fc2&spx=%2F&flow=xtls-rprx-vision#my-reality
```

ここで pbk は X25519 公開鍵、sni はターゲットから借用したドメイン、sid は **Short IDs** のひとつ、flow=xtls-rprx-vision は有効化された XTLS Vision です。2. **独自ドメインでの TLS**: TLS を選択し、**SNI** にドメイン名を入力して、証明書を追加します（ファイルパスまたはコンテンツ）。または「設定 → セキュリティ」でドメインと証明書がすでに設定されている場合は「パネルの証明書を設定」を押してください。最小/最大バージョン = 1.2 - 1.3 と uTLS = chrome のままにして、通常のブラウザのように見せます。3. 外部に公開されている inbound に **なし** モードを使用しないでください – ローカル fallback ターゲット（127.0.0.1）に対してのみ、または外部プロキシが TLS を担当する場合にのみ使用してください。4. インターフェースのアドバイス: ほとんどの詳細フィールドには「デフォルト設定のままにすることを推奨します」というヒントがあります – 影響を理解している場合のみ変更してください。

8. クライアント

クライアントとはVPNユーザーアカウントのことです。UUID またはパスワードからなる認証情報のセットであり、1つまたは複数の inbound に紐付けられ、独自のトラフィッククォータ・有効期限・同時接続数制限を持ちます。このフォークではクライアントは独立したエンティティ（テーブル `clients`）として扱われます。同一のクライアントを複数の inbound に同時に紐付けることができ、共通の UUID/パスワードおよび共通のトラフィックカウンターが維持されます。クライアント セクションでは、inbound に関係なくパネルのすべてのアカウントが、検索・フィルター・並べ替え・一括操作とともに表示されます。

8.1. クライアントのフィールド

以下では、クライアントを追加 / クライアントを編集 エディターの各フィールドを説明します。

クライアントフォームは2つのタブに分かれています。基本 タブ（email、inbound への紐付け、制限、有効期限、グループ、コメント、リバースタグ）と 認証情報 タブ（UUID/パスワード/auth、Flow、VMess Security）です。フィールドラベルではクォータが **トラフィック上限 (GB)** として、期間が **期間 (日)** および **自動更新 (日)** として表示されます。トラフィック上限 (GB) および IP 上限 フィールドには、0 が「無制限」を意味することを説明するツールチップがあります。既存クライアントの編集時はランダム email 生成ボタンが非表示になり、IP ログボタンが IP 上限 フィールドの横に直接配置され、記録済みアドレス数が表示されます。

フィールド	JSON キー	デフォルト	説明
Email	<code>email</code>	–（必須）	クライアントの一意識別子
UUID	<code>id</code>	自動生成	VMess/VLESS 用識別子
パスワード	<code>password</code>	自動生成	Trojan/Shadowsocks 用パスワード
認証	<code>auth</code>	自動生成	Hysteria 用パスワード
Flow	<code>flow</code>	空	Flow control (XTLS) 、VLESS のみ
VMess Security	<code>security</code>	<code>auto</code>	VMess 暗号化方式
IP 上限	<code>limitIp</code>	0（無制限）	同時接続 IP の最大数
合計送受信量 (GB)	<code>totalGB</code>	0（無制限）	トラフィッククォータ
有効期限	<code>expiryTime</code>	0（無期限）	期限切れ日時
自動更新	<code>reset</code>	0（無効）	トラフィックリセット周期（日）
Telegram ユーザー ID	<code>tgId</code>	0（なし）	数値 Telegram ID
サブスクリプション ID	<code>subId</code>	自動生成	サブスクリプション識別子
グループ	<code>group</code>	空	論理グループラベル
コメント	<code>comment</code>	空	任意のメモ
有効	<code>enable</code>	<code>true</code>	アカウントが有効かどうか

Email (識別子)

Email フィールドはクライアントの主要かつ必須の識別子です。名称に関わらず、必ずしもメールアドレスである必要はありません。任意のテキストラベル（ユーザー名、番号など）を使用できます。値はパネル内で一意である必要があり、既存の email でクライアントを作成しようとするすると拒否されます（email already in use）。ただし、subId も一致する場合は例外で、同一クライアントへの紐付けとして扱われます。

Email は空にできません（client email is required）。また、スペース、/、\、制御文字を含めることができません（「Email にはスペース、/、\ または制御文字を含めることができません」）。Email はトラフィック集計・IP ログ・オンラインリスト・操作名に使用されるため、後から変更することは推奨されません。

UUID / パスワード / 認証 (認証情報)

使用する認証情報フィールドは、クライアントが紐付けられる inbound のプロトコルによって異なります。フィールドを空にすると値が自動的に設定されます。

- **UUID**（フィールド id）－ プロトコル **VMess** および **VLESS** 用。未設定の場合、ランダムな UUID v4 が生成されます。
- **パスワード**（フィールド password）－ **Trojan** および **Shadowsocks** 用。Trojan のデフォルトはハイフンなし UUID。Shadowsocks では inbound の暗号化方式に応じて必要な長さの Base64 キーが生成されます。2022-blake3-aes-128-gcm では 16 バイト、2022-blake3-aes-256-gcm および 2022-blake3-chacha20-poly1305 では 32 バイト、その他の方式ではハイフンなし UUID。手動で入力したキーが 2022-blake3 方式に適合しない場合は生成されたものに置き換えられます。
- **認証**（フィールド auth）－ **Hysteria** 用パスワード。デフォルトはハイフンなし UUID。

1つのクライアントが異なるプロトコルの複数の inbound に紐付けられる場合、クライアントレコードに UUID・パスワード・auth がすべて同時に存在することがあります。各プロトコルはそれぞれ対応するフィールドを使用します。

例：クライアントの認証情報が各 inbound の settings にどのように現れるか。 同一クライアントが VLESS-inbound では id で、Trojan では password で、Shadowsocks では password（Base64 キー）で識別されます。

```
// VLESS-inbound の settings.clients のフラグメント
{ "id": "b831381d-6324-4d53-ad4f-8cda48b30811", "email": "user-a", "flow": "xtls-rprx-vision" }

// 同じクライアント (Trojan-inbound)
{ "password": "b831381d63244d53ad4f8cda48b30811", "email": "user-a" }

// 同じクライアント (Shadowsocks-inbound、方式: 2022-blake3-aes-256-gcm)
{ "password": "GPY0aA3f7C00az53eaQ8eqMfRDjmbL0h7v1u3+Z+pHk=", "email": "user-a" }
```

Flow

Flow（フィールド `flow`）は XTLS のフロー制御です。**VLESS のみ**、かつ inbound が XTLS Vision 用に設定されている場合（security が `tls` または `reality` の TCP トランスポート上の VLESS）にのみ適用されます。有効な値は `xtls-rprx-vision`（および歴史的な `xtls-rprx-vision-udp443`）で、空値は flow なしを意味します。

inbound が XTLS-flow をサポートしていない場合、設定された flow はクライアント保存時に黙ってリセットされます。複数の inbound に紐付けられた同一クライアントについて、flow は許可される inbound にのみ適用されます。変更するのは VLESS-Vision を意図的に使用している場合のみにしてください。

VMess Security

VMess Security（フィールド `security`）は VMess のペイロード暗号化方式です。デフォルト値は `auto`（Xray が暗号方式を自動選択）です。有効な値は VMess の標準値である `auto`、`aes-128-gcm`、`chacha20-poly1305`、`none`、`zero` です。他のプロトコルではこのフィールドは使用されません。

IP 上限（同時接続数）

IP 上限（フィールド `limitIp`）は、クライアントが同時に接続できる異なる IP アドレスの最大数です。デフォルト値は `0`（制限なし）です。正の値を設定すると、パネルはクライアントのアクティブ IP を追跡し、上限を超えた場合にバックグラウンドタスクがアカウントを無効化します。（3.3.1 以降、IP のカウントは Xray コアのオンライン統計 API を通じて行われ、アクセスログは不要です。古いコアバージョンではパネルがアクセスログの読み取りにフォールバックするため、その場合はアクセスログを有効にする必要があります。）1つのサブスクリプションを多数のデバイスで共有することを禁止するために使用します。例： `2` で2台のデバイスを許可。

IP 上限は **Fail2ban** によって適用されるため、**IP 上限** フィールドは Fail2ban がインストールされ動作している場合のみ有効です（パネルは `GET /panel/api/server/fail2banStatus` でステータスを確認します）。Fail2ban がインストールされていない場合、クライアントエディター（および一括追加フォーム）のフィールドが無効化され、ホバー時に x-ui bash メニューから Fail2ban をインストールするよう促すツールチップが表示されます（「Fail2ban is not installed, so the IP limit cannot be enforced. Install Fail2ban from the x-ui bash menu to enable this option.」）。Windows ではツールチップに Fail2ban が利用不可であることが表示され（「Fail2ban is not available on Windows, so the IP limit cannot be enforced.」）、サーバーで機能が無効化されている場合は「The IP limit feature is disabled on this server.」と表示されます。Fail2ban なしのサーバーでパネルを更新すると、実際には適用されていなかったため、クライアントの保存済み IP 上限がワンタイムマイグレーションによってゼロにリセットされます。

値の例。 `limitIp: 0` - 制限なし。 `limitIp: 1` - 同時に厳密に1台のデバイスのみ。 `limitIp: 3` - 最大3つの異なる IP。4番目のアクティブ IP が現れると、バックグラウンドタスクが **IP 上限をリセット** を実行するまでクライアントを無効化します（`enable = false`）。

関連する操作：**IP ログ** はクライアントの記録済み IP リストを表示します。各エントリには IP アドレスに加えて、最後のアクセス時刻と接続が記録されたノードのラベル（@ ノード名）が含まれます。マルチパネル構成では、どのノード経由でクライアントが接続したかを確認できます。**IP 上限をリセット** は蓄積さ

れた IP ログをクリアし、記録が自然に期限切れになるのを待たずにクライアントが再接続できるようにします。

合計送受信量 (GB) – トラフィッククォータ

合計送受信量 (GB) (フィールド `totalGB`) は合計トラフィッククォータ (送信 + 受信) です。デフォルト値 `0` は**無制限**を意味します。クォータに達すると (`up + down >= total`)、クライアントは**枯渇** (depleted) 状態とみなされ、無効化されます。UI では通常ギガバイト単位で入力され、データベースではバイト単位で保存されます。

クライアントリストのトラフィック列には使用量の色付きバーが表示されます。消費済みトラフィック量、上限ラベル (無制限の場合は ∞ 記号)、送信/受信の内訳と残量のホバーツールチップが含まれます。同じコンパクトなインジケーターがスマートフォンのクライアントカードにも表示されます。

有効期限 (Expiry)

有効期限 (フィールド `expiryTime`) はアカウントの期限切れ時刻を設定します。このフィールドには3つのモードがあります。

- **無期限** – `0`。クライアントは時間による期限切れが発生しません。
- **特定日時** – 正の Unix タイムスタンプ (ミリ秒)。期限に達すると (`expiryTime <= 現在時刻`) クライアントは期限切れ (expired) とみなされ無効化されます。UI では通常、日付の選択または日数 (期間、単位: 日) で設定します。
- **初回使用後に開始** – 期間をエンコードした**負**の値。クライアントがバイトを送信するまで、期限は負のまま (「遅延開始」) です。最初のトラフィック集計タイミングで、パネルが絶対日時に変換します: `現在時刻 + |期間|`。これにより、クライアントがいつ有効化されるかを事前に知らなくても「初回接続から30日間」として販売できます。変換は email ごとに1回実行され、紐付けられたすべての inbound が同じ期限を受け取ります。

期限のエンコード例。 固定日時 2026年3月1日 00:00 UTC → `expiryTime: 1772323200000` (ミリ秒の正のタイムスタンプ)。「初回接続から30日間」→ `expiryTime: -2592000000` (負の値、 $30 \times 24 \times 60 \times 60 \times 1000$)。最初のトラフィックバイトでパネルが `現在時刻 + 2592000000` に置き換えます。無期限 → `expiryTime: 0`。

自動更新 (クライアントのトラフィックリセット周期)

自動更新 (フィールド `reset`) は日数単位の自動更新/リセット周期です。ツールチップ: 「終了後の自動更新。(0 = 無効) (単位: 日)」。

- `0` – 自動更新**無効** (デフォルト値)。期限切れ後、クライアントは単に枯渇状態になります。
- `> 0` – バックグラウンドタスクが期限切れ時にトラフィックカウンターを**ゼロにリセット** (`up = down = 0`) し、新しい期限が未来になるまで必要に応じて複数の周期分だけ**有効期限**を `reset` 日延長し、必要に応じてクライアントを再び**有効化**します。これにより定期的なサブスクリプション (月次など) が実現します。自動更新はノードの inbound (`node_id IS NOT NULL`) には**適用されません**。

重要な注意: `reset > 0` のクライアントは一括削除操作における「枯渇」概念から除外されます。そのトラフィック/期限は自動更新によってリセットされることが想定されており、アカウントが削除候補にはなりません。

Telegram ユーザー ID

Telegram ユーザー ID (フィールド `tgId`) は、パネルの組み込み Telegram ボットへの紐付け (通知、セルフサービス統計表示) のための数値 Telegram ユーザー識別子です。ツールチップ: 「数値 Telegram ユーザー ID (0 = なし)」。デフォルト値 `0` は紐付けなしを意味します。このフィールドでのフィルタリング (あり / なし) が可能です。

サブスクリプション ID (`subId`)

サブスクリプション ID (フィールド `subId`) は、クライアントがサブスクリプション (subscription) に含まれる際の識別子です。同じ `subId` を持つすべてのクライアントが1つのサブスクリプションリンクで配信されます。作成時にフィールドを空にすると、パネルが自動的にランダムな `subId` (UUID) を生成します。値は別の email を持つクライアント間で一意である必要があります (`subId already in use`)、email と同じ文字制限に従います (「サブスクリプション ID にはスペース、'/'、'\' または制御文字を含めることができません」)。

`subId` がない場合、クライアントのサブスクリプションリンクは利用できません (「このクライアントには `subId` がないため、共有リンクは利用できません。」)。

Links タブ (外部リンクとサブスクリプション)

クライアントエディターには **基本** と **認証情報** タブに加え、3番目の **Links** タブがあります (ツールチップ: 「Add third-party share links and remote subscription URLs to include in this client's subscription.」)。このタブでは **Add External Link** ボタンでサードパーティの共有リンク (`vless://`、`vmess://`、`trojan://`、`ss://`、`hysteria2://`、`wireguard://`) を追加でき、**Add External Subscription** ボタンでリモートサブスクリプション URL (例: `https://provider.example/sub/...`) を追加できます。

これらはすべてこのクライアントのサブスクリプション出力 (raw、JSON、Clash 形式) にミックスされます。リンクはそのまま追加され、リモートサブスクリプションはパネルが定期的にダウンロード (キャッシュと短いタイムアウト付き) して設定を自分のものとマージします。これにより、1つのクライアントサブスクリプションリンクで自分のサーバーと外部設定を合わせて配信できます。

グループ

グループ (フィールド `group`) は関連クライアントをまとめるための論理ラベルです。ツールチップ: 「関連クライアントをグループ化するための論理ラベル (例: チーム、顧客、リージョン)。ツールバーでフィルタリング可能。」、プレースホルダー: 「例: customer-a」。フィールドは任意 (デフォルトは空) です。グループでリストをフィルタリングしたり一括操作を実行したりできます。クライアントからラベルを削除するには **グループ解除** アクションを使用します。

グループはクライアントエディター内で直接解除することもできます。**グループ** フィールドをクリアして保存すると、ラベルが正しく削除され、クライアントは以前のグループの下に表示されなくなります。

コメント

コメント（フィールド `comment`）は管理者向けの任意のテキストメモです（デフォルトは空）。内容は検索対象に含まれ、フィルタリング（コメントが **あり** / **なし**）が可能です。

有効

有効（フィールド `enable`）はアカウントのアクティビティフラグです。デフォルトは**有効**（`true`）で、作成時にフラグが渡されなくてもパネルが強制的に `true` に設定します。無効化されたクライアント（`enable = false`）は接続できず、サマリーでは**非アクティブ**（`deactive`）に分類されます。クォータを消費したクライアント、期限切れのクライアント、IP 上限を超えたクライアントはパネルが自動的に無効化します。

読み取り専用フィールド

クライアントカードには補助フィールドも表示されます。**作成日時**（`created_at`）と **更新日時**（`updated_at`）は作成・最終変更のタイムスタンプで、自動的に設定され編集はできません。**リバースタグ**（`reverse`）フィールドはシンプルな VLESS リバースプロキシ用のオプションのリバースタグです（「オプションのリバースタグ」）。

8.2. inbound への紐付け

各クライアントは少なくとも1つの inbound に紐付けられている必要があります。作成時に最低1つが必要です（`at least one inbound is required`）。エディターでは **紐付けられた着信** フィールドに **1つまたは複数の着信を選択** というツールチップが表示されます。

- **紐付け** – 選択した inbound にクライアントを追加します（同じ UUID/パスワードと共有トラフィック）。既存の紐付けは保持されます。
- **紐付け解除** – 選択した inbound からクライアントを削除します。クライアントレコード自体は保持されます（完全な削除には **削除** を使用してください）。クライアントが紐付けられていないペアは静かにスキップされます。

複数の inbound に紐付けられたクライアントを保存する際、特定のプロトコル/トランスポートと互換性のないフィールド（例：VLESS-Vision 以外での Flow）は、各 inbound に対して自動的に許容される値に調整されます。

inbound 選択リストの上部（クライアントフォーム、クライアント一括追加、一括紐付け/解除ウィンドウ）に **すべて選択** と **クリア** ボタンがあります。これらのリストでは各 inbound が備考（`remark`）でラベル付けされ、設定されていない場合は inbound のタグが使用されます。

8.3. クライアントの操作

個別クライアントに対して（**クライアント情報** カードまたは **アクション** コンテキストメニューから）以下の操作が利用可能です。

情報・QRコード・リンクの表示

- **クライアント情報** – すべてのフィールド、使用済み/残りトラフィック（残量）、有効期限、紐付けられた inbound を含むカード。3.5.0 から、グループが設定されている場合、カードにはクライアントの **グループ** も表示されます（「コメント」の上の、ラベル付きの「グループ」行）。

API 経由のクライアント取得（GET /panel/api/clients/get/:email）では、`client` と `inboundIds` フィールドに加えて `usedTraffic`（実際に消費されたトラフィック：送信 + 受信、ノードのデータを含む）も返されるため、消費量をクォータ `totalGB` と照合しやすくなっています。

- **QRコード** および **リンク** – クライアントアプリへのインポート用クライアント設定リンク。サポートされているプロトコルを持つすべての紐付け済み inbound から生成されます（GET /links/:email）。適切なリンクがない場合：「共有リンクがありません – 先にサポートされているプロトコルの着信にクライアントを紐付けてください。」
- **サブスクリプションリンク** – `subId` によるサブスクリプション URL（GET /subLinks/:subId）。クライアントに `subId` があり、**パネル設定** → **サブスクリプション** でサブスクリプションサービスが有効な場合のみ利用可能です（無効の場合は「サブスクリプションサービスが無効です。」）。**JSON サブスクリプション URL** も合わせて提供されます。

トラフィックをリセット

トラフィックをリセット（POST /resetTraffic/:email）は特定クライアントの送受信カウンター（`up`、`down`）をゼロにリセットします。クォータ（`totalGB`）と有効期限は**影響を受けません**。リセットされるのは使用済みの量のみです。トースト：「トラフィックをリセットしました」。クライアントが inbound に紐付けられていない場合：「先にこのクライアントを着信に紐付けてください。」

トラフィックをリセット ボタンはクライアント編集フォームの下部パネル（**キャンセル** / **保存** の横）からも利用可能です（リセット前に確認が求められます）。トラフィック枯渇によってクライアントが無効化されていた場合、リセット（単独でも一括でも）が自動的にクライアントを再度**有効化**し（`enable = true`）、この変更をノードに即座に配信します。マスターおよびノードでクライアントを手動で再有効化する必要はなくなりました。

IP 上限をリセット

クライアントの蓄積された IP ログをクリアします（POST /clearIps/:email）。同時接続数の上限超過による一時的なブロックを解除するためです。トースト：「ログがクリアされました」。

削除

削除（POST /del/:email）はクライアントを完全に削除します。確認ダイアログ：タイトル「クライアント {email} を削除しますか?」、テキスト「クライアントはすべての紐付け済み着信から削除され、トラフィックレコードが破棄されます。この操作は元に戻せません。」。削除するとクライアントがすべての inbound から削除され、トラフィックレコードが破棄されます。トースト：「クライアントを削除しました」。

8.4. 一括操作

クライアントリストで複数のレコードを選択できます（すべて選択、すべてクリア）。カウンター：「{count} 件選択中」。選択した項目に対して以下が利用可能です。

- **削除 ({count})** (POST /bulkDel) – グループ削除。確認：「{count} 件のクライアントを削除しますか?」、「選択した各クライアントはすべての紐付け済み着信から削除され、トラフィックレコードが破棄されます。この操作は元に戻せません。」。トースト：「{count} 件のクライアントを削除しました」、一部失敗の場合：「削除: {ok}、失敗: {failed}」。
- **編集 ({count}) / 調整** (POST /bulkAdjust) – 有効期限やクォータの一括変更。ダイアログ「{count} 件のクライアントを編集」にツールチップ「正の値は追加、負の値は減算します。無制限の有効期限またはトラフィックを持つクライアントは対応するフィールドでスキップされます。」。フィールド：**日数を追加**、**トラフィックを追加 (GB)**、**Set flow**。ロジック：
 - **有効期限**：無期限 (expiryTime == 0) のクライアントはスキップされます（「unlimited expiry」）。日付付きのクライアントは指定日数だけ期限が延長されます。「初回使用后」モード（負の期限）のクライアントは待機期間の長さが調整されます。残余期間を超える減算はスキップされます（「reduction exceeds remaining time/delay window」）。
 - **トラフィック**：無制限 (totalGB == 0) のクライアントはスキップされます（「unlimited traffic」）。それ以外はクォータが指定量だけ変更され、ゼロ以下にはなりません。
 - **Flow**： **Set flow** ドロップダウンリストで、選択したすべてのクライアントの XTLS flow を一括で設定またはリセットできます。デフォルトは **No change**（変更なし）。**Disable (clear flow)** で flow をリセットし、xtls-rprx-vision と xtls-rprx-vision-udp443 で対応する vision-flow を設定します。vision-flow の設定は flow をサポートする inbound にのみ適用され、対象外の inbound は変更されずスキップ済みとしてマークされます。flow のリセットは常に許可されます。
 - **自動有効化 (3.4.2 から)**： **枯渇のみが原因**で無効化されていたクライアント（期限切れまたはクォータ超過）は、延長によって制限内に戻る場合、自動的に再び有効化されます – ローカルおよびそのノードの両方で。手動で無効化されたクライアントや、まだ枯渇しているクライアントは無効のままです（enable フィールドが明示的に書き込まれるようになったため、inbound を持たないクライアントでも状態が保持されます）。
 - **日数・トラフィック・flow のいずれも指定されていない場合**：「適用前に日数、トラフィック、または flow を指定してください。」。トースト：「{count} 件を変更しました」/「変更: {ok}、スキップ: {skipped}」。

例：選択したクライアントを 30 日延長して 50 GB 追加。編集 ダイアログで **日数を追加** = 30、**トラフィックを追加 (GB)** = 50 を指定します。逆に1週間減らしてクォータを 10 GB 削減するには、負の値を入力します：**日数を追加** = -7、**トラフィックを追加 (GB)** = -10（無期限または無制限のクライアントは対応するフィールドでスキップされます）。

- **紐付け ({count}) / 紐付け解除 ({count})** (POST /bulkAttach / bulkDetach) – 選択したクライアントを選択した inbound に一括で紐付け/解除します。対象はマルチユーザー inbound のみです。紐付け解除の結果：「{detached} 件を解除、{skipped} 件をスキップしました。」3.5.0 から、クライアントフォームの inbound セレクターは **無効化された inbound を非表示** にし（クライアントがすでに紐付けられているものを除く）、「紐付け解除」ウィンドウでの同一クライアントの重複エントリの蓄積が解消されました（データはパネル起動時に自動的に修復されます）。

- **Sub リンク ({count})** – 選択したクライアントのサブスクリプション URL と JSON サブスクリプション URL の一覧テーブル（すべてコピー ボタン付き）。誰も subId を持っていない場合：「選択したクライアントにはサブスクリプション ID がありません。」
- **グループに追加 と グループ解除** – グループラベルの付与と削除。
- **有効化 ({count}) / 無効化 ({count})**（POST /bulkEnable / bulkDisable）– 選択したクライアントの一括有効化・無効化。**有効化** は選択した各クライアントをすべての紐付け済み inbound で有効にします。トラフィッククォータが枯渇しているまたは期限切れのクライアントは自動的に再度無効化されます。**無効化** は即座にクライアントのアクセスを停止しますが、レコードと蓄積されたトラフィックは保持されます。実行前にパネルが確認を求め、操作後に処理されたクライアント数と失敗数（ある場合）の通知が表示されます。

ステータスによるトラフィックリセットと削除

- **すべてのクライアントのトラフィックをリセット**（POST /resetAllTraffics）– パネルのすべてのクライアントの up / down カウンターをゼロにリセットします。確認：「すべてのクライアントのトラフィックをリセットしますか?」と「すべてのクライアントの送受信カウンターがゼロにリセットされます。クォータと有効期限は影響を受けません。この操作は元に戻せません。」。トースト：「すべてのクライアントのトラフィックをリセットしました」。
- **枯渇したクライアントを削除**（POST /delDepleted）– **クォータが枯渇**（total > 0 and up + down >= total）**または期限切れ**（expiry_time > 0 and expiry_time <= 現在時刻）で、かつ reset = 0 のすべてのクライアントを削除します（自動更新があるクライアントは対象外）。確認：「枯渇したクライアントを削除しますか?」、「トラフィッククォータが枯渇または期限切れのすべてのクライアントが削除されます。この操作は元に戻せません。」。トースト：「{count} 件の枯渇したクライアントを削除しました」。

エクスポート、インポート、紐付けなしクライアントの削除

何も選択されていない状態では、クライアント ページの **その他** メニューから3つの操作が利用可能です。

クライアントをエクスポート（GET /clients/export）は {client, inboundIds} 形式のすべてのクライアントの JSON リストをビューアーで開きます。コピーとダウンロードボタン付き（ファイル clients-export.json）。**クライアントをインポート**（POST /clients/import）は同じ JSON を貼り付けて **Import** を押すエディターを開きます。inboundIds 付きのクライアントは作成されて inbound に紐付けられ、紐付けなしのクライアントは独立した「裸の」レコードとして復元されます。既存の email は上書きされません。スキップリストに入ります。トースト：「{count} clients imported」、「{ok} imported, {failed} skipped」。

紐付けなしクライアントを削除（POST /clients/delOrphans）– 危険な操作：どの inbound にも紐付けられていないすべてのクライアントを、トラフィックレコード・IP ログ・外部リンクとともに削除します。確認：「Delete clients without an inbound?」、「Removes every client that is not attached to any inbound, along with its traffic record. This cannot be undone.」。トースト：「{count} unattached clients deleted」。この操作は取り消せません。

8.5. 検索・フィルター・並べ替え

リストの上部に検索バーがあります（「email、コメント、サブ ID、UUID、パスワード、auth、Telegram ID を検索...」）。email・コメント・subId・UUID・パスワード・auth、そして（3.5.0 から再び）**Telegram ID** で検索できます。結果カウンター：「{total} 件中 {shown} 件を表示」。

クライアントリストは自動的に更新されます。パネルが数秒ごとに現在のページの最新情報を取得するため、新たに接続したクライアントや並べ替え順の変更は手動更新なしに反映されます（バックグラウンドポーリング中の読み込みインジケータは点滅しません）。

3.5.0 から、テーブルには「速度」列（「トラフィック」と「残量」の間）があります: クライアントのライブ速度が青いラベル **↑ アップロード / ↓ ダウンロード** で表示されます（約 5 秒間の移動平均、5 秒ごとに更新）。トラフィックがない場合はグレーのダッシュが表示されます。モバイルでは速度はクライアントカード内の行として表示されます。

クライアントフィルター パネルでは、ステータス（カテゴリー）・プロトコル・紐付け済み inbound・有効期限の範囲・使用済みトラフィックの範囲・自動更新の有無（あり/なし）・Telegram ID とコメントの有無・グループでフィルタリングできます。ノードを持つパネルでは **ノード マルチセレクト** が表示されます。選択したノードのクライアントにリストを絞り込めます。**ローカルパネル** という個別項目でノードへの紐付けなしの inbound のクライアントを絞り込めます（フィルターはノードがある場合のみ表示）。並べ替え: 古い順/新しい順、最近更新、最近オンライン、**Email A→Z / Z→A**、トラフィック多い順、残量多い順、期限切れが近い順。

8.6. バッジとステータス

ステータスの優先順位: 枯渇/期限切れ → 非アクティブ → まもなく期限切れ → アクティブ。

- **オンライン / オフライン** – アクティブな接続があり（現在のオンラインリストに存在）、かつ有効なクライアント。オンラインリストは個別のリクエストで更新されます（`/onlines`、`/onlinesByGuid`）。
- **枯渇 (depleted)** – クォータ消費済み（`up + down >= totalGB`）または期限切れ（`expiryTime <= 現在時刻`）。このようなクライアントは自動的に無効化され、**枯渇したクライアントを削除** の対象となります。
- **まもなく期限切れ / 残量僅少 (expiring)** – 有効なクライアントで、期限切れまでの残り時間がしきい値の間隔未満、またはクォータ枯渇まで残量がしきい値の量未満（しきい値はパネル設定で設定）。クライアントがすでに枯渇/無効化されている場合はカウントされません。
- **非アクティブ (deactive)** – `enable = false`（手動またはバックグラウンドタスクによって無効化）のクライアント。
- **アクティブ (active)** – 有効、枯渇していない、期限切れでない、しきい値まで余裕がある。

9. クライアントグループ

これはこの 3X-UI フォーク固有の機能です。オリジナルの 3x-ui (MHSanaei) には「クライアントグループ」という概念はなく、このフォークでは専用のグループテーブル、パネルメニューのグループページ、および対応する API メソッドが追加されています。設定をオリジナルの 3x-ui に移行する場合、グループラベルはどこにも反映されません。

9.1. クライアントグループとは何か、なぜ必要か

グループとは、1 人または複数のクライアントに付与できる名前付きの論理ラベルです。新しい接続方法を作成するものでも、inbound でもノードでもなく、クライアントのフィルタリングや一括処理に便利な、純粋に組織的なラベルです。

このフォークにおけるクライアントモデルの核心的なアイデアは、**クライアントとは email によって識別されるトップレベルのエンティティである**（`clients` テーブルの `email` フィールドには一意インデックスがある）ということです。同一のクライアント（同じ email と同じ認証情報）は、複数の inbound、さらには複数のノードに同時に所属でき、異なるプロトコルを使う場合も含まれます。グループラベルはクライアントごとに 1 つ保存されるため、そのクライアントのすべての inbound バインディングに自動的に伝播します。

グループラベルはグループ化のための論理ラベルです：

レイヤー	保存場所	フィールド
クライアントレコード (DB)	<code>clients</code> テーブル	<code>group_name</code> (デフォルトは空文字列 '')
グループ辞書 (DB)	<code>client_groups</code> テーブル	<code>name</code> (一意インデックス、空不可)
inbound 設定 (Xray)	JSON <code>settings.clients[].group</code>	クライアントが所属するすべての inbound の各クライアントオブジェクトに複製される

実際にどのような場面で役立つか：

- **複数の inbound/ノードにまたがる 1 つのクライアント。** クライアントが複数の inbound へのアクセスとして「販売」される場合（例：異なるプロトコルや異なるノード）、グループを使うことでそのクライアントを 1 つの単位として管理できます。トラフィックのリセット、削除、ラベルの変更をすべての inbound に対して一括で行えます。
- **一括操作とフィルタリング。** ****クライアントページではグループでリストを絞り込めます。グループページではグループのすべてのメンバーに対して一括操作が可能です。
- **大規模なクライアント群の整理。** `vip`、`trial`、`team-A` のようなラベルを使うことで、inbound を増やさずに何千ものクライアントを論理的なバケツに分類できます。

9.2. グループとクライアント、inbound、ノード、プロトコルの関係

ラベルの同期が単純ではないため、このサブセクションは理解において最も重要です。

グループはクライアントを表すものであり、inbound を表すものではありません。ラベルはクライアントのレコード (`clients.group_name`) に保存されます。クライアントが複数の inbound に所属している場合、グループを変更するとパネルはそのクライアントが所属するすべての inbound を巡回し、それらの Xray 設定 (`settings.clients[]`) 内の `group` フィールドを設定または削除します。技術的には次のように動作します: クライアントの email から所属するすべての inbound を検索し、各 inbound の JSON 設定の中でその email を持つクライアントオブジェクトを修正します。このため:

- グループはプロトコルに依存しません。同じ email が、ある inbound では VLESS クライアントとして、別の inbound では Hysteria クライアントとして存在できます。それでもグループラベルは 1 つであり、両方に適用されます (各プロトコルの認証情報はそれぞれ個別に保存されます)。
- グループはノードをまたいで有効です。ノードに属する inbound は、`nodeId` フィールドによってメインパネルの inbound と区別されます (メインパネルの inbound ではこのフィールドは `null/0` です)。グループラベルは、メインの inbound かノードの inbound かにかかわらず、その email を持つクライアントが所属するすべての inbound のクライアントオブジェクトに伝播します。

グループラベルはノードとの同期や設定の再構築に対して耐性があります。これは意図的な設計です:

- ノードがトラフィックスナップショットを送信するとき、そのデータはパネル DB のクライアントの `group_name` および `comment` を上書きしません。グループとコメントは「パネルローカル」フィールドとして扱われ、ノードはそれらを管理しません。
- inbound 設定の再構築時に、受信データの `group` が空であっても、保存済みのラベルはリセットされません。グループ管理はグループページを通じて行われるものであり (inbound の Xray 設定の編集を通じてではなく)、通常の設定再構築における「空のグループ」は「触らない」として解釈され、「クリアする」とは解釈されません。

実質的な結論: ラベルを意図的にクリアする操作は、グループの削除とグループからのクライアントの明示的な削除 (後述) のみです。通常の inbound 編集やノードとのバックグラウンド同期ではグループは失われません。

9.3. グループ辞書と「空のグループ」

ページに表示されるグループのリストは 2 つのソースをマージして作成されます:

1. **派生グループ (derived)** – クライアントに実際に設定されているすべての空でない `group_name` の値と、クライアント数のカウント。
2. **保存済みグループ (stored)** – `client_groups` テーブルのレコード。

このマージにより重要な効果が生まれます: グループはクライアントが 1 人もいない状態で存在できます。このようなグループは「グループを追加」ボタン (`client_groups` へのレコード挿入) で明示的に作成され、リストでは 0 というカウンターで表示されます。これらのレコードが空のグループです。リストは常に大文字小文字を区別せずに名前ですортされます。

ページの集計カウンター:

フィールド	表示内容
グループの総数	グループの総数 (保存済みと派生の合計)。

フィールド	表示内容
グループのあるクライアント	空でないグループラベルを持つクライアントの数。
空のグループ	クライアントが存在しないグループの数（カウンター <code>0</code> ）。
グループのクライアント数	特定のグループ内のクライアント数（テーブルの列）。

9.4. グループのフィールドと列

`client_groups` テーブルのグループレコードには以下が含まれます：

フィールド	型	デフォルト	説明
<code>Id</code>	int	自動インクリメント	グループレコードの主キー。
<code>Name</code>	string	– (必須)	グループ名。一意インデックス、空不可。UI では名前列として表示されます。
<code>CreatedAt</code>	int64 (ms)	作成時刻	グループレコードの作成日時。
<code>UpdatedAt</code>	int64 (ms)	変更時刻	最終変更日時。

ページのテーブルには少なくとも名前列とグループのクライアント数列、および操作ボタン（後述）が表示されます。

9.5. グループの作成

グループを追加ボタン。

手順：

1. **グループを追加**をクリックします。
2. グループ名を入力します。
3. 確定します。

バックエンドの動作（`POST /panel/api/clients/groups/create`、本文 `{"name": "..."}` ）：

- 名前は前後の空白がトリムされます。空の名前は「group name is required」というエラーで拒否されます。
- 同名のグループが既に存在する場合は「group already exists」というエラーになります。
- 成功すると `client_groups` にレコードが作成されます（初期状態ではクライアントがいない空のグループです）。

成功メッセージ：「グループ「`{name}`」が作成されました。」

例：API を使って空のグループを作成する。 クライアントを追加する前にラベルのセットを事前に準備する：

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/create' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"name": "vip"}'
```

成功時のレスポンス:

```
{ "success": true, "msg": "Группа «vip» создана.", "obj": null }
```

同じ名前で再度呼び出すと `"success": false` と `group already exists` というメッセージが返されます。

空のグループを事前に作成しておく、ラベルのセットを準備してから「クライアントを追加...」でクライアントを後から追加したい場合に便利です。

9.6. グループの名前変更

名前を変更ボタン、ダイアログのタイトルは**「{name} の名前を変更」**。

動作（POST /panel/api/clients/groups/rename、本文 `{"oldName": "...", "newName": "..."}`）:

- 両方の名前は空白がトリムされます。古い名前が空の場合は「old group name is required」、新しい名前が空の場合は「new group name is required」というエラーになります。
- 新しい名前が古い名前と同じ場合は何も行われません（影響するクライアント数は 0）。
- それ以外の場合、名前変更はアトミックに実行されます:
 - `client_groups` のレコードの名前が変更されます。
 - `group_name = oldName` のすべてのクライアントのフィールドが `newName` に更新されます。
 - 影響するクライアントが所属するすべての **inbound**（ノードの inbound を含む）で、Xray 設定の `group` の値が古い名前から新しい名前に変更されます。
- 名前変更後、パネルは Xray を再起動が必要な状態としてマークし、クライアント変更の通知を送信します。

メッセージ:

- 成功: 「{count} 件のクライアントのグループ名が変更されました。」
- UI での名前の競合: 「名前「{name}」のグループは既に存在します。」

9.7. グループへのクライアントの追加

クライアントを追加...ボタン、タイトルは「グループ「{name}」にクライアントを追加」。

ダイアログ内のヒントテキスト（原文のまま）:

「このグループに追加するクライアントを選択してください。inbound への既存の割り当ては保持され、グループラベルのみが変更されます。すでにこのグループに所属しているクライアントは表示されません。」

追加するクライアントがない場合は**「追加できる他のクライアントがありません。」**と表示されます。

動作（POST /panel/api/clients/groups/bulkAdd、本文 {"emails": [...], "group": "..."}）：

- グループ名は必須です（指定しないと「group name is required」エラー）。email のリストが空の場合、操作は何も行いません。
- 該当するグループが client_groups にも派生グループとしても存在しない場合、自動的に作成されます。
- 選択された email のクライアントに group_name = group が設定されます。inbound へのクライアントの割り当ては変更されず、ラベルのみが変更されます。その後、これらのクライアントのすべての inbound に group フィールドが設定されます。
- 影響したクライアントレコード数が返され、Xray は再起動が必要な状態としてマークされます。

成功メッセージ：「{count} 件のクライアントを {name} に追加しました。」

例：複数のクライアントを 1 つのリクエストでグループにラベル付けする。クライアントは email で指定し、inbound への割り当ては変更されません：

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/bulkAdd' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"emails": ["alice@example.com", "bob@example.com"], "group": "vip"}'
```

グループ vip が存在しない場合は自動的に作成されます。リクエスト後、これらのクライアントのレコードに group_name = "vip" が設定され、各自の inbound の Xray 設定のクライアントオブジェクトに "group": "vip" フィールドが追加されます：

```
{ "id": "6f1b...", "email": "alice@example.com", "group": "vip", "enable": true }
```

9.8. グループからのクライアントの削除（クライアント自体は削除しない）

クライアントを削除...ボタン、タイトルは「グループ「{name}」からクライアントを削除」。

ダイアログ内のヒントテキスト（原文のまま）：

「このグループから削除するメンバーを選択してください。クライアント自体は保持されます（完全に削除するには「グループのクライアントを削除」を使用してください）。」

動作（POST /panel/api/clients/groups/bulkRemove、本文 {"emails": [...]}）：技術的には、空のグループへの「グループに追加」と同等です。選択されたクライアントの group_name がクリアされ、そ

これらの inbound の Xray 設定から `group` フィールドが削除されます。クライアント自体と inbound への割り当ては残ります。

成功メッセージ: 「{count} 件のクライアントを {name} から削除しました。」

9.9. グループのトラフィックリセット

トラフィックをリセットボタン。

確認ダイアログ:

- タイトル: 「グループ {name} のトラフィックをリセットしますか? 」
- テキスト: 「グループのトラフィックカウンターのみがリセットされます。個々のクライアントのカウンターには影響しません。」

動作: グループ自体の表示カウンターのみ がゼロになります – パネルはグループの現在の `up/down` 合計を基準点として記憶し、以降グループ一覧では `max(0, 合計 - 基準点)` を表示します。個々のクライアントの `up/down` カウンター、クォータ、`enable` フィールドは **変更されず**、Xray の再起動も不要です。これは、グループのすべてのクライアントのトラフィックがリセットされていた従来の動作からの変更です。

リクエスト: `POST /panel/api/clients/groups/resetTraffic`、本文 `{"name": "<グループ名>"}`。

成功メッセージ: 「グループ {name} のトラフィックをリセットしました。」

9.10. グループの削除とグループのクライアントの削除

このページには本質的に異なる 2 種類の削除操作があります。混同しやすいため、その違いは重要です。

9.10.1. グループを削除する（クライアントは保持）

ボタン**「グループを削除する（クライアントは保持）」**。

ダイアログ:

- タイトル: 「グループ {name} を削除しますか? 」
- テキスト: 「これにより、グループが削除され、{count} 件のクライアントからそのラベルがクリアされます。クライアント自体は削除されません。」

動作 (`POST /panel/api/clients/groups/delete`、本文 `{"name": "..."}`): グループのレコードが `client_groups` から削除され、そのすべてのクライアントの `group_name` がクリアされ、それらの inbound から `group` フィールドが削除されます。クライアント自体、その接続、トラフィックは保持されます。Xray は再起動が必要な状態としてマークされます。

成功メッセージ: 「{count} 件のクライアントのグループをクリアしました。」

9.10.2. グループのクライアントを削除する（完全削除）

ボタン**「グループのクライアントを削除する」**。

ダイアログ:

- タイトル: 「{name} 内のすべてのクライアントを削除しますか? 」
- テキスト: 「これにより、{count} 件のクライアントとそのトラフィック記録が完全に削除されます。グループラベルもクリアされます。この操作は取り消せません。」

これは破壊的な操作です: クライアント自体 (email による一括削除、エンドポイント `POST /panel/api/clients/bulkDel`) をトラフィック記録とともに削除し、すべての inbound から除去します。

メッセージ:

- 成功: 「{count} 件のクライアントを削除しました。」
- 部分的な結果: 「{ok} 件削除、{failed} 件スキップ」

グループが空の場合、メンバーに対する操作は利用できず、**「このグループにはまだクライアントがいません。」**と表示されます。

9.11. 「クライアント」 ページとの連携

グループラベルはグループページ以外でも表示・使用されます:

- クライアントのコンパクトレコードには `group` フィールドがあるため、クライアントリストにグループ所属が表示されます。
- クライアントリスト (`/panel/api/clients/list/paged`) は `group` フィルターパラメーターを受け付けます。1つの名前またはカンマ区切りで複数の名前を渡すことができます。フィールド内での照合は「OR」で行われ、大文字小文字は区別されません。特殊なケース: フィルターグループリスト内の空要素は「グループのないクライアント」(`group` が空のもの)を意味します。
- クライアントページのレスポンスには `groups` 配列が含まれています。これは既存のグループ名の完全なリストであり、UI がフィルターのドロップダウンリストを構築するために使用されます。

例: グループでクライアントをフィルタリングする。このリクエストは `vip` または `trial` ラベルを持つクライアントのみを返します (複数の名前はカンマ区切り、「OR」):

```
GET /panel/api/clients/list/paged?group=vip,trial
```

グループがないクライアントを取得するには、リストに空要素を渡します。例えば、フィルター値 `group=` (空文字列) または `group=vip,` (ラベル `vip` のクライアントとグループなしのクライアント) のように指定します。

9.12. API エンドポイントのまとめ

すべてのグループルートは `/panel/api/clients` 以下にマウントされています:

メソッドとパス	目的	リクエスト本文
GET /panel/api/clients/groups	クライアント数カウンター付きのグループ一覧	–
GET /panel/api/clients/groups/:name/emails	グループの全メンバーの email (email でソート)	–
POST /panel/api/clients/groups/create	空のグループを作成	{"name"}
POST /panel/api/clients/groups/rename	グループ名を変更	{"oldName","newName"}
POST /panel/api/clients/groups/delete	グループを削除し、クライアントは保持 (ラベルのクリア)	{"name"}
POST /panel/api/clients/groups/bulkAdd	クライアントをグループに追加 (email で指定)	{"emails":[...],"group"}
POST /panel/api/clients/groups/bulkRemove	クライアントをグループから削除 (ラベルのクリア)	{"emails":[...]}
POST /panel/api/clients/groups/bulkDel	クライアントの完全削除 (「グループのクライアントを削除する」で使用)	{"emails":[...],"keepTraffic"}

例：API を使ったグループの典型的なライフサイクルシナリオ。

```
# 1. trial ラベルを作成
curl -s ../panel/api/clients/groups/create -d '{"name":"trial"}'

# 2. 2 人のクライアントに付与
curl -s ../panel/api/clients/groups/bulkAdd -d '{"emails":["u1@example.com","u2@example.com"],"group":"trial"}'

# 3. すべてのメンバーのトラフィックをリセット (/groups/trial/emails からメールを取得)
curl -s ../panel/api/clients/groups/bulkRemove -d '{"emails":["u2@example.com"]}'

# 4. グループを削除し、クライアントは保持 (ラベルのクリアのみ)
curl -s ../panel/api/clients/groups/delete -d '{"name":"trial"}'
```

ステップ 4 はグループレコードを削除し、そのクライアントの `group_name` をクリアしますが、クライアント自体、その接続、トラフィックは残ります。クライアント自体を完全に削除するには、代わりに `bulkDel` を使用します。

クライアントのラベルを変更する操作 (`rename` 、 `delete` 、 `bulkAdd` 、 `bulkRemove`) は Xray を再起動が必要な状態としてマークし、クライアント変更の通知を送信します。

9.13. グループ別のトラフィック

バージョン 3.3.0 の新機能： **グループセクション** (「クライアント」ページのグループ管理タブ) のグループテーブルに、各グループのクライアント数だけでなく、グループの合計使用トラフィックも表示されるようになりました。この列の見出しは**「使用トラフィック」**です。

列に表示される内容

各グループの行に、そのグループに所属するすべてのクライアントのトラフィックの合計、つまりすべてのメンバーの **up + down**（送信トラフィック + 受信トラフィック）の合計が表示されます。これにより、「グループ全体でどれだけのトラフィックをやり取りしたか」を、クライアントを 1 人ずつ開いて手動で合計することなく素早く確認できます。

グループテーブルに併せて表示される列:

列	意味
名前	グループ名
クライアント	このグループにラベル付けされたクライアントの数（以前はこの列を「グループのクライアント数」と呼んでいました）
送信済み	グループの全クライアントの合計 up （送信トラフィック）
受信済み	グループの全クライアントの合計 down （受信トラフィック）
使用トラフィック	グループの全クライアントの合計 up + down

送信と受信のトラフィックはそれぞれ**送信済み**列と**受信済み**列として個別に表示され、**使用トラフィック**列がその合計を示します。クライアント数の列は単に**クライアント**と呼ばれます。

テーブル上部の集計にはすべてのグループの集計も追加表示されます: 「**グループの総数**」と「**グループのあるクライアント**」、そして合計トラフィックは 2 枚のカードに分かれます: 「**総送信 / 受信**」（上下矢印付き – すべてのグループの送信と受信トラフィックを個別に表示）と**「**総トラフィック**」**（グラフアイコン付き – それらの合計）。

集計の仕組み

集計はクライアントテーブルにトラフィック集計テーブルを **LEFT JOIN** した 1 つの SQL クエリで実行されます:

- グループラベルフィールド (**group_name**) によってクライアントがグループ化され、その数がカウントされます。これが「グループのクライアント数」です。
- トラフィックは結合された **client_traffics** テーブルから **up + down** の合計として取得されます。つまり、各クライアントの送信 (**up**) と受信 (**down**) バイト数が合算されます。
- email** はクライアントテーブルとトラフィックテーブルの両方で一意であるため、結合によって 1 人のクライアントのトラフィックが二重にカウントされることはありません。

値に関する特記事項:

- トラフィック記録のないクライアント**はメンバーカウンターにはカウントされますが、合計への加算は 0 となるため、新しく作成されたグループのトラフィックは **0** と表示されます。
- 空のグループ**（作成されたがクライアントがいない）もリストに表示され、カウンターとトラフィックは両方ともゼロです: クライアントのラベルから「導出された」グループに加えて、明示的に保存されたグループも結果に含まれ、その後リストが大文字小文字を区別せずに名前ですべてソートされます。

- グループラベルのないクライアント（`group_name` が空）は計算に含まれません。

関連するアクション

グループテーブルからは引き続きグループ全体に対するアクションを実行でき、**「トラフィックをリセット」**もその1つです。これは **グループ自体のカウンターのみ**（基準点）をゼロにし、個々のクライアントのトラフィックには触れません（[9.9 参照](#)）。このリセット後、そのグループの「使用トラフィック」列は `0` と表示されます。

10. サブスクリプション (Subscription)

サブスクリプション (subscription) とは、クライアントに1つの固定URLを発行するメカニズムです。VPN クライアントはそのURLから設定の全セットを自動でダウンロードし、定期的に更新します。各 inbound へのリンクを個別にユーザーへ手動で送る代わりに、`https://ドメイン:ポート/sub/<subId>` 形式の単一アドレスを渡します。このアドレスにアクセスすると、パネルは当該クライアントに紐付けられたすべての設定をその場で組み立て、クライアントが求めるフォーマットで返します。サーバー側の設定が変わった場合（新しいアドレス、Reality キーのローテーション、inbound の追加など）、クライアントは次の自動更新時に最新の設定を取得でき、ユーザー側での操作は不要です。

サブスクリプションは、パネル内部の独立した HTTP/HTTPS サーバーが担当します。このサーバーはウェブパネルとは独立して起動し、専用のポートでリッスンします。これはセキュリティ上の理由からで、パネル自体のポートを公開せずにサブスクリプションポートだけを外部に開放できます。

10.1. subId とは何か、リンクの構成方法

inbound の各クライアントには `subId`（インターフェース上では「サブスクリプション ID」）フィールドがあります。この値がサブスクリプションのキーです。パネルはすべての inbound から `subId` がリクエストされた値と一致するクライアントを検索し、それらの設定を1つのレスポンスにまとめます。

- 複数のクライアント（同じ inbound または異なる inbound）に同一の `subId` が設定されている場合、それらの設定は1つのサブスクリプションにまとめられます。これは、1人のユーザーに複数のサーバー / プロトコルを1つのリンクで提供する標準的な方法です。

例: 1ユーザー、2サーバーを1リンクで。 2つの inbound（サーバー A の VLESS とサーバー B の Trojan）があるとして、ユーザーに両方の設定を1つのリンクで提供するには、そのユーザーの両クライアントに同じ `subId` を設定します。

```
Inbound 1 (VLESS): email = ivan@vpn, subId = ivan2025
Inbound 2 (Trojan): email = ivan@vpn, subId = ivan2025
```

すると `https://sub.example.com:2096/sub/ivan2025` にアクセスすると、パネルが両方の設定を一度に返します。後から同じ `subId` で3つ目の inbound を追加した場合、新しいリンクを送ることなく、次のサブスクリプション自動更新時にユーザーの設定に反映されます。

- クライアントの `subId` フィールドが空の場合、共有リンクを発行できません。インターフェース上ではツールチップに「このクライアントには subId がありません。共有リンクは利用できません。」と表示されます。

クライアントの外部リンクとサブスクリプション（「Links」タブ）

クライアントのフォームには「Links」タブがあり、特定のクライアントに追加の設定ソースを紐付けることができます。これはそのクライアントのサブスクリプションにのみ混在します（RAW、JSON、Clash 形式）。

- **Add External Link** – サードパーティの共有リンク（`vless://`、`trojan://`、`ss://` など）。そのまま出力に追加され、JSON/Clash では設定として追加でパースされます。
- **Add External Subscription** – 外部サブスクリプションのアドレス。パネルが自身でダウンロードし（キャッシュと短いタイムアウトあり）、取得した設定をクライアントの全体リストに統合します。

これは、同じ1つのリンクで inbound に加えてクライアントに追加サーバーを提供する際に便利です。リモートサブスクリプションのレスポンスが大きすぎる場合、サイレントに切り捨てられることはなくなり、パネルはエラーを返し、最後に正常にキャッシュされた値を引き続き使用します。

- `subId` の値は任意には設定できません。保存時に、スペース、`/`、`\`、制御文字が含まれていないかチェックされます。バリデーションのツールチップ：「サブスクリプション ID にはスペース、`/`、`\`、制御文字を含めることができません」。

最終的なリンクは `<ベース>/<subPath>/<subId>` の形式で構成されます（サブスクリプションサーバー設定と「リバースプロキシ URI」フィールドに関するセクションを参照）。`subId` に一致するクライアントが見つからない場合（クライアントが削除された、`subId` が存在しないなど）、サーバーはボディなしで HTTP 404 を返します。内部エラーの場合は HTTP 500 を返します。VPN クライアントはレスポンスコードのみを参照するため、エラーボディは意図的に空になっています。

サブスクリプション内の inbound リンクの順序

各 inbound には「サブスクリプション内の順序」（`subSortIndex`）フィールドがあります。これは1以上の数値で、サブスクリプション出力における当該 inbound のリンクの位置を指定します。値が小さいものが先になり、同じ値の場合は作成順（id 順）が保持されます。この順序はすべての出力形式（プレーンテキスト、サブスクリプションページ、JSON、Clash）に適用されます。パネル自体の inbound の順序には影響しません。

このフィールドは inbound フォームの共有アドレス設定の隣で編集でき、通常のルールに従ってノードに同期されます。少なくとも1つの inbound の順序が1以外の場合、Inbounds リストにコンパクトな「順序」カラムが表示されます。

10.2. サブスクリプションサーバーの設定

サブスクリプションのすべてのパラメーターは、パネル設定の「サブスクリプション」タブにあります。以下では各パラメーターを説明し、括弧内に内部設定キーとデフォルト値を記載します。

このセクション自体はタブに分かれています：「パネル設定」、「情報」、「プロフィール」、「証明書」、「Happ」、「Clash / Mihomo」。サブスクリプションヘッダー、サポート URL、プロフィールページ、お知らせ、テーマカタログのフィールドは「プロフィール」タブにあります。Happ と Clash/Mihomo のルーティングルールはそれぞれの名前のタブにあり、サブスクリプション更新間隔は「情報」タブにあります。

基本パラメーター

フィールド (UI)	キー	デフォルト値	説明
サブスクリプションを有効にする	subEnable	true (有効)	独立したサブスクリプションサーバーを起動します。ツールチップ: 「個別設定を持つサブスクリプション機能」。無効にするとサブスクリプションサーバーが起動せず、いずれのリンクも機能しません。
リスニング IP	subListen	空	サブスクリプションサーバーが接続を受け付ける IP アドレス。ツールチップ: 「デフォルトでは空のままにして、すべての IP アドレスをリッスンします」。
サブスクリプションポート	subPort	2096	サブスクリプションサーバーの TCP ポート。ツールチップ: 「サブスクリプションサービスを提供するポート番号はサーバーで使用されているではありません」 – ポートは空いており、パネルや Xray と競合しないことが必要です。
URI パス	subPath	/sub/	通常のサブスクリプションが提供されるパス。ツールチップ: 「/」で始まり「/」で終わる必要があります」。
リスニングドメイン	subDomain	空	サブスクリプションへのアクセスを許可するドメイン (Host バリデーション)。ツールチップ: 「デフォルトでは空のままにして、すべてのドメインと IP アドレスをリッスンします」。設定されている場合、異なる Host のリクエストは拒否されます。

セキュリティ上の重要事項: デフォルトパス `/sub/` (および JSON 用の `/json/`) は広く知られており、簡単に推測されます。パネルは「デフォルトのサブスクリプションパス `/sub/` は広く知られています – 変更してください。」および JSON についても同様の警告を表示します。独自のわかりにくいパスを設定することをお勧めします。

TLS / 証明書

フィールド (UI)	キー	デフォルト	説明
サブスクリプション証明書の公開鍵ファイルパス	subCertFile	空	証明書ファイル (<code>.crt</code> / <code>fullchain</code>) のフルパス。ツールチップ: 「/」で始まるフルパスを入力してください」。
サブスクリプション証明書の秘密鍵ファイルパス	subKeyFile	空	秘密鍵ファイルのフルパス。ツールチップ: 「/」で始まるフルパスを入力してください」。

両方のパスが設定され、証明書が正常に読み込まれた場合、サブスクリプションサーバーは **HTTPS** で動作します。フィールドが空であるか証明書の読み込みに失敗した場合、サーバーは **HTTP** にフォールバックします (エラーはログに記録されます)。有効な TLS の有無は、ベース URL の生成にも影響します。TLS ありでポート 443、TLS なしでポート 80 の場合、リンクのポート番号は省略されます。

更新間隔

フィールド (UI)	キー	デフォルト	説明
サブスクリプション更新間隔	subUpdates	12	クライアントアプリケーションがサブスクリプションを再取得する頻度（時間単位）。ツールチップ: 「クライアントアプリケーションでの更新間隔（時間単位）」。3.5.0 から、フィールドは 0-525600 を受け付けて範囲を表示します（従来のインターフェース上限 168 はサーバーと一致せず、2.x からのアップグレード後にあらゆる設定の保存をブロックしていました）。

この値は HTTP レスポンスヘッダー `Profile-Update-Interval` でクライアントに送信されます。最新のクライアントはこれを設定の自動更新周期として使用します。

レスポンスのフォーマットと情報

フィールド (UI)	キー	デフォルト	説明
エンコード	subEncrypt	true	ツールチップ: 「サブスクリプションで返された設定を暗号化する」。技術的にはこれは暗号化ではなく、通常のサブスクリプション本文全体の Base64 エンコード です（ほとんどのクライアントが期待するフォーマット）。オフにするとリンクはプレーンテキストで1行ずつ返されます。
使用状況情報を表示	subShowInfo	true	ツールチップ: 「設定名の後に残りのトラフィックと有効期限を表示する」。有効にすると、各設定の名前（remark）に残りトラフィック（ ) と有効期限（例: 5D, 3H ）のマークが追加されます。有効期限切れ / 利用不可のクライアントには N/A と表示されます。
名前にメールを含める	subEmailInRemark	true	ツールチップ: 「クライアントのメールをサブスクリプションプロフィール名に含める」。クライアントのメールをプロフィールの remark に追加します。

備考テンプレート (Remark Template)

サブスクリプションの各設定の表示名（remark）は **備考テンプレート** によって構成されます。これはサブスクリプション設定の「情報」タブの「備考テンプレート」フィールド（`remarkTemplate`）です。以前の備考モデルコンストラクター（`inbound/email/external proxy` の各部分と区切り文字を個別に選択する方式）はインターフェースから削除されました。代わりに、任意の名前フォーマットを記述し、変数を挿入します。デフォルト値は `{{INBOUND}}-{{EMAIL}}|{{TRAFFIC_LEFT}}|{{DAYS_LEFT}}D`（つまりデフォルトではプロフィール名にクライアントのメールが含まれます）。フィールドを空のままにすると、以前の（インターフェースから設定できない）備考モデルが使用されます。

変数は **Client**、**Traffic**、**Time & status** セクションにグループ化されており、フィールドの隣にクリック可能な `{{VAR}}` チップとしてホバー時のツールチップとともに表示されます。クリックするとトークンがテンプレートに挿入され、リアルタイムプレビューが利用できます。各変数はサブスクリプション生成時に特定のクライアントに対して個別に置換されます。単一括弧による省略記法（`{DATA_LEFT}`、

{EXPIRE_DATE}、{PROTOCOL}、{TRANSPORT} など）も使用でき、パネルが内部フォーマット {{...}} に変換します。

利用可能な変数:

- **クライアント識別:** {{EMAIL}} (同義語 - {{USERNAME}})、{{INBOUND}} (inbound 自体の remark)、{{HOST}} (ホストの remark)、{{ID}} (UUID)、{{SHORT_ID}} (UUID の最初の8文字)、{{SUB_ID}}、{{COMMENT}}、{{TELEGRAM_ID}}、{{PROTOCOL}}、{{TRANSPORT}}。
- **トラフィック:** {{TRAFFIC_USED}}、{{TRAFFIC_LEFT}}、{{TRAFFIC_TOTAL}} (および正確なバイト数の *_BYTES バリエーション)、{{UP}}、{{DOWN}}、{{USAGE_PERCENTAGE}}。
- **有効期限とステータス:** {{DAYS_LEFT}}、{{TIME_LEFT}}、{{EXPIRE_DATE}} (YYYY-MM-DD)、{{JALALI_EXPIRE_DATE}} (ジャリリ暦による日付)、{{EXPIRE_UNIX}}、{{CREATED_UNIX}}、{{RESET_DAYS}}、{{STATUS}} (active / expired / disabled / depleted)、{{STATUS_EMOJI}}。
- **接続 (Connection):** {{PROTOCOL}} - プロトコル (VLESS、VMess、Trojan など)、{{TRANSPORT}} - トランスポートネットワーク (tcp、ws、grpc など)、{{SECURITY}} - トランスポートセキュリティ (TLS、REALITY、NONE; 大文字で表示)。トラフィックおよび有効期限の変数と同様に、これら3つの変数はサブスクリプション本文内でのみ機能し、パネルの表示リンク (QR/「情報」) およびサブスクリプション情報ページの remark から自動的に除外されます。

テンプレートは縦棒 | でセグメントに分割できます。変数が「無制限」の値 (∞) を返すセグメント (例: 制限のないクライアントの {{TRAFFIC_LEFT}} や {{DAYS_LEFT}}) は自動的に非表示になります。また、トラフィック使用量と有効期限のブロックは、各設定に重複して表示されないよう、クライアントの最初のリンクにのみ一度表示されます。

例。テンプレート {{EMAIL}}|{{TRAFFIC_LEFT}}|{{DAYS_LEFT}}D は、残り42 GBで7日のクライアントには ivan@vpn 42.00GB 7D のような名前を生成し、無制限のクライアントには単純に ivan@vpn と表示されます (∞ のセグメントは省略されます)。

バージョン3.4.2 から、変数 {{EMAIL}} (および同義語 {{USERNAME}}) は、サブスクリプション本文内でクライアントの **最初** のリンクにのみ出力されます - トラフィック/期限の残量ブロックと同様です; 同じクライアントの他のリンクでは省略されます。

パネルの表示リンク (「クライアント」ページの QR コードおよび「情報」ウィンドウ) およびサブスクリプション情報ページでは、クライアントのメールがプロフィール名に含まれます。ホストが設定されている場合は「inbound-host-email」、ホストなしの場合は「inbound-email」の形式です。トラフィックおよび有効期限の情報 (および「接続」グループの変数) は、これらの表示名には挿入されません。VPN クライアントが受け取るサブスクリプション本文内でのみ機能します。

inbound の削除と再作成後にクライアントのトラフィック統計行が「孤立」した場合、{{TRAFFIC_USED}} (およびその他の使用量指標) は 0.00B を表示しなくなります。パネルがクライアントのメールで統計を追加検索し、正確な使用済みトラフィックを代入します。|備考テンプレート| remarkTemplate | {{INBOUND}}|{{TRAFFIC_LEFT}}|{{DAYS_LEFT}}D | {{VAR}} 変数置換による各設定の表示名 (remark) の自由テンプレート。サブスクリプション生成時に各クライアントに対して個別に置換されます。以前の「備考モデル」コンストラクター (inbound/email/external proxy と区切り文字の選択) はインターフェースから削除され、フィールドを空のままにした場合のフォールバックとしてのみ使用されます。詳細は下記「備考テンプレート (Remark Template)」を参照してください。|

プロフィールメタデータ（レスポンスヘッダー）

これらの文字列はレスポンスの HTTP ヘッダーでクライアントに送信され、VPN クライアントにプロフィールのメタデータとして表示されます。すべてデフォルトは空です。

フィールド (UI)	キー	ヘッダー	説明
サブスクリプションヘッダー	<code>subTitle</code>	<code>Profile-Title</code> (Base64 形式)	「VPN クライアントでクライアントに表示されるサブスクリプション名」。Clash では <code>Content-Disposition</code> でインポートするプロフィール名としても使用されます。
サポート URL	<code>subSupportUrl</code>	<code>Support-Url</code>	「VPN クライアントに表示されるテクニカルサポートへのリンク」。
プロフィール URL	<code>subProfileUrl</code>	<code>Profile-Web-Page-Url</code>	「VPN クライアントに表示されるウェブサイトへのリンク」。設定されていない場合、実際のサブスクリプションリクエスト URL が使用されます。
お知らせ	<code>subAnnounce</code>	<code>Announce</code> (Base64 形式)	「VPN クライアントに表示されるお知らせのテキスト」。3.5.0 から、テキストは（空でない場合）サブスクリプション情報ページ上部の情報バナー としても表示され、カスタムテンプレートでは変数 <code>announce</code> が利用可能です。

さらに、各レスポンスにはクライアントの集計トラフィックデータ（`upload`、`download`、`total`、`expire`（秒単位の有効期限））を含む `Subscription-Userinfo` ヘッダーが送信されます。クライアントはこれを使って残りトラフィックと有効期限を表示します。

ルーティング（Happ クライアント専用）

フィールド (UI)	キー	デフォルト	説明
ルーティングを有効にする	<code>subEnableRouting</code>	<code>false</code>	「VPN クライアントでルーティングを有効にするためのグローバル設定（Happ 専用）」。 <code>Routing-Enable</code> ヘッダーで送信されます。
ルーティングルール	<code>subRoutingRules</code>	空	「VPN クライアントのグローバルルーティングルール（Happ 専用）」。 <code>Routing</code> ヘッダーで送信されます。

| サーバー設定を非表示 | `subHideSettings` | `false` | 「サブスクリプションのサーバー設定を非表示にする（Happ 専用）」。有効にすると、Happ クライアントでサーバーパラメーターの表示と変更が非表示になります。このオプションは Happ クライアントにのみ適用されます。|

Incy ルーティング（Incy クライアント専用）

VPN クライアント **Incy** 用に、サブスクリプション設定に専用の「**Incy**」タブがあります。2つのフィールドがあります。「ルーティングを有効にする」スイッチ（`subIncyEnableRouting`、デフォルトはオフ）と、`incy://routing/onadd/<base64>` 形式の「ルーティングルール」テキストフィールド

(`subIncyRoutingRules`) です。ルーティングが有効でフィールドが入力されている場合、この文字列はサブスクリプション本文 (raw 形式) の最後に別途追加されます。これにより、Happ クライアントの `Routing` ヘッダーと競合することなく、ルーティングプロフィールが Incy クライアントに配信されます。これらの設定は Incy クライアントにのみ適用されます。

リバースプロキシ URI

フィールド (UI)	キー	デフォルト	説明
リバースプロキシ URI	<code>subURI</code>	空	「プロキシサーバー経由で使用するためのサブスクリプション URL のベース URI を変更する」。

フィールドが空の場合、パネルはサブスクリプションのドメインとポートからベースアドレスを自動生成します (TLS を考慮)。サブスクリプションが別のドメインまたはパスの外部リバースプロキシ/CDN 経由で提供される場合、このフィールドに最終的なベース URI を設定することで、すべてのリンクがそれを基に生成されます。JSON (`subJsonURI`) と Clash (`subClashURI`) にも個別のフィールドがあります。

共通の `subURI` のみが設定され、JSON と Clash の個別フィールドが空の場合、それらのフォーマットのリンクはサブスクリプションページで `subURI` のスキームとホストを継承します (sub サーバーのポートや `http` ではなく) – リバースプロキシのアドレスと一致するようになります。

例: リバースプロキシ経由のサブスクリプション。 サブスクリプション自体は 2096 でリッスンしていますが、外部からは nginx/CDN 経由で `https://cfg.example.com/u/` としてアクセスできます。レスポンス内のリンクが内部の `ドメイン:2096` ではなく外部アドレスを基に生成されるよう、「Reverse proxy URI」フィールドに最終的なベース URI を設定します。

Reverse proxy URI: `https://cfg.example.com/u`

すると最終的なリンクは `https://cfg.example.com/u/ivan2025` の形式になります。JSON と Clash 形式でも必要に応じて `subJsonURI` と `subClashURI` の個別フィールドに同様に入力します。

10.3. 出力フォーマット

サブスクリプションは3つの独立したフォーマットで提供でき、それぞれ独自のエンドポイントを持ち、個別に有効/無効にできます。

サブスクリプション出力のサーバーアドレスとノード

サブスクリプションリンクのサーバーアドレスは、パネルの通常リンクや QR コードと同じアドレス戦略で置換されます。「listen」 – ルーティング可能なバインドアドレス、「custom」 – ユーザーが設定したカスタムアドレス (`shareAddr`)、「node」 (デフォルト) – ノードのアドレス。アドレス戦略が明示的に設定されていない inbound のサブスクリプション出力は変更されません。これにより、特定のパブリック IP にバインドされたノードの inbound がクライアントに到達可能なアドレスを提供できます。この戦略は raw、JSON、Clash 形式すべてに適用されます。

ノード名 (Node) はサブスクリプションのプロフィール名 (remark) には追加されません。クライアントアプリケーションでは、管理者が設定した inbound の remark のみが表示され、@ノード名 のような内部サフィックスは付きません。マルチノードサブスクリプションで同名のエントリを区別するには、手動で異なる remark を設定するか、独自の Remark を持つ管理対象ホスト (Hosts) を使用してください。

ノード間の同期ズレにより同一クライアントがサービス JSON inbound に重複して含まれた場合、サブスクリプション出力はすべての3つのフォーマットでメールによる重複を自動的に除去するため、出力に重複したプロフィールは現れません。

管理対象ホスト (Hosts)

Hosts セクション (サイドメニューの項目; Total/Enabled/Disabled の数とリストを表示する概要ページ) は、サブスクリプションリンクのアドレスのオーバーライドを設定します。各 inbound に1つまたは複数の **ホスト** (エンドポイント) を追加でき、**inbound 自体のアドレス、ポート、TLS パラメーターの代わりに**クライアントへのサブスクリプションリンクに挿入されます。inbound を変更せずに CDN やリレー経由でトラフィックを配信する際に便利です。

バージョン3.5.0 から、ホストは「グループ」です: 1 つのレコードが **複数の inbound と 複数のアドレス** を同時にカバーします。リストの操作 (有効化/無効化/削除/並べ替え) と API はグループ (文字列の groupId) 単位で動作します。一括エンドポイント `POST /panel/api/hosts/bulk/add` (`{ "inboundIds": [...], "hosts": [...], ... }`) が追加され、`list / get / update / del / setEnable` はグループオブジェクトを操作します。

各ホスト (グループ) には以下を設定します。

- **Remark** と説明 (Description)、**Inbounds** への紐付け (検索付きマルチ選択; 最低 1 つ)、**Enable** スイッチ、ノードへの割り当て (**Nodes**)。
- **Address** – 3.5.0 から、これは **アドレスのリスト** です (タグ入力; 区切り文字はカンマ、セミコロン、スペース; プレースホルダー `cdn.example.com`, `cdn2.example.com:443`)。各エントリには独自の埋め込み `:ポート` を含められます (角括弧付きの IPv6 `:::1:443` を含む)。ドロップダウンの補完は、他のホストで既に使用されているアドレスを提案します。リストが空の場合は inbound 自体のアドレスが継承されます (リストではオレンジ色の **Inherits** ラベル)。**Port** (0 の場合 inbound のポートを継承) は、埋め込みポートのないエントリのデフォルトポートとして機能します; **Tags** (RAW サブスクリプションのみで考慮)。
- **Security** タブ – SNI、フィンガープリント (fingerprint)、ALPN、証明書のピン留め (pinned-cert)、`allowInsecure`、ECH を含む `same / tls / none / reality`。
- **Advanced** タブ – Host ヘッダー、Path、VLESS ルート、Mux、Sockopt、Final Mask、および個別のサブスクリプション形式 (raw / json / clash) からホストを除外する設定。3.5.0 から、**ホストの Final Mask は raw リンクにも含まれます** (`fm=`; 以前は JSON/Clash のみ): ホストの TCP/UDP マスクは inbound のマスクに追加され、ホストの QUIC パラメーターは inbound に存在しない場合にのみ使用されます。**Allow insecure** スイッチは **Hysteria/Hysteria2** にも適用されるようになりました (リンクでは `insecure=1`、Clash では `skip-cert-verify: true`)。
- **Clash (mihomo)** タブ – IP バージョン、Mihomo X25519、ホストのシャッフル (Shuffle host)。

リストの **Endpoint** 列はアドレスをチップで表示し (最初の 1 つが表示され、残りは「+N」ポップオーバー内)、**Inbounds** 列はプロトコル別に色分けされた inbound のチップを表示します。3.5.0 から、ホス

トは inbound 内ではなく **グローバル**に 順序付けられます（並び順、次に備考順）。一括の有効化・無効化・削除は引き続きサポートされます。管理対象ホストは以前の External Proxy 配列の後継です。

VLESS route (VLESS ルート)。バージョン3.4.2 から、これは1つの数値 0-65535 です（ポートのリストではなくなりました; ヒント – 「UUID に埋め込む単一の VLESS route 値 (0-65535)、例 443; 空 – なし」、プレースホルダー 443）。設定した値は、生成される各サブスクリプション (raw / JSON / Clash) の UUID に実際に「埋め込まれます」: Xray は UUID のバイト 6-7 を読み取り、認証前にそれをマスクするため、クライアントは引き続き一致します。空または不正な値は UUID を変更しません。

通常リンク (SUB) – Base64 / プレーンテキスト

基本フォーマット、エンドポイント `subPath` (デフォルト `/sub/`)。サブスクリプション全体が有効な場合、常に有効です。Xray リンク (`vless://`、`vmess://`、`trojan://`、`ss://` など) のリストを1行ずつ返します。「エンコード」オプション (`subEncrypt`) が有効な場合、リスト全体が Base64 にエンコードされます。無効の場合はプレーンテキストで返されます。このフォーマットはほぼすべてのクライアント (v2rayNG、V2RayTun、Sing-box、NekoBox、Streisand、Shadowrocket、Happ など) が対応しています。

例: 「エンコード」を無効にしたレスポンス本文。 `subEncrypt = false` の場合、`/sub/` エンドポイントはプレーンテキストを返します (1行につき1リンク)。

```
vless://3c8f...@a.example.com:443?security=reality&...#srvA-ivan
trojan://p4ss@b.example.com:443?security=tls&...#srvB-ivan
```

`subEncrypt = true` (デフォルト) の場合、同じリスト全体が Base64 にエンコードされて1行で返されます。ほとんどのクライアントはこの形式を期待します。

JSON サブスクリプション (sing-box および互換クライアント)

エンドポイント `subJsonPath` (デフォルト `/json/`)、個別のチェックボックスで有効化。

フィールド (UI)	キー	デフォルト	説明
JSON サブスクリプション	<code>subJsonEnable</code>	<code>false</code>	「JSON サブスクリプションエンドポイントを独立して有効化/無効化する」。

sing-box および派生クライアント (Podkop、OpenWRT sing-box、Karing、NekoBox) が解析できる完全な JSON 設定を返します。3.5.0 から、**ネイティブ WireGuard inbound** のクライアントも JSON サブスクリプションに含まれます (`secretKey`、`address`、`publicKey` / `endpoint` / `preSharedKey` / `keepAlive` / `allowedIPs` を持つ `peers[]`、`mtu`) – 以前は通知なくスキップされていました。このフォーマットには追加パラメーターが利用できます (`subFormats` タブ)。

- **Mux** (`subJsonMux`、デフォルトは空) – JSON サブスクリプションの各ストリームのアウトバウンドに挿入されるマルチプレクシング (Mux) の JSON 設定。「1つの接続で複数の独立したデータストリームを伝送する」。

- **Final Mask** (subJsonFinalMask 、デフォルトは空) – 「各 JSON サブスクリプションストリームに追加される Xray の finalmask (TCP/UDP) および QUIC 設定。クライアント側で最新の Xray バージョンが必要」。サブフィールドで設定: 「パケット」 (packets)、 「長さ」 (length)、 「間隔」 (interval)、 「最大分割」 (maxSplit)、 「ノイズ」 (noises : 「タイプ」 / type 、 「パケット」 / packet 、 「遅延 (ミリ秒)」 / delayMs 、 「適用先」 / applyTo 、 「+ ノイズ」 ボタン)、 および 「並行数」 (concurrency)、 「xudp 並行数」 (xudpConcurrency)、 「xudp UDP 443」 (xudpUdp443)。
- **ルーティングルール** (subJsonRules 、デフォルトは空) – JSON 設定に追加されるグローバルルール。

Clash / Mihomo サブスクリプション (YAML)

エンドポイント subClashPath (デフォルト /clash/)、個別のチェックボックスで有効化。

フィールド (UI)	キー	デフォルト	説明
Clash / Mihomo サブスクリプション	subClashEnable	false	Clash および Mihomo クライアント用の YAML 設定生成を有効化する。
ルーティングを有効にする	subClashEnableRouting	false	「生成された YAML サブスクリプションにグローバル Clash/Mihomo ルーティングルールを追加する」。
グローバルルーティングルール	subClashRules	空	「各 YAML サブスクリプションの MATCH,PROXY の前に追加される Clash/Mihomo ルール」。

レスポンスは application/yaml; charset=utf-8 タイプで返されます。「サブスクリプションヘッダー」 (subTitle) が設定されている場合、Content-Disposition ヘッダー (attachment; filename*=UTF-8''<title>) にも送信され、Clash クライアントがインポートするプロフィールにその名前を付けられます。

生成されるリンクと YAML のフォーマットは最新クライアントに対して最新の状態に保たれています。Shadowsocks-2022 (SS2022) では userinfo を Base64 エンコードしなくなりました。http 難読化を使用した Shadowsocks リンクは obfs-local プラグインを使用した SIP002 フォーマットで提供されます。Clash/Mihomo サブスクリプションでは XHTTP の完全なフィールドセットが実装されています。3.5.0 から、**ネイティブ WireGuard inbound** のクライアントも Clash/Mihomo サブスクリプションに含まれます (フィールド private-key、public-key、pre-shared-key、persistent-keepalive、ip / ipv6、mtu、dns) – 以前は通知なくスキップされていました。これらに個別の設定は不要で、リンクがクライアントによって正確に認識されるようになっています。

注意: このビルドでサポートされているのは3つのフォーマットです – 通常リンク (Base64/テキスト)、JSON (sing-box 互換)、Clash/Mihomo (YAML)。サブスクリプションサーバーには Outline の独立したフォーマットはありません。

10.4. サブスクリプション情報ページと QR コード

ブラウザで 3つのサブスクリプションリンクのいずれか – raw、JSON、Clash – を開くと（または URL にパラメーター `?html=1` もしくは `?view=html` を明示的に追加するか、`Accept: text/html` ヘッダーを送信すると）、サーバーは「生の」レスポンスの代わりにビジュアルな **サブスクリプション情報ページ**（「サブスクリプション情報」）を返します。3.5.0 より前は、このように動作するのはメインリンク `/sub/` のみで、`/json/` と `/clash/` はブラウザに生の JSON/YAML を返していました。VPN クライアントは HTML を要求しないため、引き続き機械が読み取れるレスポンスを受け取ります。

ページ（Vite でビルドされたシングルページアプリケーション）には以下が表示されます。

- **サブスクリプション情報**（Descriptions ブロック）：
 - 「サブスクリプション ID」 – `subId` の値；
 - 「ステータス」 – 「有効」、「無効」、または「無制限」。クライアントが無効化されているか、トラフィック制限を超えているか、有効期限が切れている場合は「無効」ステータスが設定されます；
 - 「ダウンロード」と「アップロード」 – トラフィック量；
 - 「総制限」 – トラフィック制限、または無制限の場合は ∞ ；
 - 「有効期限」 – 終了日、または「無期限」；
 - 残りトラフィックと最終オンライン時刻。
 - 日付はパネルの「カレンダータイプ」設定（`datepicker`、デフォルトは `gregorian`）に応じてグレゴリオ暦またはジャラリ暦で表示されます。
- **サブスクリプションリンク**：有効な各フォーマットに対して、カラータグ（緑の **SUB**、紫の **JSON**、金の **CLASH**）、コピーボタン、**QR コード** ボタン（ポップアップウィンドウ、サイズ 240 px）を持つ個別の行が表示されます。JSON と CLASH の行は、対応するフォーマットが設定で有効な場合にのみ表示されます。
- **個別リンク**（「リンクをコピー」）：サブスクリプションに含まれる個別設定の全リスト。各エントリにはプロトコルタグ、コピーボタン、QR コード（ポスト量子リンクの場合は QR コードを生成しません）が付いています。
- **「すべての設定をコピー」ボタン**（個別リンクリストの上部）：1回のクリックですべての設定リンク（各リンクを改行区切り）をクリップボードにコピーします。個別にコピーする必要はありません。完了すると「すべての設定がコピーされました」という通知が表示されます。
- **アプリへのクイックインポートボタン**
（プラットフォーム別ドロップダウンメニュー）：Android 用 – `v2box`、`v2rayNG`（ディープリンク `v2rayng://install-config?url=...`）、`Sing-box`、`V2RayTun`、`NPV Tunnel`、`Happ`（`happ://add/...`）、`Incy`（`incy://add/...`）；iOS 用 – `Shadowrocket`（パラメーター `flag=shadowrocket` 経由）、`v2box`（`v2box://install-sub?url=...&name=...`）、`Streisand`（`streisand://import/...`）、`V2RayTun`、`NPV Tunnel`、`Happ`、`Incy`。これらのボタンは、サブスクリプションアドレスが事前に設定された対応アプリのディープリンクを開くか、リンクをクリップボードにコピーします。

サブスクリプション情報ページはキャッシュ無効ヘッダー（`Cache-Control: no-cache`）付きで返されるため、クライアントは常に最新のトラフィックと有効期限データを参照できます。

10.5. サブスクリプションページのカスタムテンプレート

3.3.0 以降、標準のサブスクリプションランディングページを独自の HTML テンプレートに置き換えることができます。デフォルトではサブスクリプションアドレスに組み込みページが返されますが、独自のテンプレートを含むディレクトリを指定すると、パネルがそれをレンダリングし、クライアントの最新データ（トラフィック、有効期限、リンクなど）を挿入します。

重要：パネルは既製のテンプレートを **提供していません** – 独自のテーマは自分で作成する必要があります。作成手順と利用可能な変数の一覧は [docs/custom-subscription-templates.md](#) にあります。

有効化する場所

テーマディレクトリはパネル設定で指定します。

設定 → サブスクリプション → 「サブスクリプション情報」セクション、「サブスクリプションテーマディレクトリ」フィールド（`subThemeDir`）。

インターフェースのフィールド説明：「サブスクリプションページ用のカスタムテンプレート（`index.html/sub.html`）を含むフォルダーへの絶対パス（例：`/etc/3x-ui/sub_templates/my-theme/`）。デフォルトページを使用するには空のままにしてください。」

同じセクション内には、テンプレートで利用可能な値を持つ関連設定が配置されています。

「サブスクリプションテーマディレクトリ」フィールドの説明には、独自のサブスクリプションページテーマを作成するためのドキュメントへの「[テンプレートガイド ↗](#)」リンクがあります。

- 「サブスクリプションヘッダー」（`subTitle`） – クラウドクライアントに表示される名前；
- 「サポート URL」（`subSupportUrl`） – テクニカルサポートへのリンク。

設定パラメーター

パラメーター	デフォルト値	用途
<code>subThemeDir</code>	""（空）	HTML テンプレートを含むディレクトリへの絶対パス。空 = デフォルトの組み込みページ。

独自テンプレートの設定方法

1. サーバー上にテーマ用のフォルダーを作成します（任意の場所）。例：`/etc/3x-ui/sub_templates/my-theme/`。
2. `index.html` または `sub.html` という名前の HTML ファイルを中に置きます。

例: テーマのパス。サーバー上の最終的なレイアウトと設定フィールドの値。

```
/etc/3x-ui/sub_templates/my-theme/  
└─ index.html      （または sub.html – こちらが優先されます）
```


設定 → サブスクリプション → 「サブスクリプションテーマディレクトリ」：
/etc/3x-ui/sub_templates/my-theme/

パスは **絶対パス**（/ で始まる）である必要があります。フォルダーに `index.html` も `sub.html` もない場合、パネルは組み込みページを返します。3. パネルで **設定 → サブスクリプション** を開き、「サブスクリプションテーマディレクトリ」フィールドにこのフォルダーへの **絶対** パスを入力します。4. 設定を保存します。

ファイル選択とレンダリングの動作:

- ディレクトリに `sub.html` がある場合は、それが使用されます。ない場合は `index.html` が使われます。つまり `sub.html` が `index.html` より優先されます。
- テンプレートは Go の標準エンジン `html/template` でレンダリングされます。
- パースされたテンプレートは **キャッシュされ**、ファイルの更新時刻が変わった場合にのみディスクから再読み込みされます。そのため、テンプレートの変更はパネルの再起動なしに反映されますが、リクエストごとの読み取り/パースのオーバーヘッドはありません。
- レスポンスはバッファに完全に組み立てられてからクライアントに送信されます。テンプレートの実行中にエラーが発生しても、部分的に生成された（破損した）ページがユーザーに送信されることはありません。

デフォルト動作とフォールバック

- フィールドが空 → 組み込み SPA ページが返されます（データは `window.__SUB_PAGE_DATA__` に埋め込まれます）。
- パスが存在しないか、ディレクトリでない → デフォルトページが使用されます。
- ディレクトリに `index.html` も `sub.html` もない → 「subThemeDir set but no usable template found」という警告がログに記録され、デフォルトページが返されます。
- テンプレートファイルはあるがパースに失敗する → 「custom template parse failed」というエラーがログに記録され、デフォルトページが返されます。
- テンプレート実行エラー → 「custom template execution failed」がログに記録され、デフォルトページが返されます。

つまり、カスタムテンプレートに問題があってもサブスクリプションは「壊れ」ません - パネルは常に組み込みページにフォールバックします。すべてのサブスクリプションページ（カスタムおよび標準）はキャッシュ無効ヘッダー（`Cache-Control: no-cache, no-store, must-revalidate`）付きで返されるため、クライアントは常に最新のトラフィックと有効期限データを受け取ります。

利用可能なテンプレート変数

テンプレートのコンテキストにはサブスクリプションクライアントのデータセットが渡されます。アクセスは `{{ .変数名 }}` で行います。

変数	型	説明
<code>{{ .sId }}</code>	string	サブスクリプション ID (UUID) 。
<code>{{ .enabled }}</code>	bool	クライアント/サブスクリプションが有効かどうか。

変数	型	説明
<code>{{ .download }}</code>	string	フォーマットされたダウンロード量（例：「2.5 GB」）。
<code>{{ .upload }}</code>	string	フォーマットされたアップロード量。
<code>{{ .total }}</code>	string	フォーマットされたトラフィック総制限量。
<code>{{ .used }}</code>	string	フォーマットされた使用済みトラフィック（ダウンロード + アップロード）。
<code>{{ .remained }}</code>	string	フォーマットされた残りトラフィック。
<code>{{ .expire }}</code>	int64	有効期限 – Unix 時間（秒単位）（0 = 無期限）。JS の <code>Date</code> には 1000 を掛けてください。
<code>{{ .lastOnline }}</code>	int64	最終オンライン時刻 – Unix 時間（ミリ秒単位）（0 = 一度もなし）。
<code>{{ .downloadByte }}</code>	int64	正確なバイト数でのダウンロード量。
<code>{{ .uploadByte }}</code>	int64	正確なバイト数でのアップロード量。
<code>{{ .totalByte }}</code>	int64	正確なバイト数での総制限量。
<code>{{ .subUrl }}</code>	string	サブスクリプションページの URL。
<code>{{ .subJsonUrl }}</code>	string	JSON サブスクリプション設定の URL。
<code>{{ .subClashUrl }}</code>	string	Clash/Mihomo 設定の URL。
<code>{{ .subTitle }}</code>	string	設定からのサブスクリプションヘッダー（空の場合あり）。
<code>{{ .subSupportUrl }}</code>	string	設定からのサポート URL（空の場合あり）。
<code>{{ .links }}</code>	[]string	設定文字列のリスト（VMess、VLESS など）。反復: <code>{{ range .links }}</code> ... <code>{{ end }}</code> 。
<code>{{ .emails }}</code>	[]string	サブスクリプションに関連するメールのリスト。
<code>{{ .datepicker }}</code>	string	現在のパネルカレンダー形式: <code>gregorian</code> または <code>jalali</code> （「カレンダータイプ」設定から取得; 空の場合は <code>gregorian</code> ）。

テンプレート変数の一部を使用した最小限のテンプレート本文の例:

```
<h1>{{ .subTitle }}</h1>
<p>使用済み: {{ .used }} / {{ .total }} (残り {{ .remained }}) </p>
{{ range .links }}<div>{{ . }}</div>{{ end }}
```

例: **expire** からの有効期限日付。 `{{ .expire }}` フィールドは Unix 時間（秒単位）なので、JavaScript では 1000 を掛けます。値 0 は「有効期限なし」を意味します:

```
<script>
  var exp = {{ .expire }};
  document.write(exp === 0
    ? '無期限'
    : '期限: ' + new Date(exp * 1000).toLocaleDateString());
</script>
```

注意: `{{ .lastOnline }}` はすでに ミリ秒単位で設定されているため、1000 を掛ける必要はありません。

11. Xray: ルーティング、outbounds、DNS、および拡張機能

「**Xray 設定**」セクションは Xray-core の設定テンプレートエディターであり、パネルはこのテンプレートに基づいてコアを起動するための最終的な `config.json` を生成します。セクションテンプレートのヒント: 「このテンプレートに基づいて Xray 設定ファイルが作成されます。」 inbounds (データベースに個別に保存され、設定ビルド時にテンプレートに挿入される) とは異なり、その他のすべて——ログ、ルーティング、outbounds、DNS、ポリシー、統計——はここで設定します。

重要: テンプレートの値は `xrayTemplateConfig` キーのもとデータベースに保存されます。保存時、パネルはそれをいくつかの自動変換処理にかけます (11.11 参照)。構文的に不正な JSON は「`xray template config invalid`」エラーで拒否されます。

メニュー内の場所: 「Outbounds」と「ルーティング」

「**Outbounds**」 (**Outbounds**) と「**ルーティング**」 (**Routing**) はサイドメニューの個別の項目であり (「**ホスト**」の直下、「**パネル設定**」の上)、それぞれのアドレスは `/outbound` と `/routing` です。これらのページへの直接リンクおよびページのリロードは期待どおりに動作します。「**Xray 設定**」サブメニューには、基本、バランサー、DNS、高度なテンプレートのみが残ります。以下の説明では、セクション 11.3 と 11.4 が「ルーティング」と「Outbounds」のページに対応します。

11.1. エディターの構造: タブ/モード

エディターはテンプレートの複数の表示モード (JSON セクション別フィルター) を提供します:

モード	表示内容
基本	基本セクション (ログ、基本ルーティング、一般設定)
高度なテンプレート	完全な Xray JSON テンプレート
すべて	すべてのセクションを同時表示

エディター内の設定の論理グループ:

- 一般設定 (ヒント: 「これらのパラメーターは一般設定を説明します」)。
- ログ (11.10 参照)。
- 基本接続: ブロックと直接ルート。
- Inbounds (ヒント: 「特定のクライアントを接続するための設定テンプレートの変更」)。
- Outbounds (11.4 参照)。
- バランサー (11.5 参照)。
- ルーティング (ヒント: 「各ルールの優先順位が重要です!」、11.3 参照)。
- DNS / Fake DNS (11.6 参照)。

11.2. 一般設定 (General)

Freedom Protocol Strategy

フィールド	ラベル	説明	デフォルト
FreedomStrategy	Freedom プロトコルストラテジーの設定	直接 (freedom) outbound のネットワーク出力ストラテジー。ヒント: 「Freedom プロトコルにおけるネットワーク出力ストラテジーを設定します」。freedom プロトコルの outbound の settings 内の domainStrategy フィールドを制御します。	基準テンプレートでは、direct という freedom outbound の domainStrategy は AsIs (アドレスは解決されず、元の形式で渡される)。

freedom の domainStrategy (Xray-core の値): AsIs (サーバー側でドメインを解決しない)、および UseIP / UseIPv4 / UseIPv6 ファミリーとその「強制」バリエーション ForceIP* (出力サーバーにドメインを解決させて取得した IP で接続させる)。出力サーバーに IPv6 がない場合や IPv4 のみで接続したい場合は UseIPv4 に変更してください。

Freedom Happy Eyeballs (IPv4/IPv6)

フィールド	ラベル	説明
FreedomHappyEyeballs	Freedom Happy Eyeballs (IPv4/IPv6)	ヒント: 「直接 (freedom) outbound のデュアルスタック接続—IPv4 と IPv6 の両方を持つ出力サーバーで便利です。」 freedom outbound に対して Happy Eyeballs アルゴリズム (両方のアドレスファミリーへの同時試行) を有効にします。

個別の Happy Eyeballs は 特定の outbound の dial 設定 (sockopt) にもあります: 3.5.0 から実際に有効化されるようになりました (以前は設定が無効として保存され、スイッチが「外れて」いました)。「Try delay (ms)」のデフォルトは 250 になりました。明示的に指定した 0 (= 無効) は保持されます。Prioritize IPv6、interleave (1)、maxConcurrentTry (4) も利用できます。|try delay| (ヒント) |「別のアドレスファミリーを試行するまでのミリ秒数。150~250 ms が良い出発点です。」代替アドレスファミリーへの切り替え前の遅延。推奨範囲は 150~250 ms。|

Overall Routing Strategy

フィールド	ラベル	説明	デフォルト
RoutingStrategy	ドメインルーティング戦略の設定	ルーティング用の全体的な DNS 解決ストラテジー。ヒント: 「DNS 解決のグローバルルーティングストラテジーを設定します」。routing.domainStrategy フィールドを制御します。	基準テンプレートでは routing.domainStrategy = AsIs。

routing.domainStrategy は、IP ルーティングルールがドメインリクエストとどのようにマッチするかを決定します: AsIs (ドメインルールのみ、解決なし)、IPIfNonMatch (ドメインがルールにマッチしな

かった場合—解決して IP ルールを確認）、IPOnDemand（IP ルールに遭遇したらすぐに解決）。ドメインリクエストに対して IP ルール（例: geoip:*）を機能させるには、通常 IPIfNonMatch が必要です。

Outbound Test URL

フィールド	ラベル	説明	デフォルト
outboundTestUrl	Outbound テスト用 URL	outbound テスト時に接続確認に使用する URL。ヒント: 「Outbound の接続確認用 URL」。テンプレートとは別に xrayOutboundTestUrl キーのもとに保存されます。	https:// www.google.com/ generate_204

この値はサニタイズされます。実際の outbound テスト時には、さらに公開 URL として検証されます—これは SSRF 対策であり、クライアントが任意の（内部を含む）URL を渡すことを防ぎます。テスト URL は常にサーバーの設定から取得されます。保存/テスト時に空の値はデフォルトの generate_204 に置き換えられます。

Default Outbound (3.5.0 から)

「基本ルーティング」タブの最初の項目は **Default Outbound** セクターです: 「どのルーティングルールにもマッチしなかったトラフィックは、この outbound を使用します（Xray はリストの最初の outbound を使用します）」。リストには direct、blocked、およびすべての既存タグが表示され、デフォルトは direct です。選択すると、指定した outbound がテンプレートの **先頭に移動されます**。存在しない direct / blocked はオンザフライで作成され、切り替え時にも削除されません。

Block BitTorrent

フィールド	ラベル	説明
Torrent	BitTorrent を ブロック	routing.rules に protocol: ["bittorrent"] のトラフィックを blocked outbound に送るルールを追加します。基準テンプレートではこのルールがデフォルトで含まれています。

接続制限 (Connection Limits)

ヒント: 「レベル 0 ユーザーの接続レベルポリシー。Xray のデフォルト値を使用するにはフィールドを空のままにしてください。」これらのパラメーターは policy.levels.0 に書き込まれます。

フィールド	ラベル	説明	デフォルト
connIdle	アイドルタイムアウト (秒)	「指定した秒数のアイドル後に接続を閉じます。値を減らすと、高負荷サーバーでメモリとファイルディスクリプターがより早く解放されます（Xray のデフォルト: 300）。」	空 → Xray デフォルト 300

フィールド	ラベル	説明	デフォルト
bufferSize	バッファサイズ (KB)	「接続ごとの内部バッファサイズ (KB) 。RAM が少ないサーバーでのメモリ使用を最小化するには 0 を設定します (Xray のデフォルト値はプラットフォームによって異なります) 。」 プレースホルダー: 「auto」 。	空 → プラットフォームに依存; 0 → 最小化

例 (`policy.levels.0`) 。 このグループのフィールドはレベル 0 ポリシーに書き込まれます。RAM が少ない高負荷サーバーでは、次のようにリソースの解放を高速化できます:

```
"policy": {
  "levels": {
    "0": {
      "connIdle": 120,
      "bufferSize": 0
    }
  }
}
```

ここでは接続はアイドル後 120 秒で閉じられ (デフォルトの 300 の代わりに) 、 `bufferSize: 0` はバッファのメモリ消費を最小化します。フォームで空のままにしたフィールドは JSON に含まれず、Xray はデフォルト値を適用します。

11.3. ルーティングルール (routing)

`routing.rules` のルール一覧。順序が重要 (「各ルールの優先順位が重要です!」) : ルールは上から下に評価され、最初にマッチしたものが適用されます。ヒント: 「ドラッグして順序を変更」。順序制御ボタン: 先頭へ、末尾へ、上へ移動、下へ移動。

各ルールには `type: "field"` があります。ボタン: ルールを作成、ルールを編集。リストフィールドへのヒント: 「カンマ区切りの要素」。

「ルーティング」ページでは、「ルールをインポート」と「ルールをエクスポート」ボタンが「Outbounds」ページと同様に「more」(その他) ドロップダウンメニューにまとめられています。「ルールをエクスポート」ボタンはすぐにファイルをダウンロードするのではなく、JSON プレビューと「コピー」および「ダウンロード」ボタンを含むモーダルウィンドウを開きます—保存前に内容を確認できます。「Outbounds」ページの outbound エクスポートも同様に動作します。

Route Tester (ルートテスター)

ルーティングタブには **Route Tester** サブタブがあります—実際のトラフィックを送信せずに、特定の接続をどの outbound が処理するかを実行中の Xray に問い合わせます。ドメインまたは IP、ポート、ネットワーク (TCP/UDP) 、必要に応じて inbound と傍受されたプロトコル (`http/tls/quic/bittorrent`) を指定して **Test Route** をクリックします。判定はライブルーティングエンジンから直接取得されます。

応答にはマッチした outbound が表示され、バランサーを使用している場合はバランサータグも表示されます。どのルールもマッチしなかった場合、テスターはトラフィックがデフォルト outbound

（outbounds リストの最初）に流れることを通知します。これはルールの順序を信頼する前に確認するのに便利です。

個々のルールの有効化と無効化

個々のルーティングルールを削除せずに一時的に **無効化** するには、切り替えスイッチを使用します。ルールテーブルには「**有効**」列があり、スイッチ（Switch）があります。ルールフォームにも「**有効**」フィールドがあり、これも切り替えスイッチです。無効化されたルールは最終的な Xray 設定には含まれませんが、テンプレートに保存され、いつでも再度有効化できます。

統計サービスルール（inboundTag: ["api"] → outboundTag: "api"）は無効化できません—そのスイッチはロックされており、パネルのトラフィック統計を壊さないようになっています（[11.11](#) 参照）。

ルールフォームのフィールド

フォームフィールド	ラベル	JSON フィールド	説明
ソース	ソース	source	送信元 IP アドレス/サブネット。カンマ区切りリスト。
ソースポート	ソースポート	sourcePort	送信元ポート。
宛先	宛先	domain + ip + port	宛先ドメイン、IP、ポート。ドメインは domain:、full:、regexp:、keyword: プレフィックスおよび geosite:* をサポート; IP は geoip:* と CIDR。
ネットワーク	—	network	tcp、udp、または tcp,udp。3.4.2 から、設定には小文字（tcp / udp）で書き込まれ、ルートテーブルでは大文字（TCP / UDP）で表示されます; 空 — 任意のネットワーク。
プロトコル	—	protocol	http、tls、bittorrent（sniffing で検出）。
ユーザー	ユーザー	user	ユーザーのメール/識別子によるフィルター。
属性 / 値	属性 / 値	attrs	マッチングに使用する HTTP ヘッダー属性。
VLESS route	VLESS route	—	VLESS の route フィールドによるルーティング。
Inbound タグ	Inbound タグ	inboundTag	ルールが適用される 1 つまたは複数の inbound タグ（組み込みの api や DNS 設定の DNS タグを含む）。inbound リストでは、inbound に別の備考が設定されている場合は「tag (remark)」として表示され、そうでない場合は単にタグとして表示されます。保存されたルールにはタグのみが保存されます。
Outbound タグ	Outbound タグ / 送信接続	outboundTag	マッチしたトラフィックの送信先。
balancer タグ	balancer タグ / balancer	balancerTag	ヒント: 「設定された負荷 balancer の 1 つを通じてトラフィックをルーティングします」。

`outboundTag` と `balancerTag` の相互排他性: 「`balancerTag` と `outboundTag` を同時に使用することはできません。同時に使用した場合、`outboundTag` のみが機能します。」 1 つのルールには `outbound` タグかバランサータグのいずれかを設定してください。

基準テンプレートの組み込みルール

標準の `config.json` では、`routing` セクションに次の 3 つのルールが含まれています（この順序で）：

1. `inboundTag: ["api"]` → `outboundTag: "api"` — パネルの gRPC API 統計のためのサービスルール。
2. `ip: ["geoip:private"]` → `outboundTag: "blocked"` — プライベートアドレス範囲のブロック。
3. `protocol: ["bittorrent"]` → `outboundTag: "blocked"` — BitTorrent のブロック。

`api` → `api` ルールは保存時に常にポジション 0 に自動的に昇格します（[11.11](#) 参照）。これにより、統計リクエストが上位の `catch-all` ルールに「飲み込まれる」ことを防ぎます。

ルールの例。 ロシアのサイトとプライベートネットワークへのすべてのトラフィックをプロキシを迂回して直接送信し、残りをバランサーに送ります。順序が重要: 「直接接続」ルールは `catch-all` の上に置く必要があります。 `routing.rules` 内:

```
{
  "type": "field",
  "domain": ["geosite:category-ru", "domain:example.ru"],
  "ip": ["geoip:ru", "geoip:private"],
  "outboundTag": "direct"
}
```

ドメインリクエストに対して IP ルール（`geoip:ru`）を機能させるには、通常ルーティングのトップレベルで `routing.domainStrategy: "IPIfNonMatch"` が必要です（[11.2](#) 参照）。

事前設定済みルーティンググループ（基本接続）

「基本接続」モードでは、パネルは既製のリストから一般的なルールを組み立てるのに役立ちます：

グループ	フィールド	ヒント
プロトコル/サイトによるブロック	—	「クライアントが特定のプロトコルにアクセスできないように設定します」
国によるブロック	ブロックされた IP アドレス、ブロックされたドメイン	「これらの設定は宛先の国に応じてトラフィックをブロックします。」
直接接続	直接 IP アドレス、直接ドメイン	「直接接続とは、特定のトラフィックが別のサーバーを経由しないことを意味します。」
IPv4 ルール	—	「これらの設定により、クライアントは IPv4 のみを通じて宛先ドメインにルーティングされます」

グループ	フィールド	ヒント
WARP ルール	—	「これらのオプションは、特定の宛先に応じてトラフィックを WARP 経由でルーティングします。」
NordVPN ルーティング	—	「これらのオプションは、特定の宛先に応じてトラフィックを NordVPN 経由でルーティングします。」

MTProto inbound: Xray 経由の Telegram トラフィックルーティング

MTProto inbound には「**Route through Xray**」スイッチ（デフォルトはオフ）と任意の **Outbound** 選択があります。有効にすると、パネルは inbound 自体のタグを持つループバック SOCKS ブリッジを Xray 設定に追加し、mtg はその経由で Telegram トラフィックをルーティングします。その後、送信 Telegram トラフィックはルーターによって制御されます。ルーティングタブで inbound タグによる通常のルールでマッチングするか、**Outbound** フィールドで選択した outbound またはバランサーに強制的に送ることができます。**Outbound** を空のままにすると、ルーティングルールが判定を行います。

11.4. Outbounds（送信接続）

`outbounds` のリスト。ボタン: [送信接続を作成](#)、[送信接続を編集](#)。ヒント: 「このサーバーの送信接続を定義するための設定テンプレートの変更」。

基準テンプレートには 2 つの必須 outbound があります:

- `protocol: "freedom"`、`tag: "direct"` — インターネットへの直接出口（`domainStrategy: "AsIs"` と `finalRules: [{action: "allow"}]` を含む）;
- `protocol: "blackhole"`、`tag: "blocked"` — ブロックされたトラフィックの「ブラックホール」。

outbound フォームの共通フィールド

フィールド	ラベル	説明
タグ	タグ（ヒント: 「一意のタグ」）	outbound の一意の識別子。プレースホルダー: 「 <i>unique-tag</i> 」。検証: 「タグは必須です」、「タグはすでに別の outbound で使用されています」。
プロトコル	—	outbound のタイプ（以下参照）。
アドレス / ポート	アドレス / ポート	接続先。アドレスとポートは必須です。
経由で送信	経由で送信	送信インターフェースのローカル IP アドレス（ <code>sendThrough</code> ）。プレースホルダー: 「ローカル IP」。

フィールド	ラベル	説明
Target Strategy	Target Strategy	3.5.0 の新機能。 接続前に宛先ドメインをどのように解決するか。11 の値: AsIs (デフォルト、解決しない)、 UseIP 、 UseIPv4 、 UseIPv6 、 UseIPv6v4 、 UseIPv4v6 (fallback 付きの解決)、 ForceIP 、 ForceIPv6v4 、 ForceIPv6 、 ForceIPv4v6 、 ForceIPv4 (解決の成功が必須)。空 = AsIs (設定には書き込まれません)。 freedom では、値は従来の domainStrategy への fallback 付きで読み取られます (レガシーキーは古いコア向けに引き続き書き込まれます)。
Dialer proxy (チェーン)	—	ヒント: 「別の <i>outbound</i> (タグで指定) を通じてこの <i>outbound</i> を接続し、プロキシチェーンを構築します。直接接続の場合は空にしてください。」 プレースホルダー: 「チェーン用の <i>outbound</i> を選択」。 streamSettings.sockopt.dialerProxy を通じて実装されます。

Dialer Proxy ドロップダウンリストにはローカル *outbound* だけでなく、サブスクリプションの *outbound* タグも表示されます—サブスクリプションで取得した出口を通じてチェーンを構築できます。リストから **blackhole outbound** と現在編集集中の *outbound* は引き続き除外されます。直接接続の場合はフィールドを空にしてください。

サポートされている outbound プロトコル

フォームでサポートされているプロトコル:

- **freedom** — 直接出口。フィールド **settings.domainStrategy**、**finalRules** (以下参照)、**Happy Eyeballs**。テスト不可 (「*Outbound has no testable endpoint*」)。
- **blackhole** — トラフィックを破棄します。フィールド **レスポンスタイプ**。テスト不可。
- **socks**、**http** — **address / port** を持つ **settings.servers[]** リスト; フィールド **認証パスワード**。**http** プロトコルでは、**Username/Password** フィールドの下に **Headers** (ヘッダー) エディターがあります—上流 HTTP プロキシに送信される **CONNECT** ヘッダーのキー/値ペア。これらのヘッダーは *outbound* を再度開いて保存しても保持されます (以前は失われていました)。注意: 設定レベルのヘッダー (**settings.headers**) のみが適用されます; 個々のサーバーレベルのヘッダーは **xray-core** では無視されます。
- **vmess** — **settings.vnext[]** (**address / port**)。
- **vless** — **settings.address / settings.port**。
- **trojan**、**shadowsocks** — **settings.servers[]**。
- **wireguard** — **endpoint** を持つ **settings.peers[]**、プラスキー (11.8 参照)。
- **hysteria** — **settings.address / settings.port** (UDP トランスポート)。

loopback タイプの *outbound* では、*inbound* と同じパラメーターを持つ **Sniffing** ブロックが利用可能です: 有効化、**destOverride**、**Metadata Only**、**Route Only**、および除外ドメインのリスト。

Hysteria2 の **UDP** マスク (**FinalMask**) では追加モードが利用可能です。**Salamander** マスクには **Mode** セレクターがあり、値は **Salamander** と **Gecko** です: **Gecko** モードは **Min/Max** サイズフィールド (**packetSize**、範囲 1–2048、デフォルト 512–1200) を持つランダムパケットパディングを追加します—これにより、パケット長によるフィンガープリンティングを防ぎます。**Realm** マスク (UDP ホールパンチング) には任意の **TLS Config** ブロックが追加され、フィールドは **Server Name** (SNI)、**ALPN** (**h3 / h2 / http/1.1**)、**Fingerprint** (uTLS)、および **Allow Insecure** スイッチです。

3.5.0 から、新しい TCP マスク タイプ **XMC (Minecraft)** (`xmc`) が追加されました: ストリームを Minecraft プロトコルに偽装します。フィールド: **Hostname** (任意; 「ハンドシェイクで模倣されるサーバーアドレス」)、**Usernames** (任意のリスト; 「プロローバーに提示されるプレイヤー名; デフォルトではコアは Dream を使用します」)、および必須の **Password** (16 文字、`u` ボタンによる自動生成; 「難読化パスワード」)。重要: **TCP Finalmask** と **REALITY** の組み合わせは保存時に拒否されます – 初回接続時に Xray-core をクラッシュさせていました (エラー: 「Finalmask is not supported with REALITY security...」; Xray-core#6453 参照)。REALITY との UDP マスクは許可されます。以前に保存された不正な組み合わせは、設定の生成時に自動的に「修復」されます (マスクが破棄されます)。

例: 上流 SOCKS 経由のチェーン。 `outbound upstream` は外部 SOCKS5 プロキシに接続し、`chained` は `dialerProxy` を通じてそのトラフィックを経由させ、チェーンを形成します。 `outbounds` 内:

```
[
  {
    "tag": "upstream",
    "protocol": "socks",
    "settings": {
      "servers": [{ "address": "203.0.113.10", "port": 1080 }]
    }
  },
  {
    "tag": "chained",
    "protocol": "freedom",
    "streamSettings": {
      "sockopt": { "dialerProxy": "upstream" }
    }
  }
]
```

これにより、`outboundTag: "chained"` を持つルーティングルールは `upstream` 経由でトラフィックをインターネットに送出します。

共有リンクからの outbound インポート

outbound は共有リンク (`vless://`、`vmess://` など) からインポートできます。インポート時には、リンクの `extra=` ブロックで渡された **xmux** (XHTTP) マルチプレクサーの設定も保持されます: インポート後、その値は作成された outbound の **XMUX** サブフォームに入力されます。

Mux (多重化) フィールド

最大並列数、最大接続数、最大再利用数、最大リクエスト数、最大再利用秒数、キープアライブ期間。これらのパラメーターは outbound の mux/XUDP 動作を設定します。

Sockopts (ソケット設定)

Sockopts グループ: キープアライブ間隔、Mark (`fwmark`)、インターフェース、IPv6 のみ、**proxy protocol** を受け入れる、**Proxy protocol**、TCP ユーザータイムアウト (ms)、TCP キープアライブアイドル (s)。ここでチェーンの dialer proxy も設定します。

Freedom finalRules（プライベート IP ブロックのオーバーライド）

freedom outbound には ファイナルルール グループが利用可能です:

フィールド	ラベル	説明
overrideXrayPrivateIp	Xray のデフォルトプライベート IP ブロックをオーバーライド	プライベート IP への送信に関する Xray の組み込み禁止を解除します。
action	アクション	allow（基準テンプレートのように: finalRules: [{action: "allow"}]）、redirect（Redirect）、その他。
blockDelay	ブロック遅延（ms）	接続を破棄する前の遅延。
redirect / fragment	Redirect / Fragment	トラフィックのリダイレクトとフラグメンテーションアクション。

fragment マスク: フラグメントごとの Lengths と Delays

fragment マスク（FinalMask の fragment タイプ、TCP 用）では、単一の Length と Delay フィールドが Lengths と Delays のリストに置き換えられています: 各セグメントに個別の長さの範囲（例: 100-200）とミリ秒単位の遅延（例: 10-20 または 0）を設定できます。リスト行は追加・削除可能; 以前に保存された単一の値は自動的に 1 要素の配列に移行されます。

その他のフォームフィールド

- UDP over TCP と UoT バージョン --shadowsocks 系プロトコル用。
- gRPC ヘッダーなし、アップリンクチャンクサイズ --gRPC トランスポートパラメーター。
- TLS/uTLS フィールド: ピア名の検証、Pinned SHA256、Short ID、Vision testpre、サーバー名プレースホルダー。

outbound のテスト

ボタン: テスト、すべてテスト。状態: 接続をテスト中...、テスト成功、テスト失敗、outbound 接続をテストできませんでした。結果: テスト結果、ミリ秒単位の遅延。

2 つのモード（ヒント: 「TCP: 高速な dial のみのプローブ。HTTP: xray 経由の完全なリクエスト。」）:

- TCP（mode=tcp）-- host:port へのシンプルな dial、すべてのエンドポイントを並列に実行、タイムアウト約 5 秒。TCP 到達可能性のみを確認し、プロキシプロトコルは検証しません。freedom / blackhole / blocked タグに対しては「Outbound has no testable endpoint」を返します。
- HTTP（mode=http またはなし）-- 一時的な Xray インスタンスを起動し、実際の HTTP リクエスト（プローブ URL = サーバーの outboundTestUrl）を実行し、実際の遅延を測定します。信頼性が高いが重いモード: グローバルロックによってシリアル化されます（「別の outbound テストがすでに実行中です、お待ちください」）。1 回の試行のタイムアウトは 10 秒、結果待機ウィンドウは 15 秒（低速またはトンネル接続で正常な outbound が「Failed」とマークされないよう延長されています）

す)。失敗した場合、実際の原因 (DNS エラー、connection refused、デッドライン超過、TLS エラーなど) はパネル/Xray のログに書き込まれ、一般的なタイムアウトメッセージはそこを参照します。

- **Real delay (3.5.0 の新機能)** — 切り替えの 3 番目のモード。同じ一時的な Xray インスタンスを使用しますが、トンネル確立を含む「コールド」リクエストの合計時間を報告します — クライアントアプリが表示する数値に近い値です。切り替えのヒント: 「TCP: 高速な dial プローブ。HTTP: xray 経由の完全なリクエスト。Real delay: 接続確立を含む合計時間」。通常の **HTTP** モードは 3.5.0 から「ウォーム」(keep-alive) 接続で測定されるため、その数値はより低く、単一リクエストの遅延に近くなります。

UDP プロトコル (wireguard、hysteria) と UDP トランスポート (kcp、quic、hysteria) は、TCP が要求された場合でも常に HTTP モードでテストされます — 素の UDP dial は「生きている」エンドポイントと「死んでいる」エンドポイントを区別できません。wireguard のテスト設定では `noKernelTun: true` が強制的に設定されます。

Egress と Country 列 (3.5.0 から)

HTTP または **Real delay** テストの成功後、パネルは同じ一時ルート経由で Cloudflare trace のメタデータを取得し、outbound リストに 2 つの列を表示します: **Egress** — 出口 IP (IPv4/IPv6、それぞれ「目」アイコンの背後に隠されています; テストがまだの場合はダッシュと「出口 IP と国を表示するには HTTP テストを実行してください」というヒント)、および **Country** — 出口の国旗と国名です; 出口が Cloudflare WARP 経由の場合はオレンジ色の WARP ラベルが追加されます。TCP テストではこれらの列は埋まりません。モバイルではデータは outbound カードに表示されます。

バッチ確認とフェーズ別内訳

HTTP モードの **テスト** と **すべてテスト** は、outbound のバッチに対して 1 つの共通一時 Xray インスタンスを起動し、各 outbound にループバック SOCKS inbound とルールを作成して、その経由で実際の HTTP リクエストを並列送信します; **すべてテスト** は outbound をバッチで確認します。**すべてテスト** はサブスクリプションから取得した outbound (読み取り専用の「サブスクリプションから」テーブル) もテストします — それらの行もテスト結果でハイライトされます。この際、`freedom` (「direct」) と `dns` の outbound はいずれのモードでもテストされません (これらはプロキシではないため): それらのテストボタンは利用不可であり、**すべてテスト** はスキップし、サーバー側の保護は直接 API 呼び出し時でも HTTP テストを禁止します。成功/エラーに加えて、ポップアップ結果は HTTP レスポンスステータスとフェーズ別の時間内訳を表示します: **Proxy connect** (プロキシへの接続)、**TLS via outbound** (outbound 経由の TLS)、**First byte** (最初のバイトまでの時間) — これにより、どのステップで遅延や障害が発生したかを理解できます。

outbound のトラフィック統計

パネルはタグごとのトラフィックカウンター (up / down / total) を管理します。リセットボタンは特定のタグまたはすべて (tag = "-alltags-") のカウンターをリセットします。**アカウント情報** と **Outbound 接続ステータス** フィールドはサマリーを表示します。

11.5. バランサー (Balancers)

`routing.balancers` のリスト。ボタン: [バランサーを作成](#)、[バランサーを編集](#)。

バランサータブにはライブ状態の列があります: **Live Target** は実行中の Xray でのバランサーの現在のアクティブターゲットを表示し、**Override** は手動でターゲットの選択をオーバーライドできます (**Auto (strategy)** 値はストラテジーによる選択に戻します)。状態は個別のボタンで更新されます。バランサーが実行中の Xray でまだアクティブでない場合、パネルは最初に変更を保存するか Xray を起動するよう提案します。

フィールド	ラベル	説明
タグ	タグ (ヒント: 「一意のタグ」)	一意の識別子。プレースホルダー: 「 <i>unique balancer tag</i> 」。検証: 「タグは必須です」、「タグはすでに別のバランサーで使用されています」。
セクター	セクター	バランサーが出口を選択する outbound タグのリスト (部分文字列マッチ)。少なくとも 1 つ選択する必要があります: 「少なくとも 1 つの <i>outbound</i> を選択してください」。
Fallback	Fallback	セクターがマッチしなかった場合の予備タグ。3.5.0 から、別のバランサーも選択できます (リストではそのような候補に青い「Balancer」ラベルが付き、ヒントは「別のバランサーを fallback として選択」)。Xray はネイティブにはこれをサポートしないため、パネルが非表示の loopback outbound <code>_bl_<ターゲット></code> とルーティングルールを自動的に構築します (経路: バランサー → loopback → サーバー → ターゲットバランサー → outbound; 余分な「ホップ」に関する警告がフォームに表示されます)。循環参照からの保護 (エラー「選択できません - 循環依存が発生します」、リスト内の候補は循環経路のヒント付きでブロックされます) と、使用中のバランサーの削除からの保護 (「削除できません - 次の fallback として使用されています: ...」) があります。タグのプレフィックス <code>_bl_</code> は予約されています (タグでは禁止)。サービス用の <code>_bl_</code> オブジェクトは Outbounds/Routing のリストから非表示になり、Fallback/Live Target/Override 列は loopback タグではなくターゲットバランサーの名前を表示します。
ストラテジー	ストラテジー	選択アルゴリズム (以下参照)。

ストラテジーと観測パラメーター

ストラテジー (`strategy.type`) は、バランサーがセクターから outbound を選択する方法を決定します。Xray-core の値: `random` (ランダム)、`roundRobin` (ラウンドロビン)、`leastPing` (observatory の結果による最小遅延)、`leastLoad` (最小負荷)。`leastLoad / leastPing` では `strategy.settings` のパラメーターが使用されます:

フィールド	ラベル	説明
<code>expected</code>	期待値	プレースホルダー: 「最適なノード数」 -- 目標とするアクティブノード数。
<code>maxRtt</code>	最大 RTT	候補選択時の許容最大 RTT 上限。
<code>tolerance</code>	許容差	遅延/負荷を比較する際の許容差。
<code>baselines</code>	Baselines	ノードのグルーピング用の遅延しきい値。

フィールド	ラベル	説明
costs	Costs	個々のタグの重み係数（cost）。

ストラテジーの例。 `strategy` ブロックはバランサー内に存在します（JSON では `tag` と `selector` の隣）：

```
"strategy": { "type": "random" }      // セレクターからランダムに選択
"strategy": { "type": "roundRobin" }  // 順番に
"strategy": { "type": "leastPing" }   // 最小遅延（オブザーバーが必要）
```

`leastLoad` のパラメーターは `settings` で設定します：

```
"strategy": {
  "type": "leastLoad",
  "settings": {
    "expected": 2,
    "maxRTT": "1s",
    "tolerance": 0.05,
    "baselines": ["500ms", "1s", "2s"],
    "costs": [
      { "regex": false, "match": "proxy-premium", "value": 0.1 },
      { "regex": true, "match": "^proxy-cheap-.+$", "value": 5 }
    ]
  }
}
```

動作の仕組み（例）。オブザーバーが各出口の遅延を測定した結果: `A = 250 ms`、`B = 280 ms`、`C = 700 ms`、`D = 1500 ms`。上記の設定での選択の流れ：

1. **maxRTT: "1s"** — 遅延が1秒を超える出口は除外されます: `D` (1500 ms) が脱落。残るのは `A`、`B`、`C`。
2. **baselines + expected** — 出口は遅延しきい値でグルーピングされ、`expected` 以上の出口が含まれる最小しきい値が選択されます。しきい値 `500ms` にはすでに `A` と `B` が含まれており、これは2 (= `expected`) なので、グループ { `A`、`B` } が選択されます。`C` (700 ms) は速い出口が十分にある限り選択に入りません（これは「ホットスタンバイ」です）。
3. **tolerance: 0.05** — 選択されたグループ内で、遅延が5% 以内の差に収まる出口は同等とみなされ、負荷が均等に分散されます。`A` (250) と `B` (280) の差は約12% (> 5%) なので、他の条件が等しければより速い `A` が優先されます; 差が5% 以内であれば、`A` と `B` の両方にトラフィックが流れます。
4. **costs** — 比較前に個々の出口の「コスト」を調整します: 値が小さいほど出口は魅力的になり、大きいほど逆です。例では `proxy-premium` は `0.1` (「安く」なり、より積極的に選択される) を取得し、すべての `proxy-cheap-*` (正規表現、`regex: true`) は `5` (「高く」なり、最後に使用される) を取得します。これにより、出口を完全に除外することなく優先順位をソフトに設定できます。

結果: トラフィックは主に `A` を通じて流れ（遅延が近い場合は `B` と均等に）、`C` はスタンバイのまま、`D` は RTT が `maxRTT` を下回るまで除外されます。

オブザーバー: observatory と burstObservatory (leastPing / leastLoad 用の計測)

ストラテジー `leastPing` と `leastLoad` は自ら何も測定しません—各 outbound の遅延と可用性に関するデータが必要です。これらを収集するのがオブザーバー (observatory) です: 定期的に各監視対象 outbound を「ピング」し、応答時間と可用性を記録します。同じデータが「**Observatory**」タブに表示されます (状態 **アクティブ** / 利用不可、「最後のアクティビティ」、「最後の試行」)。

バージョン3.4.2 から「 balancer 」ページは「 balancer 設定 」 (テーブルと追加) タブと「**Observatory**」タブに分割され、オブザーバーは **構造化フォーム** で編集します (従来の生の JSON エディターは削除されました)。パネルは必要に応じてオブザーバーを自動的に作成・削除します: 通常の `observatory` は **Least Ping** ストラテジー用、拡張版の `burstObservatory` は **Least Load** 用、および `fallbackTag` を指定した **Random/Round-robin** 用です (これは xray-core が要求します; 最後の `fallbackTag` をクリアするとオブザーバーは削除されます)。オブザーバーの作成または削除には Xray の完全な再起動が必要です。

2つのオプションが利用可能です:

- **observatory** — シンプル: `subjectSelector` + `probeURL` + `probeInterval`。
- **burstObservatory** — 拡張版、`pingConfig` による詳細なピング設定; 複数の出口に便利。

`burstObservatory` ブロックの例:

```
{
  "subjectSelector": ["WS-SE", "WS-FR", "WS-PL"],
  "pingConfig": {
    "destination": "https://www.google.com/generate_204",
    "interval": "1m",
    "connectivity": "http://connectivitycheck.platform.hicloud.com/generate_204",
    "timeout": "5s",
    "sampling": 2
  }
}
```

フィールドの説明:

フィールド	設定内容
<code>subjectSelector</code>	監視対象 outbound の タグプレフィックス のリスト。Xray は指定された文字列で始まるタグを持つすべての outbound を取得します。例では <code>WS-SE...</code> 、 <code>WS-FR...</code> 、 <code>WS-PL...</code> が監視されます。これらのタグは balancer の セクター で選択されたものと一致する必要があります。
<code>pingConfig.destination</code>	遅延測定のために各 outbound 経由でリクエストされる URL。本文なしで <code>204</code> 応答を返す「軽い」ページを使用します—例: <code>https://www.google.com/generate_204</code> 。応答までの時間が測定遅延です。
<code>pingConfig.interval</code>	各 outbound をピングする頻度。期間文字列: <code>"1m"</code> は 1 分ごと、 <code>"30s"</code> 、 <code>"5m"</code> なども可。頻度が高いほどデータが新鮮ですが、バックグラウンドトラフィックが増えます。

フィールド	設定内容
pingConfig.connectivity	(任意) サーバー自体の 基本接続性 チェック URL。アクセスできない場合――サーバーネットワークの問題を意味し、オブザーバーは outbound を利用不可とは マークしません (ローカル障害時の誤検知を防ぐ)。通常、 204 応答のエンドポイント。
pingConfig.timeout	1 回のピングの応答待機時間 (例: "5s") ―この時間を超えると試行は失敗とみなされます。
pingConfig.sampling	各 outbound に対して保持・平均化する最新測定数。2 は最後の 2 回のピングを考慮します (ランダムなスパイクを平滑化)。
pingConfig.httpMethod	3.4.2 で追加。プローブの HTTP メソッド: HEAD (デフォルト) または GET。

「**Observatory**」タブでは、同じパラメーターを (JSON ではなく) フォームで設定します。通常の observatory の場合、これは **Watched Outbounds** (subjectSelector、読み取り専用)、**Probe URL** (probeURL、デフォルト `https://www.google.com/generate_204`)、**Probe Interval** (probeInterval、1m)、**Enable Concurrency** (enableConcurrency、デフォルトで有効) です。burstObservatory の場合は、上記のフィールドに加えて新しい「**HTTP メソッド**」があります。 balancer が両方のオブザーバーを持つ場合、Observatory と Burst のフォームの切り替えはセグメントスイッチで行います。

すべてを連携させる方法:

1. 「**Observatory**」タブでオブザーバーのパラメーターを設定します (選択したストラテジーに応じて自動的に作成されます; subjectSelector は balancer のセクターに従います)。
2. balancer を作成します: **ストラテジー** = leastPing、**セクター**に同じ outbound タグ (WS-SE、WS-FR、WS-PL) を指定します。
3. ルーティングルールでそこにトラフィックを向けます (**balancer タグ**フィールド、[11.3](#) 参照)。
4. Xray を再起動します。「**Observatory**」タブに出口のステータスが表示され、balancer は生きている最速のものを選択し始めます。

1 つのルールで balancerTag と outboundTag を同時に設定することはできません――outboundTag のみが機能します。

outbound または balancer を削除したときの動作 (3.4.2 から)

outbound または balancer を削除すると、パネルは同じ操作で **ルーティング内のそれへの参照をクリーンアップ** し、確認ダイアログに **影響のプレビュー** を表示します (タイトル「削除すると、ルーティングも更新されます:」、続いて各ルールについて「削除 (宛先が残っていません)」または「保持 (現在は ... を使用)」、そして「balancer ... は削除されました (ターゲットが残っていません)」)。これにより、以前は次の再起動時に xray-core が起動できなくなる原因となっていた「宙ぶらりんの」参照 (balancerTag / outboundTag / dialerProxy) が防止されます。balancer の削除: ルールは自身の outboundTag を保持し、それがなければ削除されます。outbound の削除: そのタグは balancer のセクターと fallbackTag から除去され、他の outbound の dialerProxy がクリアされ、空になった balancer は対象を失ったルールとともに削除され、オブザーバーは再構築されます。

11.6. DNS

`dns` セクション。有効化: **DNS を有効にする**（ヒント: 「内蔵 DNS サーバーを有効にします」）。

3.5.0 から、プライベート IP 上の DNS サーバー（例: 同じ docker ネットワーク内の自分の AdGuard/Pi-hole）は「箱から出してすぐ」動作します: パネルは、ブロックする `geoip:private` ルールの前に管理された許可ルール（DNS サーバーのポートに限定された `direct`）を自動的に挿入・維持し、`dns.servers` の変更時にそれを同期します。以前は Xray 自身の DNS リクエストがこのルールによって通知なくブロックされ、新しいドメインごとに約 4 秒の遅延が追加されていました。手動ルールはもう不要です（手動で作成されたものは変更されません）。

DNS 一般パラメーター

フィールド	ラベル	JSON	説明 / ヒント
<code>tag</code>	DNS タグ名	<code>dns.tag</code>	「このタグはルーティングルールで <i>inbound</i> タグとして利用可能になります。」 DNS リクエスト自体を <code>inboundTag</code> 経由でルーティングできます。
<code>clientIp</code>	クライアント IP	<code>dns.clientIp</code>	「DNS リクエスト中に指定された IP の場所をサーバーに通知するために使用されます」（EDNS Client Subnet）。
<code>strategy</code>	クエリストラテジー	<code>dns.queryStrategy</code>	「ドメイン名解決のグローバルストラテジー」。値: <code>UseIP</code> 、 <code>UseIPv4</code> 、 <code>UseIPv6</code> 。
<code>disableCache</code>	キャッシュを無効にする	<code>dns.disableCache</code>	「DNS キャッシュを無効にします」。
<code>disableFallback</code>	フォールバック DNS を無効にする	<code>dns.disableFallback</code>	「フォールバック DNS リクエストを無効にします」。
<code>disableFallbackIfMatch</code>	マッチ時のフォールバック DNS を無効にする	<code>dns.disableFallbackIfMatch</code>	「DNS サーバーのドメインリストがマッチした場合のフォールバック DNS リクエストを無効にします」。
<code>enableParallelQuery</code>	並列クエリを有効にする	—	「より高速な解決のために複数のサーバーへの並列 DNS クエリを有効にします」。

フィールド	ラベル	JSON	説明 / ヒント
useSystemHosts	システム Hosts を使用	dns.useSystemHosts	「インストールされたシステムの hosts ファイルを使用します」。

dns ブロックの例。 Google ドメインへのリクエストは Cloudflare の DoH サーバー経由で解決され、それ以外は 1.1.1.1 経由; Google リクエストではプライベートでない IP のみが期待されます。設定のトップレベルで:

```
"dns": {
  "tag": "dns-inbound",
  "queryStrategy": "UseIPv4",
  "servers": [
    {
      "address": "https://cloudflare-dns.com/dns-query",
      "domains": ["geosite:google"],
      "expectIPs": ["geoip:!private"]
    },
    "1.1.1.1"
  ]
}
```

文字列サーバー（"1.1.1.1"）はフィールドなしで、他のすべてのドメインのデフォルトサーバーです。タグ dns-inbound は、DNS リクエスト自体を必要な outbound 経由でルーティングするために、ルーティングルールで inboundTag として使用できます。

古いレコードのキャッシュ

フィールド	ラベル	説明
serveStale	古いものを使用	「バックグラウンド更新中にキャッシュから古い結果を返します」。
serveExpiredTTL	古いものの TTL	「古いキャッシュレコードの有効期限（秒）; 0 = 無制限」。

DNS サーバー（dns.servers リスト）

ボタン: **DNS を作成**、**DNS を編集**、**すべて削除**（確認: 「すべての DNS サーバーがリストから削除されます。このアクションは元に戻せません。」）。テンプレート: **テンプレートを使用**、**DNS テンプレートウィンドウ**（ファミリー プリセットを含む）。

DNS サーバーレコード（Fake DNS レコードと同様）で **DNS を編集** をクリックすると、編集ウィンドウにはデフォルト値ではなく保存されたサーバーの値が入力されます。

DNS サーバーのフィールド:

フィールド	ラベル	説明
address	—	DNS アドレス（IP、DoH URL、localhost、fakedns など）。
domains	ドメイン	このサーバーを使用するドメインのリスト。

フィールド	ラベル	説明
expectIPs	期待 IP	IP がリストに含まれる場合のみ応答を受け入れます。
unexpectedIPs	非期待 IP	指定した IP を含む応答を破棄します。
skipFallback	フォールバックをスキップ	このサーバーをフォールバックとして使用しません。
finalQuery	ファイナルクエリ	サーバーをチェーンの最終サーバーとしてマークします。
timeoutMs	タイムアウト (ms)	サーバーへのリクエストタイムアウト。

Hosts (静的レコード)

Hosts グループ (`dns.hosts`)。ボタン **Host を追加**; 空の状態 **Host が定義されていません**。フィールド: ドメイン (プレースホルダー: 「ドメイン (例: `domain.example.com`) 」) と値 (プレースホルダー: 「IP またはドメイン—入力して `Enter` を押してください」)。

DNS ログ

[11.10 参照](#): ログセクションの **DNS ログ フラグ** (`dnsLog`)。

11.7. Fake DNS

`fakedns` セクション。ボタン: **Fake DNS を作成**、**Fake DNS を編集**。

フィールド	ラベル	説明
ipPool	IP プールサブネット	偽の IP が割り当てられる CIDR 範囲 (例: <code>198.18.0.0/15</code>)。
poolSize	プールサイズ	循環プールに保持するアドレス数。

Fake DNS は inbound での sniffing と組み合わせて使用します: コアはクライアントに偽の IP を発行し、ドメイン↔IP の対応を記憶し、ルーティング時にドメインを復元します。Fake DNS が機能するには、アドレス `fakedns` を持つ DNS サーバーを DNS サーバーリストに追加する必要があります。

例: Fake DNS + DNS サーバーの連携。まず偽アドレスのプールを設定し、次に DNS サーバー `fakedns` を追加して、ドメインリクエストがこのプールから IP を取得するようにします:

```
"fakedns": [
  { "ipPool": "198.18.0.0/15", "poolSize": 65535 }
],
"dns": {
  "servers": [
    { "address": "fakedns", "domains": ["geosite:geolocation-!cn"] },
    "1.1.1.1"
  ]
}
```

さらに inbound では `destOverride: ["fakedns"]` を持つ sniffing を有効にする必要があります。そうしないと、コアは実際のドメインを復元するための情報を持ちません。

11.8. WireGuard / WARP / NordVPN

WireGuard フィールド (`wireguard`)

フィールド	ラベル	説明
<code>secretKey</code>	シークレットキー	ローカルインターフェースの秘密鍵。
<code>publicKey</code>	パブリックキー	ピアの公開鍵。
<code>psk</code>	共有鍵	PreShared Key (任意)。
<code>allowedIPs</code>	許可された IP アドレス	トンネルにルーティングされる範囲。
<code>endpoint</code>	エンドポイント	ピアの <code>host:port</code> 。
<code>domainStrategy</code>	ドメインストラテジー	WireGuard outbound の解決ストラテジー。

Cloudflare WARP (`warp`)

統合は API `https://api.cloudflareclient.com/v0a4005` (client-version `a-6.30-3596`) を使用します。コントローラーのアクション (`/xray/warp/:action`): `config`、`reg`、`license`、`data`、`del`。

手順:

1. **WARP アカウントを作成** → `reg`: パネルは秘密鍵 (`privateKey`) と公開鍵 (`publicKey`) を生成/受け取り、Cloudflare にデバイスを登録し、`access_token`、`device_id`、`license_key`、`private_key` (および `client_id`) を `warp` 設定に保存します。
2. **WARP / WARP+ ライセンスキー** → `license`: 26 文字の WARP+ キーを設定します (プレースホルダー: 「26 文字の WARP+ キー」)。エラー時: 「WARP ライセンスの設定に失敗しました。」 設定がまだ取得されていない場合: 「最初に WARP 設定を取得してください。」
3. **アカウント情報**: デバイス名、デバイスモデル、デバイス有効、アカウントタイプ、ロール、**WARP+ data**、クォータ、使用量。
4. **outbound を追加** → 取得したキーと Cloudflare エンドポイントを使って WireGuard outbound を作成します。
5. **アカウントを削除** → `del`: 保存された WARP データをクリアします。

NordVPN (`nord` / `nordvpn`)

統合は NordLynx (= WireGuard) を使用します。コントローラーのアクション (`/xray/nord/:action`): `countries`、`servers`、`reg`、`setKey`、`data`、`del`。

手順:

1. **アクセストークン** → `reg`: パネルは `api.nordvpn.com` から NordLynx 認証情報を要求し、`nordlynx_private_key` を抽出します。 `private_key` と `token` を `nord` 設定に保存します。代替手段 → `setKey`: 秘密鍵を直接入力 (空にはできません)。
2. **国** → `countries` で国のリストを読み込みます; **都市** (または**すべての都市**)。

3. サーバー → `servers` で選択した国のサーバーを読み込みます（`countryId` はインジェクション対策として数値として検証されます）。フィルター: 負荷 > 7% のサーバーのみ表示。サーバーが見つからない場合: 「選択した国のサーバーが見つかりません」。サーバーに NordLynx 公開鍵がない場合: 「選択したサーバーは NordLynx 公開鍵を報告していません。」
4. outbound の作成/更新: トースト 「NordVPN outbound が追加されました」 / 「NordVPN outbound が更新されました」。

IPv4 優先とユーザースペース TUN

WARP と NordVPN ウィザードで生成された WireGuard outbound は、`ForceIP` の代わりに `domainStrategy: "ForceIPv4v6"`（IPv6 のみのホストでは IPv6 にフォールバックする IPv4 優先）を使用します—これにより、Cloudflare エンドポイントの AAAA レコードが選択された場合に半分設定された IPv6 を持つホストでハンドシェイクが「フリーズ」する問題が解消されます。さらに、カーネル TUN（`noKernelTun: true`）の代わりにユーザースペース TUN が有効化されます: カーネル TUN は権限と fwmark ルーティングを必要とし、多くの VPS でサイレントに失敗しますが、パネルの組み込み接続チェックは常にユーザースペース TUN 経由でテストします—これにより実際のトラフィックとチェックが同じパスを通ります。この変更は新しく追加またはリセットされた outbound にのみ適用されます;すでに保存されたテンプレートはその設定を保持します。

11.9. リバースプロキシと TUN

Reverse（リバースプロキシ）

Xray 設定の `reverse` セクション。outbound フォームにはリバースプロキシタイプへの切り替えがあります。ボタン: リバースプロキシを作成、リバースプロキシを編集。

フィールド	ラベル	説明
タイプ	タイプ	Bridge または Portal —Xray リバースプロキシの 2 つのロール。
ドメイン	ドメイン	bridge [?] portal ベアのサービスドメインラベル。
タグ/接続	タグ/接続	bridge と portal を連携させるタグ。
Reverse Tag	リバースプロキシタグ	ヒント: 「シンプルな VLESS リバースプロキシの <i>outbound</i> タグ。無効にするには空にしてください。」 プレースホルダー: 「 <i>outbound</i> タグ (空 = 無効)」 。簡略化された VLESS リバースを実装します。

outbound フォームにはバックフローフィールドもあります: バックフロー **sniffing**、ワーカー、予約済み、最小ロード間隔 (ms)、最大ロードサイズ (バイト)。

TUN (tun)

フィールド	ラベル	説明	デフォルト
name	—	「TUN インターフェース名。」	xray0
mtu	—	「最大転送単位。データパケットの最大サイズ。」	1500
userLevel	ユーザーレベル	「この inbound ストリームを通じて確立されたすべての接続はこのユーザーレベルを使用します。」	0

11.10. ログと統計 (Stats, metrics)

ログ (log)

ヒント: 「ログはサーバーの動作を遅くする可能性があります。必要な場合にのみ必要なログの種類を有効にしてください! 」 基準テンプレートの log セクション: `access: "none"`、`error: ""`、`loglevel: "warning"`、`dnsLog: false`、`maskAddress: ""`。

フィールド	ラベル	JSON	説明	デフォルト
logLevel	ログレベル	loglevel	「エラーログのジャーナルレベル...」 値: <code>debug</code> 、 <code>info</code> 、 <code>warning</code> 、 <code>error</code> 、 <code>none</code> 。	warning
accessLog	アクセスログ	access	「アクセスログファイルのパス。特殊値 <code>none</code> はアクセスログを無効にします。」	none
errorLog	エラーログ	error	「エラーログファイルのパス。特殊値 <code>none</code> はエラーログを無効にします。」	"" (デフォルト)
dnsLog	DNS ログ	dnsLog	「DNS クエリログを有効にします」	false
maskAddress	アドレスマスキング	maskAddress	「有効にすると、実際の IP アドレスがログでマスクアドレスに置き換えられます。」	"" (無効)

統計 (stats / policy)

統計グループ。 `policy.system` と `policy.levels` のカウンターを有効にします。基準テンプレートでは: `statsInboundUplink: true`、`statsInboundDownlink: true`、`statsOutboundUplink: false`、`statsOutboundDownlink: false`; レベル 0 では— `statsUserUplink: true`、`statsUserDownlink: true`。

フィールド	ラベル	説明	デフォルト
statsInboundUplink	Inbound アップリンク統計	「すべての inbound プロキシの送信トラフィックの統計収集を有効にします。」	true

フィールド	ラベル	説明	デフォルト
<code>statsInboundDownlink</code>	Inbound ダウンリンク統計	「すべての <i>inbound</i> プロキシの受信トラフィックの統計収集を有効にします。」	true
<code>statsOutboundUplink</code>	Outbound アップリンク統計	「すべての <i>outbound</i> プロキシの送信トラフィックの統計収集を有効にします。」	false
<code>statsOutboundDownlink</code>	Outbound ダウンリンク統計	「すべての <i>outbound</i> プロキシの受信トラフィックの統計収集を有効にします。」	false

クライアントと inbound のトラフィック統計（アップリンク/ダウンリンク）はダッシュボードとクライアントのトラフィック表示の基盤です; 無効化することは推奨されません。Outbound 統計はデフォルトで無効であり、outbound タグ別にトラフィックを追跡する場合にのみ必要です。

Metrics

基準テンプレートには `metrics` セクション（`listen: "127.0.0.1:11111"`、`tag: "metrics_out"`）と対応する API `metrics_out` が含まれています。パネルはこのリスナーをメトリクスと observatory スナップショットの収集に使用します: テンプレートから `metrics.listen` を解析し、`/debug/vars` をポーリングして、タグ別の遅延履歴を集計します。`metrics.listen` のアドレス/ポートを変更すると、パネルは新しいアドレスにアクセスします; `metrics` セクションを削除すると observatory グラフの収集が無効になります。

HTTP モードでの outbound テストは、ランダムポートに独自の `metrics` リスナーを持つ別の一時的な Xray インスタンスを起動します—これはメイン設定のリスナーとは異なります。

11.11. 保存、再起動、および自動変換

ボタン

ボタン	アクション
保存	POST <code>/xray/update</code> : テンプレート + <code>outboundTestUrl</code> を検証して保存します。
Xray を再起動	保存された設定でサービスをリロードします。確認: 「 <i>xray</i> を再起動しますか?」 / 「保存された設定で <i>xray</i> サービスをリロードします。」

トースト: 成功—「*Xray* が正常に再起動されました」、「*Xray* が正常に停止されました」; エラー—「*Xray* の再起動中にエラーが発生しました。」、「*Xray* の停止中にエラーが発生しました。」 **Xray 再起動** 出力ウィンドウにはコアの診断出力が表示されます。

ホット適用（完全な再起動なし）

inbounds、outbounds、ルーティングルールへの変更は「ライブ」で適用されます: 保存をクリックすると、パネルは古い設定と新しい設定の差分を計算し、gRPC API Xray (HandlerService/RoutingService) を通じて変更された部分のみを適用します（プロセスを再起動しません）。完全な再起動は、ホットリロード API のないセクション（log、dns、policy、observatory など）が変更された場合にのみ自動的に実行されます。したがって、Xray ページに個別の「再起動」ボタンは不要です—保存自体が変更を適用します。必要に応じたコアの再起動は引き続き自動的に実行されます（サブスクリプション更新時の自動リロードと WARP ローターションも参照）。

デフォルトテンプレートの復元

エンドポイント `GET /xray/getDefaultJsonConfig` は基準テンプレート（バイナリに組み込まれた `config.json`）を返します。これを使って設定を工場出荷状態にリセットできます。

保存時の自動変換

Xray 設定の保存時、パネルは（この順序で）次の処理を実行します:

1. **ラッパーの削除** — 値に誤って含まれた `{ "xraySetting": <設定>, "inboundTags": ..., "outboundTestUrl": ... }` 形式のラッパーを削除します（そうしないと、保存のたびにレイヤーが蓄積されます）。最大 8 レイヤーが削除されます。
2. **設定の検証** — JSON が Xray 設定構造に解析されます; エラーの場合 — 「*xray template config invalid*」で拒否されます。
3. **統計ルールの保証** — ルール `inboundTag: ["api"] → outboundTag: "api"` が `routing.rules` のポジション 0 に強制的に昇格されます（存在しない場合は追加されます）。これにより、パネルの gRPC 統計リクエストが上位の catch-all ルールによってインターセプトされないことが保証されます（そうでないと、プロキシが動作中でもクライアントがゼロトラフィックでオフライン表示される可能性があります）。

項目 3 により、`api → api` ルールを削除したり移動しようとししないでください—パネルは次の保存時に元の位置に戻します。これはサービス統計インフラストラクチャであり、ユーザールートではありません。

11.12. サブスクリプションからの Outbound（自動更新付き）

バージョン 3.3.0 以降、パネルはサブスクリプション URL から直接 `outbound` をインポートできます—VPN プロバイダーがクライアントアプリケーションに提供するものと同じ形式です。サブスクリプションはバックグラウンドで定期的に再読み込みされるため、設定テンプレートを手動で編集することなく、サーバー上の `outbound` セットが最新の状態に保たれます。3.5.0 から、クエリパラメーター（`?plugin=`、`?type=`）や末尾の `/` を持つ `ss://` リンク（SIP002）のポートが正しく解析されるようになりました（以前は通知なく `0` になっていました）。実際に不正なポートを持つリンクは、壊れた状態でインポートされるのではなくスキップされます。

このセクションは「**Outbound サブスクリプション**」と呼ばれ、説明は次のとおりです:「リモートサブスクリプション URL (vmess/vless/trojan/ss/...) から outbound をインポートします。タグはバランサーとルーティングルールで使用するために変更されません。更新は自動的に実行されます。」セクションは Xray ページの **outbound** 設定パネルの上に配置されています。

仕組み

サブスクリプションは Xray 設定テンプレートとは別に保存されます。テンプレートは**一切書きされません**: サブスクリプションから取得した **outbound** は、Xray 設定生成時にオンザフライで最終設定に追加されます。

サブスクリプションの追加

「サブスクリプションを追加」フォームには次のフィールドがあります:

フィールド	キー	デフォルト	目的
サブスクリプション URL	<code>url</code>	-- (必須)	サブスクリプションアドレス。プレースホルダー: 「https://... (base64 のリンクリスト)」。HTTP(S) のみ受け付けます; アドレスはセキュリティチェックが行われます。
備考	<code>remark</code>	空	任意のラベル (プレースホルダー「例: HK ノード」)。
タグプレフィックス	<code>tagPrefix</code>	<code>subN-</code>	インポートされた outbound タグが始まるプレフィックス。空のままにすると、パネルは <code>sub1-</code> 、 <code>sub2-</code> などの形式で最小の空き番号を自動的に選択します。
更新間隔	<code>updateInterval</code>	600 秒 (10 分)	サブスクリプションが再読み込みされる頻度。UI では時間/分で設定します。
有効	<code>enabled</code>	はい (true)	有効なサブスクリプションのみが設定に含まれ、自動的に更新されます。
プライベートアドレスを許可	<code>allowPrivate</code>	いいえ (false)	localhost、LAN、プライベート IP への URL を許可します。SSRF 対策としてデフォルトで無効--信頼できるローカルソースにのみ有効にしてください。
手動 outbound の前に	<code>prepend</code>	いいえ (false)	有効にすると、このサブスクリプションの outbound はテンプレートの手動 outbound の前に配置され、その 1 つがデフォルト outbound になる可能性があります。そうでない場合は後に追加されます。

「**プレビュー**」ボタン (POST /outbound-subs/parse) を使うと、保存前に URL をダウンロードして解析し、どの **outbound** とタグが得られるかを確認できます; この時点ではデータベースには何も書き込まれません。URL から何も認識されなかった場合、「この URL に outbound が見つかりませんでした。」と表示されます。

複数のサブスクリプションの **outbound** リスト全体での順序は優先度 (**priority**) によって決まり、上/下矢印 (POST /outbound-subs/:id/move) で変更できます。

受け付けられるサブスクリプション形式

URL のレスポンス本文は次のように処理されます:

- コンテンツはまず **base64** (標準および URL セーフのバリエーション、パディングの自動補完とスペース/改行の削除あり) として試みられます。base64 であればデコードされ、そうでなければそのまま使用されます。
- 次に本文が行に分割されます。# で始まらないすべての空でない行がリンクとして解析されます。認識されない行 (コメント、サポートされていないプロトコル) は無視されます。
- サポートされているリンクスキーム: `vmess://`、`vless://`、`trojan://`、`ss://` (Shadowsocks)、`hysteria2://` / `hy2://`、`wireguard://` / `wg://`。

つまり、ほとんどのプロバイダーのような「base64 エンコードされたリンクリスト」の一般的なサブスクリプションに適しています。

安定したタグ

各リンクに対して安定した「アイデンティティ」が計算されます (フラグメント備考なしのコア URI; vmess の場合は `ps` フィールドなしの内部 JSON)。 「アイデンティティ→タグ」のマッピングが保持され、次の更新時に同じサーバーは備考やマイナーパラメーターが変更されても同じタグを取得します。これはバランサーとルーティングルールが更新後も機能し続けるために特別に設計されています:

- バランサー/ルールの正確なタグは引き続き同じサーバーを指します。
- プレフィックス/ワイルドカードセクター (例: `hk-*`) は、サブスクリプションが後で返す新しいサーバーを自動的に取り込みます—これがプールに「サブスクライブ」する推奨方法です。
- サーバーがサブスクリプションから消えると、そのタグは単純に最終的な `outbound` 配列から消えます; バランサーに `fallbackTag` がある場合、Xray はそれを使用します。
- プロバイダーがサーバーの UUID/ホスト/認証情報を変更した場合、アイデンティティが変わります—これは新しいタグを持つ新しい `outbound` とみなされます。

1 つのダウンロード内では、タグは `-N` サフィックスで重複排除されます。サブスクリプションのタグは非 ASCII 文字 (例: キリル文字) を保持し、読みやすいままです: Unicode の文字と数字はスラッグに保持され、句読点はハイフンに置き換えられます—キリル文字の名前からのタグはもはや数字だけに収縮しません。

自動更新の仕組み

- サブスクリプション更新のバックグラウンドタスクは **5 分ごと** にスケジュールに従って実行されます。
- 各実行時に、すべての有効なサブスクリプションを反復処理し、独自の間隔が経過したもののみを更新します: サブスクリプションは一度も更新されていないか、最後の更新から `updateInterval` 以上が経過している場合に更新されます。つまり、タスクは頻繁にサブスクリプションをチェックしますが、各サブスクリプションは自身の `updateInterval` (デフォルト 10 分) より頻繁には再読み込みされません。UI にはそれに対応するヒントが表示されます。
- 更新: URL はパブリックとして (プライベートアドレスは `allowPrivate` が設定されていないサブスクリプションに対してブロックされます) セキュリティチェックが再度行われ、リクエストはパネル

のプロキシクライアントによって `User-Agent: 3x-ui-outbound-sub/1.0` ヘッダーで送信されます。リダイレクトチェーンは 10 ホップに制限されており、各ホップもプライベート性がチェックされます (SSRF 対策)。HTTP 200 が期待されます; それ以外の場合はエラーが記録されます。

- 正常な解析後、結果が保存され、最終更新時刻が設定され、エラーがクリアされます。エラーの場合、そのテキストは UI に「最後のエラー」として表示され、以前に取得した `outbound` は引き続き有効です。
- 少なくとも 1 つのサブスクリプションが実際に更新された場合、タスクは Xray に再起動フラグを立て、UI 無効化を送信して、インターフェースが新しい `outbound` を取得するようにします。Xray の実際のリロードはマネージャーの次の 30 秒サイクルで行われます。

1 つのサブスクリプションの手動更新——「**今すぐ更新**」ボタン (`POST /outbound-sub/:id/refresh`) ; これも Xray に再起動フラグを立てます。サブスクリプションの追加、変更、削除も Xray 再起動フラグを設定します (削除の場合、その `outbound` は次のリロードで設定から消えます)。UI のヒント: 「追加または更新後、outbound をアクティブにするために Xray を再起動してください (または次の自動リロードを待ってください) 。」

Xray 設定への組み込み方法

Xray 設定の各生成時に、アクティブなサブスクリプション `outbound` は 2 つのグループに分けられます——`prepend` (「手動 `outbound` の前に」フラグ) とその他——テンプレートと結合されます:

[サブスクリプションの `prepend`] + [テンプレートの `outbound`] + [その他のサブスクリプション]。各グループ内でサブスクリプションは優先度順に並びます。テンプレートの手動 `outbound` はこの処理では変更されません; テンプレートの `outbound` 配列が何らかの理由で解析できない場合、サブスクリプションの `outbound` はそこに混入されません (手動 `outbound` を失わないため)。

インポートされた `outbound` はパネルの `outbound` パネル自体でも別のブロック「**Outbound サブスクリプションから (読み取り専用)**」として表示されます——そこでは編集できず、管理は「Outbound サブスクリプション」セクションのみで行います。

11.13. WARP での IP ローテーション

3X-UI では WARP `outbound` を設定できます——Cloudflare WARP への WireGuard 送信接続 (Xray 設定の `warp` タグ)。パネル自体が Cloudflare サーバーにデバイスアカウントを登録し、WireGuard キーとアドレスを取得して、`warp` タグを持つ `outbound` に設定します。このような `outbound` を通じてトラフィックは Cloudflare WARP の IP アドレスのもとインターネットに出ます。バージョン 3.3.0 の新機能——WARP アカウントを手動で再作成せずに、手動またはスケジュールに従ってこの `outbound` IP を変更できます。

管理は **Xray** セクションの WARP カード内にあります (「WARP アカウントを作成」をクリックして設定を取得した後; それ以前はアクションが利用できません——パネルは「最初に WARP 設定を取得してください」とヒントを表示します)。

IP 変更時に何が起こるか

「**IP を変更**」ボタンが IP 変更を開始します。ロジック:

1. 新しい WireGuard キーペアが生成されます。

2. 新しいキーで Cloudflare サーバーに WARP デバイスが再登録されます（新しい `device_id`、`access_token`、アドレス、ピアデータ）。
3. 新しいデータが Xray 設定の WARP outbound に書き込まれます: `secretKey`、`address`（v4 /32 と v6 /128）、`reserved`（`client_id` から）、およびピアの `publicKey` と `endpoint` が更新されます。
4. 以前に WARP+ ライセンスキー（26 文字以上）が設定されていた場合、新しいアカウントに自動的に再設定されます。失敗した場合はログへの警告のみ—IP 変更はキャンセルされません。
5. 変更が成功した後、新しい outbound が有効になるように Xray に再起動フラグが立てられます。

成功するとインターフェースに「WARP IP アドレスが正常に変更されました!」と表示されます。

スケジュールによる自動ローテーション

WARP カードには「IP アドレスの自動更新」スイッチと「間隔（日）」フィールドがあります。ヒント:「0 = 無効。IP アドレスを自動的に変更します。」

パラメーター	値
DB 設定	<code>warpUpdateInterval</code> （整数、 ≥ 0 ）
デフォルト値	0（自動ローテーション無効）
単位	日
0	自動変更を無効にする
> 0	N 日ごとに IP を変更する

間隔の保存は `warpUpdateInterval` を保存し、値が 0 より大きい場合は「最後の更新時刻」を現在の時刻にリセットします—そうしないと、スケジューラーが次のティックで即座に IP を変更するためです。

スケジュールは1時間ごとに実行されるバックグラウンドタスクによって実行されます—つまりパネルは1時間ごとにローテーションが必要かどうかを確認します。確認アルゴリズム:

- 間隔 ≤ 0 の場合—何もしません;
- 「最後の更新時刻」が 0 の場合（例: 間隔が DB への直接書き込みで設定された）—これは初回実行です: タスクはベースタイムスタンプを記録するだけで、即座に IP は変更しません;
- 最後の更新から 間隔 $\times 24 \times 3600$ 秒以上経過した場合—同じ IP 変更が実行され、タイムスタンプが更新され、Xray の再起動がスケジュールされます。

重要な詳細:「IP を変更」ボタンによる手動変更も最後の更新のタイムスタンプをリセットします。したがって、手動ローテーション後、自動間隔のカウントがリセットされ、予定された変更が直後にトリガーされることはありません。

「パネルプロキシ経由」

3.3.1 で変更。「パネルネットワークプロキシ」（`panelProxy`）の個別設定は削除されました。パネル自体の送信トラフィック（WARP API へのリクエストを含む）は、選択したパネルトラフィック

用 **outbound**—Xray outbound またはバランサー—経由でルーティングされるようになりました (13 セクション参照)。以下の説明はバージョン 3.3.1 以前のものです。

Cloudflare WARP API へのすべてのリクエスト（登録、設定取得、ライセンス設定、IP 変更）は直接ではなく、タイムアウト 15 秒のパネルの HTTP クライアントを通じて行われます。このクライアントはパネル設定の「**パネルネットワークプロキシ**」（panelProxy）設定を尊重します。

設定の説明より: プロキシはパネル自身の送信リクエスト（geo データベースの更新、Xray/パネルのバージョン確認、Telegram、そして WARP への通信）をルーティングします—サーバーフィルタリングを回避するために。socks5:// または http(s):// 形式のアドレスが受け付けられます（例: Xray 自身のローカル SOCKS inbound）。フィールドが空またはプロキシが正しく設定されていない場合—直接接続が使用されます（動作は壊れません）。

WARP での利点: サーバーが api.cloudflareclient.com に直接アクセスできない場合、以前は登録とローテーションが失敗していました。panelProxy に動作するプロキシ（Xray 自身の inbound を含む）を指定することで、WARP API の可用性と手動ボタンおよびスケジュールされたローテーションの両方の動作を保証できます。

これが役立つ場面

- WARP 経由で通信する outbound の定期的な送信 IP 変更—1 つのアドレスによるブロックやトラッキングのリスクを減らします。
- 現在の Cloudflare アドレスがブラックリストに載っているか低速な場合に、手動で IP を「リフレッシュ」します。
- Cloudflare WARP API への直接アクセスがないサーバー: panelProxy 経由のリクエストルーティングにより、登録とローテーションが機能するようになります。

12. ノード（マルチパネル、マスター/スレーブ）

ノードセクションは、通常の 3X-UI インストールを中央（マスター）パネルに変換し、他の（子）3X-UI パネルをリモートで監視・管理できるようにします。各ノードはそれぞれのサーバーに独立してインストールされた 3X-UI であり、マスターはそのパネルの HTTP API を通じてアクセスし、状態をポーリングし、割り当てられた inbound とクライアントを同期します。これが**マルチパネル**機能です。各パネルに個別にアクセスする代わりに、すべてのサーバーを一つのリストで確認し、一元的に管理できます。

重要な原則として、ノードはエージェントではなく、**完全な 3X-UI パネル**です。マスターはノード上に何かを「インストール」するのではなく、トークンを使ってその API に接続するだけです。リストからノードを削除すると監視のみが停止し、リモートパネル自体には影響しません（ヒント：「これによりノードの監視が停止します。リモートパネル自体は影響を受けません」）。

12.1. リスト上部のサマリー

ノードテーブルの上に集計カウンターが表示されます：

フィールド	説明
総ノード数	リスト内のノードの総数。
オンライン	online ステータスのノード数。
オフライン	offline ステータスのノード数。
平均遅延	ノードへの平均遅延（ping）（ミリ秒単位）。

12.2. ノードの追加と編集

ノードを追加ボタンとノードを編集ボタンはノードフィールドのフォームを開きます。

必須フィールド（ヒント：「名前、アドレス、ポート、API トークンは必須です」）は**名前、アドレス、ポート、API トークン**です。

「保存」をクリックすると（追加時と編集時の両方）、パネルはまず 6 秒のタイムアウトで**ノードの到達可能性を確認**します。ノードが応答しない場合、レコードは保存されずエラーが表示されます。つまり、明らかに到達不能なノードを追加することはできません。

フォームフィールド

フィールド	デフォルト	許容値	説明
名前	－（必須）	空でない文字列、一意	ノードの内部名。名前列には一意性が適用されており、同じ名前を持つ 2 つのノードを作成することはできません。プレースホルダーのヒント： 例：de-frankfurt-1。保存時に前後のスペースはトリミングされます。
メモ	空	任意の文字列	ノードのオプションのメモ/説明。動作には影響しません。

フィールド	デフォルト	許容値	説明
スキーム	https	http / https	リモートパネルへの接続プロトコル。空白または無効な値を指定した場合、正規化により https が設定されます。ノードが通常の HTTP で応答しているにもかかわらずスキームが https に設定されている場合、パネルはわかりやすいヒントを返します: 「the server speaks HTTP, not HTTPS; set the node scheme to http」。
アドレス	– (必須)	ホストまたは IP	リモートパネルのアドレス。プレースホルダー: panel.example.com または 1.2.3.4。アドレスは正規化されます; SSRF 保護のため、デフォルトではプライベート/ローカルアドレスは禁止されています。「プライベートアドレスを許可」を参照してください。
ポート	– (必須)	整数 1-65535	リモートノードのウェブパネルのポート。範囲外の値は拒否されます (「node port must be 1-65535」)。
ベースパス	/	パス文字列	リモートパネルのベースパス (web base path) が設定されている場合はそれを指定します。正規化されます: 先頭と末尾に / が保証されます (空の値 → /)。パネルはポーリング時にこれに panel/api/server/status を追加します。
API トークン	– (必須)	リモートパネルのトークン	ノードの API にアクセスするための Bearer トークン。Authorization: Bearer <token> ヘッダーで送信されます。プレースホルダー: 「リモートパネルの設定ページのトークン」。ヒント: 「リモートパネルの設定 → API トークンセクションで API トークンが表示されます」。つまり、トークンはノード自体で (設定 → API トークン) 作成し、ここに貼り付ける必要があります。
有効	true	はい/いいえ	ノードの監視と同期を有効にします。無効化されたノードはバックグラウンドタスク (heartbeat と traffic-sync はそれらをスキップ) によってポーリングされず、パネルの一括更新にも参加しません。
プライベートアドレスを許可	false	はい/いいえ	SSRF 保護を解除し、プライベート/ローカルアドレスでノードへの接続を許可します。ヒント: 「プライベートネットワークまたは VPN 内のノードにのみ有効にしてください」。ノードが実際にプライベートネットワーク内にあるか VPN 経由でアクセス可能な場合のみ有効にしてください。

ノード側でのトークンの取得と再生成

トークンはリモートパネルの **設定 → API トークン** セクションで取得します。そこで再発行することもできます: **トークンを再生成** ボタンには警告が表示されます: 「再生成すると現在のトークンが無効化されます。それを使用している中央パネルは更新するまでアクセスを失います。続行しますか?」。再生成後、マスターパネルの古いトークンは機能しなくなります。ノードフォームで更新する必要があります。

接続 outbound (Connection outbound)

Connection outbound (接続 outbound、outboundTag) フィールドは、マスターからこのノードの API への接続トラフィックがサーバーをどのように出るかを設定します。Xray outbound のタグを選択すると、パネルからノードへのリクエストは直接ではなく、指定された outbound を経由するようになります。パネルは自動的に loopback にブリッジ inbound を実行中の設定に追加し、再起動なしにライブで適用します。ヒント: 「Route this node's panel API traffic through the selected Xray outbound. A loopback bridge inbound is added to the running config automatically and applied live. Leave empty for a direct connection」。

セクターはパネルの outbound 選択と同様に機能します: タグは **Outbounds** (通常の outbound) と **Balancers** (バランサー) にグループ化され、blackhole outbound はリストから非表示になります。空の値 (プレースホルダー「Direct connection」) = ノードへの直接接続。

inbound のインポート (同期する inbound の選択)

ノードフォームには、2 つのモードを持つ **inbound インポート** (inboundSyncMode) 設定があります: **すべての inbound** (all 、デフォルト) と **選択済み** (selected)。デフォルトでは、マスターはこのノードが選択されているすべての inbound をノードに同期します。既存のノードは「すべての inbound」モードで動作し続けます。

選択済みモードでは、フィールドの下に inbound タグのマルチセクターが表示されます。**inbound を読み込む**をクリックすると、マスターは入力済み (まだ保存されていない) の接続パラメーターを使用してノードから inbound のリストを要求し (エンドポイント POST /panel/api/nodes/inbounds)、タグを表示します。必要なものをチェックしてください。パネルはチェックされたタグのみをノードに同期・デプロイし、ノード自体に直接存在する他の inbound はそのまま残されます。マスターはそれらを削除したり管理したりしません。

例: 選択インポートのためにノードの inbound リストを要求する。 リクエストボディにはまだ保存されていない接続パラメーターが含まれ、レスポンスにはノードで利用可能な inbound のタグが含まれます:

```
POST /panel/api/nodes/inbounds
Content-Type: application/json

{ "name": "de-fra-1", "scheme": "https", "address": "node1.example.com",
  "port": 2053, "basePath": "/", "apiToken": "abcdef..." }
```

12.3. TLS 検証 (https ノード向け)

このフィールドグループは、マスターがノードの HTTPS 証明書を検証する方法を設定します。これらの設定は** https スキームにのみ適用**されます。 http ノードでは無視されます。

TLS 検証 ドロップダウンリスト、ヒント: 「パネルがノードの HTTPS 証明書を検証する方法。自己署名証明書の場合はピン留めまたはスキップ (https ノードのみ)」。

モード	値	デフォルト	説明
検証 (標準 CA)	verify	はい (デフォルト)	信頼できる CA による標準的な証明書チェーンの検証。パブリック/Let's Encrypt 証明書を持つノードに適しています。すべての http ノードにも使用されます。
証明書をピン留め (SHA-256)	pin	—	CA チェーンは検証されませんが、ノードのリーフ証明書の SHA-256 が保存されたフィンガープリントと照合されます (定数時間比較)。自己署名証明書の MITM 保護を維持します。フィンガープリントフィールドの入力が必要です。

モード	値	デフォルト	説明
検証をスキップ	skip	—	証明書の検証が完全に無効化されます。警告: 「検証のスキップは中間者攻撃に対する保護を削除します。API トークンが傍受される可能性があります。代わりに証明書をピン留めしてください」。

3.4.0 では 3 つのモードに加えて 4 番目のモード – **Mutual TLS (クライアント証明書)** (`mtls`) が追加されました。他のモードと同様に `https` スキームでのみ使用可能です。

モード	値	デフォルト	説明
Mutual TLS (クライアント証明書)	mtls	—	ノードの証明書検証に加えて、マスターはノードの独自 CA によって発行された クライアント証明書 を使ってノードに自身を証明します。このモードでは、ノードの API トークン はオプションになります – ノードは証明書でマスターを識別します。モードを選択すると、次のヒントが表示されます: 「This node authenticates the panel with a client certificate. Copy this panel's CA from the Node mTLS section onto the node, set its Trusted parent CA, then restart it」。

ノードの Mutual TLS を有効にするには: ノード側でモードを **Mutual TLS** に設定し、**Node mTLS** セクション (後述) から管理パネルの CA をコピーし、それをノードの **信頼できる親 CA** として設定してノードを再起動します。

`skip`、`pin`、`mtls` 以外の値を選択すると、正規化により強制的に `verify` が設定されます。

証明書のピン留め

証明書をピン留めを選択すると、次が表示されます:

- **ピン留めされた証明書の SHA-256** – 入力フィールド。フィンガープリントは **base64** 形式 (Xray の `pinnedPeerCertSha256` 形式) またはコロンあり/なしの **hex** (`openssl -fingerprint` スタイル) で受け付けられます。ヒント: 「ノードの証明書の SHA-256 (base64 または hex)。「取得」をクリックして今すぐノードから読み取ります」。プレースホルダー: 「base64 または hex の SHA-256」。 `pin` を選択した場合、空または無効なフィンガープリントは保存時に検証エラーが発生します。

例: 同じフィンガープリントの 2 つの形式。 フィールドはどちらの形式も受け付けます – どちらも同じ証明書を意味します:

```
# base64 (Xray の pinnedPeerCertSha256 形式)
607TNg3l2k0pq8R1sT2uV3wX4yZ5a6B7c8D9e0F1g2=

# コロンあり hex (openssl x509 -fingerprint -sha256 スタイル)
E8:E2:D3:60:DE:5D:9A:4D:29:AB:CF:11:B2:7C:34:...
```


フィンガープリントがまだわからない場合は、**取得**をクリックします。マスターが HTTPS 経由でノードに接続し（証明書検証なし）、現在のリーフ証明書の SHA-256 を読み取り（エンドポイント `POST /certFingerprint`）、フィールドに入力します。

- **取得ボタン** – 証明書検証なしで HTTPS 経由でノードに接続し、現在のリーフ証明書の SHA-256 を読み取り（エンドポイント `POST /certFingerprint`）、フィールドに入力します。成功した場合 – 「ノードの現在の証明書を取得しました」；失敗した場合 – 「証明書の取得に失敗しました」。https ノードでのみ使用可能です。

Node mTLS（パネル間の相互 TLS 認証）

ノードページには別個の **Node mTLS** セクションがあります – ノード間の「パネル → ノード」呼び出しに対して API トークンに加えて第 2 要素（クライアント証明書）を追加する相互 TLS 認証の設定です。Mutual TLS はオプションです；セクションのフィールドが空の場合、ノードは以前のスキーム – **API トークンのみ**で動作します（ヒント：「Mutual TLS adds a client-certificate factor on top of the API token for node-to-node calls. It is opt-in: leave it empty to keep token-only auth」）。セクションには 2 つの操作があります：

- **このパネルの CA をコピー**（`POST /panel/api/nodes/mtls/ca`）– このパネルのルート証明書（CA）をクリップボードにコピーします。この CA を管理対象ノードに渡して、パネルのクライアント証明書を信頼させる必要があります。ノード自体では TLS 検証モードを **Mutual TLS** に設定します（ヒント：「Hand this CA to the nodes this panel manages, then set their TLS verification to Mutual TLS」）。コピー後 – 「CA 証明書をクリップボードにコピーしました」。
- **信頼できる親 CA**（Trusted parent CA、`POST /panel/api/nodes/mtls/trustCA`）– このパネル自体が上位（管理）パネルのノードとして機能する場合に使用するフィールドです。管理パネルの CA をここに貼り付けてクライアント証明書を要求し、**Save trust CA** をクリックします。この変更には**パネルの再起動**が必要です（ヒント：「When this panel is itself a node, paste the managing panel's CA here to require its client certificate. Restart the panel to apply」）。

12.4. 各ノードに表示される内容

テーブルの列とノードカードのフィールド（heartbeat ポーリングごとに更新される観測された状態）：

フィールド	説明
ステータス	<code>online / offline / unknown</code> – 後述を参照。
CPU	リモートサーバーのプロセッサ使用率（パーセント）。
メモリ	RAM 使用率（パーセント、 <code>current/total*100</code> として計算）。
稼働時間	サーバーの継続稼働時間（秒単位）。
遅延	最後のポーリングに対するノードの応答時間（ms）。
最終 ping	最後に成功した heartbeat のタイムスタンプ（Unix 秒； <code>0</code> = 「なし」；最近の値は「たった今」と表示）。
Xray バージョン	ノードで実行中の Xray コアのバージョン。

フィールド	説明
パネルバージョン	ノードの 3X-UI バージョン – 更新インジケーターのために最新バージョンと比較されます。
(inbound)	このノードに物理的に配置されている inbound の数。
(クライアント)	ノードの inbound 上のクライアント数。
(オンライン)	現在オンラインのノードのクライアント数。
(使い切り)	トラフィック制限を超過または期限切れのノードのクライアント数。手動で無効化されたクライアントはこのカウンターに含まれません。
(速度)	ノードに配置された inbound の現在（ライブ）の転送速度。

inbound/クライアント/オンラインのカウンターは、ローカル id ではなくノードの安定した GUID（panelGuid）でノードに関連付けられます。これにより、サブノードのクライアントが中間ノードではなくそのサブノードに適切に帰属されます。

ノードに配置された inbound については、ページはオンラインクライアント、カウンター、および**現在の転送速度**を表示します。安定した GUID による関連付けにより、同じ panelGuid を持つ「クローン」ノードも正しく区別されます。

ノードのステータス

ステータス	説明	設定されるタイミング
online	オンライン	ノードが panel/api/server/status ポーリングに success=true で応答した。
offline	オフライン	ノードが応答しなかった、HTTP エラーを返した、success=false、または認識できない応答を返した。
unknown	不明	ノードがまだポーリングされていない初期値。

ポーリング失敗時にはエラーテキストが保存され、わかりやすい形式で表示されるため、「offline」の原因を診断するのに役立ちます。

12.5. ノードに対するアクション

- **接続テスト**（POST /test） – ノードフォームで、入力済み（まだ保存されていない）パラメーターを使って 6 秒のタイムアウトで接続を確認します。結果：「接続が正常です（{ms} ms）」または「接続に失敗しました」。保存前にアドレス/ポート/トークン/TLS のデバッグに便利です。
- **今すぐ確認**（「今すぐ確認」ボタン、POST /probe/:id） – 既に保存されているノードの予定外のポーリング；ステータスとメトリクス（CPU/メモリ/稼働時間/遅延/バージョン）を即座に更新し、heartbeat を記録します。失敗した場合 – 「確認に失敗しました」。

例：マスターの API を使ってノードを確認・ポーリングする。 「接続テスト」はフォームからまだ保存されていないパラメーターをテストします：

```
POST /panel/api/nodes/test
Content-Type: application/json
```

```
{ "scheme": "https", "address": "de-frankfurt-1.example.com", "port": 2053,
  "basePath": "/", "apiToken": "eyJhbGciOi...", "tlsMode": "verify" }
```

id が 7 の既に保存されているノードの予定外のポーリング:

```
POST /panel/api/nodes/probe/7
```

- **パネルを更新** (ボディ `{ids:[...]}` の `POST /updatePanel`) – ノードで標準の自己更新ツールを起動します: ノードは最新の 3X-UI リリースをダウンロードしてそこで再起動します。**選択済みを更新 ({count})** ボタンは、チェックされた複数のノードに対してこれを実行します。各ノードの横にインジケータが表示されます: **更新が利用可能**または**最新** (ノードのパネルバージョンと最新バージョンの比較に基づく)。

例: 1 つのリクエストで複数のノードを更新する。 ボディにはチェックされたノードの id が含まれます; 有効で `online` のノードのみ更新され、他はスキップされたものとして返されます。

```
POST /panel/api/nodes/updatePanel
Content-Type: application/json
```

```
{ "ids": [3, 7, 12] }
```

「2 つのノードで更新が開始されました、1 つ失敗しました」のような応答: ノード 12 はオフラインだったためスキップされた可能性があります。

- 確認: 「{count} ノードを最新バージョンに更新しますか? 各選択されたノードは最新リリースをダウンロードして再起動します。有効かつオンラインのノードのみ更新されます」。
- 有効かつ `online` ステータスのノードのみ更新されます。無効なノードは結果で「node is disabled」、オフラインは「node is offline」とマークされます。結果: 「{ok} ノードで更新が開始されました、{failed} つ失敗しました」。適切なノードが選択されていない場合 – 「少なくとも 1 つの有効なオンラインノードを選択してください」。

更新確認ダイアログ (単一ノードと一括の両方) には、**開発チャンネル (最新コミット) に更新する** チェックボックスがあります。これをチェックすると、選択されたノードは安定リリースの代わりに dev-latest ローリングビルド (main ブランチの最新コミット) をインストールします; チェックを外した場合、ノードは通常のチャンネルで更新されます。チェックを入れると、下に警告が表示されます: 「開発ビルドは main の各コミットを追跡し、安定リリースではありません – 自動ロールバックはありません」。dev フラグは `POST /panel/api/nodes/updatePanel` を通じてノードに渡され、ノードは dev チャンネルで更新を起動します。

- **Set Cert from Panel** (補助的、`GET /webCert/:id`) – ノードに inbound を作成する際、ファイルがそのノードに存在するように、中央パネルではなくノード自体のウェブ TLS 証明書へのパスを設定できます。ノードが有効で到達可能である必要があります。
- **ノードを削除** (`POST /del/:id`) – 確認: 「ノード「{name}」を削除しますか? これによりノードの監視が停止します。リモートパネル自体は影響を受けません」。ノードレコードとその累積トラ

フィック統計を削除します；リモートパネルは通常通り動作し続けます。**ノードはすべての inbound が取り外された後にのみ削除できます。** 少なくとも 1 つの inbound がノードに（`node_id` を通じて）まだ関連付けられている場合、パネルは「cannot delete node: N inbound(s) still attached to it; detach or delete them first」のようなエラーで削除を拒否します – まずそれらの inbound を切り離すか削除してから、ノードを削除してください。これにより、削除されたノードへの参照がぶら下がる「孤立した」inbound が防がれます。

12.6. メトリクス履歴

履歴ボタン/グラフは `GET /history/:id/:metric/:bucket` にアクセスします。利用可能なメトリクスは `cpu` と `mem` – 成功した heartbeat ごとに蓄積されます。集計間隔のサイズ（`bucket`、秒単位）はホワイトリストによって制限されます：

例：履歴のリクエスト。 ノード 7 の CPU 使用率グラフを 60 秒間隔で集計したもの（最大 60 ポイントが返される）：

```
GET /panel/api/nodes/history/7/cpu/60
```

メモリと「リアルタイム」モード（2 秒）については、それぞれ `.../7/mem/60` と `.../7/cpu/2`。ホワイトリスト外の値は拒否されます（「invalid metric」/「invalid bucket」）。

Bucket (秒)	用途
2	リアルタイムモード
30	30 秒間隔
60	1 分間隔
120	2 分間隔
180	3 分間隔
300	5 分間隔

最大 60 ポイントが返されます。無効なメトリクスまたは bucket は拒否されます（「invalid metric」/「invalid bucket」）。

12.7. inbound とクライアントの同期方法

inbound は `node_id` フィールドを通じてノードに「属します」（inbound エディターでノードを選択）：

例：ノードフォームのトークン。 トークンは子パネル（設定 → API トークン）で取得し、マスターの API トークンフィールドに貼り付けます。各ポーリング時にマスターはヘッダーで送信します：

```
GET https://panel.example.com:2053/panel/api/server/status
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.abc123...
```

子パネルにベースパス (web base path) が設定されている場合、例えば `/secret/`、マスターは `panel/api/server/status` の前に自動的に挿入します → `https://panel.example.com:2053/secret/panel/api/server/status`。

1. **設定のデプロイ (reconcile)**。 ノードに関連付けられた inbound/クライアントが変更されるたびに、ノードは「ダーティ」としてマークされます。バックグラウンドタスクは、変更がある場合に `** online ステータス**`の各有効なノードに対して、ノードの inbound (node_id による) をデプロイし、その後「ダーティ」フラグをリセットします。無効、オフライン、または「ダーティ」なノードは「待機中」と見なされます。デプロイは接続が回復するまで延期されます。
2. **トラフィック収集**。 同じタスクがノードからトラフィックスナップショットを要求し、ローカル統計にマージします。マージされたトラフィックに基づいて、制限/期限の枯渇チェックが実行され、必要に応じてクライアントが無効化されます；ノードの「使い切り」カウンターはこれを反映します。ノードが到達不能な場合、そのオンラインクライアントはクリアされます。

複数のパネルに同時に関連付けられているクライアントに対して、マスターは同じタスクで追加的に**すべてのパネルにわたる**そのクライアントの合計トラフィック消費量をノードに配布します (ノード上の別テーブル、キーはマスターの GUID; 各送信で上書きされるため、マスター側でのリセットも伝播します)。ノードはクライアントのトラフィックとして、ローカルと受信したもののうち大きい方の値を表示し、合計クォータを超過すると**ノード自体でローカルにクライアントを無効化**します (自動無効化時の Xray 再起動と同じメカニズムで、既に確立された接続を切断します)。これにより、ノードが自分のトラフィック分だけを見て消費を過小評価し、合計制限を既に超過したクライアントのサービスを継続するという状況が解消されます。トラフィックのリセット、自動更新、またはクライアントの削除時に、送信済みカウンターはクリアされます。

ノードに配置された inbound の**最初の同期時** (新しいノードの追加または inbound の再インポート)、マスターはクライアントのトラフィックカウンターをノードの実際の値で初期化します。以前は、この状況では inbound の合計カウンターは正しく転送されていましたが、個々のクライアントカウンターはリセットされ、マスターはノード接続前に蓄積されたすべての履歴に対してクライアントの消費量を過小報告していました。現在、inbound が同じ同期で作成された場合、新しい `client_traffics` 行はノードのカウンター値を継承します (ベースラインがそれと等しく設定されるため、次のデルタはゼロになりトラフィックが二重にカウントされない)。カウンターのシードは同じパスで作成された inbound にのみ適用されます: 既存の inbound の下に再び現れたクライアントは引き続きゼロから開始し (「ファントム」トラフィックに対する保護)、削除されたばかりのクライアントはその inbound を再作成しても「復活」しません。

3. **Heartbeat**。 別のバックグラウンドタスクが (同時実行を制限しながら) すべての**有効なノード**を定期的に `panel/api/server/status` でポーリングし、ステータス/メトリクス/バージョンを更新し、ウェブクライアントがある場合は WebSocket で更新されたノードツリーを配布します。

12.8. ノードチェーン (サブノード/トランジティブノード)

トポロジはフラットである必要はありません: ノード自体が独自のノードのマスターになることができます。このような下位パネルはサブノードとして表示されます – これらは直接ノードから取得された読み取り専用の投影です。

- ヒント: 「読み取り専用: {親} を通じてアクセス可能な従属ノード。{親}自身のパネルから管理してください」。つまり、ここでサブノードを編集、削除、更新することはできません – すべての操作はその直接の親パネルから実行されます。
- サブノードのアイデンティティはその GUID によって決まります。これにより、オンラインクライアントと inbound は、Node1 → Node2 → Node3 のチェーン (マスターは各直接ノードを通じて 1 レベル深くまで「走査」します) であっても、それらをホストする物理ノードに正確に帰属されます。
- 直接ノードが到達不能になると、そのサブノードキャッシュはクリアされ、接続が回復するまでサブノードはツリーから消えます。

12.9. 3.5.0 の修正 (ノードの安定性)

3.5.0 の修正パッケージのうち、ノードのオペレーターに関係するもの:

- **変更なしのクライアント保存がノードのトラフィックを壊さなくなりました。** 以前は、クライアントを保存するだけで (「開いて保存した」だけでも)、Xray ハンドラーの再作成を伴う inbound の完全な更新がノードに送信されていましたが – ノードはオンラインと表示されるのに、手動で再起動するまでトラフィックを通しませんでした。現在、no-op の保存は何も行わず、実際の変更は再作成なしの 1 回の軽量の更新として送信されます。
- **ノードの Host オーバーライドがマスターに取り込まれます:** ノードの inbound を初めて承認する際、その Hosts 行 (SNI、フィンガープリント、ALPN など) がインポートされ、サブスクリプションが正しい TLS パラメーターを持つようになります。
- **ノード上の自動更新が新しいクォータウィンドウを開きます** (「30 日ごとに 100 GB」のようなパッケージが再び全量を付与します)。
- **マスターでのクライアント削除がノード上でも完全に削除します** (レコード + トラフィック)。紐付け解除だけではなくになりました。過去の一括削除の残骸は、ノード上の「紐付けなしクライアントを削除」アクションで取り除けます。
- **ノードの inbound は初回承認前に一掃されません:** 追加したばかりのノードを保存しても、その稼働中の inbound が削除されることはなくなりました。
- **1 つの不正な inbound がノードのトラフィック同期を止めません** (あわせて、ノード上のレガシー socks inbound は mixed にリネームされます)。
- **ポート競合チェックは自分のノードに限定されました** – ノード上の inbound のリッスンアドレスの編集が、別のノード上の同じポートを理由に拒否されることはなくなりました (「port ... already used by inbound ...」)。

12.10. ノード: 3.3.0 での新機能

バージョン 3.3.0 では、ノードセクションに 3 つの注目すべき改善が加われました: 多段 (マルチホップ) トポロジでのトラフィックとオンラインクライアントの正確な帰属、ノード間の client-IP 同期、そしてノードのパネルが稼働中でもその Xray コアがクラッシュした場合の専用ステータスインジケータ。

1. マルチホップ：サブノードチェーンにわたる正確なトラフィック帰属

以前、カウンター（inbound 数、オンラインクライアント、使い切り）は「直接」ノードレベルで計算されていました。マスター → ノード1 → ノード2 → ノード3 のようなチェーンがある場合、ノード2 / ノード3 に物理的に存在するものはすべて、マスターに届いた ノード1 に誤って帰属されていました。3.3.0 では帰属は実際のソースに従います。

仕組み：

- サブノードが別個の行として表示されるようになりました。各パネルは直接ノードのリストを公開します；既知の Guid を持つノードのみが含まれます – 安定したアイデンティティは、ノードを 1 つ「上」に帰属させるために必要です。マスターは heartbeat タスクから定期的にこれらのリストを取得してキャッシュし、直接ノードに「トランジティブ」なサブノードを追加します。
- トランジティブノードは読み取り専用です。UI では**「サブノード」**としてヒントとともにマークされます：「読み取り専用：{親} を通じてアクセス可能な従属ノード。{親}自身のパネルから管理してください。」そのような行には管理ボタンがありません – ノードはその直接の親パネルから管理されます。
- GUID による階層。直接ノードの ParentGuid はマスター自身の GUID；トランジティブノードの場合は親ノードの GUID。これがツリーを構築します。
- カウンターの真実のソースは inbound の origin_node_guid です。これはこの inbound を物理的に保持するノードの panelGuid です。ノードからの inbound 同期時に設定され、以降のホップでもそのまま保持されるため、深くネストされた inbound は中間ノードではなく実際のノードに帰属されます。この GUID によって inbound 数、オンラインクライアント、使い切りクライアントのカウンターが再計算されます。キー選択ロジック：

inbound の状態	帰属先
origin_node_guid が設定されている	この GUID（実際のソースノード）
空だが node_id が設定されている	ノードの合成 GUID（古いビルド、まだ panelGuid を報告していない）
空かつ node_id も空	マスター自身の GUID（ローカル Xray 上の inbound）

オンラインクライアントも GUID でグループ化されるため、各ノード行には実際にそのノードに接続しているクライアントのみが表示されます。

ユーザーが見るもの：フラットなトポロジー（マスターの直下にあるノード）では何も変わりません – GUID と id によるカウンターは一致します。しかし、ノードのチェーンが現れると、「サブノード」行がリストに表示され、各ノードの inbound/オンライン/使い切りの数は、トランジットされたすべてのものの合計ではなく、そのノード自身の負荷を反映するようになります。

2. ノード間での access.log から client-IP 同期

IP 制限（クライアントの `limitIp`）は Xray が `access.log` に書き込むアドレスに依存します。以前、各ノードは自分への接続のみを見ていたため、「クライアントあたり最大 N 個の IP」制限はクラスター内では機能しませんでした：クライアントは異なるノードに接続して制限を回避できました。3.3.0 では、観測された IP がクラスター全体で同期されます。

仕組み：

- 各ノードで、バックグラウンドタスクが `access.log` をパースし、各行から IP、クライアントの email、タイムスタンプを抽出し、ローカルテーブルに保存します（email ごとに 1 レコード、IP は JSON 配列 `{ip, timestamp}` として保存）。ローカルアドレス `127.0.0.1` と `:::1` は除外されます。
- 10 秒ごとの同期が、オンラインの各有効なノードで双方向の交換を実行します：ノードから IP を取得してローカルテーブルにマージし、マスターの統合ビューをノードに送信します。
- マージは古い観測と受信した観測を複数のノードで見られた同じ IP を二重にカウントすることなく、古いレコードを復活させることなく結合します：ローカルタスクと同じ鮮度しきい値が適用されます – 30 分。各 IP については最新のタイムスタンプが保持されます。他のノードからのレコードには新しいローカル id が付与されます（ノードの id 空間は独立しています）；同じ email の同時挿入は重複から保護されます。
- 制限を計算する際、「ライブ」の IP は、ローカルスキャンで現在観測されているか、同期されたデータベースから非常に新しいタイムスタンプ（2 分以内）を持つものです。これにより、アドレスが別のノードで観測された場合でも、制限がクラスター全体で機能するようになります。制限を超えると、最も古い「ライブ」IP が `fail2ban` ログに送られ、接続が強制的に切断されます（Xray API を通じたクライアントの削除/再追加）。

ユーザーが見るもの： IP 数の制限が各ノード個別ではなくクラスター全体に適用されるようになりました；パネルでは任意のノードで観測された IP がクライアントに表示されます（30 分のウィンドウ内）。これ用の別ボタン/設定はありません – 同期はバックグラウンドで自動的に実行されます。ただし、ノードで `access.log` が有効かつアクセス可能である必要があります（制限自体にはノードで `Fail2Ban` も必要です）。

3. 専用ステータスインジケーター：ノードのパネルがオンラインでも Xray がクラッシュしている場合

以前、ノードのステータスは基本的に「オンライン/オフライン」でした。パネルが応答すればノードはオンラインと見なされました – Xray コアが動作していなくてクライアントが実際に接続できない場合でも。3.3.0 ではパネルの正常性と Xray コアの正常性が分離されました。

仕組み：

- ノードをポーリングする際、マスターはリモートの `/panel/api/server/status` レスポンスから `xray.state` と `xray.errorMessage` フィールドを取得してノードに保存します。これらのフィールドはコアが異常な場合でもパネルの ping が成功する際に入力されます – パネルの可用性と Xray の状態を区別するためです。
- `xray.state` の値：`"running"`（動作中）、`"stop"`（停止）、`"error"`（エラー）。

- これらの値はノードのステータスに変換されます。馴染みのあるものに加えて新しいものが追加されました：

ステータスキー	表示ラベル	表示されるタイミング
online	「オンライン」	パネルが応答し、Xray が動作中 (running)
offline	「オフライン」	パネルが到達不能/ping 失敗
unknown	「不明」	状態がまだ決定されていない
xrayError	「Xray エラー」	パネルがオンラインだが Xray コアが error 状態 (errorMsg あり)
xrayStopped	「停止中」	パネルがオンラインだが Xray が停止中 (stop)

- このような状態では UI で**専用の紫色のインジケーター**が使用されます（オンラインの緑とオフラインの赤とは異なる色）。紫色は明確に示します： ノードには到達できますが、問題はネットワークやパネル自体ではなく Xray コア自体にあります。

ユーザーが見るもの： コアがクラッシュしても誤解を招く「グリーン」の代わりに、ノードはステータス**「Xray エラー」または「停止中」で紫色**にハイライトされます。これはすぐに、ノードの到達可能性を調査するのではなく、ノードの Xray を修正する必要があること（コアを再起動し、errorMsg を確認する）を示します。同じ xrayState / xrayError はトランジティブなサブノード（項目 1 参照）にも伝播されるため、コアの異常な状態がチェーン全体で表示されます。

13. パネル設定

「設定」セクション（ページタイトル – **設定**、英語 *Panel Settings*）は、3X-UI ウェブパネル自体の動作を管理します。パネルがどのアドレスとポートでリッスンするか、どのように保護されているか、Telegram ボットや外部サービスとどのように連携するか、スケジュールされたタスクをどのタイムゾーンで実行するかを設定します。各パラメータはデータベースの `settings` テーブルにキーと値のペアとして保存されます。データベースに値がない場合はデフォルト値が適用されます。

重要 – 変更の適用。 このページのすべての変更は **Сохранить** (Save) ボタンで保存し、変更を有効にするためにパネルを再起動する必要があります。ヒントの文言：「変更を保存してパネルを再起動し、変更を適用してください。」保存時に「設定が変更されました」という通知が表示されます。

13.1. パネルの保存と再起動

要素	目的
Сохранить (Save)	フォームのすべてのフィールドをデータベースに書き込みます（POST <code>/panel/setting/update</code> ）。書き込み前に値が検証されます。不正なアドレス、ポート、またはパスは拒否され、パネルはエラーを返します。
Перезапустить панель (Restart Panel)	パネルのウェブサーバーを再起動します（POST <code>/panel/setting/restartPanel</code> ）。再起動は 3 秒の遅延で実行されます。ヒント：「パネルを再起動してもよいですか？ 確認すると、3 秒後に再起動されます。パネルが利用できない場合はサーバーログを確認してください。」成功時 – 「パネルが正常に再起動されました。」
Восстановить настройки по умолчанию (Reset to Default)	データベースに保存されているすべての設定を削除します。その後、パネルはデフォルト値を使用します。管理者の認証情報はこの操作でリセットされません。

再起動はパネルプロセスへの `SIGHUP` シグナル送信（または登録された再起動フック経由）で実行されます。Windows ではシグナルによる自動再起動はサポートされていません。**リッスンパラメータの変更**（IP、ポート、パス、ドメイン、証明書、タイムゾーン）はパネルの再起動後にのみ適用されます。

13.2. 一般設定（「パネル」タブ / *General*）

インターフェース言語 (Language)

パネルのウェブインターフェースの言語。利用可能な言語：`en-US`（英語）、`ru-RU`（ロシア語）、`zh-CN`、`zh-TW`、`fa-IR`、`ar-EG`、`es-ES`、`id-ID`、`ja-JP`、`pt-BR`、`tr-TR`、`uk-UA`、`vi-VN`。これは表示設定であり、Xray の動作には影響しません。

カレンダータイプ (Calendar Type)

- キー: `datepicker`
- デフォルト値: `gregorian`（グレゴリオ暦）。

- **目的:** 日付選択で使用するカレンダーのタイプ（クライアントの有効期限設定時など）。ヒント：「スケジュールされたタスクはこのカレンダーに従って実行されます。」代替値はペルシャ暦（ジャリ暦）で、イラン向けのユーザーに需要があります。

ページサイズ (Pagination Size)

- **キー:** `pageSize`
- **デフォルト値:** 25
- **許容値:** 0 から 1000 の整数。
- **目的:** テーブル（接続 / inbound のリスト）の 1 ページあたりの行数。ヒント：「接続テーブルのページサイズを設定します。無効にするには 0 を設定してください。」 - 0 の場合はページネーションが無効になり、すべてのレコードが単一リストで表示されます。
- **パネルの再起動は不要**（表示設定）。

自動無効化後に Xray を再起動 (Restart Xray After Auto Disable)

- **キー:** `restartXrayOnClientDisable`
- **デフォルト値:** `true`
- **目的:** クライアントが自動的に無効化された場合（有効期限切れまたはトラフィック制限到達時）、そのクライアントの既存接続を切断するために Xray が再起動されます。ヒント：「クライアントが有効期限切れまたはトラフィック制限によって自動的に無効化された場合、Xray を再起動してください。」機能自体は変わっていません - 切り替えスイッチは「パネル」タブ (General) に他の一般設定と並んで配置されています。

備考モデルと区切り文字 (Remark Model & Separation Character)

- **キー:** `remarkModel`
- **デフォルト値:** `-ieo`
- **目的:** サブスクリプション内の設定名 (remark) の生成方法を定義します。文字列は**最初の文字**（区切り文字）と、その後の**順序を示すアルファベットのシーケンス**で構成されます：
 - `i` - inbound の備考 (*inbound remark*);
 - `e` - クライアントのメールアドレス;
 - `o` - 追加ラベル (*extra*).

デフォルト値 `-ieo` では区切り文字が `-` で、順序は inbound → メール → extra（例： `MyInbound-user@mail-extra`）。空のパーツはスキップされます。インターフェースの「サンプル備考」フィールド (*Sample Remark*) では生成される名前のプレビューが表示されます。名前へのメールの組み込みは、サブスクリプション設定の「名前にメールを含める」パラメータにも依存します（サブスクリプションのセクションを参照）。

例: `remarkModel` の値が設定名に与える影響。inbound 名が `VLESS-Reality`、クライアントのメールが `alex@vpn`、追加ラベルが `RU` の場合：

フィールドの値	結果の名前 (remark)
-ieo (デフォルト)	VLESS-Reality-alex@vpn-RU
_ie	VLESS-Reality_alex@vpn
-ei	alex@vpn-VLESS-Reality
o (スペース区切り、ラベルのみ)	RU

文字列の最初の文字は常に区切り文字です。残りのアルファベットで、どのパーツをどの順序で名前に含めるかを指定します。

13.3. パネルへのアクセス: IP、ポート、パス、ドメイン、証明書

このグループはパネルのネットワーク入口ポイントを定義します。ここでのすべての変更はパネルの再起動後にのみ適用されます。

フィールド	キー	デフォルト値	説明
パネル管理 IP アドレス (Listen IP)	webListen	"" (空)	ウェブパネルがリッスンする IP。空 = すべての IP でリッスン。ヒント: 「任意の IP から接続するには空のままにしてください。」指定する場合は有効な IP アドレスである必要があります (そうでなければ保存が拒否されます)。
パネルドメイン (Listen Domain)	webDomain	"" (空)	リクエストをドメインで確認するためのパネルのドメイン名。空 = 任意のドメインおよび IP からの接続を受け入れ。ヒント: 「任意のドメインおよび IP から接続するには空のままにしてください。」
パネルポート (Listen Port)	webPort	2053	パネルが動作するポート。ヒント: 「パネルが動作するポート。」1-65535 が許容されます。ポートは空いている必要があり、パネルとサブスクリプションサービスが同じ IP:ポート のペアを同時に使用することはできません。
URI パス (URI Path)	webBasePath	/	パネル URL のベースパス (basePath)。ヒント: 「/」で始まり「/」で終わる必要があります。」保存時、先頭と末尾の / がない場合はパネルが自動的に追加します。パスに使用できない文字は拒否されます。

パネル証明書 (TLS / HTTPS)

フィールド	キー	デフォルト値	説明
パネル証明書の公開鍵ファイルパス (Public Key Path)	webCertFile	""	証明書 (チェーン) ファイルへのフルパス。ヒント: 「/」で始まるフルパスを入力してください。」
パネル証明書の秘密鍵ファイルパス (Private Key Path)	webKeyFile	""	秘密鍵ファイルへのフルパス。ヒント: 「/」で始まるフルパスを入力してください。」

証明書 / キーのパスのうち少なくとも 1 つが指定されている場合、保存時にパネルは「証明書 + キー」のペアのロードを試みます。エラー (存在しないファイル、キーと証明書の不一致) の場合は保存が拒否さ

れます。両方の正しいパスが指定されている場合、パネルは HTTPS に切り替わります。両方のフィールドが空の場合 = パネルは通常の HTTP で動作します。

セキュリティ警告 (Security warnings)。パネルは安全でない設定が検出された場合、「パネルが公開されている可能性があります: 」というブロックを警告とともに表示します:

- 通常の HTTP での動作 – 「本番環境では TLS を設定してください」;
- 標準ポート 2053 – 「ランダムなポートに変更してください」;
- デフォルトのベースパス / – 「ランダムなパスに変更してください」;
- 標準のサブスクリプションパス /sub/ および JSON サブスクリプション /json/ – 「変更してください」。これらは推奨事項であり、ブロックではありません。

13.4. セッション、パネルプロキシおよびトラステッドプロキシ (「プロキシとサーバー」タブ / *Proxy and Server*)

セッション期間 (Session Duration)

- キー: sessionMaxAge
- デフォルト値: 360 (分、つまり 6 時間)。
- 許容値: 1 から 525600 分 (1 年) まで。
- 目的: 管理者が再ログインなしで認証状態を維持できる期間。単位は分。ヒント: 「システムセッションの期間 (値: 分)。」

パネルトラフィックの outbound (Panel Traffic Outbound)

- キー: panelOutbound
- デフォルト値: "" (空 = 直接接続)。
- 目的: パネルが自身のリクエスト (バージョン確認とパネル / Xray のダウンロード、Telegram へのアクセス、通常の geo ファイル更新) を送信する際に使用する Xray **outbound** を設定します。これにより、GitHub / Telegram のサーバーサイドフィルタリングを回避できます。フィールドはドロップダウンリストです。Xray 設定テンプレートの outbound、outbound サブスクリプションの outbound、およびルーティングバランサー (別グループ) が一覧表示されます。blackhole タイプの outbound はリストから除外されています – ダウンロードを「ブラックホール」にルーティングしても意味がないためです。ヒントの文言: 「パネル自身のリクエスト (バージョン確認とパネル / Xray のダウンロード、Telegram、通常の geo ファイル更新) を、GitHub / Telegram のサーバーサイドフィルタリングを回避するために、この Xray outbound 経由でルーティングします。ローカルブリッジ inbound は実行中の設定に自動的に追加され、即時に適用されます。Xray 組み込みの Geodata 自動更新は影響を受けません。独自のダウンロード用 outbound があります。直接接続するには空のままにしてください。」

動作の仕組み。 outbound を選択すると、パネルは動作中の設定にサービス用ループバック inbound (タグ panel-egress の SOCKS ブリッジ) とルーティングルールを自動追加し、パネル自身の HTTP トラフィックを選択した outbound にリダイレクトします。バランサーが選択された場合、

ルールには `balancerTag` が設定され、パネルのトラフィックがメンバー間で分散されます。ブリッジとルールは即時に適用され、パネルの完全な再起動は不要です。直接接続するにはフィールドを空のままにしてください。Xray 組み込みの geo データ自動更新はこの設定の影響を受けません – Xray ルーティング内に独自の outbound があります。

- **形式:** `socks5://` (または `socks5h://`) あるいは `http(s)://`、必要に応じて `socks5://user:pass@host:port` 形式の認証情報付き。サポートされるスキームは厳密に: `socks5`、`socks5h`、`http`、`https` – その他のスキームは無効とみなされ、パネルは直接接続にフォールバックします。典型的な例は Xray 自身のローカル SOCKS inbound です。
- ヒントの文言: 「パネル自身の送信リクエスト (geo 更新、Xray / パネルのバージョン確認、Telegram) を、GitHub / Telegram のサーバーサイドフィルタリングを回避するために、このプロキシ経由でルーティングします。socks5:// または http(s):// を受け付けます (例: Xray のローカル SOCKS inbound)。直接接続するには空のままにしてください。」
- 無効なプロキシは保存エラーになりません – パネルは単に直接接続を使用し、ログに警告を書き込みます。

フィールド値の例。 サーバーでポート `10808` にローカル SOCKS Xray inbound がすでに起動している場合、パネル自身のリクエストをそれ経由でルーティングします:

```
socks5://127.0.0.1:10808
```

認証情報付きの外部 HTTP プロキシの場合:

```
http://user:pass@proxy.example.com:8080
```

保存して再起動後、パネルは geo データベースの更新の取得、バージョン確認、および Telegram へのアクセスを指定したプロキシ経由で行います。

トラステッドプロキシ CIDR (*Trusted proxy CIDRs*)

- **キー:** `trustedProxyCIDRs`
- **デフォルト値:** `127.0.0.1/32,::1/128` (ローカルホストのみ)。
- **形式:** IP アドレスまたは CIDR サブネットのカンマ区切りリスト (例: `10.0.0.0/8, 192.168.1.5`)。各要素は IP または CIDR として検証され、不正な値は保存時に拒否されます。
- **目的:** `X-Forwarded-Host`、`X-Forwarded-Proto`、およびクライアントの実 IP ヘッダーの設定が許可されるソースのリスト。ヒント: 「転送されたホスト、プロトコル、クライアント IP ヘッダーの設定が許可される IP / CIDR (カンマ区切り)。」パネルがリバースプロキシ (nginx、Caddy など) の背後で動作している場合、クライアント IP とスキームを正しく判断するために設定が必要です。

例: リバースプロキシの背後にあるパネル。 nginx が同じホストにあり、パネルにリクエストをプロキシしている場合、ローカルホストへの信頼のみを残します (デフォルト値):

```
127.0.0.1/32,::1/128
```


プロキシが内部ネットワーク `10.0.0.0/8` の別サーバーにある場合は、そのサブネットを追加します。そうしないとパネルは転送されたヘッダーを無視し、実際のクライアントではなくプロキシの IP を表示します:

```
127.0.0.1/32,::1/128,10.0.0.0/8
```

実際の IP とスキームを転送する対応する nginx ブロックの例:

```
proxy_set_header X-Real-IP      $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header X-Forwarded-Host $host;
```

13.5. Telegram ボット (「Telegram ボット」 タブ / *Telegram Bot*)

Telegram ボットを有効にする (*Enable Telegram Bot*)

- キー: `tgBotEnable`
- タイプ / デフォルト: ブール値、`false`。
- 目的: Telegram ボットの動作を有効にします。ヒント: 「Telegram ボット経由でパネルの機能にアクセスします。」

Telegram トークン (*Telegram Token*)

- キー: `tgBotToken`
- デフォルト: `""`。
- 目的: ボットのトークン。ヒント: 「Telegram のボットマネージャー `@botfather` でトークンを取得する必要があります。」
- セキュリティの特徴: トークンは機密性の高い値です。パネルの設定読み取りのレスポンスではトークンは返されません (フィールドはクリアされ、「設定済み / 未設定」フラグのみが返されます)。保存時にフィールドを空のままにすると、以前に保存されたトークンは**保持されます** (上書きされません)。

Telegram ボットの言語 (*Telegram Bot Language*)

- キー: `tgLang`
- デフォルト: `en-US`。
- 目的: ボットメッセージの言語 (ウェブインターフェースの言語とは独立)。利用可能な言語リストはパネルの言語と同じです。

ボット管理者のユーザー ID (*Admin Chat ID*)

- キー: `tgBotChatId`
- デフォルト: `""`。
- 形式: カンマ区切りの 1 つまたは複数の数値 Telegram ユーザー ID。

- **目的:** 通知の受信者および、ボット経由でパネルを管理する権限を持つ管理者。ヒント: 「Telegram ボット管理者の 1 つまたは複数のユーザー ID。ユーザー ID を取得するには @userinfobot またはボットの '/id' コマンドを使用してください。」

通知頻度 (Notification Time)

- **キー:** tgRunTime
- **デフォルト:** @daily (1 日 1 回)。
- **形式:** Crontab 形式の文字列 (標準 cron 式および @daily、@hourly、@every 1h などの短縮形をサポート)。ヒント: 「Crontab 形式で通知間隔を指定してください。」ボットの定期レポートを制御します。

フィールド値の例。

値	ボットがレポートを送信するタイミング
@daily	1 日 1 回、真夜中 (デフォルト)
@hourly	毎時
@every 6h	6 時間ごと
0 9 * * *	毎日 09:00
30 8 * * 1	毎週月曜日 08:30

時間は設定 13.6 「タイムゾーン」のタイムゾーンで計算されます。

SOCKS プロキシ (SOCKS Proxy)

- **キー:** tgBotProxy
- **デフォルト:** ""。
- **目的:** Telegram への接続専用のボット用 SOCKS5 プロキシ。ヒント: 「Telegram への接続にプロキシが必要な場合は、ガイドに従って Socks5 パラメータを設定してください。」ボットのトラフィックにのみ適用されます (13.4 の一般的な「パネルネットワークプロキシ」とは異なります)。

Telegram API サーバー (Telegram API Server)

- **キー:** tgBotAPIServer
- **デフォルト:** "" (標準サーバー api.telegram.org を使用)。
- **形式:** http(s)://... の URL。保存時に URL の正確性が検証されます – 無効なアドレスは拒否されます。ヒント: 「使用する Telegram API サーバー。デフォルトサーバーを使用するには空のままにしてください。」自己ホスト型 Telegram Bot API サーバーが必要な場合に使用します。

ボット通知（「通知」グループ / *Notifications*）

フィールド	キー	デフォルト	説明
データベースバックアップ (<i>Database Backup</i>)	tgBotBackup	false	レポートとともに DB バックアップファイルを Telegram に送信する。ヒント：「データベースバックアップファイルとともに通知を送信します。」
ログイン通知 (<i>Login Notification</i>)	tgBotLoginNotify	true	パネルへのログイン試行時に通知する。ヒント：「誰かがパネルにログインしようとしたときのユーザー名、IP アドレス、時刻を表示します。」
セッション有効期限通知の遅延 (<i>Expiration Date Notification</i>)	expireDiff	0	クライアントの有効期限が切れる何日前に通知を送るか。0 – 無効。0 以上が許容されます。ヒント：「しきい値に達する前のセッション期限切れ通知の受信（値：日）。」
トラフィック上限通知のしきい値 (<i>Traffic Cap Notification</i>)	trafficDiff	0	通知を送るトラフィック残量のしきい値。ヒント：「しきい値に達する前のトラフィック枯渇通知の受信（値：GB）。」 0–100 が許容されます。
CPU 負荷通知のしきい値 (<i>CPU Load Notification</i>)	tgCpu	80	CPU 負荷がしきい値（%）を超えた場合に管理者に通知する。0–100 が許容されます。ヒント：「CPU 負荷がこのしきい値を超えた場合に Telegram の管理者に通知します（値：%）。」

13.6. 日付と時刻（「日付と時刻」タブ / *Date and Time*）

タイムゾーン (*Time Zone*)

- **キー:** timeLocation
- **デフォルト値:** Local（サーバーのシステムタイムゾーン）。
- **形式:** IANA tz データベースのゾーン名（例：Europe/Moscow、UTC、Asia/Tehran）。
- **目的:** パネルがスケジュールされたタスク（ボットのレポート、トラフィックのリセット / 確認、有効期限）を実行するタイムゾーン。ヒント：「スケジュールされたタスクはこのタイムゾーンの時間に従って実行されます。」
- **検証:** 保存時にゾーンが検証されます – 存在しないゾーンは拒否されます。後でデータベースに不正な値が含まれている場合、パネルは実行時に Local にフォールバックし、それも利用できない場合は UTC にフォールバックします。

13.7. 外部トラフィックと Xray の動作（「外部トラフィック」タブ / *External Traffic*）

フィールド	キー	デフォルト	説明
外部トラフィック通知 (<i>External Traffic Inform</i>)	externalTrafficInformEnable	false	トラフィック更新のたびに外部 API に通知する。ヒント：「トラフィック更新のたびに外部 API に通知します。」

フィールド	キー	デフォルト	説明
外部トラフィック通知 URI (<i>External Traffic Inform URI</i>)	<code>externalTrafficInformURI</code>	<code>""</code>	パネルがトラフィック更新を送信する URL。保存時に URL の正確性が検証されます。ヒント: 「トラフィック更新はこの URI に送信されます。」
自動無効化後に Xray を再起動 (<i>Restart Xray After Auto Disable</i>)	<code>restartXrayOnClientDisable</code>	<code>true</code>	クライアントが有効期限切れまたはトラフィック制限超過で自動的に無効化された場合に Xray を再起動する。ヒント: 「クライアントが有効期限切れまたはトラフィック制限によって自動的に無効化された場合、Xray を再起動してください。」切り替えスイッチは「パネル」タブ (<i>General</i>) にあります – 13.2 を参照。ここには網羅性のために記載しています。

13.8. その他: Xray 設定テンプレートとテスト URL

Xray 設定テンプレート (*xrayTemplateConfig*)

- キー: `xrayTemplateConfig`
- デフォルト: ビルドに同梱された組み込み (embedded) JSON テンプレート。
- 目的: パネルが inbound / outbound を構築する際のベースとなる Xray コアの JSON 設定テンプレート。この値は通常の設定一覧には含まれず、パネル設定のフィールド一覧ではなく、専用の Xray 設定ページで編集されます。デフォルトの標準テンプレートは `GET /panel/setting/getDefaultJsonConfig` で取得できます。

outbound テスト URL (*xrayOutboundTestUrl*)

- キー: `xrayOutboundTestUrl`
- デフォルト: `https://www.google.com/generate_204`
- 目的: outbound 接続の動作確認に使用される URL。設定時に HTTP(S) URL としてサニタイズされます。

13.9. 管理者アカウントと API トークン

これらのパラメータは隣接するタブ（「アカウント」 / *Authentication*）にあり、セキュリティに関するセクションで詳しく説明されています。ここではキーの簡単な概要を示します。

- **認証情報の変更**（「現在のログイン」、「現在のパスワード」、「新しいログイン」、「新しいパスワード」フィールド）は個別のリクエスト `POST /panel/setting/updateUser` で保存されます。現在の正しいログインとパスワードが必要です。新しいログインとパスワードは空であってはなりません。メッセージ: 「管理者の認証情報を正常に変更しました。」 / 「ユーザー名またはパスワードが正しくありません。」
- **二要素認証 (2FA)** – キー `twoFactorEnable` (デフォルト `false`) と秘密の `twoFactorToken`。トークンは機密です: 2FA が有効な場合、保存時に空のフィールドは既存のトークンを上書きしませ

ん。2FA を初めて有効にすると、パネルは現在のセッションを無効化します（「ログインエポック」が増加します）。

- **API トークン**は個別のエンドポイント（`/panel/setting/apiTokens...`）で管理されます：一覧、作成（`apiTokens/create`）、削除、有効化 / 無効化。トークン自体は**作成時に一度だけ表示**され、読み取り可能な形式では保存されません：「このトークンを今すぐコピーしてください。セキュリティ上の理由から、トークンは読み取り可能な形式では保存されず、再度表示されることはありません。」

2FA、パスワード、LDAP 同期、サブスクリプション形式（JSON / Clash、fragmentation、noises、mux）の詳細は、マニュアルの対応する個別セクションに記載されています。

13.10. 3.3.0 での API の変更（インテグレーションに重要）

バージョン 3.3.0 でサーバー API のパス構造が変更されました。HTTP 経由でパネルにアクセスする外部インテグレーション（スクリプト、ボット、集中管理パネル、CI タスク）がある場合は、**修正が必要です**。そうしないと動作しなくなります。

 **BREAKING CHANGE:** `/panel/setting/*` および `/panel/xray/*` エンドポイントが `/panel/api` 配下に移動

以前はパネル設定と Xray 設定の管理が `/panel/setting/*` および `/panel/xray/*` パスで別々に行われていました。現在は両方のセットが共通 API グループ `/panel/api` 内に登録されています。古いパスは**完全に削除**されました – それらへのリクエストは 404 を返します。

この変更の理由: `/panel/api` グループ全体が統一されたアクセスチェックを通過します。つまり、これらのエンドポイントは他の API と同じ `Authorization: Bearer <token>` ヘッダーを受け付けるようになりました。API トークンは完全な管理者アクセスであり、このようにして API サーフェース全体が統一されました。

変更されていないもの: ウェブインターフェースのページ（SPA ルート）`/panel/settings` および `/panel/xray` はそのままです – サーバーサイドの API エンドポイントのみの変更です。

パスの対応表（旧 → 新）

以下のすべてのパスのプレフィックス – `/panel/` の後に `api/` が追加されただけです。

旧 (≤ 3.2.x)	新 (3.3.0)	メソッド
<code>/panel/setting/all</code>	<code>/panel/api/setting/all</code>	POST
<code>/panel/setting/defaultSettings</code>	<code>/panel/api/setting/defaultSettings</code>	POST
<code>/panel/setting/update</code>	<code>/panel/api/setting/update</code>	POST
<code>/panel/setting/updateUser</code>	<code>/panel/api/setting/updateUser</code>	POST
<code>/panel/setting/restartPanel</code>	<code>/panel/api/setting/restartPanel</code>	POST
<code>/panel/setting/getDefaultJsonConfig</code>	<code>/panel/api/setting/getDefaultJsonConfig</code>	GET

旧 (≦ 3.2.x)	新 (3.3.0)	メソッド
/panel/setting/apiTokens	/panel/api/setting/apiTokens	GET
/panel/setting/apiTokens/create	/panel/api/setting/apiTokens/create	POST
/panel/setting/apiTokens/delete/:id	/panel/api/setting/apiTokens/delete/:id	POST
/panel/setting/apiTokens/setEnabled/:id	/panel/api/setting/apiTokens/setEnabled/:id	POST
/panel/xray/	/panel/api/xray/	POST
/panel/xray/update	/panel/api/xray/update	POST
/panel/xray/getDefaultJsonConfig	/panel/api/xray/getDefaultJsonConfig	GET
/panel/xray/getXrayResult	/panel/api/xray/getXrayResult	GET
/panel/xray/getOutboundsTraffic	/panel/api/xray/getOutboundsTraffic	GET
/panel/xray/resetOutboundsTraffic	/panel/api/xray/resetOutboundsTraffic	POST
/panel/xray/testOutbound	/panel/api/xray/testOutbound	POST
/panel/xray/warp/:action	/panel/api/xray/warp/:action	POST
/panel/xray/nord/:action	/panel/api/xray/nord/:action	POST
/panel/xray/outbound-subs (および /outbound-subs/*)	/panel/api/xray/outbound-subs (および /outbound-subs/*)	GET/POST/DELETE

サブパスの名前、リクエストボディ、レスポンス形式は変更されていません – プレフィックスのみが変更されました。

既存のインテグレーションの修正方法

1. スクリプト / 設定ファイル内の `/panel/setting/` および `/panel/xray/` のすべての出現箇所を検索します。
2. プレフィックスを置換します: `/panel/` の直後に `api/` を追加します (例: `/panel/setting/all` → `/panel/api/setting/all`)。
3. リクエストボディ、パラメータ、レスポンス形式の変更は不要 – URL のみ変更されます。
4. 設定と Xray 設定が `/panel/api` 配下になったため、`/panel/api/inbounds/*` などの他のエンドポイントと同じ API トークン `Authorization: Bearer <token>` でアクセスできます (また、そうすべきです)。 `/panel/api` グループ全体で有効になっている CSRF ミドルウェアを忘れずに。

例: API を使用してすべての設定を読み取る。旧 (≦ 3.2.x) :

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/setting/all" \
-H "Authorization: Bearer <token>"
```

新 (3.3.0) – `/panel/` の後に `api/` を追加:

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/api/setting/all" \
-H "Authorization: Bearer <token>"
```

同様にパネルの再起動: `POST /panel/api/setting/restartPanel`。古いパス `/panel/setting/restartPanel` は現在 404 を返します。

型付き API: スキーマとドキュメント (Swagger / OpenAPI)

3.3.0 では OpenAPI 仕様が完全に型付きになりました。以前は型付きレスポンスが空オブジェクト `{}` で記述されていましたが、現在はコンポーネントとスキーマ (`components.schemas`) がデータモデルから直接生成されます。これにより:

- Swagger UI が実際のデータモデルを表示し、汎用的なプレースホルダーではなくなりました。
- 外部ジェネレーター (`openapi-generator` など) が仕様から任意の言語の完成したクライアントを構築できます。
- 各型付きレスポンスには特定のモデルへの `$ref` が添付され、レスポンスの例が含まれています。

API ドキュメントの参照場所:

- **組み込み Swagger ページ**。パネルメニューの「API ドキュメント」項目 (SPA ルート `/panel/api-docs`)。ここではすべてのエンドポイントが説明、リクエストボディ、レスポンス例とともにインタラクティブに一覧表示されます。
- **生の OpenAPI 3.0 仕様**は `/panel/api/openapi.json` で提供されます。この URL を Postman、Insomnia、または `openapi-generator` に直接渡せます。仕様はビルド時にバイナリに組み込まれています。パネルが非標準の `webBasePath` で動作している場合、仕様内の `servers` フィールドは現在のベースパスに自動的に書き換えられ、「Try it out」ボタンと外部ジェネレーターが正しいプレフィックスにアクセスできます。

14. Telegramボット

3X-UIパネルには組み込みのTelegramボットが含まれており、サーバーとクライアントの状態に関する通知を受け取ったり、メッセージャーから直接個々のクライアントを管理したりできます。ボットはlong polling（Telegramへの継続的なポーリング）技術で動作するため、外部ドメインや開放ポートは不要で、Telegramサーバーへのアウトバウンドアクセスさえあれば十分です。

ボットは2種類の対話相手を区別します：

- **管理者** – ボット設定の「ボット管理者のUser ID」フィールドにTelegram User IDが指定されているユーザー。サーバー統計、バックアップ、クライアント管理、Xray再起動など、すべての機能にアクセスできます。
- **クライアント** – Telegram User IDが特定のinboundクライアントに紐付けられている（クライアントの tgId フィールド）その他のユーザー。自分のサブスクリプション情報のみ閲覧できます。

例：クライアントとTelegramの紐付け。 ユーザーが自分のサブスクリプションの統計情報を受け取れるよう、そのユーザーの数値Telegram User IDをクライアントの tgId フィールドに記録します。クライアントのJSON設定では次のようになります：

```
{
  "email": "ivan",
  "id": "6f1e6b1a-0c3d-4f2a-9b7e-1a2b3c4d5e6f",
  "tgId": "123456789",
  "enable": true,
  "limitIp": 2,
  "totalGB": 53687091200,
  "expiryTime": 0
}
```

これにより、User ID 123456789 を持つユーザーがボットに `/usage ivan` とリクエストすると、自分の統計情報を確認できます。同じIDを管理者はクライアントカードの「 TelegramユーザーIDを設定」ボタンから設定できます。JSONを手動で編集する必要はありません。

14.1. ボットの有効化と設定

ボットのすべてのパラメーターはパネルの **設定** → **Telegramボット** セクションで設定します。設定を変更したら保存するだけで、パネルがすぐに適用します。パネルの再起動は不要です。有効化フラグ

（tgBotEnable）、トークン、管理者のUser ID、またはAPIサーバーアドレスが変更された場合、パネルは自動的に旧ボットを停止し、新しいパラメーターで再起動します。トークン変更後にパネルの再起動が必要だという以前のルールは廃止されました。

フィールド (UI)	設定キー	デフォルト値	説明
Telegramボットを有効にする	tgBotEnable	false	メインスイッチ。ヒント：「Telegramボットを通じてパネル機能にアクセス」。無効の間はボットが起動せず、通知タスクもスケジュールされません。

フィールド (UI)	設定キー	デフォルト値	説明
Telegram トークン	tgBotToken	(空)	ボットのトークン。ヒント: 「Telegramのボットマネージャー @botfatherからトークンを取得してください」。空でないトークンがなければボットは起動しません。
SOCKSプロキシ	tgBotProxy	(空)	Telegramへの接続に使用するプロキシ。ヒント: 「TelegramへのアクセスにSocks5プロキシが必要な場合は、マニュアルに従って設定してください」。
Telegram API サーバー	tgBotAPIServer	(空)	代替のTelegram APIサーバー。ヒント: 「使用するTelegram APIサーバー。デフォルトのサーバーを使用する場合は空のままにしてください」。
ボット管理者のUser ID	tgBotChatId	(空)	カンマ区切りの1つ以上のTelegram管理者User ID。ヒント: 「User IDの取得には@userinfobotを使用するか、ボットに /id コマンドを送信してください」。
ボットから管理者への通知頻度	tgRunTime	@daily	crontab形式の定期レポートスケジュール。ヒント: 「Crontab形式で通知間隔を指定してください」。
データベースのバックアップ	tgBotBackup	false	ヒント: 「データベースのバックアップファイルと共に通知を送信する」。定期レポートにバックアップを添付します。
ログイン通知	tgBotLoginNotify	true	ヒント: 「パネルへのログイン試行時にユーザー名、IPアドレス、時刻を表示する」。
通知するCPU 負荷のしきい値	tgCpu	80	CPU使用率のしきい値 (パーセント) (0~100の検証)。ヒント: 「CPU負荷がこのしきい値を超えた場合にTelegramで管理者に通知する (単位: %)」。0の場合はCPUチェックが無効になります。
Telegramボットの言語	—	—	ボットがすべてのメッセージを生成する際の言語。

@BotFatherからのトークン取得

1. Telegramで **@BotFather** とのダイアログを開きます。
2. /newbot コマンドを送信して、指示に従います (ボット名と bot で終わる一意の username)。
3. BotFatherが 123456789:AA... 形式のトークンを発行します。それを **Telegramトークン** フィールドにコピーします。

管理者のUser ID取得

User IDはアカウントの数値識別子 (usernameではありません) です。2つの方法で確認できます:

- **@userinfobot** にメッセージを送る。
- 既に設定済みのボットを起動して **/id** コマンドを送信する - ボットがあなたのIDを返します。

取得した数値を **ボット管理者のUser ID** フィールドに入力します。複数の管理者を指定するには、IDをカンマ区切りで列挙します（例： 11111111,22222222 ）。各IDは整数として検証されます。無効な値はボットの起動エラーを引き起こします。

例：「**ボット管理者のUser ID**」フィールドの値。管理者が1人の場合は単純な数値：

123456789

2人の管理者をカンマ区切りで（スペースは不要）：

123456789,987654321

各値は整数でなければなりません。@username や 123 456 （数値内にスペース）のような形式は使用できません – ボットが起動しません。

プロキシ

socks5:// 、 http:// 、 https:// の形式がサポートされています。プロキシフィールドが空の場合、ボットはパネルの共通プロキシ（設定されており、形式がサポートされている場合）を使用しようとします。サポートされていない形式または無効な構文のURLは無視され、ボットは直接接続します。プロキシは、サーバーからTelegram APIへの直接アクセスがブロックされている場合に便利です。

メール通知 (SMTP)

Telegram以外に、同じイベントをメールで受け取ることもできます。このチャンネルは **設定 → Email** セクションの **SMTP Settings** タブで設定します：

フィールド (UI)	設定キー	デフォルト値	説明
Enable Email Notifications	smtpEnable	false	SMTPによるメール通知のメインスイッチ。
SMTP Host	smtpHost	(空)	SMTPサーバーのホスト（例： smtp.gmail.com ）。
SMTP Port	smtpPort	587	SMTPサーバーのポート。
SMTP Username	smtpUsername	(空)	SMTP認証のユーザー名。送信者アドレス (From) としても使用されます。
SMTP Password	smtpPassword	(空)	SMTP認証のパスワード。非表示で保存されます。パスワードが既に設定されている場合、フィールドには「設定済み」のサインが表示され、現在の値を維持するために空のままにできます。
Recipients	smtpTo	(空)	カンマ区切りの受信者リスト（例： admin@example.com, ops@example.com ）。
Encryption	smtpEncryptionType	starttls	接続の暗号化タイプ： none （暗号化なし）、starttls (STARTTLS) 、 tls （暗黙のTLS）。

Send Test Email ボタンはテストメールを送信し、**Connection**（接続）、**Authentication**（認証）、**Send**（送信）の各段階で結果を表示します。問題がある場合、診断はエラーが発生した段階を示します（例：「Authentication failed – check username and password」または「Server requires STARTTLS – change encryption type」）。これによりパラメーターの調整が容易になります。

2番目のタブ（**Notifications**）では、メールで通知するイベントを選択します。Telegramと同じカードグループを使用します（14.5のセクション「イベントバスと通知の選択」を参照）。

Telegram APIサーバー

デフォルトでは、ボットは公式のTelegram APIに接続します。**Telegram APIサーバー** フィールドに、独自のBot APIサーバー（`telegram-bot-api`）のアドレスを指定できます。URLはセキュリティ上の理由で検証されます。ブロックされたまたは無効なアドレスは破棄され、デフォルトのサーバーが使用されます。

14.2. メインメニューとボタン

メニューは `/start` コマンドで呼び出します。ボタンはメッセージに添付されたインラインキーボードです。ボタンのセットは、あなたが管理者かクライアントかによって異なります。

管理者メニュー

ボタン	動作
 トラフィック使用量ソートレポート	トラフィック順に並べたすべてのクライアントの一覧と各自の使用量を表示します。データの無い余分なメールは「結果なし」とマークされます。
 サーバー状態	サーバーの概要（14.5セクション参照）。「更新」ボタンでデータを再描画します。
 全トラフィックをリセット	すべてのクライアントのトラフィックカウンターをリセットします。確認（「本当によろしいですか？」）を求め、各クライアントに対して「成功」または「失敗」を表示し、最後に「全クライアントのトラフィックリセットが完了しました」を表示します。
 DBバックアップ	データベースファイルと <code>config.json</code> を送信します（14.6セクション参照）。
 BANログ	IPリミット超過によりBANされたIPアドレスのログファイルを送信します。
 Inbound接続	全inboundの概要：Remark、ポート、トラフィック、クライアント数、有効期限。
 もうすぐ終了	トラフィックまたは期限がもうすぐ切れるinboundとクライアントの一覧（14.5セクション参照）。
 コマンド	管理者コマンドのヘルプを表示します。
 オンライン	オンラインのクライアント数と一覧。メールをタップするとクライアントカードが開きます。「更新」ボタン。
 全クライアント	inboundの選択→クライアント一覧の順で開き、閲覧・管理できます。
 新規クライアント	クライアント追加ウィザードを起動します（inbound選択→下書き→確認）。
サブスクリプション設定 / 個別リンク / QRコード	サブスクリプションリンク、個別リンク、またはQRコードを取得するためのinboundとクライアントの選択。

クライアントメニュー

クライアントには限定されたボタンセットが用意されています：

ボタン	動作
クライアント統計	クライアントのTelegram User IDに紐付けられた全サブスクリプションのデータを表示します。
コマンド	クライアントコマンドのヘルプを表示します。
サブスクリプション設定	自分のクライアントを選択→サブスクリプションリンクを表示。
個別リンク	自分のクライアントを選択→個別リンクを表示。
QRコード	自分のクライアントを選択→QRコードを表示。

ユーザーのTelegram User IDに紐付けられたクライアントが1つもない場合、ボットは次のように返信します：「 あなたの設定が見つかりません！ 管理者にあなたのTelegram User IDを設定に使用するよう依頼してください。 あなたのUser ID: ...」。このIDを管理者に伝え、クライアントのフィールドに入力してもらう必要があります。

14.3. ボットのコマンド

バージョン3.5.0 から、Telegramの「/」メニューには **8つ** のコマンドが登録されています：

コマンド	説明（メニューより）	アクセス	動作
/start	メインメニューを表示	すべて	ウェルカムメッセージ。管理者には追加で「  <ホスト>管理ボットへようこそ！」とメインメニューが表示されます。
/help	ボットのヘルプ	すべて	一般的なウェルカムメッセージとメニュー項目を選択するよう促すメッセージを表示します。
/status	ボットのステータスを確認	すべて	 「ボットは正常に動作しています」と返信します。
/id	あなたのTelegram IDを表示	すべて	 「あなたのUser ID: ...」を返します。自分のUser IDを確認するのに便利です。
/usage	クライアントの使用量を表示: /usage email	すべて (下記参照)	3.5.0 でメニューに追加されました。
/inbound	inbound を検索: /inbound remark (admin)	管理者	3.5.0 でメニューに追加されました。
/restart	Xray コアを再起動 (admin)	管理者	3.5.0 でメニューに追加されました。
/clearall	全クライアントのトラフィックをリセット (admin)	管理者	3.5.0 の新機能: 確認を求め（「キャンセル」/「トラフィックリセットを確認」ボタン）、全クライアントのトラフィックをゼロにします。

引数コマンドの詳細:

- **/usage [Email]** – メールでクライアントを検索します。
 - **管理者** にはクライアントの完全なカード（管理ボタン付き）が表示されます。
 - **クライアント** には、指定したメールの自分のサブスクリプションのみ（Telegram User IDの紐付けによる）が表示されます。引数なしの場合、ボットはメールを指定するよう促します: 「 検索するメールアドレスを指定してください」。
- **/inbound [接続名]** – 管理者専用。RemarkでInboundを検索し、全クライアントの統計情報とともにパラメーターを表示します。引数なし（またはクライアント）の場合 – 「 不明なコマンド」。
- **/restart** – 管理者専用。Xray Coreを再起動します。可能な返答: 「 Xrayコアが正常に再起動されました」、 「 Xray Coreが起動していません」（コアが動作していない場合）、 「 Xray-coreの再起動中にエラーが発生しました。<エラー>」。 **/restart** の後に引数を付けると、不明なコマンドのメッセージと **/restart** のヒントが表示されます。

グループチャットでは、**/コマンド@botusername** 形式のコマンドは、usernameが現在のボットの名前と一致する場合にのみ処理されます。

さらに 3.5.0 では3つの変更があります: **オンラインクライアント** のリストでは、ボタンに **email - inbound** の備考 というラベルが付きます（異なる inbound の同名クライアントが混同されなくなりました）。**バックアップ** と **BANログ** のメッセージは **Hostname==...** の行で始まります – 1つのボットが複数のパネルを担当する場合に便利です。**Telegram ID** によるクライアントカードの検索は、設定 JSON のフォーマットに依存せず機能します（以前は、ノードやインポート由来の「コンパクトな」レコードが見つかりませんでした）。

管理者ヘルプ（「コマンド」ボタン）:

 Xray Coreを再起動するには: **/restart**
 メールでクライアントを検索するには: **/usage [Email]**
 Inbound接続を検索するには（クライアント統計付き）: **/inbound [接続名]**
 あなたのTelegram User ID: **/id**

クライアントヘルプ:

 サブスクリプション情報を確認するには: **/usage [Email]**
 あなたのTelegram User ID: **/id**

14.4. クライアント管理（管理者のみ）

クライアントカードを開くと（「全クライアント」、「オンライン」、「もうすぐ終了」、または **/usage** 経由）、管理者はクライアントの情報（メール、紐付けられたinbound、「アクティブ」状態、接続状態、有効期限、トラフィック使用量）と管理用インラインボタンを確認できます:

ボタン	目的
 更新	クライアントカードを再読み込みします。

ボタン	目的
 トラフィックをリセット	クライアントのトラフィックカウンターをリセットします。  「 トラフィックリセットを確認しますか? 」の確認が必要です。
 トラフィックリミット	トラフィック制限を設定します。プリセット値:  無制限 (0)、1/5/10/20/30/40/50/60/80/100/150/200 GB、または「 カスタム 」 - 組み込み数字キーボードで数値を入力 (0~9のボタン、「 0 」 - 0にリセット、「 0 」 - 最後の数字を削除、  「 確認: N 」)。値はギガバイト単位で設定します。
 有効期限を変更	プリセット:  無制限、「 カスタム 」、7/10/14/20日を追加、1/3/6/12ヶ月を追加。正の数値は期限を延長します (現在の有効期限または期限が切れている場合は「 現在 」に日数を加算します)。0は期限の制限を解除します。
 IPログ	クライアントの記録されたIPアドレス (タイムスタンプ付き、ある場合) を表示します。ログから「 更新 」と「 IPをクリア 」 ( 「 IPのクリアを確認しますか? 」の確認付き) を利用できます。
 IPリミット	同時接続IP数の制限。オプション:  無制限 (0)、1~10、または「 カスタム 」 (数字キーボード)。
 Telegram ユーザーIDを設定	クライアントに紐付けられた現在のTelegram User IDを表示します。紐付けを解除できます (「 Telegramユーザーを削除 」、確認付き)。新規ユーザーの紐付けはTelegramの連絡先選択システムを通じて行います。
 有効化/無効化	クライアントを有効または無効にします。  「 ユーザーの有効化/無効化を確認しますか? 」の確認が必要です。

設定を変更するすべての操作 (トラフィック/IPリミット、有効期限、Telegramユーザーの紐付け/解除、有効化/無効化) は、必要に応じてXrayを再起動のマークを付けて変更を反映させます。操作が成功すると、ボットは「 <メール>: ...」の形式で確認メッセージを表示し、カードを再表示します。

ウィザードでの数値入力 < 999999 の値に制限されています。

14.5. 通知とレポート

通知はすべての管理者 (tgBotChatId のすべてのUser ID) に送信されます。

イベントバスと通知の選択

通知は単一のイベントバス上に構築されており、配信チャンネルは **Telegram** と **メール (SMTP)** の2つです。各チャンネルで、どのイベントについて通知を受け取るかを個別に選択します。**設定** → **Telegram** では **Notifications** タブで、**設定** → **Email** では同名のタブで設定します。

イベントはカードにグループ化されています。各グループには有効なイベント数のカウンター (n/合計) と、一部のみ選択されている場合の中間状態を持つマスタースイッチがあります。利用可能なグループ:

- **Outbound** - 「Down」 (outbound.down) と 「Up」 (outbound.up) : outboundの停止と復旧。
- **Xray Core** - 「Crash」 (xray.crash) : Xrayコアの異常終了。
- **Nodes** - 「Down」 (node.down) と 「Up」 (node.up) : ノードが利用不可になったか復旧した。

- **System** – 「CPU high (%)」 (`cpu.high`) と 「Memory high (%)」 (`memory.high`) : 高CPU使用率と高メモリ使用率。両方のイベントの横にしきい値のインラインフィールド（パーセント単位）があります。
- **Security** – 「Login attempt」 (`login.attempt`) : パネルへのログイン試行。

有効なイベントのセットは個別に保存されます: Telegramは `tgEnabledEvents`、メールは `smtpEnabledEvents`。デフォルトでは、両チャンネルで「Login attempt」と「CPU high」が有効です (値: `login.attempt,cpu.high`)。

パネルへのログイン通知

ログイン通知 チェックボックス (`tgBotLoginNotify`、デフォルトで有効) で管理します。Webパネルへのログイン試行のたびに、管理者にメッセージが届きます:

- 成功時: 「 パネルへのログインに成功しました。」 + ホスト、ユーザー名、IP、時刻。
- 失敗時: 「 パネルへのログインに失敗しました。」 + ホスト、理由 (例: 二要素認証エラーの場合「2FAエラー」)、ユーザー名、IP、時刻。

CPUとメモリの高負荷

パネルは1分ごとにCPUとメモリの使用率をチェックします。 `tgCpu` のしきい値が0より大きく、1分間の平均CPU使用率がそれを超えた場合、管理者に「 CPU使用率がN%でしきい値M%を超えています」と送信されます。同様に、しきい値 `tgMemory` (デフォルト80%) に対してRAM使用率もチェックされます – 「Memory high (%)」 イベント。

両方のしきい値は、Notificationsタブの **System** グループの「CPU high (%)」と「Memory high (%)」 イベントの横のインラインフィールドで設定します (「イベントバスと通知の選択」を参照)。メールチャンネルには個別のキー `smtpCpu` と `smtpMemory` が使用されます。しきい値が0の場合、対応するチェックが無効になります。

定期レポート (スケジュール)

通知頻度 フィールド (`tgRunTime`、デフォルト `@daily`) のcron式に従ってスケジュールされます。値が空または無効な場合は `@daily` が使用されます。レポートには以下が含まれます:

スケジュールビルダー

ボットから管理者への**通知頻度** フィールドは文字列を手動入力するのではなく、スケジュールビルダーを通じて設定します。まずドロップダウンリストでモードを選択します:

- **@every** – 間隔で繰り返す – 数値フィールドと単位の選択 (秒 / 分 / 時) が表示されます。結果は `@every 6h` のような式に組み立てられます。
- **@hourly** – 毎時、**@daily** – 毎日00:00、**@weekly** – 毎週、**@monthly** – 毎月 – 対応するマクロ (`@hourly`、`@daily`、`@weekly`、`@monthly`) として保存されるプリセット。
- **カスタム (crontab)** – 独自のcrontab式のフィールド。パネルのスケジューラーは秒が有効な状態で動作するため、カスタム式は6つのフィールドで構成されます: 秒、分、時、日 (月内)、月、曜日

(例: 0 30 8 * * * - 毎日08:30:00)。このモードに切り替えると、フィールドに現在の選択の crontab相当が入力されるので、そこから編集を始められます。

例: 「通知頻度」フィールド (`tgRunTime`) の値。プリセットの省略形とフルcrontab形式の両方がサポートされます:

値	トリガータイミング
@daily	毎日深夜0時 (デフォルト値)
@hourly	毎時
@every 6h	6時間ごと
0 9 * * *	毎日09:00
0 9 * * 1	毎週月曜日09:00
0 */12 * * *	12時間ごと (00:00と12:00)

crontabのフィールド順: 分、時、日 (月内)、月、曜日。

1. 「 予定レポート: <スケジュール>」の行と現在の日時。
2. サーバー状態 (以下参照)。
3. inboundとクライアントの「もうすぐ終了」ブロック。
4. Telegram User IDが紐付けられたクライアントへの個別通知 – 管理者以外の各クライアントに、トラフィック/期限が間もなく切れるサブスクリプションの一覧 (無効化されているものも含む) が送信されます。
5. データベースのバックアップ (`tgBotBackup`) が有効な場合 – 管理者にDBバックアップを送信。

サーバー状態 には以下が含まれます: ホスト、3X-UIとXrayのバージョン、IPv4/IPv6、稼働時間 (日数)、平均負荷 (Load1/2/3)、RAM (現在/合計)、オンラインクライアント数、TCP/UDP接続カウンター、合計ネットワークトラフィック (↑/↓)、Xrayのステータス。

「もうすぐ終了」 に表示されるもの:

- inboundごと: 無効化された数と「間もなく切れる」数、その後そのようなinboundの列挙 (Remark、ポート、トラフィック、有効期限)。
- クライアントごと: 同じ情報、さらにクライアントカードとメールボタン (タップするとクライアントカードが開きます)。

「間もなく切れる」のしきい値はパネルの共通設定から取得されます: トラフィックの余裕 (GB単位) と期限の余裕 (日数)。残りトラフィックがしきい値を下回る、または残り日数がしきい値を下回る inbound/クライアントが「間もなく切れる」と見なされます。

14.6. バックアップとログ

- **DBバックアップ** ( DBバックアップ」ボタンまたは定期レポートのチェックボックス): ボットはバックアップ時刻、データベースファイル (`x-ui.db`、またはPostgreSQLの場合は `x-ui.dump`)、Xray設定ファイル `config.json` を送信します。

ボットが送信するバックアップファイルの名前はサーバーアドレスから生成されます：**webDomain** の値が使用され、設定されていない場合はサーバーの公開IPが使用されます。複数のパネルからバックアップを収集する際に、どのサーバーからファイルが届いたかを識別するのに役立ちます。アドレスを特定できない場合は、汎用名が使用されます。

- **BANログ**（「 BANログ」ボタン）：IPリミット超過によりBANされたIPアドレスの現在のログファイルと前のログファイルを送信します。空のファイルは送信されません。

14.7. 動作の特性

- **長いメッセージ** は複数のパートに分割されます（しきい値：約2000文字）。インラインキーボードは最後のパートに添付されます。
 - **並行処理**：コマンドとボタンのタップは並行して処理されます（最大10個の同時ハンドラーのプール）。
 - **送信の信頼性**：接続エラー時には指数バックオフ（1秒/2秒/4秒、最大3回の試行）でメッセージが再送されます。
 - **キャッシング**：「サーバー状態」データはキャッシュされ、「更新」を頻繁に押してもシステムに負荷がかかりません。
 - **ボットの再起動**：ボットに影響する設定（有効化フラグ、トークン、管理者のUser ID、またはAPIサーバーアドレス）を保存すると、パネルは自動的に前のポーリンググループを停止し、最新のパラメーターで新しいループを起動します。パネルの再読み込みは不要です。更新受信のインスタンスは同時に1つだけ動作します。
-

15. ジオデータベース (geoip / geosite およびカスタム)

ジオデータベースとは、Xray-core がトラフィックを国別 (IP アドレス範囲) またはドメインカテゴリ別に振り分け・フィルタリングするために使用するバイナリ `.dat` ファイルです。パネルは標準のジオファイル一式と、URL で指定した任意のカスタムソースの両方を取得・更新できます。すべてのファイルは Xray バイナリと同じ `bin` ディレクトリに保存されます (デフォルトパスは `bin`、環境変数 `XUI_BIN_FOLDER` で変更可能)。

15.1. geoip.dat と geosite.dat とは

- **geoip.dat** – 「IP アドレス → 国・地域コード」の対応データベース。ルーティングルールで `geoip:<コード>` の形式 (例: `geoip:ru`、`geoip:cn`) や、特別なラベル `geoip:private` (プライベート/ローカルネットワーク) として使用されます。「この IP はどの国にあるか」を答えるものです。
- **geosite.dat** – 「ドメイン → カテゴリ/リスト」の対応データベース。 `geosite:<カテゴリ>` の形式 (例: `geosite:category-ads-all` (広告ドメイン)、`geosite:google`、`geosite:ru`) で使用されます。グループ化されたドメインリストです。

これらのファイルは、「ロシアの IP/ドメイン宛てのトラフィックはすべて直接接続、それ以外は outbound 経由」といったルールを構築するために必要です。ルール自体は Xray のルーティングセクションで設定し、ジオデータベースはそのためのデータを提供するにすぎません。最新のジオファイルがなければ、`geoip:/geosite:` を参照するルールは機能しないか、古いリストに基づいて動作します。

例: 「ロシアのドメインと IP を直接接続」ルール。 ルーティングセクションにあるこのルールは、ロシアのリソースへのすべてのトラフィックを `direct` タグの outbound に向けます:

```
{
  "type": "field",
  "domain": ["geosite:category-ru"],
  "ip": ["geoip:ru"],
  "outboundTag": "direct"
}
```

15.2. 標準ジオファイルとその更新

パネルには、ダウンロード元が固定された 6 つの標準ファイルからなる固定の allowlist があります。更新は `POST /panel/api/server/updateGeofile/:fileName` (ファイル名なしで全ファイルを一括更新) で実行します。

例: API による単一ファイルおよび全ファイルの更新。 `geoip_RU.dat` のみを更新:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile/
geoip_RU.dat' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

6 つの標準ファイルをすべて一度のリクエストで更新 (ファイル名を指定しない):

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

成功時のレスポンス:

```
{ "success": true, "msg": "Geofile updated successfully", "obj": null }
```

ファイル名	ソース (リリースリポジトリ)
geoup.dat	github.com/Loyalsoldier/v2ray-rules-dat (geoup.dat)
geosite.dat	github.com/Loyalsoldier/v2ray-rules-dat (geosite.dat)
geoup_IR.dat	github.com/chocolate4u/Iran-v2ray-rules (geoup.dat)
geosite_IR.dat	github.com/chocolate4u/Iran-v2ray-rules (geosite.dat)
geoup_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geoup.dat)
geosite_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geosite.dat)

標準ファイル更新時の注意事項:

- **単一ファイルの更新ボタン。** ダウンロード前に確認ダイアログが表示されます: 「ジオファイルを更新しますか?」と「#filename# ファイルが更新されます」 (英: *Do you really want to update the geofile? This will update the #filename# file.*)。成功すると「ジオファイルが正常に更新されました」 (英: *Geofile updated successfully*) という通知が表示されます。
- **「すべて更新」ボタン** (英: *Update all*) は 6 つのファイルすべてをダウンロードします。確認メッセージ: 「すべてのジオファイルが更新されます」 (英: *This will update all geofiles.*)。
- **条件付きダウンロード。** ローカルファイルがすでに存在する場合、リクエストにはファイルの更新時刻を含む `If-Modified-Since` ヘッダーが付与されます。サーバーからの `304 Not Modified` レスポンスはファイルが変更されていないことを意味し、再ダウンロードは行われず、ファイルのタイムスタンプのみが更新されます。
- **ファイル名のセキュリティ。** `allowlist` に含まれるファイル名のみが受け付けられます。ファイル名には `..`、パス区切り文字 `/` や `\`、絶対パスが含まれていないことが確認され、`^[a-zA-Z0-9._-]+\\.dat$` のパターンに一致する必要があります。リスト外のファイル名は「Invalid geofile name」エラーで拒否されます。
- **Xray の再起動。** ジオファイルのダウンロード後、Xray-core は更新されたデータベースを読み込むために再起動されます。再起動に失敗した場合は、エラーメッセージに対応する情報が追加されます。

コマンドラインからのジオデータベース更新 (x-ui)

ジオデータベースはパネルを使わずに更新することもできます。x-ui のインタラクティブメニュー (ジオファイル更新項目)、またはノンインタラクティブコマンド `x-ui update-all-geofiles` を使用します。各ファイルセット (IR および RU セットを含む `geoup/geosite`) について、「更新済み」「既に最新」「ダウンロードエラー」のいずれかのステータスが個別に表示されます。ダウンロードに失敗した場合、誤った成功メッセージは表示されません。Xray の再起動 (および既存接続の切断) は、少なくとも 1 つのファイルが実際に更新された場合にのみ行われます。すべてのファイルが変更なし (すべて `304 Not Modified` を返した) 場合は、パネルも Xray も再起動されません。

15.3. Xray による Geodata 自動更新 (Geodata Auto-Update)

任意の URL による追加の `.dat` ソースは、パネルの機能ではなく Xray-core のネイティブな `geodata` セクションで追加します。該当セクションは Xray 更新モーダルウィンドウ（ダッシュボード → Xray 更新、`xrayUpdates`）内の「Geodata Auto-Update」タブに配置されています。パネルはここで Xray 設定テンプレートの `geodata` キーを編集するだけで、ファイルのダウンロード・検証・ホットリロードは Xray コア自身が行います。

セクション上部には次のヒントが表示されます：「Xray はこれらのファイルをスケジュールに従ってダウンロードし、再起動なしにホットリロードします。URL は HTTPS である必要があります。Xray が更新を行うには、事前に `bin` フォルダにファイルが存在している必要があります」（英: *Xray downloads these files on schedule and hot-reloads them without a restart. URLs must be HTTPS. Each file must already exist in the bin folder once before Xray can update it.*）。

フィールドの説明

- **スケジュール (cron)** (英: *Schedule (cron)*) – 5 フィールドの cron 文字列。デフォルト値は `0 4 * * *`（毎日 04:00）。保存時に 5 フィールドが含まれているか検証され、不正な場合は「Cron は 5 フィールドを含む必要があります（例: `0 4 * * *`）」というエラーが表示されます。
- **outbound 経由でダウンロード (任意)** (英: *Download through outbound (optional)*) – 利用可能な outbound タグ（サブスクリプションの outbound を含む）のドロップダウンリスト。`blackhole` プロトコルの outbound はリストに含まれません。空のままにすると直接接続が使用されます。この設定はパネル自身のリクエスト用 outbound (§11 参照) とは独立しており、`geodata` 自動更新には専用の outbound が使われます。
- **ファイルリスト** – 各行が「URL + ファイル名」（英: *File name*）のペアを定義します。URL は `https://` で始まる必要があります（そうでなければ「各ファイルに HTTPS URL が必要です」）。ファイル名はパスや区切り文字を含まないシンプルな形式で指定し、使用できる文字は `^[A-Za-z0-9._-]+$` のみです（そうでなければ「ファイル名はシンプルな形式にしてください（例: `geosite_custom.dat`、パスは不可）」）。URL 入力時、パネルはパスの最後のセグメントからファイル名を自動補完しようとします。「ファイルを追加」ボタン（英: *Add file*）で行を追加し、ゴミ箱ボタンで削除します。

リストが空の場合は次のヒントが表示されます：「ファイルが設定されていません。ルーティングルールでは `ext:geosite_custom.dat:category` の形式でファイルを参照してください」（英: *No files configured. Reference files in routing rules as ext:geosite_custom.dat:category.*）。

保存

「保存して Xray を再起動」ボタン（英: *Save & Restart Xray*）を押すと「`geodata` 設定を保存しますか?」という確認が表示され、「Xray 設定テンプレートが更新され、Xray が再起動されます」（英: *Save geodata settings? This updates the Xray config template and restarts Xray.*）という説明が添えられます。保存後、`geodata` キーが設定テンプレートに書き込まれ（`POST /panel/api/xray/update`）、Xray が再起動されます（`POST /panel/api/server/restartXrayService`）。ファイルリストが空の場合、`geodata` キーはテンプレートから削除されます。

重要な注意事項:

- **ファイルは事前に `bin` に存在する必要があります。** Xray は起動時点で `bin` フォルダに存在する `.dat` ファイルのみを更新します。そのため、新しいカスタムファイルはまず手動で `bin` に配置し（あるいは少なくとも適切な名前で空/古いバージョンを作成し）、その後 Xray がスケジュールに従って最新状態を維持するようになります。
- **ホットリロード。** 定期的なダウンロード後、Xray はプロセスを完全に再起動することなく更新されたデータベースを再読み込みします。
- **互換性。** 以前にダウンロードしたジオファイル（標準・カスタムともに）は、`ext:` 構文を使用したルーティングルールで変更なく引き続き機能します。

リストが空の場合は次のヒントが表示されます: 「カスタム geo ソースがまだありません – 「追加」をクリックして作成してください」 (英: *No custom geo sources yet – click Add to create one*) 。

テーブルのカラムとソースフィールド

フィールド (UI)	JSON	デフォルト値	説明
タイプ (Type)	<code>type</code>	– (必須)	リソースタイプ: <code>geosite</code> または <code>geoip</code> のみ。最終的なファイル名を決定します。
エイリアス (Alias)	<code>alias</code>	– (必須)	ソースの短い識別子。タイプとともにファイル名を構成します。
URL (URL)	<code>url</code>	– (必須)	<code>.dat</code> ファイルへの直接リンク (http/https) 。
有効 (Enabled)	–	–	リスト内でのソースのアクティブ状態。
更新日時 (Last updated)	<code>lastUpdatedAt</code>	<code>0</code>	最後に正常に更新された時刻 (Unix 時刻、 <code>0</code> は未更新) 。
ルーティング (ext:...) (Routing (ext:...))	–	–	ルーティングルール用の既成文字列: <code>ext:<ファイル.dat>;tag</code> 。
操作 (Actions)	–	–	「編集」「削除」「今すぐ更新」ボタン。

データベースにはさらに補助フィールドも保存されます: `localPath` (`bin` ディレクトリ内の実際のファイルパス)、`lastModified` (条件付きダウンロードに使用されるサーバーからの `Last-Modified` ヘッダー値)、`createdAt`、`updatedAt` 。

ファイルの命名規則

最終的なファイル名はタイプとエイリアスから自動的に生成されます:

- タイプ `geoip` → `geoip_<alias>.dat` ;
- タイプ `geosite` → `geosite_<alias>.dat` 。

例えば、タイプ `geosite`、エイリアス `myads` のソースは `geosite_myads.dat` というファイルを作成します。

例: API によるソースの追加。 `myads` というエイリアスで広告ドメインの独自リストを `geosite` リソースとして追加する:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/customGeo/add' \
-H 'Cookie: 3x-ui=<session-cookie>' \
-H 'Content-Type: application/json' \
-d '{
  "type": "geosite",
  "alias": "myads",
  "url": "https://example.com/lists/myads.dat"
}'
```

パネルはファイルを `geosite_myads.dat` として `bin` ディレクトリにダウンロードし、レコードを保存して Xray を再起動します。

ボタンと操作

- **追加** (英: *Add*) – 「ソースを追加」フォーム (英: *Add custom geo*) を開きます。保存ボタンは「保存」 (英: *Save*)。API: `POST /add`。
- **編集** (英: *Edit*) – 「ソースを編集」フォーム (英: *Edit custom geo*)。API: `POST /update/:id`。タイプまたはエイリアスを変更した場合、古いファイルは削除され新しいファイルが再ダウンロードされます。
- **削除** (英: *Delete*) – 「このカスタムソースを削除しますか?」 (英: *Delete this custom geo source?*) の確認。データベースからレコードと `.dat` ファイル自体を削除します。API: `POST /delete/:id`。成功時: 「カスタム geo ファイル「<名前>」が削除されました」。
- **今すぐ更新** (英: *Update now*) – 特定のソースを再ダウンロードしてタイムスタンプを更新します。API: `POST /download/:id`。成功時: 「Geofile「<名前>」が更新されました」。
- **すべて更新** – すべてのカスタムソースをまとめて更新します。API: `POST /update-all`。完全成功時: 「すべてのカスタムソースが更新されました」 (英: *All custom geo sources updated*)。1 つでも更新に失敗した場合は部分的な失敗とみなされ、「1 つ以上のカスタムソースの更新に失敗しました」 (英: *One or more custom geo sources failed to update*) というメッセージとともに成功・失敗したソースの一覧がレスポンスに含まれます。

いずれの操作後 (追加、編集、削除、更新、成功を含む全体更新) も Xray-core が再起動されます。

手順: ソースの追加

1. 「追加」をクリックします。
2. 「タイプ」フィールドで `geosite` または `geoip` を選択します。
3. 「エイリアス」フィールドに識別子を入力します (半角英小文字、数字、`-`、`_` のみ。プレースホルダーヒント: `a-z 0-9 _ -`)。
4. 「URL」フィールドに `.dat` ファイルへの直接リンクを入力します (`http://` または `https://` で始まる必要があります)。
5. 「保存」をクリックします。パネルはただちにファイルを `bin` ディレクトリにダウンロードし、レコードを保存して Xray を再起動します。

15.4. バリデーションと制限事項

ソースの作成および編集時には厳格なチェックが行われます。エラーメッセージ一覧:

条件	メッセージ (日本語)	メッセージ (EN)
タイプが geosite / geoip 以外	タイプは geosite または geoip である必要があります	Type must be geosite or geoip
エイリアスが空	エイリアスを入力してください	Alias is required
エイリアスに使用不可の文字 (^[a-z0-9_-]+\$ 以外)	エイリアスに使用できない文字が含まれています	Alias must match allowed characters
エイリアスが予約済み	このエイリアスは予約されています	This alias is reserved
URL が空	URL を入力してください	URL is required
URL がパースできない	URL が正しくありません	URL is invalid
スキームが http/https 以外	URL は http または https を使用する必要があります	URL must use http or https
ホストが空/不正、または SSRF 保護によりブロック	URL のホストが正しくありません	URL host is invalid
「タイプ + エイリアス」が重複	このタイプでそのエイリアスはすでに使用されています	This alias is already used for this type
ソースが見つからない	ソースが見つかりません	Custom geo source not found
ダウンロードエラー	ダウンロードに失敗しました	Download failed

フォームのヒント (クライアントサイドバリデーション): 「エイリアスには半角英小文字、数字、- および _ のみ使用できます」 (Alias may only contain lowercase letters, digits, - and _) および 「URL は http:// または https:// で始まる必要があります」 (URL must start with http:// or https://)。

追加の技術的制限:

- **予約済みエイリアス。** 標準ファイルと競合するエイリアスは使用できません。予約済みエイリアス (大文字小文字を区別せず、ハイフンはアンダースコアと同一視): `geoip`、`geosite`、`geoip_ir`、`geosite_ir`、`geoip_ru`、`geosite_ru`。例えば `geosite-ru` は `geosite_ru` として拒否されます。
- **SSRF 保護。** URL のホストが IP に解決される際、プライベート/内部アドレスを指している場合はダウンロードがブロックされます (ユーザーには「URL のホストが正しくありません」と表示されます)。これにより、パネルが内部サービスへのアクセスに使用されるのを防ぎます。
- **パストラバーサル対策。** ファイルの最終パスはシンボリックリンクを解決した上で `bin` ディレクトリ内に収まる必要があります。ディレクトリ外への脱出は拒否されます。
- **最小ファイルサイズ。** ダウンロードされたファイルは 64 バイト以上の場合のみ有効とみなされます。小さすぎるファイルはダウンロードエラーとして拒否されます。
- **プロキシと条件付きダウンロード。** パネル設定にプロキシが設定されている場合はそれを経由してダウンロードが行われ、それ以外の場合は SSRF セーフなトランスポートで直接接続します。標準ファイルと同様に、`If-Modified-Since / 304 Not Modified` (変更なしのファイルは再ダウンロードしない)

い) が適用されます。ダウンロードのタイムアウトは 10 分、URL の到達可能性チェック (HEAD、必要に応じて部分的な GET) は 12 秒です。

15.5. パネル起動時の自動チェック

パネルの起動時、すべてのカスタムソースをループして各ソースのローカルファイルの存在と整合性をチェックします (ファイルが存在しない、ディレクトリである、または 64 バイト未満)。ファイルが存在しないか破損している場合は、ソースへの疎通確認と再ダウンロードが試みられます。これにより、再インストール後や `bin` ディレクトリを紛失した後でも、カスタムジオファイルが自動的に復元されることが保証されます。

15.6. ルーティングルールでのジオデータベースの使用

Xray のルーティングルールでは、ジオデータベースは `domain / ip` などのフィールドでプレフィックスを使って参照します:

- **geoip:** IP ベースは `geoip:<コード>`。例: `geoip:ru`、`geoip:cn`、`geoip:private`。`geoip.dat` (またはルールが特定ファイルを指す場合は `geoip_RU.dat` など) から取得されます。
- **geosite:** ドメインベースは `geosite:<カテゴリ>`。例: `geosite:category-ads-all`、`geosite:google`、`geosite:ru`。`geosite.dat` から取得されます。

例: geosite による広告ブロック。 すべての広告ドメインを「ブラックホール」に送るルール (`blocked` タグで `blackhole` プロトコルの `outbound` が存在することを前提とします):

```
{
  "type": "field",
  "domain": ["geosite:category-ads-all"],
  "outboundTag": "blocked"
}
```

カスタムファイルには `ext:` 外部ファイル構文を使用します。UI のヒント: 「ルーティングルールでは、値の列を `ext:ファイル.dat:タグ` として使用してください (タグを置き換えてください)」 (英: *In routing rules use the value column as `ext:file.dat:tag` (replace tag).*)。形式:

```
ext:<ファイル名.dat>:<タグ>
```

ここで `<ファイル名.dat>` は `geoip_<alias>.dat` または `geosite_<alias>.dat`、`<タグ>` はファイル内の特定リスト/カテゴリです。パネルの「ルーティング (`ext:...`)」カラムには `ext:geosite_myads.dat:tag` のような既成テンプレートが表示されます。`tag` を必要なタグに置き換えるだけです。このファイル名は「Geodata Auto-Update」セクション (§15.3 参照) の「ファイル名」フィールドで指定します (例: `geosite_custom.dat`)。ルーティングルールでは `ext:geosite_custom.dat:category` として参照します。

例: カスタムファイルに基づくルール。 タイプ `geosite`、エイリアス `myads` のソースが追加されており、`.dat` ファイル内のリストが `ads` タグでラベル付けされている場合、ルーティングルールは次のようになります:

```
{
  "type": "field",
  "domain": ["ext:geosite_myads.dat:ads"],
  "outboundTag": "blocked"
}
```

IP ソース（タイプ `geoip`、エイリアス `mycorp`、タグ `office`）の場合、フィールドは `"ip": ["ext:geoip_mycorp.dat:office"]` になります。

16. 運用：バックアップ、ログ、更新、CLI

このセクションでは、パネルの日常的なメンテナンスを説明します。データベースのバックアップと復元、パネルおよびXrayのログ閲覧、サービスの再起動と停止、XrayおよびパネルのUI本体の更新、定期タスク（cron）、パネルのアンインストールを対象とします。一部の操作はWebインターフェース（「ダッシュボード」および「パネル設定」ページのタブ）から、一部はサーバー上のコンソールメニュー `x-ui` から実行します。

16.1. データベースのバックアップと復元

パネルのすべてのデータ（inbound、クライアント、グループ、ノード、設定）は1つのデータベースに格納されています。バックアップの管理は「ダッシュボード」ページの「バックアップ」タブにある「バックアップと復元」ブロックから行います。

パネルは2種類のデータベースエンジンをサポートしており、バックアップの動作はそれによって異なります。

- **SQLite**（デフォルト） – データはファイル `x-ui.db` に保存されます。
- **PostgreSQL** – パネルがPostgreSQLで設定されている場合、ブロックに次の説明が表示されます：

「このパネルはPostgreSQLで動作しています。「バックアップ」はpg_dumpアーカイブ（.dump）をダウンロードし、「復元」はpg_restoreを使用してそれをアップロードします。復元はSQLiteデータベース（.db）またはSQLiteマイグレーションダンプも受け付け、そのデータをPostgreSQLにインポートします。サーバーにPostgreSQLクライアントツール（pg_dumpおよびpg_restore）がインストールされている必要があります。」

エクスポート（コピーの作成）

「データベースをエクスポート」ボタン（英語：Back Up）はバックアップファイルをデバイスにダウンロードします。

データベースエンジン	ファイル名	サーバーでの処理
SQLite	<code>{host}_YYYY-MM-DD_HHMMSS.db</code>	まずWALチェックポイントを実行してファイルに最新の記録が含まれるようにし、その後ファイル全体を読み込んでダウンロードとして提供します
PostgreSQL	<code>{host}_YYYY-MM-DD_HHMMSS.dump</code>	<code>pg_dump</code> を実行し、アーカイブをダウンロードとして提供します

バージョン3.4.2 から、ダウンロードされるバックアップファイルの名前には、パネルを開いたアドレスと日時が含まれます：`{host}_YYYY-MM-DD_HHMMSS` に拡張子 `.db`（SQLite）または `.dump`（PostgreSQL）が付きます – 例 `panel.example.com_2026-06-27_000000.db`。これにより、ファイルはサーバーごとにグループ化されて時刻順にソートされ、同じ日の複数のコピーが互いに上書きされません。同じ名前は

Telegram ボットが送信するバックアップにも使われます（host はパネルのドメイン/IP から取得され、フォールバックは x-ui ）。

インターフェースの説明：

- SQLite：「クリックして、現在のデータベースのバックアップを含む .db ファイルをデバイスにダウンロードします。」
- PostgreSQL：「クリックして、現在のデータベースのPostgreSQLダンプ (.dump) をデバイスにダウンロードします。」

技術的には、エクスポートは GET /panel/api/server/getDb リクエストです。添付ファイル名はエンジンに応じてサーバーが Content-Disposition で決定します。

バックアップファイルの名前は固定の x-ui.db / x-ui.dump ではなく、サーバーアドレスに基づいて生成されます。ブラウザからダウンロードする場合はアドレスバーのパネルアドレス（リクエストのホスト）から取得し、それ以外の場合は設定済みのWebドメインから、ない場合はサーバーのパブリックIP（まずIPv4、次にIPv6）から、最終的には x-ui にフォールバックします。これにより、異なるサーバーからのバックアップを簡単に区別できます。拡張子はSQLiteの場合 .db、PostgreSQLの場合 .dump のままです。Telegram経由のバックアップも同じドメイン/IPの命名規則に従います。

例：API経由のバックアップダウンロード。 コンソールのリクエストで同じエクスポートを取得することもできます。例えば自動バックアップスクリプト用です。認証済みセッション（ログインcookie）が必要です：

```
# 1) ログインしてセッションのcookieを保存する
curl -s -c cookies.txt \
      -d 'username=admin&password=admin' \
      https://panel.example.com:2053/panel/login

# 2) データベースファイルをダウンロードする（名前はサーバーが決定：x-ui.db または x-ui.dump）
curl -s -b cookies.txt -OJ \
      https://panel.example.com:2053/panel/api/server/getDb
```

パネルがベースパス（Web Base Path）付きで開かれている場合、URLにそれを追加する必要があります： ...:2053/<base_path>/panel/api/server/getDb。

インポート（復元）

「データベースをインポート」 ボタン（英語： Restore ）はファイル選択を開き、復元のためにサーバーにアップロードします（ POST /panel/api/server/importDB、フォームフィールド db ）。

インターフェースの説明：

- SQLite：「クリックして、データベースを復元するために、デバイスから .db ファイルまたはマイグレーションダンプ (.dump) を選択してアップロードします。」
- PostgreSQL：「クリックして、データベースを復元するために、PostgreSQLバックアップ (.dump)、SQLiteデータベース (.db)、またはSQLiteマイグレーションダンプを選択してアップロードします。これにより現在のすべてのデータが置き換えられます。」

3.5.0 から、ファイルの種類は拡張子ではなく内容で判定されます（PGDMP – pg_dumpアーカイブ；「SQLite format 3」ヘッダー – .db データベース；先頭の PRAGMA / BEGIN TRANSACTION – テキストSQL ダンプ）。選択ダイアログは両方のDBMSで .dump, .db を受け付けます。PostgreSQLパネルにアップロードされたSQLiteファイルは、単一ランザクションでPostgreSQLにインポートされます（3.14参照）。インポート前に、ファイルが本物のパネルDBであることが検証され、古いスキーマは自動的にマイグレーションされます。pg_dump アーカイブの復元前に、パネルは（Xrayを停止する前に）事前にその読み取り可能性を確認します：ダンプがより新しいPostgreSQLで作成されている場合、正確なコマンドを含むエラーが表示されます – 例えば「...run 'x-ui pgclient 17'...」。

SQLiteのインポートプロセス（アトミックでロールバック付きであることを理解することが重要です）：

1. アップロードされたファイルは形式チェックされます – 有効なSQLiteデータベースである必要があります。そうでない場合は「Invalid db file format」エラーが返されます。
2. ファイルは一時的な x-ui.db.temp に保存され、整合性チェックが行われます。
3. データベースの置き換え前にXrayが停止されます。
4. 現在のデータベースはバックアップ用の x-ui.db.backup（フォールバック）に名前変更されます。
5. 一時ファイルが作業データベースの場所に移動され、スキーマの初期化とマイグレーション、その後 inboundのマイグレーションが実行されます。
6. いずれかのステップでエラーが発生した場合 – ロールバックが実行されます： x-ui.db.backup から以前のデータベースが復元され、古いデータでXrayが再起動されます。
7. 成功した場合、フォールバックファイルは削除され、Xrayは復元されたデータで自動的に再起動されます。

インターフェースの結果メッセージ：

結果	テキスト
成功	「データベースが正常にインポートされました」
インポートエラー	「データベースのインポート中にエラーが発生しました」
ファイル読み取りエラー	「データベースの読み取り中にエラーが発生しました」

復元により現在のデータが完全に置き換えられます。Xrayはプロセス中に一時的に停止するため、インポート中は既存のクライアント接続が中断されます。

エンジン間のマイグレーションファイル（SQLite ⇄ PostgreSQL）

通常のバックアップとは別に、「マイグレーションファイルをダウンロード」機能（Download Migration、リクエスト GET /panel/api/server/getMigration）があります。これにより、別のデータベースエンジンへの移行用のポータブルファイルが生成されます：

3.5.0 から、このボタンは **PostgreSQLパネルにのみ** 表示されます（SQLiteでは通常の .db バックアップがそのままPostgreSQLに直接復元できるようになったため、個別のエクスポートは不要です）：

現在のエンジン	ダウンロードされるもの	ファイル名	目的
PostgreSQL	PostgreSQLのデータから構築されたSQLiteデータベース	x-ui.db	パネルをSQLiteに戻す

説明: 「クリックして、PostgreSQLデータからビルドされたSQLiteデータベース (.db) をダウンロードします。SQLiteでパネルを実行する準備が整っています。」

SQLiteの `.db ⇄ .dump` 変換は `x-ui migrateDB [file]` CLIコマンドからも実行できます (セクション16.7参照)。また 3.5.0 から、エンジン間のマイグレーションは **トランザクション化され、損失がなくなりました** (クライアントグループやグローバルトラフィックカウンターも移行されます; 失敗したインポートは移行先DBを変更しません)。PostgreSQLクライアントツールの更新のために、コマンド `x-ui pgclient [バージョン]` と PostgreSQLメニューの項目 **10. Install/Upgrade client tools (pg_dump/pg_restore)** が追加されました (必要に応じて公式PostgreSQLリポジトリが接続されます)。

Telegram botを通じたバックアップ

Telegram botが設定されている場合 (通知のセクション参照)、管理者のチャットに直接バックアップを送信できます。Telegram経由のバックアップには **2つのファイル** が含まれます: データベース自体 (`x-ui.db`、またはPostgreSQLの場合は `x-ui.dump`) と Xrayの設定 `config.json` です。メッセージには「**[?] バックアップ時刻: ...**」という行が先行します。

Telegramでバックアップを受け取る方法は2つあります:

1. リクエストによる。botメニューの「 **DBバックアップ**」ボタン - botは現在のチャットにすぐにファイルを送信します。
2. レポートと一緒に自動的に。bot設定には「**データベースバックアップ**」 (Database Backup) トグルがあり、「バックアップファイルを含む通知を送信する」という説明が付いています。これを有効にすると、定期的なレポート配信のたびに、botはレポートの後にすべての管理者にバックアップを送信します。レポートの送信間隔はbotのcronスケジュールで設定されます (セクション16.6参照)。Telegramの制限を超えないよう、ファイル間および管理者間でbotは一時停止を入れます。

bot経由のバックアップはbotが実行中の場合にのみ送信されます。PostgreSQLの場合、サーバー上に `pg_dump` が存在することも必要です。

16.2. ログの閲覧

パネルには2つの独立したログビューアがあり、どちらも「ダッシュボード」の「**ログ**」タブから開きます。各ウィンドウは更新 (ヘッダーの「更新」アイコン) と、表示されている内容をファイル `x-ui.log` にダウンロード (ダウンロードアイコンボタン) する機能を持っています。

パネル (アプリケーション / syslog) のログ

パネルログウィンドウ (`POST /panel/api/server/logs/{count}`)。コントロール:

要素	デフォルト値	説明
行数	20	ドロップダウンリスト: 20 / 50 / 100 / 500 / 1000
レベル	Info	最小レベル: Debug / Info / Notice / Warning / Error
SysLog (チェックボックス)	無効	ログの取得元: アプリケーションバッファまたはシステムジャーナル
自動更新 (チェックボックス)	無効	5秒ごとにログを再読み込みする (下記参照)

動作は **SysLog** チェックボックスによって異なります:

- **無効 (デフォルト)**: ログはパネルの内部リングバッファから取得され、選択したレベルでフィルタリングされます。エントリはレベル (DEBUG / INFO / NOTICE / WARNING / ERROR) とソース付きで表示されます: **X-UI:** はパネル自身のメッセージ、**XRAY:** はXrayから転送されたメッセージです。

タイムスタンプとレベルのない単純な通知 (例: Windowsでの「Syslog is not supported」などのシステムメッセージ) は、そのまま全体として表示されます。YYYY/MM/DD LEVEL - 本文 という形式のみが厳密に認識されます。それ以外はパースせずに出力されるため、そのような行はもはや切り捨てられません (以前は最初の3語が誤って日付/時刻/レベルとして解釈されていました)。

- **有効**: パネルはサーバーで `journalctl -u x-ui --no-pager -n <count> -p <level>` を実行し、**x-ui** サービスのシステムジャーナルを表示します。有効な行数は1から10000まで、レベルはsyslogの値 (emerg/0、alert/1、crit/2、err/3、warning/4、notice/5、info/6、debug/7) を受け付けます。Windowsではsyslogモードはサポートされていません - チェックボックスを外してアプリケーションログを使用するよう警告が表示されます。systemd/サービスが利用できない場合は、`journalctl` 起動エラーメッセージが表示されます。

例: サーバーコンソールから同じジャーナルを参照。 パネルが利用できない場合 (例: 起動しない)、システムジャーナルを直接読み取ることができます - これはパネルがsyslogモードで実行するまさにそのコマンドです:

```
# warningレベル以上の最後の100行
journalctl -u x-ui --no-pager -n 100 -p warning

# リアルタイムでジャーナルを監視
journalctl -u x-ui -f
```

このウィンドウのレベルは出力をフィルタリングします。コンソール/syslogに書き込まれる最小レベルはパネルのログレベル (環境変数、デフォルトは Info) によって決まります。パネルはファイルには常に DEBUG レベルで書き込みます。

Xrayアクセスログ (アクセスジャーナル)

Xrayのアクセスログ用の別ウィンドウ (`POST /panel/api/server/xraylogs/{count}`)。Xrayのアクセスログの行を解析し、テーブルとして表示します: **Date**、**From**、**To**、**Inbound**、**Outbound**、**Email**。

バージョン3.4.1からは、このウィンドウとXrayステータスカードの呼び出しボタンには「アクセスログ」（Access Logs）という表示がされます – 以前は単に「ログ」という名称でした。この名称変更は、Xrayのaccess-logビューアと、以前同じ名称だったパネル自身のログビューアを混同しないようにするために行われました。

要素	デフォルト値	説明
行数	20	20 / 50 / 100 / 500 / 1000
フィルター	空	部分文字列によるテキスト検索（Enterで適用）
自動更新（チェックボックス）	無効	5秒ごとにログを再読み込みする（下記参照）
Direct（チェックボックス）	有効	直接接続を表示（freedom-outbound経由のトラフィック）
Blocked（チェックボックス）	有効	ブロックされた接続を表示（blackhole-outbound宛のトラフィック）
Proxy（チェックボックス）	有効	プロキシされたトラフィックを表示

イベントタイプは、ログ行の送信接続タグによって自動的に決定されます：freedomタグに一致 → 「DIRECT」（緑）、blackhole → 「BLOCKED」（赤）、その他すべて → 「PROXY」（青）。api → apiの行と空の行はスキップされます。

自動更新。 両方のログウィンドウ（「ログ」と「アクセスログ」）には「自動更新」（Auto Update）チェックボックスがあります。これを有効にすると、現在のウィンドウ設定（選択した行数、レベル/フィルター、Direct / Blocked / Proxyのチェックボックス）を維持しながら、ログの内容が5秒ごとに自動的に再読み込みされます。ウィンドウが閉じられるかチェックボックスが外されると、ポーリングが停止します。

このウィンドウにエントリを表示するには、Xrayでファイルパス付きの **アクセスジャーナル** が有効になっている必要があります（none 以外） – 下記参照。access-logが無効またはファイルが利用できない場合、ウィンドウは空（「No Record...」）になります。

16.3. Xrayのログレベルとログ設定

Xray自体のログパラメータは「**Xray設定**」ページの「ログ」（Log）ブロックで設定します。次の警告が表示されます：

「ログはサーバーのパフォーマンスを低下させる可能性があります。必要なログのみ、必要に応じて有効にしてください！」

フィールド	翻訳	デフォルト値	説明
ログレベル (logLevel)	Log Level	warning	Xrayエラーログの詳細レベル。有効な値：debug、info、notice、warning、error。説明：「記録すべき情報を示すエラーログのログレベル。」

フィールド	翻訳	デフォルト値	説明
アクセスログ (accessLog)	Access Log	none	アクセスログファイルへのパス。特別な値 none はアクセスログを無効にします。説明: 「アクセスログファイルへのパス。特別な値「none」はアクセスログを無効にします。」
エラーログ (errorLog)	Error Log	空 (デフォルトパス)	エラーログファイルへのパス。none は無効にします。説明: 「エラーログファイルへのパス。特別な値「none」はエラーログを無効にします。」
DNSログ (dnsLog)	DNS Log	false (無効)	DNSクエリのログを有効にします。説明: 「DNSクエリのログを有効にする。」
アドレスマスキング (maskAddress)	Mask Address	空 (無効)	有効にすると、ログ内の実際のIPアドレスが自動的にマスクされたアドレスに置き換えられます。説明: 「有効にすると、ログ内の実際のIPアドレスがマスクされたアドレスに置き換えられます。」

デフォルトでは「アクセスログ」 = none であるため、「Xrayアクセスログ」ウィンドウ (セクション16.2) は最初は空です。機能させるには、ここでaccess-logへのパスを設定してXrayを再起動してください。

空のaccess-logはこのウィンドウにのみ影響することに注意してください。「ダッシュボード」のオンラインクライアントリストとクライアントフォームのIPリミットは、access-logに **依存しません** – パネルはXrayコアのonline-stats API (接続統計) を通じてオンラインクライアントを特定しIPアドレスをカウントします。このAPIが存在しない古いバージョンのコアでは、パネルは自動的に以前の方法 (access-logの読み取り) にフォールバックし、その場合はIPリミットのためにここでaccess-logへのパスが引き続き必要になります。

IPリミットとfail2ban。 クライアントのIPリミット (クライアントフォームと一括追加の「IP Limit」フィールド) は、fail2ban がインストールされている場合にのみサーバー上で適用されます – fail2banがリミットを超えたアドレスをブロックします。パネルはfail2banの存在をチェックし (GET /panel/api/server/fail2banStatus)、存在しない場合、「IP Limit」フィールドは無効になり説明付きのツールチップが表示されます (Windowsでは別のメッセージ)。そのようなサーバーでは以前に設定されたリミットはいずれにせよ機能していなかったため、自動的にゼロにリセットされます。fail2banのブロックはTCPとUDPの両方に適用されます。3.5.0 から、「死んだ」接続 (正常なTCPクローズなしにクライアントが消えた場合) は、10秒ごとのスキャンのたびではなく **1回だけ** 記録されます: [LIMIT_IP] 行と切断サイクルは実際の再接続時にのみ繰り返されるため、fail2banのカウンターが不必要に増加せず、maxretry を大きめに設定する必要はもうありません (ログの形式とfailregexは変更されていません)。通常のサーバーでは、パネルのインストールおよび更新時にfail2banが自動的にインストールされるようになりました (セクション16.5参照)。

例: 「Xrayアクセスログ」ウィンドウにエントリが表示される log ブロック。 XrayのJSON設定では次のようになります:

```
{
  "log": {
    "loglevel": "warning",
    "access": "./access.log",
    "error": "",
    "dnsLog": false,
    "maskAddress": ""
  }
}
```

重要なのは `"access": "none"` をファイルへのパス（例: `./access.log`）に置き換えることです。保存後にXrayを再起動すると、「Xrayアクセスログ」ウィンドウのテーブルに行が表示されます。

16.4. Xrayの管理：停止と再起動

Xrayの状態は「ダッシュボード」のXrayカードから管理します。現在の状態は次のいずれかの値として表示されます：**実行中**（Running）、**停止中**（Stopped）、**不明**（Unknown）、**エラー**（Error）。エラーの場合、「Xrayの起動エラー」というツールチップが表示されます。

ボタン	翻訳	エンドポイント	アクション
停止	Stop	POST /panel/api/server/stopXrayService	Xrayプロセスを停止します。成功した場合 – 警告通知「Xray service has been stopped」。
再起動	Restart	POST /panel/api/server/restartXrayService	現在の設定を適用してXrayを再起動（または起動）します。成功した場合 – 通知「Xray service has been restarted successfully」。

どちらの操作後もパネルはWebSocketで新しい状態をブロードキャストするため、「ダッシュボード」のステータスはページをリロードせずに更新されます。操作でエラーが発生した場合、Xrayの状態は「エラー」になり、エラーテキストが通知に表示されます。

手動での再起動に加えて、パネルはXrayを再起動する必要があるかどうかを自動チェックし（30秒ごとのバックグラウンドタスク）、プロセスがクラッシュしていないかどうかを確認します（1秒ごとのチェック） – セクション16.6参照。

トンネルヘルスマニター（Xray自動再起動）

バージョン3.4.1ではオプションの **トンネルヘルスマニター** が追加されました。有効にすると、パネルは指定されたURLへのアクセス可能性を定期的にチェックし、連続して複数回の障害確認が発生した後にXrayコアを自動的に再起動します – これはトラフィックを通過させなくなったトンネルを回復するのに役立ちます。デフォルトではモニターは **無効** であり、**サービスの環境変数のみで設定** されます（Webインターフェースには設定がありません – 開発者による意図的な設計です）。

モニターを有効にする変数は `XUI_TUNNEL_HEALTH_MONITOR=true` です。変数 `XUI_TUNNEL_HEALTH_PROXY` はローカルのxray-inbound（例: `socks5://127.0.0.1:1080`）を指定する必

要があります – こうするとプローブはXray自体を通して行われ、トンネルを正確にチェックします。指定しない場合はホストの接続性のみがチェックされ、サーバーのネットワーク接続の問題は再起動では解決できません。その他の変数はチェックパラメータを設定します：

変数	目的	デフォルト
<code>XUI_TUNNEL_HEALTH_MONITOR</code>	モニターを有効にする（オン/オフ）	<code>false</code>
<code>XUI_TUNNEL_HEALTH_PROXY</code>	プローブが経由するプロキシ（ローカルのxray-inboundを指定）	空
<code>XUI_TUNNEL_HEALTH_URL</code>	チェックするURL	<code>https://www.cloudflare.com/cdn-cgi/trace</code>
<code>XUI_TUNNEL_HEALTH_INTERVAL</code>	チェックの間隔	<code>30s</code>
<code>XUI_TUNNEL_HEALTH_TIMEOUT</code>	1回のチェックのタイムアウト	<code>10s</code>
<code>XUI_TUNNEL_HEALTH_FAILURES</code>	再起動までの連続障害数	<code>3</code>
<code>XUI_TUNNEL_HEALTH_COOLDOWN</code>	再起動間の最小休止時間	<code>5m</code>

Xrayを再起動すると接続中のすべてのクライアントの接続が切断されるため、偶発的な1回のプローブ障害が不必要な再起動を引き起こさないよう、間隔と障害しきい値は十分に大きな値に設定しておくことが望ましいです。

16.5. パネルの再起動と更新

パネルの再起動

「パネル設定」ページには「パネルを再起動」（`Restart Panel`、`POST /panel/api/setting/restartPanel`）アクションがあります。確認するとパネルは **3秒後に** 再起動されます。

メッセージ：

- 確認：「パネルを再起動してもよいですか？ 確認すると3秒後に再起動が行われます。パネルが利用できない場合は、サーバーのログを確認してください。」
- 成功：「パネルが正常に再起動されました。」

技術的に、Linuxでの再起動はパネルプロセスに `SIGHUP` シグナルを送信することで行われます（または登録されたフックを通じて）。Windowsでは `SIGHUP` の送信はサポートされていません。

パネルの自己更新（Update Panel）

「ダッシュボード」には「パネルを更新」（`Update Panel`）機能があります – WebインターフェースからDirectlyに3X-UIを最新リリースに更新します。

更新前にパネルはバージョンを比較し（`GET /panel/api/server/getPanelUpdateInfo`）、GitHubから最新の3x-uiリリースをリクエストします：

フィールド	翻訳
現在のパネルバージョン	Current panel version
最新のパネルバージョン	Latest panel version
パネルは最新です / 「最新」	Panel is up to date / Up to date – 新しいバージョンがない場合に表示されます

更新の開始 – POST /panel/api/server/updatePanel 。確認ダイアログ:

- 「本当にパネルを更新しますか? 」
- 「これにより3X-UIがバージョン #version# に更新され、パネルサービスが再起動されます。」

開始後 – ポップアップメッセージ「パネルの更新が開始されました」 (Panel update started) 。バージョンチェックが失敗した場合 – 「パネルの更新チェックが失敗しました」 (Panel update check failed) 。

サーバーでの動作: 自己更新は **Linuxのみでサポートされています** (他のOSでは「panel web update is supported only on Linux installations」エラーが返されます)。パネルはGitHubから公式スクリプト `update.sh` をダウンロードし (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`) 、別プロセスとして実行します: 可能であれば `systemd-run` を使用して別ユニット (`x-ui-web-update-<timestamp>`) として、systemdがない場合は分離した別プロセスとして実行されます。完了後、スクリプトはコンポーネントを更新してパネルサービスを再起動します。実行には `bash` が必要です。

更新中にスクリプトが新しいランダムなWebパネルのベースパス (Web Base Path) を生成した場合、新しいパスがすぐに機能するよう `x-ui` サービスが自動的に再起動されます。(再起動しなければ、サーバーは古いパスを提供し続け、インターフェースは新しいパスを表示するため、手動で再起動するまで新しいアドレスにアクセスできなくなります。)

Dev チャンネル (rolling) 、3.4.2 から。「ダッシュボード」のパネルバージョンボタンが常に更新ウィンドウを開くようになり (以前は最新の安定版ビルドではGitHubのリリースページへ移動していました)、「**Dev チャンネル**」トグル (`devChannelEnable`) が常に表示されるようになりました – 安定版ビルドでも表示されます。これを有効にすると、次の更新時に「rolling」の dev チャンネル (`main` への各コミットごとのビルド) へ移行できます。有効化すると警告が表示されます: 「Dev ビルドは main への各コミットを追跡しており、安定版リリースではありません – 自動ロールバックはありません」。ユーザーが明示的に操作しない限り、何も更新されません。

Devチャンネルへの更新 (コミットごとのローリングビルド)

安定版リリースへの通常の更新に加えて、オプションの「**開発チャンネル**」 (Dev) があります。このスイッチはパネルの更新ウィンドウにdev-buildの場合 (個別のコミットでビルドされたCIビルド) にのみ表示されます。安定版リリースには表示されません。有効にすると、パネルは `dev-latest` というローリングビルドに更新されます。これは `main` ブランチの各コミットを追跡しており、安定版リリースではありません – devビルドは不安定で自動ロールバックはないという警告が表示されます。devモードではウィンドウにバージョン番号の代わりに「現在のコミット」/「最新のコミット」が表示されます。この機能はsystemdを搭載したLinuxでのみ利用可能です。

dev-buildではパネルのバージョンは誤解を招く安定版番号の代わりに `dev+<短いコミット>` として表示されます – サイドパネルバッジ、「ダッシュボード」カード、更新ウィンドウ、Telegram botのステータス

レポート、および `x-ui -v` コマンドの出力で確認できます。安定版リリースではバージョン表示は変わりません。

ノード（ノード）上のパネルの同じ3x-uiは `POST /panel/api/nodes/updatePanel` を通じて一元的に更新されます – ノードに関するセクション参照。

fail2banの自動インストール

クライアントのIPリミット（セクション16.3）がすぐに機能するよう、通常のサーバーへのパネルのインストールおよび更新時に `fail2ban` が自動的にインストールおよび設定されるようになりました（以前はDockerイメージ内でのみ行われていました）。動作は環境変数 `XUI_ENABLE_FAIL2BAN` で制御されます。変数が設定されていないか `true` に等しい場合にセットアップが実行されます。手動での実行はコマンド `x-ui setup-fail2ban` で行えます。`fail2ban`のセットアップの失敗はパネルのインストールまたは更新を中断しません。

IPv6のみのホストへのインストールと更新

`install.sh` と `update.sh` スクリプトは、IPv6のみのサーバーでも正しく機能するようになりました。リリース、スクリプト `x-ui.sh`、サービスファイルのダウンロードはもはや強制的にIPv4（`curl -4`）を使用せず、利用可能なプロトコルを使用します。そのため、IPv4アドレスのないホストにもパネルをインストールおよび更新できます。

3.5.0 でのスクリプトの修正

- **RHEL 系**（Rocky/Alma/RHEL/Oracle）：メニューからのローカルPostgreSQLインストールが動作するようになりました（`ident`の代わりにパスワード認証）。IPリミット用の`fail2ban`は **EPEL** からインストールされます。
- **Arch/Manjaro**: インストールと更新が完全なシステムアップグレード `pacman -Syu` を実行しなくなりました – パッケージデータベースの更新のみを行います。
- **32ビットARM**: ダウンロードされるXrayバイナリは `xray-linux-arm32` という名前になりました – パネルが起動するとおりの名前です（以前は更新が別の名前でファイルを配置し、パネルは古いコアを使い続けていました）。
- **IP証明書**: 発行前に、自動検出されたパブリックIPv4が確認のために表示されます（Enter – 承認）。拒否した場合は検証付きの手動入力になります。
- **ACMEポートの選択**: Enter（デフォルトのポート80を承認）が誤った「Your input is invalid」を表示しなくなりました。

XUI_PORT 変数によるパネルポートの上書き

Webパネルのリスニングポートは環境変数 `XUI_PORT` で上書きできます – これは現在のプロセスの実行時間中にのみ有効であり、データベースに保存されている `webPort` の値は **変更しません**。有効な値は `1` から `65535` まで。空の値、不正な値、または範囲外の値は無視され（`webPort` が使用されます）、ログに警告が記録されます。これはデプロイ時、特にDockerで便利です。ブリッジネットワークを使用してい

る場合、コンテナの公開ポートは `XUI_PORT` と一致する必要があります – 例えば `XUI_PORT=8080` と `ports: "8080:8080"` のように。

Xray-coreのバージョン更新と切り替え

「ダッシュボード」ではパネルとは別にXray-coreのバージョンを管理することもできます。

- **Xrayの更新**（Xray Updates）/**バージョン選択**（Version） – 利用可能なバージョンのドロップダウンリスト。説明：「必要なバージョンを選択してください」および警告「重要：古いバージョンは現在の設定をサポートしていない可能性があります。」
- バージョンのインストール/変更 – `POST /panel/api/server/installXray/{version}`。ダイアログ：「Xrayのバージョンを切り替える」/「本当にXrayのバージョンを変更しますか？」成功した場合 – 「Xrayが正常に更新されました。」

例：APIリクエストによるXray-coreのバージョン変更。 バージョンはXTLS/Xray-coreのリリースタグで指定します（`v` プレフィックス付き）。例えば、`v1.8.24` に切り替える場合：

```
curl -s -b cookies.txt -X POST \
  https://panel.example.com:2053/panel/api/server/installXray/v1.8.24
```

（`cookies.txt` – セクション16.1の例のcookieファイル。）インストール後、Xrayは選択したバージョンで自動的に再起動されます。

サーバーでのバージョン変更時、まずXrayが停止され、GitHubから必要なバージョンのアーカイブがダウンロードされ（XTLS/Xray-core）、バイナリが展開されて置き換えられ、その後アーカイブ/バイナリのチェックサム検証とともにXrayが再起動されます。

16.6. 定期タスク（cron）

パネルは起動時にいくつかのバックグラウンドタスクを登録します。スケジュールは固定されています（Telegramレポートのスケジュールとldap同期を除き、UIでは設定できません）。以下は運用に関連するタスクです。

タスク	スケジュール	目的
Xrayの動作確認	1秒ごと	Xrayプロセスが実行中であることの監視
Xray再起動の必要性確認	30秒ごと	設定が変更済みとしてマークされている場合に再起動
Xrayトラフィックの収集	5秒ごと（起動後5秒から開始）	inbound/クライアントのトラフィック計算
クライアントIPの確認	10秒ごと	ログによるIPリミットの監視
ノードのハートビートとトラフィック同期	5秒ごと	ノードとの通信

タスク	スケジュール	目的
ログのクリーンアップ	毎日（@daily）	IPリミットログと永続アクセスログをクリアし、現在のログを *.prev.log にローテーション。3.5.0 から、毎日のクリーンアップは Xray の error ログも 対象になりました（以前は access のみ）
Xray ログの肥大化制限	10分ごと（@every 10m）	3.5.0 の新機能。Xray の access ログと error ログのいずれかが 64 MiB を超えた場合に切り詰めます。無効化されたログ（none /空）には触れません
期間によるトラフィックリセット	@hourly、@daily、@weekly、@monthly	対応する自動リセット期間が設定されているinbound（およびそのクライアント）のトラフィックカウンターをリセット
Telegramレポート	bot設定で指定（デフォルト @daily）	管理者へのレポート配信。オプションが有効な場合 – データベースバックアップファイルを添付（セクション16.1）
TelegramハッシュストアのリセットSE	2分ごと	botが有効な場合のみ
TelegramのCPU負荷監視	10秒ごと	CPUしきい値 > 0 が設定されている場合のみ

補足：

- **定期的なトラフィックリセット** は、対応する自動リセットモード（毎時/毎日/毎週/毎月）が選択されているinboundのみに対して実行されます。タスクはinbound自体とそのすべてのクライアントのトラフィックをリセットします。
- **期限切れと枯渇の確認**。クライアントの期限切れとトラフィックリミット枯渇による無効化はトラフィック計算の中で実行されます： expiry_time が切れているまたはトラフィック量を使い切っているクライアントはマークされ無効化されます。必要に応じて次の期限が計算されます（循環リミットと「最初の使用からカウント」モードの場合）。「ダッシュボード」とリストではこれが「期限切れ」/「枯渇」/「まもなく終了」のステータスとして反映されます。
- **Telegram自動バックアップ** – これはレポートタスクの副作用であり、バックアップのみのための別のcronスケジュールはありません。そのため、自動バックアップの頻度はbotレポートの頻度と同じです。

16.7. コンソールメニューとCLI（x-ui）

サーバーではパネルは x-ui コマンドで管理されます。引数なしでインタラクティブメニュー「3X-UI Panel Management Script」が開きます。引数を指定すると特定のサブコマンドが実行されます。運用に関連するメニュー項目：

メニュー番号	項目	アクション
1	Install	パネルのインストール（install.sh をダウンロードして実行）
2	Update	データを失わずにx-uiのすべてのコンポーネントを最新バージョンに更新し、自動的に再起動

メニュー番号	項目	アクション
3	Update to Dev Channel (latest commit)	確認付きでローリングビルド <code>dev-latest</code> (<code>main</code> ブランチの最新コミット) に更新 (セクション16.5参照)
4	Update Menu	メニュースクリプト <code>x-ui</code> 自体のみを更新
5	Legacy Version	入力したバージョン番号 (例: <code>2.4.0</code>) で指定の (古い) バージョンのパネルをインストール
6	Uninstall	パネルとXrayの完全削除 (セクション16.8参照)
7	Reset Username & Password	管理者のログイン/パスワードのリセット
8	Reset Web Base Path	Webパネルのベースパスのリセット
9	Reset Settings	設定をデフォルト値にリセット
10	Change Port	パネルポートの変更
11	View Current Settings	現在の設定の表示
12-14	Start / Stop / Restart	パネルサービスの起動、停止、再起動
15	Restart Xray	Xrayのみの再起動
16	Check Status	サービスの現在のステータス
17	Logs Management	ログの表示とクリア (下記参照)
18-19	Enable / Disable Autostart	OS起動時のサービス自動起動の有効化/無効化
27	Update Geo Files	ジオファイル (GeoIP/GeoSite) の更新
25	PostgreSQL Management	PostgreSQLの管理

メニュー番号はバージョン3.4.1で変更されました: 項目3「Update to Dev Channel」の追加により、それ以降のすべての項目が1つずつずれました。項目の合計は28になり、選択は [0-28] の範囲で入力します。

CLIでのログ管理 (項目16)

「Logs Management」サブメニューは現在項目 17 (以前は16) で開きます:

- **Debug Log** – サービスジャーナルのストリーミング表示:
`journalctl -u x-ui -e --no-pager -f -p debug` (Alpineでは `/var/log/messages` を `grep`) 。
- **Clear All logs** – システムジャーナルのクリア: `journalctl --rotate + journalctl --vacuum-time=1s`、その後サービスが再起動されます。(Alpineでは利用不可。)

`x-ui` の直接サブコマンド

利用可能なすべてのサブコマンド:

コマンド	説明
<code>x-ui</code>	管理メニューを開く
<code>x-ui start</code>	パネルを起動
<code>x-ui stop</code>	パネルを停止
<code>x-ui restart</code>	パネルを再起動
<code>x-ui restart-xray</code>	Xrayを再起動
<code>x-ui status</code>	現在のステータス
<code>x-ui settings</code>	現在の設定を表示
<code>x-ui enable</code>	OS起動時の自動起動を有効化
<code>x-ui disable</code>	自動起動を無効化
<code>x-ui log</code>	ログを表示
<code>x-ui banlog</code>	Fail2banのバンログを表示
<code>x-ui setup-fail2ban</code>	IPリミット用のfail2banをインストールして設定（セクション16.5参照）
<code>x-ui update</code>	パネルを更新

| `x-ui update-dev` | パネルを開発チャンネル（ローリングビルド `dev-latest`）に更新 || `x-ui update-all-geofiles` | すべてのジオファイルを更新（その後再起動） || `x-ui migrateDB [file]` | データベースの `.db` ⇄ `.dump` 変換（SQLite） || `x-ui legacy` | 古いバージョンをインストール || `x-ui install` | パネルをインストール || `x-ui uninstall` | パネルをアンインストール |

コマンド `x-ui update` は公式の `update.sh` をダウンロードして実行します（セクション16.5のWeb更新と同じ）。確認を求めます：「This function will update all x-ui components to the latest version, and the data will not be lost.」完了後、パネルは自動的に再起動されます。

setting サブコマンドの `-webCert` / `-webCertKey` フラグ。WebパネルへのSSL証明書と秘密鍵へのパスは `x-ui setting -webCert <パス> -webCertKey <パス>` サブコマンドで直接設定できます — これらのフラグのいずれかを指定すると対応するパスが保存され（別の `cert` サブコマンドと同様）、パネルはすぐにHTTPSに移行します。

CLI経由のAPIトークン取得

CLI経由のAPIトークン取得コマンド（メニュー項目/`x-ui` コマンド）は、以前に発行されたトークンを表示しません。APIトークンはハッシュ形式でのみ保存されているため、既存のトークンを平文で取得することはできません。トークンがすでに設定されている場合、コマンドはその数を報告し、パネル（**Settings** → **API Tokens**、APIトークンに関するセクション参照）でトークンを管理するよう案内し、直ちに `cli-fallback-<タイムスタンプ>` という形式の名前で **新しいフォールバックトークン** を生成して表示します。これにより、インターフェースにアクセスせずにCLIが引き続き役立ちます。

16.8. パネルのアンインストール

アンインストールはCLIから実行します。メニュー項目 **5 (Uninstall)** またはコマンド `x-ui uninstall`。アンインストール前に確認が求められます（デフォルトは「いいえ」）：「Are you sure you want to uninstall the panel? xray will also uninstalled!」

確認した場合、スクリプトは：

1. サービスを停止して自動起動を無効化し（`systemctl stop/disable x-ui`、またはAlpineでは `rc-service / rc-update`）、ユニットファイルを削除してsystemdの設定を再読み込みします。
2. データとアプリケーションのディレクトリ（`/etc/x-ui/`、インストールディレクトリ）とサービスのenvファイル（ディストリビューションに応じて `/etc/default/x-ui`、`/etc/conf.d/x-ui`、または `/etc/sysconfig/x-ui`）を削除します。
3. `x-ui` スクリプト自体を削除して「Uninstalled Successfully.」と再インストール用のコマンドを表示します。

パネルがPostgreSQLを使用していた場合（envファイルに `XUI_DB_TYPE=postgres`）、パネルファイルの削除後、スクリプトはPostgreSQLサーバー自体とそのすべてのデータベースも削除するかどうかを追加で尋ねます：「Also purge PostgreSQL and delete all of its data?」このリクエストには明示的な確認が必要であり（デフォルトは拒否）、警告が伴います：削除はマシン上の **すべての PostgreSQLデータベース**（他のアプリケーションに属するものも含む）に影響し、不可逆的です。拒否した場合、PostgreSQLとそのデータは変更されません。

アンインストールは不可逆的です：パネルとともにXrayとすべてのデータ（データベースを含む）が削除されます。データが必要な場合は、事前にデータベースをエクスポートしてください（セクション16.1）。

16.9. x-ui migrateDB コマンド

バージョン3.3.0からは、管理スクリプト `x-ui.sh` が `migrateDB` サブコマンドを受け取りました。パネルデータベースSQLiteをバイナリ形式の `.db` とポータブルなテキストダンプ `.dump`（通常のSQLテキスト）の2形式間で変換するための組み込みバイナリ `x-ui`（`x-ui migrate-db`）のラッパーです。

コマンドの動作

コマンドは2方向で動作し、方向は入力ファイルによって **自動的に** 決定されます：

方向	名称	動作
<code>.db → .dump</code>	dump（エクスポート）	SQLiteのバイナリデータベースをテキストSQLファイルにエクスポート
<code>.dump → .db</code>	restore（復元）	テキストSQLファイルからSQLiteのバイナリデータベースを再構築

内部では、スクリプトはパネルのバイナリを呼び出します：

- エクスポート： `x-ui migrate-db --src <入力> --dump <出力>`

- 復元: `x-ui migrate-db --restore <入力> --out <出力>`

呼び出し構文

```
x-ui migrateDB [file.db|file.dump] [output]
```

- **[file.db|file.dump]** – 入力ファイル（第1引数）。指定しない場合、デフォルトのパネルデータベース `/etc/x-ui/x-ui.db` が使用されます。
- **[output]** – 出力ファイルへのパス（第2引数）。省略可能: 指定しない場合、入力ファイルの隣に自動的に名前が選択されます（下記参照）。

例:

```
x-ui migrateDB                                # /etc/x-ui/x-ui.db -> /etc/x-ui/x-ui.dump
にエクスポート
x-ui migrateDB /etc/x-ui/x-ui.db backup.dump
x-ui migrateDB backup.dump restored.db        # ダンプから .db を構築
```

方向の決定方法

スクリプトは入力ファイルの拡張子を確認します:

- `*.db`、`*.sqlite`、`*.sqlite3` → **dump** モード（テキストへのエクスポート）
- `*.dump`、`*.sql` → **restore** モード（データベースの構築）

拡張子が認識されない場合、スクリプトはファイルの最初の16バイトを読み取ります: シグネチャ `SQLite format 3` はバイナリデータベース（dumpモード）を意味し、それ以外はダンプとして扱われます（restoreモード）。

第2引数が指定されない場合の出力ファイル名:

- エクスポート時 – 入力と同じ名前拡張子 `.dump`
- 復元時 – 同じ名前拡張子 `.db`

保護チェックと動作

- **バイナリの存在確認。** バイナリ `x-ui` が見つからないか実行可能でない場合 – 「x-ui binary not found ... Is the panel installed?」エラーが表示されます。
- **ビルドでの機能サポート確認。** スクリプトはバイナリが `migrate-db --dump/--restore` をサポートしているかチェックします（`x-ui migrate-db -h` を通じて）。サポートしていない場合 – まず `x-ui update` コマンドでパネルを更新するよう提案されます。
- **入力ファイルの存在確認。** 入力ファイルが存在しない場合はエラーと呼び出し構文が表示されます。
- **出力の上書き。** 出力ファイルがすでに存在する場合、確認が求められます（デフォルトは「いいえ」）。確認なしでは操作がキャンセルされます。復元時には古い出力ファイルが事前に削除されます。

- 「ライブ」データベースの保護。デフォルトデータベース `/etc/x-ui/x-ui.db` への復元時、パネルが実行中の場合、操作は拒否され、まずパネルを停止する（`x-ui stop`）か別の出力パスを選択するよう求められます。これにより実行中のサービスの動作データベースの上書きを防ぎます。
- データベース構築が失敗した場合、不完全な出力ファイルは削除されます。

用途

- バックアップ。テキスト形式の `.dump` は人間が読めるため、バージョン管理システムへの保存やデータベースの内容の差分表示に便利です。
- 移行。ダンプはマシン間での移植性があり、SQLiteファイル形式のバージョンの違いに対して堅牢です – 新しいサーバーで機能する `.db` を構築できます。
- 診断。 `.dump` からSQLiteツールなしでパネルの構造とデータを目視で確認できます。

インタラクティブモード

直接呼び出しに加えて、変換はインタラクティブメニューからも利用できます。PostgreSQLサブメニュー（`x-ui` → PostgreSQL管理セクション）には **9. Convert SQLite `.db` <-> `.dump`** という項目があります。入力ファイルへのパス（デフォルト `/etc/x-ui/x-ui.db`）と出力ファイルへのパス（自動命名の場合は空のまま）を尋ね、方向はCLIモードと同様に自動的に決定されます。

このドキュメントは3X-UIのソースコードに基づいて作成されています。お使いのバージョンでインターフェースの項目が異なる場合は、パネルの動作とUI内の説明が優先されます。